

Coursera Course Transcripts

Formal Languages And Applications

Generated on June 24, 2026

Total lectures: 193

Modules / Weeks: 1

Formal Languages and Applications

Meet Your Instructor - Prof. S.P. Vimal Video• . Duration: 1 minute

[Notes]

Hello, everyone. My name is SP Vimal. I'll be one of your instructors for the course Formal Languages and Applications. I'm an Associate Professor in the CSIS Department WLP Division, BITS Pilani. I completed my B.E. and M.E. in Computer Science and Engineering from MS University. I'm working for BITS Pilani since 2006. I teach courses on Algorithms, Compiler Design, Reinforceable Learning, Information Retrieval, etc. My research interests are in Information Retrieval, Remote Sensing, Computer Vision, and Reinforceable Learning. I consider myself a passionate educator and I take great interest in teaching Computer Science course. I hope you will enjoy the course Formal Languages and Applications.

Meet Your Instructor - Prof. Kunal Korgaonkar Video• . Duration: 1 minute

[Notes]

(No transcript found for this lecture.)

Course Introductory Video Video• . Duration: 2 minutes

[Notes]

Dear students, welcome to our course on Formal Languages and Applications. We're excited to have each one of you in our course on formal languages. Over the next 10 weeks, we'll learn about formal languages. As you know, this is a central theme in computer science. This course will uncover how the seemingly abstract concepts of formal languages gets more interesting. We'll learn some important applications of formal languages as well. Formal languages are not languages we use in our daily communication.

Course Overview Reading• . Duration: 10 minutes

[Notes]

Overview

This introductory course on Formal Languages and their applications provides a foundational understanding of formal languages and their significance in computer science. Through a blend of theory and practice, students will explore various types of languages, their properties, and methods for generating and recognising these languages. Key topics include applications in programming and natural languages, with an emphasis on hands-on exercises, such as pen-and-paper problem-solving and programming projects. By the end of the course, students will have developed

the skills to design and implement programs like lexical analysers and parsers, essential tools for processing and recognising formal languages. Additionally, students will learn to identify and analyse computational problems, applying formal language techniques to create effective, real-world solutions. The core learning materials included in the course are the videos and the readings. Each module includes videos, which deliver learning based on the objectives defined for the module. The course includes essential readings in the form of sections listed from the prescribed book and links, which are compulsory for a learner to go through. The course also includes some recommended readings which are helpful to get a deeper understanding of the concepts but are not compulsory. You are required to purchase the following book and refer to it for the course. There are formative and summative assessments in the course. All assessments have been designed to promote learner engagement and student success in the online model. Formative assessments are ungraded/practice assessments that guide and support learning. Relevant videos have practice quiz questions that help deepen your understanding of new concepts and measure your progress toward the learning objectives. Practice questions are provided for the relevant topic or video and appear as a quiz after the video. You have unlimited attempts for each practice quiz. At the end of every alternate week, we also have a graded quiz corresponding to the learning objectives. Feedback will be provided for your submission for a better understanding and learning. The graded quizzes are auto-graded and count for 30% of your overall course grades. You will have one attempt for each graded quiz. There is a staff-graded assignment in Week 3 (Part 1) and Week 6 (Part 2) of the course. For staff-graded assignment: You will work independently and work through the problems in response to the questions provided in the staff-graded assignment in Week 3 (Part 1) and Week 6 (Part 2). The graded assignment forms an important component of your final grading for the course. They count for 20% of your overall course grades. You can attempt the graded assignment only once. There are ten ungraded discussion prompts in the course, one corresponding to each week. Discussion prompts help to facilitate the sharing of ideas between students and generate peer learning. The program staff monitors these prompts to support the students with additional information and ensure that the community engagement in the discussion forums remains academic and polite. Staff-Graded Assignment (Individual Assignment) Term-End Comprehensive Examination (Online Proctored Exam) BITS Pilani follows a continuous system of internal evaluation. The concept of relative grading is applied. Letter grades A, A-, B, B-, C, C-, D, and E carrying grade points 10, 9, 8, 7, 6, 5, 4, and 2, respectively, are awarded for all regular courses. For project-type courses, qualitative grades, excellent, good, fair, and poor, are awarded. A grade sheet will be issued at the end of each semester, showing the grades and CGPA obtained. The cumulative grade point average (CGPA), calculated on a 10-point scale, will describe your overall performance. It is the weighted average of the grade points of all the letter grades received by you in the program.

$$CGPA = \frac{U_1 * G_1 + U_2 * G_2 + U_3 * G_3 + \dots + U_n * G_n}{U_1 + U_2 + U_3 + \dots + U_n}$$

where $U_1, U_2, U_3, \dots, U_n$ denote units of the course, and $G_1, G_2, G_3, \dots, G_n$ denote grade points of the letter grades awarded in that course. Non-letter grades are not considered while computing CGPA. Instructor: Prof. SP Vimal and Prof. Kunal Korgaonkar The live online classroom sessions provide you with opportunities to interact with the faculty. You will have one live classroom session every week for approximately one hour.

Introduction Video• . Duration: 2 minutes

[Notes]

(No transcript found for this lecture.)

Introducing Alphabets Video• . Duration: 2 minutes

[Notes]

Dear students, welcome to this video. In this video, I am going to introduce you to alphabets in formal language. At the end of this video, you will understand what are alphabets in formal languages and you will also see some examples of alphabets. Alphabets are the most basic element for any language. Likewise, in formal language, alphabet is the most basic element. What you see on the screen is the formal definition of what an alphabet is. An alphabet in formal languages is any finite and non-empty set of symbols.

Introducing Strings Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to introduce you to strings in formal languages. I'll do this by introducing you to a formal definition of what a string is. And I'll also give you some examples, strings in formal languages. At the end of this video, you will be able to verify if a string in alphabet is given to you, you will be able to verify if the string is valid according to a given alphabet. And you will also learn certain other simple aspects about strings. Let's get started.

Operations - Concatenation & Reverse Video• . Duration: 4 minutes

[Notes]

(No transcript found.)

Operations - Prefix, Suffix & Substring Video• . Duration: 8 minutes

[Notes]

Dear students, welcome to this video. In this video, I'll introduce you to three important string operations. One is prefix, second one is suffix, third one is a substring operation. I'll give you the formal definition of these operations and I'll also give you examples of these operations so that you can understand this better. Let's get started. The first operation is a prefix operation. The definition is, x is a prefix of a string y if a string z exists such that x is equal to y .

Kleene Star $[\Sigma^*]$ Video• . Duration: 4 minutes

[Notes]

(No transcript found.)

Properties on String Operations - A Proof Video• . Duration: 9 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to prove a simple property on a string of operations using a proof technique called as proof by induction. In this course, we are going to state a lot of properties about formal languages and we are going to prove using formal techniques. Proof by induction is one of the most important proof techniques. Treat this video as a refresher for your proof by induction proof technique. Let's get started. The property that we want to prove is on the screen.

Introducing Languages Video• . Duration: 9 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to introduce you to the formal definition of a language. And I'm going to give you some examples of language. And then I'm going to introduce you to the connection between language, grammar and automata. So let's get started. A language is any set of strings over an alphabet Σ . Let's try to understand this better.

Operations on Languages Video• . Duration: 9 minutes

[Notes]

Dear students, welcome to this video. In this video, I am going to introduce you to a set of operations on languages. At the end of this video, you will be able to perform operations on existing languages to create new language. A language is any set of strings over an alphabet. It's important to observe that a language is a set. Right? Because it's a set of strings over a given alphabet.

Introduction to Grammar Video• . Duration: 13 minutes

[Notes]

Dear students, welcome to this video. In this video, you will be introduced to the notion of grammar in the formal languages. Grammar plays a crucial role in the structure and interpretation of languages. By the end of this video, you will be able to understand 1. What are grammars in formal languages? 2. The role of grammars in formal languages.

Introduction to Automata Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, I'll introduce you to the notion of automata briefly and its role in the study of formal language. The term automaton refers to an abstract model of a computer. In this course on formal languages, we're going to study different kinds of formal languages. We define grammar for different languages. We use grammar to write or generate valid sentences of a given language. Grammar helps us to verify if a sentence is valid as per the

grammar of a language. We can assume that this takes a sentence as an input and returns an answer, yes or no. Yes means sentence is valid or accepted. No means the sentence is not valid. Let's refer grammar for the toy expression language again. These productions are known to you anyway. We know that we can generate valid sentences using this grammar. Let us now assume the task is opposite. We are given a sentence and the task is to verify if the sentence is valid as per the grammar. That is, is the sentence A plus C valid as per the grammar? Is the sentence 5 star 9 is valid as per the grammar? To do this, we need a computing machine that takes the sentence as input, goes through each word in the input until the end of the sentence and then say yes if the sentence is valid or no if the sentence is not valid. In this course on formal language, we will learn different classes of languages differing in their strength and differing in their ability to express things. Correspondingly, we need automaton that differs in capability that can work different class of languages. Let's take a quick look at the structure of an automaton. Central to the automaton is a control unit. The control unit is in one of the internal state at all times. The sentence we want to get processed or verified is placed in the input file. The control unit reads the input word by word or simple by simple and verifies the input is conforming to the grammar. Once the input is completely read, the automaton produces an output yes or no as an output. In this case, we can consider that our automaton works as an acceptor. It accepts the sentence if it is valid. At times, we use automaton to produce some of the meaningful output or transform the input given. To do this, we need a computing machine that takes the sentences input, goes through each word in this input until the end of sentences and says yes if the sentence is valid or no if the sentence is not valid. In this course on formal languages, we will learn different classes of languages differing in their strength and differing in their ability to express things. Correspondingly, we need automaton that differs in their ability that can work with those different classes of languages. Let's take a quick look at the structure of an automaton. What you see on the right side of the slide is structure of an automaton. Central to this automaton is the control unit. The control unit is in one of the internal states at all times. The sentence we want to get processed or verified is placed in the input file. The control unit reads the input word by word, symbol by symbol and verifies if the input is conforming to the grammar. Once the input is completely read and processed, the automaton produces an output yes or no. In this case, our automaton works as an acceptor. It accepts the sentence if it is valid. At times, the automaton produces some other useful input or transforms the input given. In other words, it takes its input and then produces another output, not just yes or no. In such cases, we call an automaton, an automaton works as a transducer while performing. The emphasis on automata is relatively lesser in this course. The larger emphasis is on formal languages, their corresponding grammar, properties and applications. We've come to the end of this video. I'm sure you understood the notion of automata and its role as a language acceptor. Thank you.

Summary of Module on Elements Video• . Duration: 2 minutes

[Notes]

Dear students, welcome to the video. We have come to the end of our first module. It's time we take a quick look at what we have learned in this module. In this video, I'm going to give you an overview of topics covered in this module. Let's go ahead. We started this module with alphabets. We define alphabets as any finite set of symbols. Once learning what alphabets are, we proceeded to the next higher elements of language called the strings. Strings are a finite sequence of symbols from

alphabets. We've learned about few important string operations earlier in this module. Having understood alphabets, strings, we proceeded to define what languages are. At different points, we've highlighted the difference between spoken language and formal languages. We defined language as a set of strings from a given alphabet. We performed various operations on languages. Since language is a set, we can use set operations on existing sets to define new sets, leading to new languages. Grammar is important both for spoken language and formal language. We've learned the formal definition of a grammar. Can you pause the video now and then recollect the quintuple defining the grammar. Do you recollect this? V is, P , T , P , S and do you remember what V is, what T is and what is S and what is P ? I'm sure you have a clarity on each of those elements. We use grammar as a language generator. It's like, given the syntax of various elements of a programming language, you can write so many programs for various purposes. For instance, if you are introduced to the syntax details of a programming language like C, you can write a lot of C programs. Finally, we've learned about automata. These are abstract machines that can recognize languages. Automata perform the role of language acceptor. Given a sentence in the language's input, the corresponding automata responds with yes if the sentence is valid as per the language. This is like your compiler takes a programmer's input and returns a success message if the program is syntactically correct. In this module, we've covered the foundational concepts of formal languages from alphabets to string, to string operations, to grammars and automata. These are building blocks for more advanced topics in this course. Great work. You successfully completed module one of formal language and applications course. It's fantastic to see you reaching this milestone. Keep up the momentum. Thank you.

Recording of Formal Languages and Applications: Week 1 - Live Session on 26-06-05 Video• . Duration: 1 hour 1 minute 1h 1m

[Notes]

Okay, great. Good to see you all here. Good evening. Like, welcome to this course on formal languages and applications. Okay, and we will be together for next few weeks. Okay. And we'll have these live sessions every Friday at 6.30pm, right? That's the schedule that we have got. So I hope to see you and your friends also more often. Okay. Try to make it to these sessions. These sessions we would, you know, any concerns you have, any questions you have, you can ask us. Okay. You can also post it in the discussion forum. Okay. If you post it in the discussion forum, we'll be able to answer it immediately. Maybe within like, let's say 12 to 24 hours, we'll be able to respond. Otherwise, you can ask us the live sessions as well. So once again, welcome. Good evening. And okay, I have few content to talk about. Okay. But if you have any questions, I'll prioritize your questions over what we are talking about today. Okay. Okay, so if you're wondering, who am I? I'm Swapnil Saurabh. I'll be your faculty for this topic or this subject for this trimester. And with me we have Sharad, Shubham, Devanshu, Chirag, and all five of us will be here to help you in your journey to make your journey smooth. Any questions related to anything that you might have, you can ask us. We'll be handling the discussion forum as well. Your questions related to subject, topic, you know, the examinations, grading, everything. Okay. We will take care of. Okay. So you can ask us on those things as well. All right. So yeah, if you have any questions, please let us know. Hope you are able to navigate through the, you know, the Coursera topics without any problem. Okay. And I hope by now you should have completed your module two, right? Okay. Module one is introduction

and module two is where the actual session begins. So I hope you have gone through all the videos. Now see, this live lecture is not to replace those videos lectures. Okay. So we will try not to cover everything in the video lectures. You are supposed to watch the video lectures and come to us and ask questions. Okay. And in this sessions, we will try to build on, you know, what you would have already gone through the video lectures. Okay. So don't think that this is supposed to be like revision or like repeat of the video lectures. Okay. You are supposed to watch video lectures. Okay. Your grading questions and everything will, will be related to what you learn in the recorded sessions and as well as these live sessions. Okay. And these are meant to be like extension of those recorded sessions. Okay. So we'll probably revise some topics. We'll probably take the discussion forward. We'll probably cover something else, which probably may not have been covered in a video lecture. Okay. So this is meant to supplement with you, supplement you with more content to make the subject easy. Okay. So you have to go through video lectures and also try to attend these live lectures. Anyway, these are getting recorded and would be uploaded to your LMS and Coursera as well. So you can prefer in case you are not able to make it to the live sessions. All right. With that, we'll, we'll start. Okay. I don't see any questions in the chat or hands raised. So hopefully we have gone through and you do not have any questions. That's what my assumptions are. Okay. I see a message in the chat. Shashi, question is please share contact. Okay. You can contact us through discussion forum. Okay. We don't like we are there checking discussion forum every day. Okay. So once, at least once every day we will check discussion forum. So that's the best place to reach out to us. Okay. Is that okay? Use discussion forum will be, will be able to, you know, you can reach out to us. Would it be possible for you to share extra worksheets for practice? I found that textbook questions related to grammar very tough for me. Okay. So Shwes, okay. So please do post such questions. Okay. I'll take this as a feedback on grammar. I'll try to add more stuff related to grammar. So yes, this kind of message, if you send us, okay. And especially in the discussion for, if you send, it will be recorded, right? So we would get back to you. Okay. We will share more worksheets, practice sheets to you. Okay. We'll be more than happy to do that. So that, yeah. So why don't you just start a discussion with this topic? Thanks, Shwes. So you see that they get live questions. If you, okay. If everybody starts asking questions, the live classes, right? We won't be able to cover everybody's question, right? And why should you wait for a week to ask that question? If you have a question, your question should be answered immediately. And that's why discussion forums are there. Okay. Don't wait till last minute. Okay. For, you know, for us to answer you. Discussion forum, we will, as I said, right? Within 24 hours, we, you know, we all will check at least once. Okay. So you'll have your answer on same day, if not, like worst case, like next day, as early as possible. It's okay. So use the discussion forum. That's what those are meant to be. What are the applications of formal language? How is it relevant for? Yes, exactly. That's, that's what I'm going to cover. So, Carlinton, right? Did I get your name right? Carl, Carlinton, right? Carliton. Sorry. I'm sorry if I'm pronouncing your name wrong. Carliton. I think that's, that's the right. Yeah. Perfect. So Carliton, that's the objective for today's session. Okay. So, so do let me know if you still don't get it after next 30, 30 minutes. Okay. So if you didn't still don't get the answer to your question, I'll probably come back to the question. Okay. But that's the objective. So next 30 minutes, you should be aware of the importance of this subject and why this is part of CS2Rent. Great. That's good interaction, guys. I think good responses, good questions. Okay. That's where these sessions are meant to be. Okay. Of course, you can ask us any time, Abhijeet. Okay. Before, after, during. Okay. Awesome. Great. So I don't see any other questions. So let's move on. Okay. And as Carliton asked, what's the importance of this? And we'll, we'll talk about it. Okay. We'll talk

[illegible]

you cannot have. Right. Of course. Right. You know, if you talk about Hindi or French or any language, they do have extra characters and things like that. Okay. But every language has a finite set of alphabets. Right. And the words of those languages can only be made from those finite set of alphabets. Right. So you form the word. But does it mean that any combination of alphabets will give you a word? No. There are some accepted words and there are some non accepted words. Right. For example, if I give you A and C, probably you can make can as an accepted alphabet. But I don't know, ACN. I don't think you have any word called ACN in English, isn't it? Right. So you do not have any word called ACN. So that's. That's not an accepted alphabet, right? So you have a dictionary, okay? The dictionary is kind of rule which says which are valid word and which are not valid word. So you need to understand alphabet, but all alphabets together, you know, doesn't, or any combination of alphabet doesn't give you a valid word. There is some rule and each, you know, each word has a meaning. Meaning nothing but the rule, right? Now, coming back to computers, right? So even computers has to face multiple languages, whether it's Java, Python, right? Okay, we've been developing all these languages, right? Now, how can computer understand these languages? When you write a program, okay, the computer has to understand what is that you're trying to write, okay? Any programming language that you take, okay? First, you do what you call a syntax checking, isn't it? Is the language written, programming language, written in a perfect order, right? That's the grammar that we use in our international languages, right? So first you have alphabets, then you have words, which not all, there are some valid and invalid words, and then you need to use grammar in order to form a sentence. Sentence is what is the ultimate objective, right? Okay, so ultimate objective when we speak any language. But then we have to pass through all these, right? Alphabet, strings, or I mean, words are called strings in programming. So like alphabets, words, grammar, and then you have your sentence. So if I speak a sentence, and if you understand those words, and if you understand the grammar, then you will be able to understand what I'm saying, right? If I don't form the grammar properly, you probably will not be able to understand anything, or you'll have confusion between three or four meanings that might come out from the sentence if I don't put a proper grammar, right? So what is formal languages? Formal languages is exactly what we just described as a human, how we learn. How do we make machine or computers understand information? How does we make computers understand what we are trying to convey to it? Right? So that's the purpose of formal languages. And that's why we need to learn, because if you end up into a team, or if you plan to develop a new language like Python, you need to design compilers. You need to design the software which can understand the high-level language, right? Like Python, Java, C++, whatever, and convert it into a machine-level language which machine understands. And then when machine gives back the reply to you, you have to convey it back to the user. So ChatGPT, being a computer, can only understand zero binary language. For it, doesn't matter whether you type your answer in English, or, you know, and if you have read about, you know, how, you know, or probably you will definitely do in this course, you know, in later semesters. You know, you need to understand what is tokenization, you know, how the words are actually converted into, see, all these machines, okay, algorithms that we have are all mathematical-based algorithms. And when we talk about natural languages, okay, we write in English, or Hindi, or whatever language, has to be converted into those numbers, because machine, or algorithm, or maths can only understand numbers, right? So any language that you type to ChatGPT has to be converted into some form of number, which ChatGPT will understand and reply in those numbers, and that number is again converted back into the language of your choice. All these translations happen if you have, if there is a particular rule defined. If there are no rule

defined, then you can't translate one language to another, you can't break it into different language, you can't understand the meaning. Right? So that's where learning of formal languages is important. When you go down to low-level programming, and you know, you're designing these algorithms, okay, then this will help you. Of course, when you say you are a graduate of computer science, you need to understand, okay, everything of the base, right? Okay, and then whichever field you get into, right, whichever field you choose to pursue your career, you will probably have to build on it, but then these are base fundamentals, right, which will help you to understand how computers work. Right? So AI is not just prompt engineering, okay? For non-CS people, AI could be prompt engineering, but for CS people, we need to understand what happens when somebody gives a prompt, isn't it? We should be building those kinds of machines, isn't it? So let me elaborate a little more on what I just said, okay? Okay, think about any app that you use, okay? For example, like when you want to do something, or you want to search something on Google, or let's say you want to chat with ChatGPT, or you want to send a WhatsApp message, or you want to write a Python code or Java code, right? All of them rely on a common foundation, and that is what we call as formal languages. So everything starts with simple symbols, right? Symbols, A, B, C, D, what we call as alphabets, right? So everything starts with simple symbols called alphabets. These symbols are combined to form what? Words, okay? Now, that word is called as strings in computer language, okay, so we'll call it as strings. So you start with alphabet, okay? Symbols, my mouse, yeah, you can see this. And then you arrive at second level, which are your strings, just like we form words from letters, right? Now, collection of valid strings, valid strings, not just, you know, not all strings are language, okay? Valid strings, okay? For example, for natural languages, the words that we have in a dictionary, correct? Okay, like for English, you have Oxford dictionary is supposed to be the standard. I can't create my own word and say this is what it is meant, right? Okay, I need to form words which are part of the dictionary, and only those words, valid words will form the language, right? So you have this language here, okay, which is set of strings. Now, but how do we know which strings are valid? The rules, that's where grammar comes in, isn't it? It provides rules, okay, for constructing correct expressions, okay? Alphabet, strings, languages, grammar, and then comes at last, automata. And, you know, that's what we're going to learn from next week onwards. What is automata? They are like your intelligent machines. They check whether, okay, these strings follow the rule or not. Okay, so if I write this. Sir, I have a question. Yes. How we can say that alphabets and symbols are same thing because actually alphabets are- They are same thing. Alphabets, what we say in English, is nothing but a symbol, isn't it? If I don't write A like this, if I say I want to write A like different way, will you be able to understand? No. Okay, alphabets are limited by those 26 characters, isn't it? I mean, alphabet is a kind of finite, non-empty set of symbols, right? I mean, if symbols are in this most basic thing, I mean, from symbols, we get the alphabets, right? See, when you say symbols, okay, see, I think I am getting your point, okay? See, symbols doesn't mean any symbol. Symbols are limited. You got it what I'm saying? Okay, so see, we all decided to write A in a particular format. We call it a symbol. I'm not saying that symbol would mean anything, okay? I mean, you write 27th symbol and say this is also alphabet, no. I'll talk about that in next slide, okay? So, we call it an alphabet, okay? So, if you don't like the word symbols, okay, we'll not use word symbols. Let's stick with alphabet. Is that okay? Yes, sir, okay. Okay, yeah. So, okay, yeah. So, when we say symbol doesn't mean any symbol, okay? It means those 26 symbols only. So, next slide, I'll talk about it, okay? These are, yes, those are limited. Those are finite set, okay? And those finite set of symbols are your alphabets. We'll talk about it. Okay, next slide. Okay, so yeah. So, now we talk about automata, right? So, automata is a machine, okay? It's an intelligent machine which

checks whether string follows those rules or not. Now, what do you see on your screen is a programming language, right? Now, when we write this code here, okay, the compiler has to, yeah. So, I was saying that, I mean, if we can't consider symbols and alphabets as same because if there is an infinite connection of symbols, we can't say them as alphabet. So, symbols- I'll answer your question next slide, okay? Just hold on for a few seconds. Yes, yes. Okay, just hold on for a few seconds. I'll clear it out, yeah. So, yeah. So, now let's say if I write this code, okay? Now, the compiler of this programming language has to understand, okay, if this code is correct or not. That's your automata. And like if, okay, is a valid word, string, okay? X greater than sign, okay? Like curly bracket. Curly bracket, you know, is valid in Java, not valid in Python. So, if I use a compiler for Python, probably it will not be able to understand what I'm trying to say. It'll give error, correct? But maybe if I use some other language, like, you know, which understands, let's say, you know, which has a valid curly bracket allowed, will be easy. We'll read this easily and say that, you know, and it'll give you the answer as well, right? It'll give you an answer as well. But if it is not valid, it'll not give you an answer. It'll throw error, right? Similarly, okay, if I speak English word, but if I don't put it in proper grammar, you will not be able to understand. So, what is your brain doing? Your brain is acting like automata. Whatever I'm saying, you're trying to understand what I'm trying to say, and you're trying to understand the meaning. And if you understand the meaning, you are happy. If you're not, you'll ask me the question, right? I didn't understand what do you mean by that, right? Because your automata is probably not able to process it, so you have a question. And if you're able to understand, probably you will have an extended question, or, you know, that's what communication is all about, right? So, that's what automata is. Automata is a machine which checks if the strings are valid or not. So, so far what we have seen is, okay, so formal languages provides or gives us a framework, okay, that allows computers, okay? So, okay, let me, you know, let me put things in my mind first. So, we are talking about a framework. Automata gives us a framework which allows computers to understand, generate, and process the information, right? So, from simple symbols, okay, with star, we'll talk about it, okay, to smart systems, formal languages makes your computing possible, right? For simple, right, from simple symbols to your smart systems, formal languages makes it all possible, okay? Now, okay, if you're okay, let's move to the next slide. Now, sorry, I didn't see your name, so I, you know, so, but then you asked me, right, difference between alphabets and symbols, okay? Now, see, when we say alphabets in English, we say A, B, C, right? Now, if I define a binary alphabet, binary language, for us, the symbols would be zero and one, right? If I talk about basic mathematical operator, then the set of symbols or alphabet would be plus minus star slash. Did I answer your question? Sorry, I didn't see your name. So, now, is it clear, or you still have some points to say about symbols and alphabet? So, it's clear now. Okay, perfect, thanks. Now, now, see, one problem that you will generally face, okay, for someone, I'm sure, because most of you would be learning this language for the, I mean, for the- Sir, I was not confused, I was, because according to- Not confused, like you had an opinion, right, which did not, which were different than mine, correct? So I was trying to say that, you know, I was trying to show that, you know, they're similar. And probably next slide, again, I'll come back to it. Okay, thank you, sir. So, yeah. So I was talking about, yeah, okay, yeah, subject. So I was saying that, you know, sometimes, you know, when you go through these videos or online material when you refer and prepare and things like that, okay, you'll, you know, what is more intimidating is these symbols, okay? Don't be scared of the symbols, okay? You understand what it means and you can, then you can easily, you know, understand the symbols. So when you see something like sigma or this bracket, you don't think it's too complicated, okay? In formal languages, okay, the alphabets are

represented like this sigma symbol. This is a sigma, summation, what we call it, okay? You must have done this in your mathematics before, right? Summation. So alphabets are represented as sigma. So whenever you see a sigma sign anywhere, okay, you should always connect it to alphabet. Okay, this talks about alphabet. Set of alphabet, not just alphabet, it's set of alphabet. So if you talk about English language, this sigma would be set of A to Z, A, B, C, D, X, Y, Z, okay? Now, sets are generally represented using curly bracket. And properties of sets are what? Order doesn't matter, okay? In set, okay, if you have done set theory, right? You must have done set theory in your mathematics. In set theory, okay, we generally represent curly bracket as a set. Order doesn't match whether I say 0, 1 or I say 1, 0 is same. A, B, C or C, B, A is same. That's why it's called set. One property. Second property of set is you cannot have duplicate values, right? Set does not allow duplicate. So in set, order doesn't matter. If I say, okay, name five cities of India. It doesn't really matter in which order you mention those cities, right? As long as those are cities of India, it doesn't really matter in which order you say, isn't it? So in set, order doesn't matter whether you say 0, 1, 1, 0, exactly same. Second thing is you don't write anything duplicate. Sets will always remove, will have unique. You cannot say, okay, Delhi, Delhi, Delhi, five times. It doesn't become five cities of India. It's still, you know, it's one, right? So duplicate values are not allowed in sets. So keep that in mind, okay? So sigma, curly bracket and unique set of alphabets written inside those curly bracket. Order doesn't matter. So you don't have to remember the order. So in formal languages, alphabet is a finite set of symbols. Whether it's 0, 1, whether it's ABC, whether it is plus, minus, or whether it is different set of smileys. You can even create alphabet of WhatsApp emojis, right? All the unique emoji that you see in WhatsApp is actually an alphabet. And we use them, right? When you send messages to your friends, you use these emojis. Why do you use your emojis? Because you understand the meaning of those emojis, isn't it? If you didn't understand the meaning or if let's say there's emoji which you don't understand, you will not use it. You first understand and then you will use it, isn't it? For example, let's say this is, you know, wow or cool or love, right? So we understand the meaning and then we use these emojis when we want to convey some kind of expression, right? So in this case, your emojis actually become symbols or, okay, if you are building a language using WhatsApp emojis, then these are your alphabets. Then these become your alphabets. All right? Now for me, let's assume I only understand these three things. So for me, the alphabet is a finite set of three emojis. If you understand 30, then for you it is a set of 30 alphabets. Right? So mini quiz, okay? You can type in the chat, okay, or you can speak. Now you have these three options, right? Which of these among the three that you see is an alphabet? Everybody says a wow, okay? That means this class is successful, right? So how do you know it's alphabet? Curly bracket. Curly bracket gives you a hint that it is an alphabet, okay, because it's a set. And then you have unique set of values within the curly bracket. Awesome. So let's move on, okay, to the next part. The answer is A. Okay. Now what is a word? Okay. What is a word? Word is nothing but collection of those alphabet, okay? And then you put them in a sequence, but those has to be. So we can say that because B is a string and C is infinite. We can say that because B is a string. We can say that because B is a string and C is infinite. I didn't get your question, I mean point, Shreyas. I was trying to say was that we can say A is the answer because B is a string and C is infinite. Okay, yeah, right, right. Okay, so you're giving reason why B and C are not. Okay, got it. Perfect. No, you're right. Okay. Now see, the first thing is the notation, right? So, yes, you're right. Okay. B is a word, okay, which is made up of alphabet, you know, set of alphabet. Yeah, right. Yep. Perfect. B is a word. Yeah. Okay. So you put this alphabet in a specific order. They become words. So string, okay. In machine, okay. In computers, we talk about string, not word. So string is a sequence of symbols

from an alphabet. What is alphabet? Set. Right. So if you talk about alphabet as the zero and one, then all these are your valid string. Yes. Sir, I mean, can we have strings as a symbol in alphabet? Can we have string as a? As a symbol in alphabet. I mean, if we have a set of three strings. No, no, no, no, no, no. You cannot have, so you cannot say, for example, you cannot say dog is an alphabet, right? Dog is made up of? Suppose we define a language where strings are used as symbols. Where strings are zeros and one, okay. I mean, sir, if in a binary sequence, zero and one are symbols, right? Suppose we define a language where some limited set of strings are used as symbols to define the alphabet. For example, if we have a concatenation of more than one, two, three strings, we define the alphabets actually for that language. In that context, can't we have symbols as strings? I mean, strings as symbols. No, for example, okay. So, you know, let me see if I understood your point. Okay. So let's say I define the alphabet as zero, comma, one, comma, cat. Is that what you're trying to say? Yes. So, yes. So this together is one symbol. Okay. Now, if this together is one, so, you know, how does machine understand that this together is one symbol? You cannot, like when you are writing a string later, you cannot have like C080. Because C80 together is one symbol, it has to appear together. Does it make sense? Yes, sir. Yeah, perfect. Yeah. Yes. So, yes. So, see, always think from strings to language, never come back from language to string. If you come back from language to string, you'll have questions. If you go from string to language, you will never have questions. Okay. Strings are foundation. Strings are those unique symbols. When you put them together becomes strings and those strings becomes a language. Okay. So, we can predefine a string like a word. Yes. Can you predefine like something like let cat be, let's see it, let cat be meaning something this. So, we can use it in a predefined function or something like that. You. I mean to say that like we have the option to use the word as an alphabet if you predefine it. No, no. So, listen, like, okay. So, okay. So, it means that like symbol should, it is not a condition that symbol should have single character. I mean, it can be have, it can have more than one character. Okay. So, let me give you an example. Okay. And then, then I'll come back to your, because see, some of your voices are weak and you know, it's not coming out properly, but let me, let me tell you one thing. Okay. Now let's say, you know, I define this English alphabets. Okay. A to Z. I define, let somebody said, right. Let cat play. Now tell me, is this a string? Is this a string? So, Sashi, what do you think? Is it a string? So, here I have this let cat play. Okay. And your alphabets are A to Z, all your English alphabets. Let cat play. Is it a string? Are we considering a space also in this set? I'm talking about English alphabets, right? You know, what are English alphabets, isn't it? Yeah. So, it's not a string. It's not a string. Exactly. Okay. So, you know, because you are using something which is not part of the set. Right. Now, let cat play without space. Then it becomes a string. Right. So, that's what, you see, the rules are created to clarify such questions. Does it make sense? Did I answer your point? You have some point, isn't it? When it's a multi-letter symbols. See, letter itself is symbol. Okay. Letter and symbol are same. Doesn't matter. That's what I said. Cat is one symbol, one letter. Cat is not three letter. Okay. Now, if you are trying to say that can I have 0, 1, 10? No. Then it's wrong. This is not allowed. Now, if you say no, 10 is not 0. It's 10, not 0, 1. Sorry. If I say 0, 1, 10, then it's not valid. So, it means when the order matters, they are different. No, order doesn't matter. Order doesn't matter. It has to be unique. It has to be unique and order doesn't matter. Okay, sir. Got it. So, my question regarding the languages part. I mean that like in C language or any language, we define a user function also. Like for example, like we define it like has defined like cat. Got it. Yeah. Go ahead. Let cat be equal to C. Like we define a string as an alphabet. Like a string as a function. And then we can use that string as an alphabet, as a function in the whole program. So, accordingly, can we also do that? No, no, no. String. You define. So, when you define

a function name as let's say print, for example. Right. You can define print as a function because language allows you to use any of those English words to define a function. Can I say print space first as a function? No, you cannot. Because that particular language, most of the programming languages, doesn't allow space to be used as a variable name or function name or any name. Right. But if I use underscore, most of the languages will allow. Because underscore is allowed as it's an alphabet, but space is not an alphabet. Does it make sense? Got it. So, there are rules for that. Exactly. Okay. Some rules will allow you to use dot. R programming allows you to use dot. But Python, Java, C doesn't allow you to put dot. Right. In the variable name. Because they all have their own set of rules. Okay. Perfect. Good. Then you will also see one more symbol called as epsilon. Okay. This is, you know, nicely curved written E. Okay. So, this is called as epsilon. You will hear these words. Now, epsilon means nothing. Epsilon means blank. Just like, you know, some... Sometimes your friend might say, I don't know. I don't think WhatsApp allows to send blank messages, right? You have to have at least dot, I guess, right? Will not allow you to send a blank message. If WhatsApp allow you to send blank message, blank, nothing. No, no alphabet is called epsilon, okay? So that's why length of epsilon is zero, okay? Because it, epsilon represents nothing, okay? Nothing, okay? So you will hear this, you will see this term symbol epsilon. Epsilon means nothing, or if I have to put it in set, it becomes null set, okay? There's nothing, empty set. There's no value. There's no string in it. There is no string, no alphabet in it. That's what epsilon says, okay? Okay, there's some properties of strings that you need to remember. You can concatenate. Okay, we have about eight minutes, okay? And I think we're almost done. So, you know, you have concatenate. Concatenate means what? Joining two strings together. So if you say X, Y, X, Y will be cat, dog. No space, no comma, no pull stop, right? Putting those strings together. If I say Y, X, it becomes dog, cat. Then you have like hello world, in English becomes hello world, right? Something similar. So you have concatenation as a property of string. These are properties of strings. Then you have reverse. Reverse means right to left, right? So if you have a string as a-p-p-l-e, it becomes e-l-p-p-a, right? Then you have something called as prefix and suffix. This might be a little confusing, so be careful, okay? Prefix means you are reading the strings from left to right. If I say, give me the first character, I'm talking about first prefix, C. Give me first two characters, prefix two, C-U. First three, first four, first five, first six. You cannot say C-M-U-T something. That's not prefix. Prefix means it will come from left to right. You tell me how many characters you want. You want zero characters, you get epsilon. Epsilon is there, okay? If I say, give me nothing from computer, okay? From this string, you get epsilon. You give me one character, become C. Give me two characters, become C-O. That's prefix. Suffix is from right side. Okay, give me one character, you get R. Give me two characters, two suffix, becomes E-R, so on. So you're reading from right side. Then you have something called as substring. Substring means taking a character from anywhere in that string. So is C a substring? Yes. Is U a substring? Yes. Is C-U a substring? Question. Is C-U a substring? No, correct, Suyash. C-U is not a substring. They have to be together, okay, in the valid string. So C-U, okay, if I try to find, okay, if I use find function of any language, let's say Python or, you will get nothing, okay? You will not get that found. You will get not found. You get false, you get not found, okay? I have a question. Yes. So is the word itself a prefix or suffix of itself? I mean, computer is prefix of computer and computer is suffix of computer. I mean, I'm talking about whole word is prefix of- Yeah, yeah, yeah. Whole word is all three. Prefix also, suffix also, substring also. Okay, thank you, sir. Sure. Okay. If you ask 50 characters from more than total letters prefixed from computer, would it still come? So if I ask 50 characters more than total characters prefixed from computer, will still computer or result in an error? Great question. Fatima, great question. But to

answer this question, you need to understand the automata, okay? That's where your knowledge about automata will come and you have... So it depends upon how you design your machine. It depends upon how you understand this, right? So for that purpose, you need to read or you need to understand automata. Okay, Fatima? So from next week onwards, we'll talk about different automata machines, okay? And it depends on those, okay? Yes, perfect. Okay, so we are almost near the end, right? So what do you see here is, you know, like just the revision, okay? So let's say this is your alphabet, okay? And then you have, you know, all strings ending with one. So if the language says all strings ending with one, you can say, see language is after grammar, right? It, you know, grammar creates a language. So that's the end, last part of it, okay? So here, okay, all these are then valid language. So remember, language will be represented using capital L. Sigma, alphabet, and then you have set of unique alphabet, unique symbols. Language will be represented using L, okay? And this indicates all the valid language, right? So for example, if I say password, okay? Minimum six characters, you are actually setting a rule. That's the language. You say valid password is minimum eight characters with one special symbol. You can keep on building multiple conditions on it. You can build multiple conditions on it, and every condition is actually building a, you know, language. Valid email ID, you should have at the red sign. There has to be at least one character after at the red sign language, okay? And to be very specific, the formal languages, okay? If you have done it before, fine. If you have not done it, we will do it step by step. Right? So whether you talk about typing a website address, you're writing, you know, OTP for your code, you know, credit card numbers, your vehicle numbers. Okay, when you write a code, everything has to follow the language, okay? And then you have grammar. Now, just like you have a grammar in English, right? Like if I form a sentence, I have to use subject, I have to use verb, I have to use object. Same way in formal languages also, you have a grammar. Okay, and this grammar will say which are valid, you know, language or not. So languages which followed the grammar, right grammar will only be marked as valid language. Okay? So now you see what I'm trying to make is languages are everywhere. So now those who had questions on formal languages, did you understand where and why you need to learn formal languages as a CA student? Yes? Yes, so yes, correlated, okay, great. So that's why we need to learn formal languages, okay? Now, okay, so like, you know, application. When you use the sentence correction in Word document, how does machine is able to check your grammar, okay? There are some rules. When you write a code, okay? You get this red underscore, right? Okay, this is how it checks. These are formal languages checking. So here, grammar is setting up the rule and automata, which is a machine, is like checking the answers. Okay, so if I have to compare it, right? So grammar tells you what is right, what is not right. Automata checks whether you have done it right or not, right? So if you have to teach you mathematics addition, that's a grammar. I'm telling you, this is how you should perform addition. And automata checks whether you have done addition correctly or not. That's the difference between grammar and automata. Grammar is setting the rule, automata is checking if the rule has been followed or not. Okay, and there are different ways of, you know, different types of automata. Okay, there are different type of grammar. Each grammar has its own automata kind of machine, okay? So we'll talk about those grammars from simple to the advanced concept. So yeah, so this is one such example which I drew, okay? I'll just take one more minute. So here, like, let's say this is, you know, I've created a diagram here. So if you look at this state diagram here, okay? So this, you'll have this line which indicates that it's the rule, okay? This is a rule, okay? The pictorial diagram of a rule. That this is your starting point, this is where to start. And double circle shows this is where you need to end. So if you see here, you start from here. If you get zero, you are on Q zero itself. If you get one,

then you reach Q one, the ending state. So in this case, okay, you should always start from start. And at the end, you should reach your ending state. So when say game over, and where are you? Are you an ending state? Yes, I'm an ending state. That means that is a valid grammar, or valid string rather, okay? That means this is grammar and your string follows the valid grammar. So if I say zero, for example. So if I have zero, you start from zero, great. That's the end, right? I just have zero. Are you in your final state? No. So this is not a valid string. It's not a valid string. If I say one, one, one, zero, one. So you start from here, you say one. So when you add Q zero, one is like going to Q one. Now again, one. So when it is one, you come back to Q one itself. Again, one, you come back to Q one itself. Now zero. So now zero means you have to go to Q zero. Then one, again, you come back to Q one. That's the end, okay? Game over. Where are you? I'm at Q one. Is Q one ending state? Yes, double circle. So it's a valid string. Okay? This is what is being shown on the right side. No, epsilon. Epsilon means nothing. So when you're Q zero, epsilon means you're at Q zero only. Is Q zero valid? No. So that's invalid. Zero, I gave you. One, one, zero, zero. One, one. So one, you come here. Again, one, you are here. Zero, you come here. And zero, again, you are at Q zero. Q zero is not a valid ending state. So that's why it's not a valid string. So what we say is this is the language, okay? So don't read too much about it. I just wanted to show you, okay, this thing. And in the further classes, we'll talk more about these. All right? And there are some questions, okay? Okay, so if you have time, guys, you can stay or you can leave as well. So first question is, okay, is A, B, B a valid string? So A is part of it. B is also part of it. B is part of it. A is part of it. Yes. So this A, B, B, A belongs to this alphabet. So what is the length of epsilon? We know it is what? Zero, perfect. Yes. And then if you have to concatenate A, B and C, D, what do you get? A, B, C, D. Perfect. Great. Reverse of madam. It's madam only. Exactly. Great. Which strings, okay, which comes first? Alphabet or string? Exactly. Okay. You start from alphabet, then string, then grammar, then language, then automata. That's the order. Perfect. Great. Thank you for your time. Okay. If you have any questions, please post in discussion forum. We'll be more than happy to answer you there. Okay. And I'll see you again next Friday, same time, 6.30. All right. Thank you. Thank you so much, everyone. Take care. Thank you, sir. I have one question. Not it's not a good subject. Actually, it's like, we initially we are told that we can take more than three subverts of like, at least a bit so we can can can we take like more than three subverts? I'm not the right person to answer. Okay. Yes, that's why I didn't ask in the beginning. Yeah, no problem. Okay. Yeah, the course director would be able to help you with that. Yeah. So is there is there an email or something we can we can ask the email, right? You you email to support, support depending upon the question support person will direct it. Okay. Okay. Yeah, just you have single email to send all your queries. Based on your queries, it will come to you know, if it's subject related come to me if it's related to course, you'll go to the course director. Yeah. Okay. Okay. Thank you, sir. Sure. Thank you. Take care, everyone. Bye.

Recommended Reading: Introducing Alphabets Reading• . Duration: 10 minutes

[Notes]

Brief: Alphabets are the building blocks of formal languages. In formal languages, an alphabet (denoted as Σ) is a finite and nonempty set of symbols. These symbols can be letters, digits, or any characters, but they must belong to a well-defined finite set. An example of an alphabet could be $\Sigma =$

$\{a, b, c\}$ or $\Sigma = \{0, 1\}$ for binary strings. Infinite sets of symbols are not considered alphabets. Alphabets provide the foundation for constructing strings and ultimately defining languages. Alphabet (Σ): A finite, nonempty set of symbols. Symbols: Individual elements that make up an alphabet. Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Section 1.2.

Recommended Reading: Introducing Strings Reading• . Duration: 10 minutes

[Notes]

Brief: A string is a finite sequence of symbols drawn from a given alphabet. Strings can vary in length, but they are always finite. A string can be empty (denoted as ϵ), meaning it contains no symbols. Strings play a critical role in formal languages because they are used to construct languages. The length of a string is the number of symbols it contains, which is denoted as $|w|$, where w is the string. **String:** A finite sequence of symbols from an alphabet. **Length of a string ($|w|$):** The number of symbols in the string. Example: Given $\Sigma = \{0, 1\}$, a few possible strings are "0", "10", and "1101". The length of the string "1101" is 4 because it contains four symbols. Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Section 1.2.

Recommended Reading: Operations - Concatenation & Reverse Reading• . Duration: 10 minutes

[Notes]

Brief: Operations on strings are essential for manipulating and analysing strings in formal languages. The most common operations include concatenation, reversal, and finding the prefix, suffix, and substring of a string. **Concatenation:** Combines two strings, where the symbols of one string follow the other. If $u = "ab"$ and $v = "cd"$, the concatenation of u and v is $"abcd"$. **Reverse:** Reverses the order of symbols in a string. The reverse of $"abc"$ is $"cba"$. A **prefix** is a substring that starts from the first symbol. A **suffix** is a substring that ends with the last symbol. A **substring** is any sequence of consecutive symbols within a string. Example: For the string $"abcd"$, $"ab"$ is a prefix, $"cd"$ is a suffix, and $"bc"$ is a substring. Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Section 1.2.

Recommended Reading: Operations - Prefix, Suffix & Substring Reading• . Duration: 10 minutes

[Notes]

Brief: In string operations, prefixes, suffixes, and substrings refer to specific parts of a string based on their positions. A **prefix** is a substring that starts from the first symbol. A **suffix** is a substring that ends with the last symbol. A **substring** is any sequence of consecutive symbols within a string. Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones &

Recommended Reading: Kleene Star $[\Sigma^*]$ Reading• . Duration: 10 minutes

[Notes]

Brief: In formal language theory, the Kleene star (Σ^*) and Kleene plus (Σ^+) are operations applied to an alphabet Σ .

Key concepts: Σ^* represents the set of all possible strings (including the empty string ϵ) that can be formed using symbols from Σ . Σ^+ is similar to Σ^* , but it excludes the empty string. Σ^* : All strings, including the empty string. Example: For $\Sigma = \{0, 1\}$, Σ^* includes strings like " ϵ ", "0", "1", "01", "110", and so on. Σ^+ includes all the same strings except for ϵ . Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Section 1.2.

Recommended Reading: Properties on String Operations - A Proof Reading• . Duration: 10 minutes

[Notes]

Brief: Proof techniques are critical for verifying properties of strings, such as the relationship between string lengths and operations like concatenation. One commonly used method is proof by induction.

For example, we can prove that the length of a concatenated string is the sum of the lengths of the two individual strings: $|uv| = |u| + |v|$, where u and v are strings.

The proof typically involves a base case and an inductive step. The base case considers the simplest scenario (e.g., $|u| = 1$), and the inductive step proves that if the property holds for length n , it also holds for length $n+1$.

Example: Given $u = "a"$ and $v = "bc"$, the concatenation $uv = "abc"$, and the length of "abc" is 3, which is the sum of the lengths of "a" (1) and "bc" (2).

Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Section 1.2.

Recommended Reading: Introducing Languages Reading• . Duration: 10 minutes

[Notes]

Brief: A formal language is defined as a set of strings over a given alphabet. Languages can be finite or infinite, and the rules for forming valid strings in a language are typically specified using grammars or automata. In this context, a language is simply a set of strings that follow the rules of the alphabet and grammar. These languages are fundamental in computer science as they help describe patterns, rules, and structures within computational systems. Formal languages are crucial

in building grammars and automata, tools that help us define and recognise languages within computing environments. Language (L): A set of strings formed from an alphabet. Alphabet (Σ): A defined set of symbols used to form languages. Grammar and Automata: Mechanisms to generate and verify sentences within a language. Example: For $\Sigma = \{0, 1\}$, a language L might consist of strings such as {"0", "1", "10", "11"}. Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Section 1.2.

Recommended Reading: Operations on Languages Reading• .

Duration: 10 minutes

[Notes]

Brief: Languages, like sets, can undergo operations such as union, intersection, and complement. These operations allow us to create new languages by combining or manipulating existing ones. Union ($L1 \cup L2$): The set of strings that belong to either language L1 or L2. Intersection ($L1 \cap L2$): The set of strings common to both languages. Complement (L'): The set of strings that are not in the language L. Example: Given $L1 = \{"0", "10"\}$ and $L2 = \{"10", "11"\}$, $L1 \cup L2 = \{"0", "10", "11"\}$ and $L1 \cap L2 = \{"10"\}$. Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Sections 1.1 & 1.2.

Recommended Reading: Introduction to Grammar Reading• . Duration: 10 minutes

[Notes]

Brief: In formal languages, grammars define the rules for constructing valid strings in a language. A grammar consists of variables, terminal symbols, production rules, and a start symbol. These components work together to generate the strings of a language. Grammar (G): A formal system for generating strings in a language. Production Rules: Rules that specify how symbols can be combined to form valid strings. Example: In a simple grammar for arithmetic expressions, we might have production rules like: $\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$ $\text{Term} \rightarrow 0 \mid 1$ This grammar generates expressions like "1+0" and "0+1". Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Sections 1.2 & 1.3.

Recommended Reading: Introduction to Automata Reading• . Duration: 10 minutes

[Notes]

Brief: Automata are abstract machines used to recognise languages. They provide a way to formally verify if a string belongs to a particular language. Different types of automata (finite automata, pushdown automata, etc.) are used to recognise different types of languages. Finite Automaton: A simple machine that can recognise regular languages. Example: A finite automaton for recognising the language $L = \{"0", "01", "011"\}$ might have states that transition between accepting and rejecting states based on the input string. Reference: Peter Linz, An Introduction to Formal Languages and Automata, 6th Edition, Jones & Bartlett, Chapter 1, Section 1.2.

Automata, 6th Edition, Jones & Bartlett, Chapter 1, Sections 1.2 & 1.3.

Review Problems and Practice Questions Reading• . Duration: 1 hour 1h

[Notes]

(No transcript found.)

Contact Session 1 Reading• . Duration: 10 minutes

[Notes]

(No transcript found.)

Week 1 Live Session 1 Reading• . Duration: 1 hour 1h

[Notes]

(No transcript found.)

Introducing Regular Expressions Video• . Duration: 7 minutes

[Notes]

Dear students, welcome to this video. In this video, I am going to introduce you to the world of regular expression with simple examples. At the end of this video, you will be able to understand what regular expressions are, what are their purposes and you will be able to write regular expressions for some simple languages. Let's begin. Regular expressions are notations to specify patterns shared by sentences of a given language. When a language is given to you, the language will have sentences, of course. Regular expressions are notations that specifies compactly all the sentences in the language. The notation regular expression involves symbols from the alphabet and symbols like plus, star and dot. Plus we use it for or. The simple star, when used, we mean that we are going to repeat a pattern. And we actually can use dot for concatenation. Right? Let's go ahead with examples. For all these examples, we are going to use a simple alphabet, an alphabet consisting of only two symbols, zero and one. Our first language, a language consisting of only one sentence, which is zero. And our objective is to write a pattern that generates this language. Generating this language means what? Generating this zero. So the pattern that generates zero could be just zero only. So the regular expression here, this case is zero. That generates this language. Let's go to the next language, a language consisting of two symbols. So our regular expression should be able to generate the first sentence and the second sentence. Right? In case if I want to generate a first sentence, the regular expression should generate it. In case if I want to generate the second sentence, it should generate that as well. So what could be the regular expression in this case? It should generate zero or it should generate one. So from zero or one, I can generate zero. From zero or one, I can generate one. So the regular expression here is zero plus one. Let's go to our next example. This example consisting of a language with only one

sentence of length two, zero one. So this language, how do we generate it? It consisting of a sentence of length two, in which the first symbol is always zero and the second symbol is always one, right? And the only possible sentence is zero concatenated with one. Zero dot one is the regular expression that generates this language. Correct? So here it is. Let's go to our fourth example. This is a language with two sentences. The first one is zero zero. The second one is zero one. Let's actually attempt a regular expression for this language. The first sentence is zero, zero. So zero dot zero is the one that generates it. The second sentence is zero dot one. And the language consisting of either zero zero or zero one, a plus here, right? Looks like a long expression with dots and pluses. So let's put a brackets, right, parenthesis to see that we mean zero zero or zero one. Is there a better way of writing this regular expression? If I just look at the sentences of this language, all the sentences starts with zero. And the second symbol is either zero or one, right? So you have a regular expression one and regular expression two. They both generate the same language. The first one approaches it by listing all the strings and putting a plus in between. The second one truly captures the pattern and is a more compact way of doing this, right? And the second one, while both are correct, second one is the one which we consider to be more appropriate for us, right? So here is a regular expression for this language. Let's look at our next example. This language is an infinite language, see? It has got symbols zero one, zero one, zero one, zero one, zero one, zero one, and so on. All we understand is that the pattern zero one repeats, right? When it repeats one time, you get this, when it repeats twice, you get this, when it repeats three times, you get this, when it repeats fourth time, you get the next one and it keeps going. All right? So the first symbol is an empty string. How do we look at it? If we have a pattern like zero one that repeats zero times, you get empty. Any pattern that repeats zero times gives you an empty string, right? So now we are able to describe the whole language simply as a language consisting of sentences which are string zero one repeated zero or more times. String zero one, right? Getting repeated zero or more times can be written like this, right? This symbol, a star indicates that you're repeating the pattern zero one more time, right? So while this language looks infinite, consisting of so many sentences, the pattern that generates it is actually very simple, right? Let's look at our next example, a language consisting of no strings at all. It has got no strings. What pattern can generate nothing? Then the regular expression could itself be nothing. The symbol pi indicates a language which has got no sentences at all, right? It's important to understand the difference between a language that consisting of an empty string and a language which consisting of nothing. So for this, the regular expression could be just this empty, right? By now you would have learned what regular expressions are and you've learned how to write regular expressions for some simple languages, right? I'm sure if I give you some language, you will be able to write the corresponding regular expressions. Thank you.

An Advice on Writing RE's Video• . Duration: 3 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to share with you an important piece of advice on writing a regular expression. It's important that when you write a regular expression, it unambiguously generates the intended language. It's important to understand the role of parentheses in writing a regular expression. At the end of this video, you will understand how to use parentheses appropriately in writing regular expressions. What language does the regular

expression $a.b$ plus c imply? This can be understood in two different ways. The first way to understand this is $a.b$ plus c can be viewed as a comparison of two regular expressions r_1 and r_2 , right, where r_1 is $a.b$ and r_2 is c . It's also possible that we can consider this to be a comparison of two regular expressions where r_1 is just a and r_2 is b plus c , right? These two understandings give rise to two different languages. Let's see how it is. Right? So there are two ways by which we actually interpret $a.b$ plus c . The first interpretation gives us a language a, b or c . Look at the language generated by $a.b$ plus c is nothing but a, b, c , right? So this is a regular expression with two components, either $a, a.b$ or c , right? So $a.b$ gives rise to a sentence a, b, c gives rise to a sentence c and the r says there are two possible options, right? So a, b, c is the language. Let's look at the second, the language that the second one generates is a, b, a, c , right? If you just look at the regular expression, it says it generates sentence that starts with a and ends with either b or c , right? So that is a, b, a, c . So it is important to really write a regular expression that generates one of these languages appropriately as required. The presence of parentheses makes it very clear. If the intended language is the first one, write this way. If the intended language is the second one, write it in the second possible way, right? So the parentheses helps us. It is actually possible to assume precedence rules as what is there in programming languages. These programming languages have got precedence rules while writing expressions. We have precedence rules for understanding a regular expression as well. What we are going to assume is the expressions that we are going to write, the regular expressions we are going to write as a part of this course are all going to be parenthesized, ignoring all the precedence rules. Now you would have understood why it is important to use parenthesis appropriately in writing regular expressions and you will be able to write regular expressions in ambiguously generating the given language. Thank you.

Definition of Regular Expressions Video• . Duration: 8 minutes

[Notes]

Dear students, welcome to this video. In this video, I am going to introduce you to the formal definition of regular expression. At the end of this video, you will understand what regular expressions are. And if I give you a string, you will be able to understand if the given string represents a valid regular expression or not. And you will be able to write regular expressions confidently. Let's get started. Let's assume Σ to be your alphabet. The definition I am going to present to you is constructive in nature. In the sense, I am going to introduce you to what are primitive regular expressions. And then I am going to introduce you to a set of rules with which you combine the primitive regular expressions to generate larger regular expressions. So let's go ahead. The first rule, the symbol ϕ , λ are regular expressions themselves. So if I simply write ϕ , it indicates a language that consists of no sentences. If I simply write λ , this means we have a language with just one symbol, which is empty, right? And each symbol in the alphabet are regular expressions themselves. Consider Σ to be $0, 1$, then 0 is a valid regular expression, 1 is a valid regular expression, right? What is L of 0 ? Can you pause the video and tell me? And what is L of 1 ? I'm sure you should be able to do it now. It is 0 , it is 1 , right? So let's continue rule number 2. If r_1 and r_2 are regular expressions, then r_1 plus r_2 , r_1 dot r_2 , r_1^* and (r_1) are all regular expressions. So if you're given r_1 and r_2 , which are regular expressions, which are valid regular expressions, then anything that's constructed out of these rules are also valid regular expressions. That's what this rule says. Let's look at some examples. Let us consider r_1 to be 0 dot

1 and r_2 to be 0 plus 1. If we write r_1 plus, right, and r_2 , the resultant expression is a regular expression. If we write r_1 dot r_2 , the resultant is a valid regular expression. So the use of plus and dot between any two regular expressions results in another regular expression. That's all it means. Right? Let's look at the next one. I take a regular expression and put a star over it, the resultant is a regular expression. You put a star around regular expression 1 or 2 or anything, the resultant is a regular expression. To ensure that we don't create an ambiguous regular expression, we are consistently using parenthesis. Now take any regular expression, put a parenthesis around, it's a regular expression, right? So given primitive regular expression, any regular expressions constructed out of these rules are also regular expressions. So a string is a regular expression if it can be derived from primitive regular expression by a finite number of applications of rules in two. All right? So let's put whatever we learned together in a single definition. Given Σ to be an alphabet, the simple π , λ , and every element of alphabet are all regular expressions. If r_1 and r_2 are regular expressions, then r_1 plus r_2 , r_1 dot r_2 , r_1 star, and parenthesized r_1 are all regular expressions. A string is a regular expression if it can be derived from the primitive regular expression by a finite number of applications of the rules in two. Right? So now you got what a regular expression is clearly. Let's go ahead and look at some examples, which are the following are valid regular expressions. Assuming our Σ is a, b, c . The first one is c plus π and a star around it, right? Is it a regular expression? How do we verify that it's a regular expression? Let's do it step by step, right? If this, let's actually put a box around it, right? And we call this boxes r_1 , and we put a star over r_1 , right? So if you have a regular expression and put a star over the regular expression, the resultant is a regular expression, right? So we need to verify if r_1 is a valid regular expression or not. So what is r_1 ? c plus, right, this is r_1 . So can we now consider this to be r_2 ? So this means r_1 is r_2 , put a bracket around it. So if I take any regular expression and put a bracket around it, put a parenthesis around it, it's a regular expression, right? Good. So what is r_2 ? Our r_2 is c plus π . Is it a regular expression? Right. If c is r_4 , it's a valid regular expression, right? And π is r_5 , then what is r_2 ? r_2 is nothing but a sum of two regular expressions. So r_2 is a valid regular expressions. It means the given string is a valid regular expression. Let's look at the next one. Is the string shown to you a valid regular expression or not? How do you verify that this particular string is a valid regular expression or not? Look at this very closely. We can look at it as if it consisting of two parts, right? This is r_1 and this is r_2 , they're connected by dot, right? So if r_1 is a valid regular expression and r_2 is a valid regular expression, then r_1 dot r_2 is a valid regular expression. It has to be verified whether r_1 and r_2 , both are valid regular expressions. You can actually look at it very closely, r_1 is a valid regular expression, right? So a plus b dot c is a valid regular expression, correct? a plus b dot c is a valid regular expression and c plus π is a valid regular expression, right? And since a plus b dot c is a valid regular expression, a plus b dot c , a star around it is still a valid regular expression. So r_1 and r_2 are pretty valid, you can convince yourself. Hence r_1 dot r_2 is also valid, all right? I'm sure you have a question. a plus b dot c , what exactly it is? Is it a plus b dot c this way, a plus b dot c this way, right? Here while writing this regular expression, I've assumed dot has a higher precedence over plus and it is a plus b dot c . When you write a regular expression, do not leave this without parenthesizing, right? Do the parenthesis always. Let's look at the next one. b dot c star plus, right? If I see plus, I should see r_1 in the left and r_2 in the right. I see r_1 here for sure. Is there any r_2 there in the right? No. So this is not a valid regular expression, right? So c is not a valid regular expression. So by now you understood what a regular expression is and you know how to construct regular expression from simple elements, give it a language, you will be able to construct regular expressions and you can do all these things confidently. Thank you so much.

Languages Associated with REs Video• . Duration: 7 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to explain how to identify the language associated with regular expressions. At the end of this video, you will be able to write down or identify the language associated with a given regular expression. Let's begin. If the regular expression is ϵ , then what's the language associated with? It is empty, nothing. If the regular expression is an empty string, then the language associated with is what? A set consisting of just empty strings. If the regular expression is a for every a in the alphabet, then the language associated is a set consisting of only that alphabet. Right? Cool. Let's go ahead. If R_1 and R_2 are regular expressions, then the language associated with L of R_1 plus R_2 is nothing but L of R_1 union L of R_2 . What are we saying? To write down or to identify the language associated with regular expression R_1 plus R_2 , a regular expression consisting of two parts, it is enough to identify the regular expression associated with R_1 separately and R_2 separately and do a set union, right? Let's consider this example, R_1 is 0.1 , R_2 is 0 plus 1 . Then if I try to write down the language associated directly, then it will be by looking at the regular expression, 0.1 plus 0 plus 1 . And I need to deduce the language and the language is this, correct? $01, 0, 0$. The rule says we can deduce the language associated with R_1 separately, which is just 01 . The language associated with R_2 separately, which is $0, 1$. And then just do an union of these two sets to get the result, which is $01, 0, 1$. This is same as this, right? So let's go ahead. If R_1 and R_2 are regular expressions, then the language associated with R_1 dot R_2 is nothing but the language associated with R_1 dot the language associated with R_2 . Let's look at it. Assuming R_1 and R_2 to be the same thing as we seen earlier, the language associated with R_1 dot R_2 is this, 0.1 dot 0 plus 1 . So the language consisting of sentences that always starts with 01 , which ends with either 0 or 1 , right? So that is $010, 011$. Let's look at how to apply the rule. We need to write down the language associated with R_1 separately and R_2 separately, that we know what are. The language associated with R_1 dot R_2 is nothing but L of R_1 dot L of R_2 , which is take each element of R_1 , concatenate with each element of R_2 in that order. 01 concatenated with 0 , you get 010 and 01 concatenated with the next one, you get the second sentence. Right? Simple, right? If R_1 and R_2 are regular expressions, then the language associated with R_1 with a parenthesis around is nothing but the language associated with R_1 with a parenthesis stripped off. Let's look at an example. Assuming R_1 to be 0 dot 1 , the language associated with 01 parenthesis around is same as the language associated with just 01 . That's all this rule implies. So the language associated with R_1 with parenthesis around is 01 , the language associated with just R_1 is 0 . If R_1 and R_2 are regular expressions, then L of R_1 star is nothing but L of R_1 star. Right? So let's look at this a little more closely with example. The regular expression is 01 star as per the rule is nothing but language of 0 dot 1 star around it. Right? So what is 0 language associated with 0 dot 1 ? Language associated with 0 dot 1 is just 01 . It generates only one string, right? 01 . And we need to write down a star, which is what? A null string. If the set, the elements in set is taken 0 times, elements in the sets are taken one time, $0, 1, 2$ times, $0, 1, 0, 1$, etc. And it goes. Right? That's what this language is. The language associated with R_1 star is this. And language associated with R_1 , the whole star is this and both the same thing. So what does this rule imply? It means, you just write down the language associated with the regular expression, taking the star around, taking away the star around it, and with the resultant set, apply the star. Right? Likewise, the language associated with the primitive regular expressions are there in the

table. And then if $R1$ and $R2$ are regular expressions, the language associated with each form of construction can be identified in a simple manner as what you see in this page. Right? $R1$ plus $R2$, language associated with $R1$ plus $R2$ is nothing but language associated with $R1$ union language associated with $R2$. Language associated with $R1$ dot $R2$ is nothing but language associated with $R1$ dot language associated with $R2$. Language associated with $R1$ with parenthesis around is the same as language associated with $R1$ with the outer parenthesis stripped off. The language associated with $R1$ star is nothing but the language associated with $R1$, get the set and put a star over it. Right? So these are simple. The purpose of these rules are to help you identify the language associated with complex regular expression from its constituent regular expressions and the associated languages. Now if I give you a regular expression, you must be able to write down the language associated with this regular expression in a more easier manner. Thank you.

Problem Solving – Language for RE (1) Video• . Duration: 5 minutes

[Notes]

Dear students, welcome to this video. In this video, I am going to explain how to identify the language associated with a regular expression with an example. At the end of this video, you will be able to identify the language associated with a given regular expression, applying the rules that we have learned earlier. Let's get started. The problem here for us is, identify the language associated with the regular expression $0^* \cdot 0 + 1$. Let us assume that this actually comes from an alphabet $\{0,1\}$. Okay, we need to identify the language associated with this regular expression. Let's do step by step. This regular expression given to you can be considered as having two parts, correct? $R1$ and this is $R2$. So which form it is? It is in L of $R1$ dot $R2$ form. So if you have to identify the language associated with $R1$ dot $R2$, how can we do that is simply, we can do as follows, L of $R1$, identify the language associated with $R1$ separately, and identify the language associated with $R2$ separately, alright? So let's keep this aside. This is what we are going to use, alright? So $R1$. So let's begin with $R1$ and we try to identify L of $R1$, is what? L of 0^* . Now look at this particular form, it is L of 0^* , right? L of $R1$ star can be identified as L of $R1$ whole star. So this can be written as L of 0^* , fine? This looks simple, right? And this is equal to, what is L of 0 ? It is a 0 , is it not? So the language consisting of 0 and whole star, and the result is taking 0 0 times, empty 0 0 0 , 0 0 0 , and you know it goes on, right? So L of $R1$ is identified. Let's go to L of $R2$. What is L of $R2$? L of $R2$ is L of $0 + 1$. What is L of $0 + 1$? Which is just 0 comma 1 , right? is found with a language associated with the same regular expression with the outer parenthesis stripped off, right? So that's what we've used here. Now that we've identified what is L of $R1$, and we've identified what is L of $R2$, it is time to write down the whole answer. L of $R1$ dot $R2$ is L of $R1$ dot L of $R2$, which is, L of $R1$ is $\lambda 0^* 0$, etc. Looks like I'm placing a lot of dots for etc, it's only 3 . And 0 with 1 , and 0 0 with 0 , triple 0 , and 0 0 1 , and it goes on, correct? And triple 0 with this 0 , and triple 0 with this 1 , and it goes on. That's what the language that we're looking at, correct? So L of $0^* \cdot 0 + 1$ is the following. So what we have done is to identify the language associated with the given regular expression, applying the steps that we have learned earlier. By now, you would have learned, if a regular expression is given to you, how to identify the language associated with it. And I've demonstrated how to apply the rules correctly, so that identifying language becomes easier. Thank you.

Problem Solving – Language for RE (2) Video• . Duration: 5 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to demonstrate how to identify the language associated with a given regular expression. At the end of this video, you will be more familiar how to identify the language given a regular expression. I'm going to do this with an example. The problem here is to identify the language associated with this regular expression 0^*11^*1 , assuming the alphabet is $0, 1$. Let's go ahead. You can look at this regular expression. You can readily understand that this regular expression can be considered as having parts. Correct? Depending on how do you see this, this can be considered to have two parts or three parts. For convenience, I'm assuming that this is going to have two parts. This is r_1 and this is r_2 . And we actually can write r_1 as 0^* and r_2 as 11^*1 . And we need to only identify what is $L(r_1)$ and what is $L(r_2)$. And we actually can find $L(r_1)$ and $L(r_2)$ by concatenating both sets. Let me try slightly differently, because the regular expression given to you looks much simpler. Why and how? Look at 0^* , this 0^* means even number of occurrences of zeros, right? Let's write down simply 0^* . The language associated with this is 0 times, 1 time, 2 time, 3 time and it goes so on. Right? So I can describe this language simply as 0^m , all right? Such that m is greater than or equal to 0. When m is equal to 0, you get empty. When m is equal to 1, you get double zeros. When m is equal to 2, you get four zeros. So you understood what it is. Similarly, we can write down the next part $L(11^*1)$ to be 1^n , such that n greater than or equal to 0, is that all right? Now, you might have noticed that for zeros, 0^* , I've kept a variable m . And then for 11^*1 , I kept variable n , right? So the way given regular expression generates language is such that first you generate even number of zeros, that even number could be anything. And then you generate even number of ones, that even number could be anything. For example, the number of zeros could be 10 and the number of ones could be 20, right? So we cannot actually use same variable in both place. And then there is one at the end, right? So let's try to put everything together in a nice way. So $L(0^*11^*1)$ is, the language is, it starts with 0^m , 1^n and there's one extra, right? So can we put one extra or can I take this one extra away and just put a plus one here? Correct? We can do like this. And your m is greater than or equal to 0 and n greater than or equal to 0. When m is equal to 0, it would actually generate an empty, right? And when n is equal to 0, it would generate just a 1, all right? And you actually can deduce a language from this nicely, right? So the language generated is a sequence of zeros, even number of zeros followed by odd number of ones, right? This looks simple, right? So this is expected. So by now, you would have understood one more way of identifying the language associated with a given regular expression. I hope this example gives you a little more clarity on how to identify the language associated with a regular expression. Thank you.

Problem Solving – RE for a Language (1) Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to design a DFA for a given language. Let's go ahead. The language for which we want to design a DFA is this. A language consisting of sentences from the alphabet A and B . Right? Sentences of A and B such that the string does not contain three consecutive Bs. All strings of A and B except those strings which contains three consecutive Bs. Let's see how can we do this. We need to start somewhere.

So here is our starting state. And in the starting state, we have not seen three consecutive Bs. So starting state is our accepting state. So an empty string is allowed. Right? So the language says, Sigma star, right? Sigma star that does not contain three consecutive Bs. Sigma is AB. We have not seen three consecutive Bs and we have accepting state. If you see three consecutive Bs, right? From the starting state, I've seen a B, right? I've seen a B here. I've not seen three Bs so far. And I've seen one more B. I've not seen three Bs so far. And I've seen one more B. Oh, I've seen three Bs. I should not accept it. Anything other than seeing three Bs are okay with me, right? So I would accept this and I would accept this. I have not seen three Bs. So in the beginning, the starting state, I've not seen any B so far. And the next state means I've seen just one consecutive B, just one B. And again, I've seen two consecutive Bs. Fine. It's allowed. I've seen three consecutive Bs. I'm not going to take it. Right? We need to complete the DFA. What happens if the input is A, right? In the starting state, what happens if the input is A? Fine. We get as many number of A's as possible from the starting state. It doesn't change anything because we have cared about not seeing three consecutive Bs, right? I've seen as many A's. I've seen a B, I've come here. But at this moment, I've seen A. In this particular state, I've actually remembered that I've seen one B, right? If I got one A now there, then I have to restart the counting, right? So I go back and I look for, again, consecutive Bs, right? But now the case is slightly different. I've seen two consecutive Bs. That is when I've seen an A. We want to count consecutive Bs. So I've seen two Bs and I've seen an A. What should I do? I should go back and restart my counting. And if I see three Bs, you're going to know what I accepted. Anything other than that is okay. Right? That's the question. Did we complete our DFA? Sort of. Not really. From state A, you know what happens if the input is A, right? And what happens if the input is B? For the second state, let's number this 0, 1, 2, 3, right? From state one, we know what happens if the input is A and what happens if the input is B. From state two, we know what happens if the input is A, what happens if the input is B, right? From state three, we don't know what happens if the input is A and what happens if the input is B, right? The state three is a state which is where we've actually found we've seen three Bs. So anything that I see in state three, I'm not going to accept further, right? For example, a string like AAABBB, and after that, whatever be the sequence of A and B, I'm not going to consider. I can make the transition to stay in the same state, which is never going to take me to any final state any further, right? So this completes our DFA. Now have a look at this DFA. You have four states. In state zero, I've seen zero Bs so far. In state one, I've seen one B so far. In state two, I've seen two Bs so far. In state three, I've seen three Bs so far. So state B, once if you read state B, I've already counted three consecutive Bs, whatever comes after, I'm not going to consider, right? Everything other than that, we're going to consider. So this is a simple DFA design example that should convince you how to approach designing a DFA. So we've come to the end of this video. I'm sure you understood construction of this particular DFA and be able to attempt designing a DFA for different problems on your own. Thank you.

Problem Solving – RE for a Language (2) Video• . Duration: 3 minutes

[Notes]

Dear students, welcome to the video. Regular languages are a very useful type of formal language. This is the first class of formal language we learned in this course. We have learned that the regular expression and the regular languages are connected in a particular manner. In this video, we are going to learn the connection between regular languages and DFA. In fact, we will be learning the

connection between all three, regular expressions, DFA and the regular languages. Let's get started. What you see on the screen are two things. One is a DFA-M and a regular expression R. You can observe that the language described by the regular expression R is same as the language accepted by the DFA-M. In case you need time to confirm this, pause the video and verify this. Now on the screen, you see another DFA-M and the corresponding regular expression R. Here again, be sure that the language described by the regular expression R and the language accepted by M are one and the same thing. This might raise a couple of questions in your mind. 1. Can we find a regular expression to describe the language accepted by every DFA? 2. Given a regular expression, can we always find a DFA? Happily, the answer to both are yes. It may not be as simple as looking at a regular expression and we draw the corresponding DFA. Or by looking at the DFA and we get the corresponding regular expression. It requires some work at times. But we certainly can do that. The language associated with any regular expression is called as regular language. We've seen two regular expressions now and the language defined by each of them. And these are regular languages. This means that whenever we have a regular expression to describe a language, this language is regular. Right? We've also learned that for every regular language, there is a corresponding regular expression. Let us now come to understand how regular language and DFA are. Let M be any DFA, the language accepted by M, that is L of M is a regular language. The language L is regular if and only if there exists a some DFA such that L equal to L of M. A language L is regular if and only if there exists some DFA such that L is equal to L of M. This means that for every regular language, there exists a DFA that accepts it. The language accepted by a DFA is regular. So we've come to the end of this video. We've learned that for every regular language L , one, we will be able to find a regular expression to describe it. Two, there is a DFA that accepts this language L . Thank you.

Regular Language (using RE) Video• . Duration: 2 minutes

[Notes]

Hey students, welcome to this video. This video, I am going to introduce you to the first kind of formal languages known as regular languages. Let's get started. Regular languages are very useful and important type of formal language. You will learn that regular expressions can be used to describe languages and if a language is given, you will learn how to find the regular expression if possible. Great! The language associated with any regular expression is called as regular language. This means that whenever we have a regular expression, the set of strings it defines is a regular language. Furthermore, for every regular language, there is a corresponding regular expression. So given a regular language, you should be able to find the corresponding regular expression. In a sense, regular expressions and regular languages are two sides of the same coin. Regular languages are quite a useful class of formal languages. Imagine that I am handing over a document to you, a long document, and I request you to find all the phone numbers in the document. You know for sure, the valid phone numbers form a pattern and you can certainly write a regular expression to search all of them. How do you think? The text editors or IDEs are highlighting certain kind of text, keywords, etc. They use regular expressions to find all the matching patterns and they highlight them. When you declare a variable or a constant in a programming language, when you write a numerical value or a string value, or when you use a keyword in a programming language, the compiler understands them correctly by modeling these items as regular languages. We will see

more of this shortly in this course. In network security, they help to detect patterns that might indicate a cyber attack. In data mining, they help finding useful patterns in large data sets. Very close. Regular languages are a powerful class of formal languages which has got so many applications in computer science. I am sure you will learn what regular expressions are and how regular expressions and regular languages are connected. Thank you. Happy learning.

Introducing Finite Automata Video• . Duration: 6 minutes

[Notes]

In this video, I'm going to introduce you to an important concept in our course called as Finite Automator. Let's get started. What you see on the screen is a model of working of a simple switch that we're all familiar with. The switch can be made to go from on state to off state by again pushing the button. So there are two states a switch can be in at any point in time, on or off. And the switch transitions between on and off and off and on using an external input called as push. So this simple model of working of a switch is a very basic example of an automator, a finite automator. That just says that an abstract model of computation in which the number of states is finite. Let's go ahead and have a look at one more example. What you see on the screen is an example of an automaton that recognizes then, a keyword then in the computer program. For example, assume you are writing a program, if x less than 5, then print lesser. And if it's a computer program that you're writing. Now we need a program that takes this program that you've written as input and then verifies if this program is as per the syntax. So that particular program will scan this program from left to right. It reads from left to right. It looks at if, i, f, that program will say, Oh, I've recognized it. And then it continues to read all the way up to here. Maybe it says, okay, I've seen a condition. And then from this particular point, it will look at t, h, e, n and it will announce, Oh, I've recognized then. It's a keyword, right? So the automaton that is shown to you in the screen is only for recognizing the keyword then. Okay. So you look for, you're in the starting state to begin with, you're somewhere here, you're in the starting state. And the moment you see an input t, you go to a state from starting state to a state which I name it as t, right? If the system is in the state, it means I have seen t already. And now go and look at the input text. Okay. There is a h. So I shift the system to a state named as t, h. What does it mean to be in t, h? It means I've seen up to t, h. So that's the amount of memory that we have, right? And then we look at a letter e and we go to a state t, h, e. If the system is in state t, h, e, it means I've seen t, h, e and an n comes finally. I've seen n and we go to a state called as t, h, e, n. That's a place I've recognized the keyword then and I will announce to the world, okay, I've recognized the keyword then, an accomplishment. So what you see on the screen is an abstract model of computation. In this model of computation, the state of computation will be in one of these five states, either in the starting state or in t state or in t, h state or in t, h, e state or in t, h, e, n state. If the system is in state t, h or t, h, e, it just means that I've actually seen t, h or I've seen t, h, e. Being in a particular state means the system remembers concisely whatever it has done so far, right? So system remembers whatever it needs for its processing in states. And this model of computation has few other elements. From a state t to state t, h, the system transitions on an external input h. Similarly an external input e causes the system to transition from t, h to t, h, e, right? So there is an external input that causes the system to transition from one state to other state, one kind of memory, right? To other kind of memory, which is a very limited memory. So we have a system which consisting of a finite number of states and transition between states are controlled by

external inputs, all right? And the system clearly says how the transition between the states has to be done so that the computation has to be accomplished. So finite automaton is a model of an abstract computation in which the number of states is finite, right? In the automaton that you have learned earlier, you've seen a diagram like this, where there is a control unit, right? So the set of states connected by the transition diagram and the way the transition between states in the automaton, this forms your control unit. The input that causes the control unit to start, to go from starting state to end state is t, h, e, n . The moment you reach then, you announce the output that you've seen the word then, right? So a finite automaton is an automaton in which the number of state is final. The amount of memory is limited to what it is possible to remember in one of those states in the automaton and it does not use any separate dedicated memory that can remember anything beyond what is possible to be remembered in the states. So a finite automaton summarizes a useful model of computation. We will use finite automaton to recognize a class of languages as we go ahead. So we've come to the end of this video. I'm sure you understood what finite automaton are. And we are going to learn and use the applications of finite automaton in recognizing languages as we go ahead. Thank you.

Deterministic Finite Automata (DFA) (1) Video• . Duration: 10 minutes

[Notes]

Dear students, welcome to this video. In this video, I am going to introduce you to a set of operations on languages. At the end of this video, you will be able to perform operations on existing languages to create new language. A language is any set of strings over an alphabet. It's important to observe that a language is a set. Right? Because it's a set of strings over a given alphabet. Knowing that languages are sets, it's possible that we can perform operations on languages using the known set operations and we create new languages. Let's actually take some examples. Assume we have language L_1 and L_2 on Σ which contains just 0 and 1. The language L_1 , as you can see, consisting of sentences of this format, right? n zeros followed by n ones, right? n could be anything. It could be 0, 1, 2 or anything. And L_2 , exactly the opposite, right? Sequence of ones and then sequence of zeros such that the number of ones and zeros are exactly the same thing. Now, L_1 defines a language, which is a set of all strings matching this particular pattern. L_2 is another set or another language. Now we can create a new language L_3 , which is nothing but L_1 union L_2 . L_1 union L_2 is our new L_3 . So what will L_3 be? It will have set of strings, which has got either a sequence of zeros followed by a sequence of ones or sequence of ones followed by sequence of zeros such that the number of zeros and ones are exactly the same. Alright? Now, let's look at this particular example, L_1 be consisting of these four strings, L_2 consisting of these four strings, and L_1 intersection L_2 will be a new language called as L_3 or L_2 intersection L_1 is a new language called as L_3 , right? So L_3 will contain 0, 1 and 1, 0, because 0, 1 and 1, 0 are common in both. Now let's look at another example. Let L_1 be $0^n, 1^n$ and L_2 be $1^n, 0^n$. In both cases, n is greater than or equal to 0. Now L_3 , we define a new language L_3 such that L_3 is Σ^* minus L_1 . What is Σ^* ? Σ^* consisting of all strings of length 0 or more, which are made from Σ , right? And from that huge set, you subtract L_1 , right? So a set of all strings except strings of the form $n0s$ followed by $n1s$, these are excluded. So you can actually understand what L_3 is, right? So L_3 is a new language formed by subtracting L_1 from Σ^* . So what is L_4 if L_4 is written as Σ^* minus L_2 ? I'm sure you will be able to understand what L_4 is

by now. Now assume we have same L1 and L2 and we can define L3 as W reverse of all strings such that strings are coming from L1. We're defining L3 from all strings in L1, right? And we're reversing all this and we're making L3. So what will I get? I take all the strings in L1 and I reverse each of them. So what will be L3? It would be same as L2, right? If I reverse 0 0 1 1, I get 1 1 0 0. So if I reverse all L1, all strings in L1, I get L2. So L3 will be same as L2, right? So let's assume we have two simple languages, L1 and L2. And we define a new language L3 as a concatenation of L1 and L2. The concatenation of L1 and L2 is defined as below, right? So L3 will have sentences of the form XY, where X comes from the first language and Y come from the second language. For this particular case, we'll try to write down L3. L3 will have the first part comes from L1 and the second part comes from L2, correct? Now, I can write first part come from L1, second part come from L2, 0 0, triple 1. And the first part come from L1, triple 0, the second part come from L2, right? Triple 0, double 1, triple 0, triple 1. Can we make any other sentences? I think we cannot. So this is what our L3 is. So we can form or create new languages using existing languages using operations like concatenation and reverse as well. All right, let's go ahead. We define one more operation, clean closure on L, which is L star is equal to L power 0, union L power 1, union L power 2, union L power 3. So what is this L power 0, L0, L superscript 0, L superscript 1, right? So when you say L power n, when you say L power n, it just means concatenate L with itself n times, L repeated n times, right? So L2, L repeated twice by concatenation operation. Let's take a specific example, right? So the clean star would be union of all Ls such that L starts with 0, goes all the way to infinity. Let's take a simple example. Let ELP set a language 0, 1, 0. So L star will be taking all of them 0 times, empty, taking them one at a time, right? This one directly 0, 1, 1, 0. Now going twice, taking two at a time, I can actually take 0 twice. I can take 1, 0 twice. I can take 0, 1, 0. I get this. I get 1, 0 and then 0. I get this. Now I want to take 3 times, L3 and so on. And this, this set goes longer and longer, right? Now L plus means take EL star and then remove this empty string, right? Remove this empty string that becomes your L plus, right? So this is a union of all Ls where L starts with 1 and goes all the way, right? So L star, L starts with 0, L plus, L starts with 1, right? So we've come to the end of this video. What we've learned is one, we learned that languages are sets. It means we can apply set operations on existing languages to create new languages. Also, we've learned we can use string operations on existing languages to create new languages. Further, we've also learned what is a clean closure or star closure and positive closure, right? So here L plus is positive closure, L star is clean closure. Thank you.

Deterministic Finite Automata (DFA) (2) Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to explain how a DFA accepts or rejects a string. What you see on the screen is a DFA for a language. For now, let's keep this language away and we'll look at only the DFA. The DFA has got three states. One is q0, which is an initial state. For q1, two circles means that it is an accepting state and there is one other state q2. For a starting state, we put an arrow mark like this, an arrow mark without any input symbol, signifies that it's a starting state. Now let's look at this particular input string and to see that if the DFA accepts it or not. When you are in starting state, while taking a, the system stays in q0, right? And then, after looking at next a, it's still in q0, looks at another a, it stays in q0. And again the next a, it stays in q0. Again the next a, it stays in q0. In q0, it sees a b, it goes to q1. And in q1, it sees an

a, it goes to q_2 . In q_2 , it sees an a, it goes to q_2 again. So we've consumed the entire input or in other words, we've processed the entire input. At the end of processing the entire input, the DFA, after consuming the entire input, the deterministic finite automata or DFA is in q_2 . q_2 is not an accepting state. Hence, the string four a's followed by bba is rejected, it's not accepted. Now look at the next string, four times a. The system starts with q_0 . After consuming all four a's, you can convince yourself that the system is still in q_0 . So after processing the entire input, the system is still in q_0 , which is not a final state. And hence, the input a four times is rejected. Right? This one. Can you take a pause and then verify if this is accepted or not? Four times a, as long as you see a from the starting state, still stay in q_0 . The moment you see b from q_0 , we go to q_1 . After consuming the entire input, you reached q_1 , which is a final state. And hence, the input four a's and a b is accepted. Now just look at another string, a long sequence of a's and followed by a b, you can convince yourself easily that this is also accepted. After receiving this many a's, the system would still be in a. And then seeing the last b, it goes to b. And simple after processing the entire sequence, you are in q_1 . Hence the sequence is accepted. Now the language accepted by a DFA set of all strings that it accepts. In this case, you can see that a DFA accepts as a language acceptor. You give a string to a DFA, it will tell you whether the string belongs to the language or not. Right? So you can see that this works as a language acceptor. The set of all strings accepted by this DFA defines this one, L of a power n, b, language accepted by the DFA. So we've come to the end of this video. You understood how a DFA accepts or rejects an input string. And when we say a language accepted by DFA, by now you understood what it means. Thank you.

Deterministic Finite Automata (DFA)(3) Video• . Duration: 5 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to introduce you to the formal definition of a DFA. Let's get started. What you see on the screen is a DFA for the language a power n b, where n is greater than or equal to zero. a power n signifies a repeated n times. So what you see is a DFA that accepts this language. So let's go to the formal definition of a DFA. A DFA or a deterministic finite automaton is defined by a quintuple. It's a triple with five elements. So a DFA M is defined by a quintuple. If we understand these five elements, and if you are able to specify these five elements, we are good, right? So let's try to understand one by one. The first element is Q. Q is a finite set of internal states. The set Q has to be finite, right? In this example, the set Q consisting of three states, Q_0 , Q_1 , Q_2 , right? The second element is fairly obvious for you, Σ input symbol. This language is defined in only two symbols. The input symbols are a and b. So it's a set consisting of these two elements. Third one is a transition function. A function δ defined on for each Q and Σ , right? So for each state, an input symbol, we must specify the transition, right? So here in this example, there are three states and two input symbols, right? Let's take an example. The transition function δ must be specified for each such combination. Q_0 a. What will I do? Q_0 b. What will I do? Right? Where will the system go? Right? This has to be specified. Now for Q_1 a, b, right? Similarly for Q_2 a, Q_2 b. So we've specified for each Q_0 , Q_1 , Q_2 on the input a and b. So this function has to be completely specified. This is the requirement. So third element of the quintuple is δ . I'm sure you understood it now, right? Each transition or each edge is defined by one instance of δ , right? So Q_0 comma a refers to this edge. Q_0 comma b refers to this edge, right? Q_1 comma a refers to this edge. Q_1 comma b again refers

to this edge. Q_2 comma a and Q_2 comma b refers to this edge, right? So all these combinations are specified here, right? So let's go to the fourth element. The fourth element is which of the states acts as the starting state, right? Or initial state. One particular state from the set of all states has to be the initial state. And this must be specified. There must be exactly one set state, right? So here in this example, Q_0 is an initial state. The next one is set of final states or accepting states. So the set of final states is a subset of Q . So here F is Q_0 , Q_1 , Q_2 . F has to be subset of Q , right? It's possible to have more than one final states. In this example, we have just one final state, which is Q_1 , right? We put two circles. So this is our final state. If you specify what are the set of states on what are the input symbols on which the machine is DFA is defined, or what is a complete definition of transition function, and specifically mention what is the starting state and what are accepting states, we're defining the DFA completely. Let's try to put everything together for this DFA now, right? Let's try to put what is Q , what is Σ , what is δ , what is Q_0 , what is F for this DFA. We've done it already. So Q is Q_0 , Q_1 , Q_2 , three states, Σ , A comma B , the δ function transition on A , B for each Q_0 , Q_1 , Q_2 , right? And Q_0 being the initial state and final state consisting of just one state Q_1 . This completes our definition of DFA. We come to the end of this video. In this video, you learned the formal definition of a DFA and given a DFA, how to write out the quintuple to define the DFA is also something that you've learned now. Thank you.

Language Accepted by DFA (1) Video• . Duration: 5 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to write down the language accepted by a given DFA. We are going to start with a simple DFA and we are going to try and find out what language possibly this could accept. Right? Treat this as a practice problem. So what you see on the screen is a DFA given to us. Let's look at it closely and you and I have to work together to identify the language accepted by this DFA. This DFA has got three states, q_0 , q_1 , q_2 . Of these three states, q_1 is the only accepting state. We need to identify what is the language that this DFA accepts. It's obvious that the input symbols are 0 and 1 and the states are q_0 , q_1 , q_2 and q_1 is the accepting state, q_0 is the starting state. Right? These are all pretty clear. How do we arrive at the language accepted by this DFA? So we are in state q_0 . If I simply see a 1, I can go to the final state. So 1 is certainly accepted. Right? For now, just look at, ignore the right hand side, a set of transitions in the state after the accepting state. Right? From q_0 , if I just see a 1, I go to q_1 . Done. I can also go to q_1 by taking this edge once and this edge once. Right? That gives a string 0, 1 is accepted. Now I can take this edge as many number of times, right? Generating sequence of zeros and then taking this edge, one edge once and go to the accepting state. Right? So that gives you a string, sequence of zeros and 1 is accepted. Fine. So 0^*1 is accepted. That's all we've understood about this DFA now. Now assume that you come to this, you take 0^*1 , right? Taking 0 as many number of times and to take 1s and you come to q_1 and now you see 0, right? You come to q_1 and you're seeing a 0. So if it takes you back to q_0 , then what should you do to get back to q_1 , right? You should now look at another 0^*1 to come back to q_1 . Correct? You following me? So if you see a long sequence of zeros, a sequence of zeros followed by 1, and then you see a 0, then you should take another sequence of zeros followed by 1 to come back to q_1 . That's all we understood about the left hand side or this part of the DFA, right? You can take this edge as many

number of times and you take this edge once, one way to go to q_1 or you come to q_1 , come back and then again take 0 as many times, take this edge once, come back to q_1 , right? So this is described by this example, a long sequence of zeros followed by 1, right? And then as 0, if you take 0 and a sequence of zeros and 1, right? And this sequence of 0 means 0 or more types. So that is 0^*1 , 0^*0^*1 is accepted. That's all we know. Now let's actually try a different thing. I have 0^*1 , right? I've taken this as many number of times and I've taken 1 once, right? I've come here at this point. If I take a 0, I'll go back to q_0 , but I'm not taking that. That's a previous case, right? I'll take 1. I'll take 1 again. That takes me to q_2 . So if I'm in q_2 , what will take me back to q_1 , right? Look at the structure of q_2 , sequence of zeros followed by 1, correct? Again, sequence of zeros followed by 1. So sequence of zeros followed by 1 from q_0 , that takes you to q_1 . But if you get to q_2 , again, sequence of zeros followed by 1 to take you back to q_1 , right? So the middle one, so after seeing a sequence of zeros and 1, you come to a final state. Then if you see a 0, again, you should go to the same sequence. The other option is you go through a sequence of zeros that followed by 1. If you see 1 in place of 0, again, you should see the same pattern. How do we combine? These are only ways by which you come to q_1 , correct? So how do we combine these two into 1? Everything else is same except this part, right? Can we write the language accepted by this DFA is this part, 0^*1 , sequence of zeros followed by 1, and then after that, a 0 or 1 followed by 0^*1 , right? And this can repeat as many number of times. So the language accepted by this DFA is exactly this, 0^*1 followed by 0 or 1 followed by 0^*1 , right? So this is the language that the DFA-M accepts. It will be very difficult for you to understand or accept this at this moment. It's important that you also try this a few times to see that the language accepted by this DFA is the one that I'm writing down, right? It takes a while to understand. By now, you would have understood if a DFA is given to you, how to trace the DFA with various inputs to find out what language does it accept. If I give you a different DFA, you should be able to replicate this and be able to identify the language. So it's important that you practice this with few DFAs, look at some DFAs in textbooks and start practicing and be able to write down the language accepted by DFA, right? Thank you.

Language Accepted by DFA (2) Video• . Duration: 3 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to introduce you to an extended transition function for a DFA. Also, I'm going to explain what is a language accepted by a DFA using the extended transition function. Let's get started. What you see on the left is a DFA and we can write a transition function δ for this DFA. The transition function looks like the one which is written on the right side, right? Each line in the transition function, each δ line refers to one edge or corresponds to one edge. So this says, what will you do if you are in Q_0 and see an input 0, right? You come back to Q_0 . What will you do? You are in Q_0 and you see a 1, you go to Q_1 , right? So important thing to observe is, for each state and for each input symbol, for each single input symbol, the transition function δ is defined. It's possible to define an extended notation that takes a string from a state and define a transition. And we call that transition function as δ^* , a superscript star that is defined for each state and as for all possible strings, right? So δ^* takes two things as input. The first thing is the state. The second thing is a string, not a symbol anymore, okay? What does it mean to take a string as input or how do we really identify the transition over a string from a state? Let's try to figure out. The δ^* for state Q_0 , input 01, what does it mean? It means you

start in state Q_0 and accept the input 01. It means from state Q_0 , first you apply the input 0, that takes you to Q_0 and then you apply the next input, which is 1, that takes you to Q_1 , right? So from Q_0 , if you receive 01, the transition function will mention that you're going to state Q_1 at the end of 01. It means we can define transition on longer strings as well. Now take a pause and then explain a state reached by 011 on Q_0 . From state Q_0 , you take a 0 and you take a 1, you come here and you take another 1, you go to Q_2 , correct? So this can be defined for any string or any string of any length, right? Now given the extended transition function's definition, we can write down the language accepted by a DFA is a set of all strings that takes a system from the starting state to an accepting state. I'm sure you understood what is the extended transition function for DFA and the definition of language accepted by M using extended transition function. Thank you.

Designing DFA Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to design a DFA for a given language. Let's go ahead. The language for which we want to design a DFA is this. A language consisting of sentences from the alphabet A and B. Right? Sentences of A and B such that the string does not contain three consecutive Bs. All strings of A and B except those strings which contains three consecutive Bs. Let's see how can we do this. We need to start somewhere. So here is our starting state. And in the starting state, we have not seen three consecutive Bs. So starting state is our accepting state. So an empty string is allowed. Right? So the language says, Σ^* , right? Σ^* that does not contain three consecutive Bs. Σ is AB. We have not seen three consecutive Bs and we have accepting state. If you see three consecutive Bs, right? From the starting state, I've seen a B, right? I've seen a B here. I've not seen three Bs so far. And I've seen one more B. I've not seen three Bs so far. And I've seen one more B. Oh, I've seen three Bs. I should not accept it. Anything other than seeing three Bs are okay with me, right? So I would accept this and I would accept this. I have not seen three Bs. So in the beginning, the starting state, I've not seen any B so far. And the next state means I've seen just one consecutive B, just one B. And again, I've seen two consecutive Bs. Fine. It's allowed. I've seen three consecutive Bs. I'm not going to take it. Right? We need to complete the DFA. What happens if the input is A, right? In the starting state, what happens if the input is A? Fine. We get as many number of A's as possible from the starting state. It doesn't change anything because we have cared about not seeing three consecutive Bs, right? I've seen as many A's. I've seen a B, I've come here. But at this moment, I've seen A. In this particular state, I've actually remembered that I've seen one B, right? If I got one A now there, then I have to restart the counting, right? So I go back and I look for, again, consecutive Bs, right? But now the case is slightly different. I've seen two consecutive Bs. That is when I've seen an A. We want to count consecutive Bs. So I've seen two Bs and I've seen an A. What should I do? I should go back and restart my counting. And if I see three Bs, you're going to know what I accepted. Anything other than that is okay. Right? That's the question. Did we complete our DFA? Sort of. Not really. From state A, you know what happens if the input is A, right? And what happens if the input is B? For the second state, let's number this 0, 1, 2, 3, right? From state one, we know what happens if the input is A and what happens if the input is B. From state two, we know what happens if the input is A, what happens if the input is B, right? From state three, we don't know what happens if the input is A and what happens if the input is B, right? The state three is a state which is where we've

actually found we've seen three Bs. So anything that I see in state three, I'm not going to accept further, right? For example, a string like AAABBB, and after that, whatever be the sequence of A and B, I'm not going to consider. I can make the transition to stay in the same state, which is never going to take me to any final state any further, right? So this completes our DFA. Now have a look at this DFA. You have four states. In state zero, I've seen zero Bs so far. In state one, I've seen one B so far. In state two, I've seen two Bs so far. In state three, I've seen three Bs so far. So state B, once if you read state B, I've already counted three consecutive Bs, whatever comes after, I'm not going to consider, right? Everything other than that, we're going to consider. So this is a simple DFA design example that should convince you how to approach designing a DFA. So we've come to the end of this video. I'm sure you understood construction of this particular DFA and be able to attempt designing a DFA for different problems on your own. Thank you.

RL as Language Accepted by DFAs Video• . Duration: 3 minutes

[Notes]

Dear students, welcome to the video. Regular languages are a very useful type of formal language. This is the first class of formal language we learned in this course. We have learned that the regular expression and the regular languages are connected in a particular manner. In this video, we are going to learn the connection between regular languages and DFA. In fact, we will be learning the connection between all three, regular expressions, DFA and the regular languages. Let's get started. What you see on the screen are two things. One is a DFA-M and a regular expression R. You can observe that the language described by the regular expression R is same as the language accepted by the DFA-M. In case you need time to confirm this, pause the video and verify this. Now on the screen, you see another DFA-M and the corresponding regular expression R. Here again, be sure that the language described by the regular expression R and the language accepted by M are one and the same thing. This might raise a couple of questions in your mind. 1. Can we find a regular expression to describe the language accepted by every DFA? 2. Given a regular expression, can we always find a DFA? Happily, the answer to both are yes. It may not be as simple as looking at a regular expression and we draw the corresponding DFA. Or by looking at the DFA and we get the corresponding regular expression. It requires some work at times. But we certainly can do that. The language associated with any regular expression is called as regular language. We've seen two regular expressions now and the language defined by each of them. And these are regular languages. This means that whenever we have a regular expression to describe a language, this language is regular. Right? We've also learned that for every regular language, there is a corresponding regular expression. Let us now come to understand how regular language and DFA are. Let M be any DFA, the language accepted by M, that is L of M is a regular language. The language L is regular if and only if there exists a some DFA such that L equal to L of M. A language L is regular if and only if there exists some DFA such that L is equal to L of M. This means that for every regular language, there exists a DFA that accepts it. The language accepted by a DFA is regular. So we've come to the end of this video. We've learned that for every regular language L , one, we will be able to find a regular expression to describe it. Two, there is a DFA that accepts this language L . Thank you.

Grammars for Regular Languages Video• . Duration: 8 minutes

[Notes]

Dear students, welcome to this video. In this video, we're going to learn the use of grammars to generate regular languages. Particularly, we're going to look at an example for right linear grammar to generate regular languages. Let's get started. If we have a language that can be described using regular expression, then the language is regular. Right? If a language is accepted by DFA, then the language is regular. If we have a regular language, we should be able to find a regular expression to describe. We should have a DFA that accepts it. Right? These are something that we've learned so far. Now we're going to look at a third way with which a regular languages can be described. The third way is by the use of grammars. Whenever we talk of languages, it's always of interest to identify what kind of a grammar can be used to describe this language. Right? So, regular languages can be described using grammars called as right linear grammars or left linear grammars. In this video, we're going to look at right linear grammar. The grammar $G = (V, T, S, P)$ is said to be right linear if all of the productions are of form $A \rightarrow XB$ and $A \rightarrow X$. So let's look at the grammar $G = (V, T, S, P)$. What are those elements in the quadruple? What is V ? V is set of variables. T is set of terminal symbols. Right? We can consider this T to be same as your sigma alphabet. Right? V, T, S , starting symbol, P stands for set of productions or rules. So here in this example, these are two rules. Now look at the rule. A grammar $G = (V, T, S, P)$ is said to be right linear if all the productions are of this form. Let's look at the form little closely. Right? We can consider $A \rightarrow XB$, $A \rightarrow X$. So here, X is any string derived from the set of terminals. Right? So X is element of T^* . So we are supposed to see grammars of the form, a string followed by a non-terminal. Right? Or just the string on the right hand side. Left hand side, there must be exactly one non-terminal. Right hand side, we can have a string and then one non-terminal. You will understand this better with an example. Let's go ahead with an example. What you see on the screen is an example of a right linear grammar. What is right linear? In the right hand side, we can have only one non-terminal occurring and that one non-terminal should be the last symbol. So in each production, we can have only one non-terminal occurring in the right hand side. So here, the right hand side, there's only one non-terminal occurring. Right? Which is yes. Which is the rightmost symbol. So this first production confirms to our definition. Right? It is of this form. Second production, yes goes to A . Right? A is a string derived from the alphabet and there is no non-terminal. So we are fine with that. So the grammar that you see is a right linear grammar. I'm giving you a grammar. It's always of the interest as to what language does it generate. So let's try to write a few sentences that can be derived from this grammar. Yes is a start symbol. So I can always derive from starting symbol yes or generate languages from starting symbol yes. You will learn about derivations, leftmost derivation, rightmost derivation and so on in the later modules. But let's quickly understand derivations in a very simple manner that we can use it for now. So $A \rightarrow AB$ capital S. The right-hand side has got A , which is from terminal, B , which is also from terminal, but yes, which is a non-terminal. So that S has to be replaced, right? So yes can be replaced with either this or this one of the right-hand side form, one of the, you know, RHS forms, right? So yes goes to A . So this S can be replaced with this A . So ABA . Alright? Is there any non-terminal here in the right-hand side? No. ABA . All are terminals. So this is a valid string. Right? A string too we have come up with. Now again, let's try again. Again. Right? We start with yes. That goes to AB capital S. And we know now, we have two terminal symbols and one non-terminal symbols. Let's try to replace yes with the first production. AB . This S is replaced with this RHS. Okay? So another AB capital S. We try to replace AB capital S in which S with this right-hand side. We get $ABAB$ capital S and S occurs again. What can

we do with this S? We have to replace. There's no other way. This is not a final string. Final string should have only terminals, right? So A B A B and this S is replaced with A B capital S again. Right? So if at all I have to stop this, there's only one way to stop it. A B A B A B. Replace this S with small a. Correct? That's the only way to stop this. And we got another string. With these examples, you can actually make an understanding about the language that it generates. Right? It generates A. It generates A B A. It generates A B A B A. It generates A B A B A B A. Right? So what does it actually generate? It generates A B star A. All right? We have a regular expression. So we started with the grammar. We understood what language does it generate and we're able to write a regular expression. The grammar that we have in front of us is a grammar for a regular language. Right? So right linear grammar is used to generate regular language. The language generated by this grammar is A B whole star A. And this is a regular language because what you have is a regular expression now. We've come to the end of this video. In this video, you will learn the role of grammar in describing regular languages. We've learned a particular form of grammar called as right linear grammar. And we've also seen an example of how a right linear grammar can be used to generate a regular language. Thank you.

Grammars for Regular Languages (2) Video• . Duration: 6 minutes

[Notes]

In this video, you will understand what are left linear grammars and you will learn how to use left linear grammar to describe a regular language. Let's get started. What you see on the screen is a definition of a left linear grammar. Let's try to understand this definition now. A grammar G equal to $VTSP$ is said to be left linear if all the productions are of the form A goes to BX and A goes to VX . So on this, A and B are variables or non-terminals. X is a string made up from terminal symbols, right? So if terminal symbols, if the set T consisting of A and B , then your X will be an element of T^* , right? In other words, X will be any string made from T . Important thing to note in this definition is that in the production, important thing is that in the valid productions, right? The right-hand side can be just a string made up from T or the right-hand side begins with the non-terminal and what follows is a string, all right? So these forms are allowed. Let's look at an example. What you see on the screen is an example of a left linear grammar. The grammar G has only one variable or non-terminal, right? So this is set V and set T , there are two terminal symbols A and B and there's one starting symbol. can be only one starting symbol and the set of productions are here, right? The first thing to observe from the set of productions are that in both production, we have only one non-terminal in the left and in the right, we have either a string or a non-terminal followed by the string. We've strictly confirmed to the definition of left linear grammar. What language does this grammar generate? We can try S goes to SBA , keep replacing S with again SBA , so I replace this S with this and the previous BA still remains and I replace this BS with SBA , so this BA remains, right? And finally, when I want to stop, I have to replace S with A , so AB , AB , ABA . The language that it generates will be AB^* , right? A sequence of AB s, zero or more number of AB s and A , right? AB , AB , A , or AB , A , or just A , or AB , AB , AB , A , right? So what you see is a regular expression. It means the language that this left linear grammar described is a regular language. Let's look at another example of a grammar. The grammar has got following productions. S goes to $ABES$, S goes to SBC , S goes to ASB and S goes to A . So all three rules are part of P . The question here is, is the grammar a left linear or right linear? Is the grammar left linear or right linear? Can you

pause the video and try to figure out? The first production, the non-terminal occurs in the rightmost position, right? Rightmost position of RHS. In the second production, S occurs in the leftmost position of RHS. In the third production, it occurs in the middle, right? So this grammar is neither left linear nor right linear, all right? Let's take another example, right? Let's assume S goes to ABS and S goes to SAB and S goes to A. Is it left linear or right linear? The first one, S occurs in the rightmost place. Second rule, S occurs in the leftmost place. And the third one, it perfectly conforms to the form, right? Right hand side is only A, or a string made from alphabet, that's all. Is it left linear or right linear? The first rule, it is conforming to right, second it conforms to left. So if you just look at the grammar as a whole, it is neither left linear or nor right linear. So this grammar is not guaranteed to generate a regular language, right? Strictly, either left linear grammar or right linear grammar are those valid grammars that can generate regular language. So we've come to the end of this video. In this video, you've learned what are left linear grammars, and you've learned an example of a left linear grammar. And you've also seen one example where you've verified if it is left linear or right linear. By now you will be able to understand what are left linear grammars, what are right linear grammars and the use of it. We've kept one part specifically away from the course. Given a grammar, write down or draw the automata, or given an automata, given a language, come up with right linear or left linear grammars. Those kind of exercises we've kept out of the course, right? As long as you are able to write simple grammars for languages, left linear or right linear, this serves a purpose. The rest are out of Scopeful's scope. Thank you.

Module Closure (Summary) Video• . Duration: 3 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to give you a quick summary of topics that you learned in module 2. Let's get started. We introduced regular languages in three lessons. In the very first lesson, you learned what are regular expressions, you learned the definitions, examples and given a regular expression how to identify the corresponding language, given a language how to write the regular expression, the number of possible examples that could be possible in a video lectures are limited. We will give you more exercises and we want you to practice more so that you get comfortable handling regular expressions and regular language. At the end of this lesson, you learn the connection between regular languages and regular expressions. This connection is very important. We've actually stated the connection without actually getting into proofs. The topics that are necessary to complete the proof are out of scope and hence we kept the proof outside the scope of this course. However, it's sufficient that you understand the theorem or the statement that connects regular language and regular expressions. We've then gone on to study a deterministic finite automata. In this lesson, you were introduced to particularly deterministic finite automata as a definition, how to design a DFA for some simple examples. And if I give you a DFA, how do I write down the languages? Finally, at the end of this module, we've learned the connection between regular languages and DFA. This connection is very important. In this lesson, you should focus more on solving more exercises beyond what is done in the class, taking a few languages and trying to come up with a DFA that accepts this language and so on. The third lesson that we had was on grammars. We've learned a particular form of a grammar called as left linear grammar or right linear grammar and we've learned it with an example. We've learned that grammars, particularly left linear or right linear grammars can also describe regular languages. With this third

lesson, we've learned three different ways with which regular languages can be described and all are connected. In the third lesson, we focus or we constrain ourselves only to understanding what are left linear grammar, what are right linear grammar. And you've seen an example. Right? We've not actually seen cases where given a grammar, how to arrive at an automata, what kind of an automata will it actually give you and so on. Right? The statements of connecting regular languages with each of them and connecting between them are also very important. The proofs are, as I said, left out because tools necessary to prove the theorem are out of scope for this course. Right? I'm sure you've enjoyed learning this module. Thank you.

Recording of Formal Languages and Applications: Week 2 - Live Session on 26-06-12 Video• . Duration: 56 minutes

[Notes]

Okay. Sir, is it recording the cloud? Is it correctly recording? Okay. Great. So, good evening, everyone. Okay. Welcome to week two of our session, Formal Languages. And I hope you have already gone through week one and week two lecture content. Also, next week you will have your test, weekly test. Okay. So, next week you will have your weekly test, which will be based on week one and week two content. Okay. Every alternate week, you will have what we call as graded quiz. Okay. So, we have like first quiz based on week one and week two. Then second quiz will be based on week three and four. Okay. So, that's the plan. And so, you have, I guess, 10 modules. So, you'll have five quizzes. And then you will have a couple of assignment as well. Okay. Two assignments. Okay. And then you will have one final exam at the end of trimester. So, a lot of things. Okay. A lot of tests, right? Five tests, two assignments, one final exam. Now, these are good for multiple reasons. Okay. That, you know, you will be studying regularly. And at the same time, okay, you know, let's say if you do bad one day, if you are not well one day, okay, still you will have other options to make up for your marks. Right. So, if you have just one or two, and if you don't do well for some reason, you know, you lose marks. But when you have regular test, okay, you know, you will be, you know, if one or two bad days also doesn't matter because overall it will be covered. Right. So, that's good. And these are not too difficult. Okay. Your graded quiz will be multiple choice. Your assignment will be something that you can work offline, refer books, and submit it. And of course, then you'll have final exam. Okay. Based on the instruction that you would get from the university. Okay. So, that's the plan. Okay. You'll have a couple of days to take the graded quiz. Okay. So, this window will be open for two days. Anytime within two days, you can take the exam. And you just need about 10 to 15 minutes to complete the quiz. Okay. You don't need more time. All right. Any questions on that? When time comes, the link will automatically get available. Okay. In your course, Coursera only, at the end of Module 3 or 4, I guess. Right. Module 4, I guess. You will see it available. Okay. So, somewhere on the left side, the link will automatically get visible and you can take the quiz. And because this is all driven from the backend, so exam will get closed automatically on scheduled time. Is it clear, everyone? Can you just confirm? Any of you? It's clear. Perfect. Great. So, any questions on what you have done this week? Actually, I had a couple of questions which I'd like to get some clarification on. Please go ahead. And the video that you're talking about RE for DFA, is it still in Week 2? It's still. Essentially, what happened was there were videos titled Problem-Solving RE for a Language and Problem-Solving RE for a Language 2. And later on in the same week, there was content related to DFA. So, Barry was supposed to find language for a DFA

and stuff like that. But at that point, a DFA had not yet been introduced. But in those videos had content related to DFA solving. And the same content was repeated later on while solving DFA related questions. So, would it be possible for the core staff to rectify this? Yeah, yeah, yeah. So, I don't see your name appearing, but can you do us a favor? Can you email us or at least put that in the discussion forum? I actually had posted this on a discussion forum like about 3-4 days back and have not yet received a response. So, would be possible for you to look into it? Yes, yes. Okay. So, yeah, we will take care of it. Sharath, let's take care of that, Sharath. Okay. I'm sorry. I, you know, I will go through it once again and I'll see. I'll make sure that, you know, correct video is uploaded. Yeah. Anyway, like DFA is next week itself. So, you know. Like we have a bit of DFA this week and next week it's about implementation with tables or, you know, direct implementation. So, right. Right. So, it would be relevant, but then definitely the same video should not be repeated twice. Okay. That's definitely a problem from our side. I'll just see. Okay. We'll just see and we'll rectify. I'll do that. Thanks. Thanks for bringing this up. Okay. Great. Thanks for sharing. Okay. So, if there are no other questions, then we'll move ahead with, you know, what I have planned for today. Let's move ahead. So, once again, welcome everyone to live session two of formal languages and applications. And today we'll talk about, you know, how computers recognize patterns. Okay. That's the theme for week two. And that's what we'll discuss a little bit more today. And, you know, by the end of this session, the objective is that you'll understand how computers validate inputs, recognize patterns, and decide whether a given string is valid or invalid. Okay. So, let's begin our journey into the world of pattern recognition. Okay. And, yeah, this is your team. Okay. We'll make sure that any issue that you face, we are on top of that and we get it rectified on time. Okay. We'll do that. And, yeah, I think any questions that you have, you put it on the discussion forum. We will take care of it. Actually, not to defend ourselves, but we were a little busy with creation of questions and things like that for the last couple of days. So, maybe because of that, we might have missed it, but we will definitely look into this case and we'll rectify it. Okay. So, talking about pattern recognition. Right. So, you have examples on the screen. Okay. You have like a phone number. Yeah. So, you have examples. You have, you know, phone number, email address, password. Okay. So, if I ask you, okay, let's say if I give you my number, phone number, and, you know, before you save it, you will validate it. Okay. Then tell me, how would you validate if a phone number is valid or not? Right. Definitely, there has to be some pattern. What pattern will you look into it? You would look into whether or not it's an integer, whether it's numbers, because alphabets, you can't use alphabets. Correct. And then you would also look at whether the string length matches the required length of, let's say, there are only nine numbers which can be present. It has to be nine numbers. It can't be less than or more than. Right. In our case, it is 10. So, yes, of course. Okay. You would look for 10 numbers. Right. And also, like, the mobile numbers won't begin with two or three or four. Right. So, 10 characters, and it should be like six, seven, eight, or nine. Right. Less than six, like five cannot be the first character. Yeah. So, you need to understand the pattern, and then the logic that you will write will identify that pattern before accepting that value. Right. And it happens, right? When you book a train ticket, you book a flight ticket, you enter your details while creating an account on some e-commerce platform. So, they would look at that if the name is valid or not. They would look at it or not. Right. Your, for example, email should have at the red sign. Then, you know, after at least one character, you should have a dot. After a dot, you should have at least a couple of characters. Right. So, these are patterns. Okay. And we encounter multiple patterns. We encounter multiple such situations in our daily life. Okay. And so, the machine that we are going to talk about today is mostly about the simplest form of machine. And of course, as we go along in our week, we'll talk more advanced

concept of different machines, what we call as automata. So, we're talking about the most basic. Right. So, here you have like email ID. You might have, let's say, vehicle registration number. Right. Vehicle registration number. India has two digit, first two characters, which represents state. Then, two, next two characters, which represents the RTO. Then, you'll have the number. Number will have, again, a couple of characters followed by three or four digits of vehicle number. Right. So, if you have standard or if you have OTP, right, OTP, every platform will have its length. Right. Some has four digits. Some has five. Some has six. So, when you enter this OTP, the platform will validate and it will tell you whether, you know. So, before it actually checks if the number is correct or not, we're not going to check that number is correct or not. We're looking at the validation. And most of this validation happens at the, what we say, on the client side itself. Right. So, it saves time. Right. You don't have to send it to the server and server then validates and says that it's wrong. Okay. Most of these errors should happen at the client side itself so that your response is faster, better. Right. So, that's where, you know, the patterns become important in computers. Okay. So, now the question comes, how does a compiler know whether your code is syntactically, okay, syntax wise. Okay. If your code is syntactically correct. Okay. And that's where computers use rules. Okay. That's where computers use rules. So, yeah, you have some examples on your screen like WhatsApp messages, compiler, you know, like search query when you put it in the Google. There are some patterns that they'll recognize. And what it says is, okay, all this automated or the logic behind these, we look for the The question, the last question, like, does this string follow the rules or not? Simple. Does this string follow the rules? And that's the importance of patterns in our, you know, building machines. Okay. Or the formal languages. Okay. What do we see here? Okay. So, I look at this carefully. Okay. Now, okay. So, this slide is meant to describe or, you know, convey what formal languages are, the formal language concepts, the real world applications. Right. Now, if you talk about Netflix validating OTPs. Okay. The pattern, it says, it has to be 6 digits. Okay. And it has to have 0 to 9 only. Each of those characters has to be 0 to 9 only. So, you have something called as regular expressions. So, regular expressions, what it says is that, okay, you see the bracket. It says 0 to 9. That means 0 to 9 means any value between 0 and 9 is valid and the length should be 6. That was the second number indicates that length should be 6. Okay. And then if we know. this then we can validate our correct or incorrect numbers, isn't it? OK, the invalid it correct pattern or incorrect pattern, right? So yeah, and what do you have next year? You have Google email ID, right? So email ID if you see here again your regular expressions. If you see what it says is. Uhm? Yeah, let me zoom a little bit. OK, so it says here that value can be small A to small Z. OK, you cannot stop people from typing capital letters as well, right? And when you type capital letters, email doesn't. Uh, you know, give you wrong. OK, so you know the it. It converts everything to lowercase, but then uppercase also is treated as. It will. Then you can have a dot OK and then you can have at least one character. Oh. Yes. And so these characters can repeat in between at the rate and domain name, right? So domain name can again contain alphabet characters and then at the dot. OK, after Dotdot it says to come \$1, right? That means it can be anything A to Z now. OK, and then you should have at least two characters. What it says. Now, if you put this as a regular expression. OK. Uh, Gmail or any other system which uses OK which needs to validate email can validate. Similarly, you have regular expression for for WhatsApp message as well. Something similar to that we discussed last time, right? Let's say you know you have those four and you can validate. Of course there is a comma. Now compilers, right? The last image you see compilers again. If I have to indicate a variable and if you know rule says that you have to indicate the data type of the variable. OK, then the variable name then equal to. Then. The value. And that. The line

should end with a semicolon, right? If that's the rule of the programming language, then of course, OK, you need to break it into different values like `hint` is the keyword. Your variable name is your identifier equal to `is` is an operator. Then you have the value or the number that needs to be assigned to the semicolon, which is the your delimiter or end of line indicator, isn't it? OK. So so here we use rules and every system defines a pattern. OK, and then checks whether the input follows that pattern or not. This simple idea forms the foundation of formal languages and automata theory, OK? And, you know, so if you ask me, somebody asked last time, right? Why are we studying this? What's the purpose? So your formal languages provide the rules. Automata provides the mechanism to check those rules. That's the machine. OK, and what we say is that's the reason why we actually study for one languages. OK. Moving on to the next slide. OK, this is one single slide which actually describes the entire world of formal languages. OK, now this is called as Chomsky's hierarchy. OK, and what you see at the core is the basic of formal languages. And as you move away, OK, the automata and the languages and the grammar becomes a little advanced, right? Every stage is more advanced than the previous stage. OK, so what it does is, OK, the Chomsky hierarchy actually classifies languages into different categories, OK, according to their complexity. And what do you have at the core? OK, at the core you have regular language. OK, and this is the, you know, your simplest languages that we have. And of course, we are going to, you know, this is the kind of, this is the focus, like this topic is focus of our week two. So as we go outward, as you move outward from here, OK, then you have context free languages and context free grammar. OK, so the context free. OK, and the automata that we use here is pushdown automata. OK, so we'll talk about finite automata and pushdown automata again. So pushdown automata has, you know, has memory, which finite automata lacks. We'll talk about it. OK, then after context free language, you have context sensitive languages. And finally, the, you know, the most advanced of all, you have regular languages. OK, then you have recursively enumerable languages. Right. Now, each level, as we go outward from core, it requires more powerful computational models. OK, and your regular languages can be recognized by your finite automata, OK, which is at the core. Your context free requires pushdown automata. And as we go to the last, you have Turing machines. OK, Turing machines is the most advanced kind of machines that human being, you know, have done so far. This concept came, you know, long, long time back, almost like 80, 90 years back. But still, it is valid today. When you talk about cybersecurity, AI, even if you talk about quantum, you know, computing that we are talking about, OK, everything can be explained using your Turing machine. So, OK, we'll keep that for some other day. Let's come back to the core today. OK, now, think of this hierarchy as a ladder where, you know, each step that you take, actually, you know, you gain more power. Right. Now, coming back to the core, OK, what is your regular expressions? What do you mean by? I think we have a slide here. I can talk about it a little bit here. Yeah. OK, so when we talk about regular expressions, we, you know, generally, you know, we mark it as RE for regular expression. And it's a compact way to, you know, describe a pattern. OK, so instead of listing all valid strings individually, we write a concise expression. And it represents your, you know, the pattern that we're talking about, right, which we have seen in the previous slide. So instead of listing, you know, the detailed possible, which probably is not possible to list all the valid, if I ask you, OK, what, which numbers are, what is valid OTB for Netflix, for example. So you cannot list all the possible, right, right from 0, 0, like 6 0s to 6 9s. And imagine the number of combinations that we might end up with, right. So how many combinations we end up with, computation combinations? How many? 10 to the power of 6 , right, of 6 , correct. Now, when I talk about email, valid, almost, you know, infinite kind of, you know, email IDs, you can list down, isn't it? So which is, OK, by going by the absolute number,

which is not easy. So what we need is some compact expression, which can represent the entire pattern. That is the whole objective of regular expressions. How do we represent the concept or validity using some compact, you know, expression, right. And you can see here, there are some symbols, some of the symbols we probably discussed last time as well, right. You have epsilon, which represents empty string, right. Then, OK, then it's making it so difficult to see on the phone. Let me zoom it again a little bit. So you have like epsilon, then you have regular expression, which says is represented using regular language, right. And there are some rules, right. So you have epsilon, OK, the empty string, then you talk about A. So A can be any symbol A from the input alphabet sigma, right. So we discussed this last time, right. You have sigma, which represents set of alphabets, right. You can have A or whatever, OK, which will represent, you know, when we talk about strings, so strings should have the valid alphabets, OK, which is described by sigma. If it has it, then it's a valid string, isn't it. And we can always apply different operators, can apply union, concatenation, and you have what is called as clean star, OK. You know, so your regular expressions, if you talk about the usage, OK, it is used in software engineering for searching, validation, parsing an input. If I have to parse an input and let's say parsing would be to get, let's say, you know, you pass multiple values together as a single string. So I can parse and decode, OK, the string and maybe break it into different components, right. So, you know, let's for example, you have built, and now anyway, we have standards like, you know, parsing as CSV, XML, things like that. But, you know, earlier, you would parse it, like even let's say if you parse a CSV string, you know, CSV string will have like first column as name, second column as let's say age, then you have address, phone number. So now if I have to read this CSV and, you know, able to decode each of these or get this information separately, OK, then I need to parse, right. And this is, again, where you can use regular expression. So that's another thing filtering, for example, filtering a text out of a given string. We use regular expressions. It's like whenever you are searching text in an editor, or you validate data entered in the form, right, we discussed about phone number, name, email, anything like that, you use this form of system, OK, you may use regular expression to validate such things. OK, so even though you may not, you know, create a regular expression by yourself, but the programming languages that you use, whatever language that you use, actually creates this regular expression in the backend, OK, to validate, right. So that's the usage of your regular expression, OK. Let's move on to the next slide. I hope it's making sense so far, OK. So great. And we'll see, OK, some applications we'll see before we, you know, we end the day. So now you're talking about regular expression in Google search, right. Now, suppose you enter a text like here, it's a Python tutorial, OK. Let's say you enter Python tutorial in the search, OK, in the Google, OK, or you put, say, chat PT, or you put machine learning. Google needs to determine whether these queries contain accessible, sorry, acceptable characters or not, right. And that valid string. OK, so you simplified your regular expression, OK, might allow letters and spaces, OK. And like in this case, if it is, you know, you know, so you need to make sure. that the input that Google is given, Google will validate that input before it actually performs search. That is the purpose of your regular expression. Before Google does the searching operation, it will validate and make sure the input is valid. OK. And you know, and here it says, yeah, right? You have regular expression, right? So only if this is valid, it will perform search. Otherwise, before it performs, it can say it is invalid. I don't understand. Whatever it is, it's invalid, or I don't understand the input. So yeah, again, extension of what we did before. Now, there is a pizza analogy. This is to understand the clean pattern, OK? Now, if you have to ask me, OK, can I order zero pizza? Yes, you can order zero pizza, right? If you don't want to eat, you can tell your friend, I want zero pizza, right? That means I don't want to eat. You can order 1,

200 pizza. Now, what are we talking about here, OK? So one of the concepts that we have in regular expression is your clean start, OK? Now, is there any upper limit that you can have? No, right? You can demand. I mean, whether you get it or not, different story. 500 slices of pizza, right? So there is no upper limit, but there is a lower limit, which is zero. So when you put a star sign in regular expression, the regular expression star means you are repeating from zero to any number of times, OK? You are repeating. Like when you say A star means A can be zero time, one time, 10 times, 100 times. That's the purpose of star, OK, using star. Means repeat it. Repetition can be from zero to any limit. Of course, you can't have minus. Minus is not allowed, OK? So the clean star, OK, what you see here, like A to the power of star, that's clean star. It allows us to represent infinite number of repetition. OK? So yeah, and you will see this very often in regular expression. OK, let's move on to the next one. OK, now this is your regular grammar. We know, right, before you form the regular language, you need to understand the regular grammar. Regular grammar means the rule. Rule behind the language, right? So first, we have seen alphabets, right? So we know like alphabets, but what are the valid alphabets? Or what is a valid combination of alphabets? Not valid alphabets. Alphabet is there in the sigma. But what are the valid combinations of alphabet that is allowed? OK, that's where this, your regular grammar will help. OK, now there's something called as finite automata. OK, OK, so before we talk about finite automata, let's understand this map. And then we'll talk about why this is finite or infinite. OK, so here you see what it says, $L = \{w \mid |w| \text{ is even} \}$ that belongs to A^*B^* . A^*B^* means it can contain any combination of A and B, right? But then it is made up of string w, which can have A or B star. OK, what it says is the length of w is even, right? OK, w, that means length. And you have regular expressions where it says $A \cup B$, $A \cup B$, $A \cup B$, B. OK, so this is a valid regular expression here. OK, so just like our English grammar, the regular grammar also tells us how to form your valid sentences. So what specifically formal grammar tell us, it tells us how to construct valid strings in a language. And what we see here, OK, it generates a string of even length, right? It could be 0, 2, 4, 6, 8, right? So first thing is even length. And then, OK, and the same language can be represented in multiple ways, OK? So using regular expression, using the grammar, OK, and using a finite automaton, OK? So now, what is finite automaton? These are different representations of the same language. So of course, we draw a transition diagram, OK? It has those two circles that you see, OK? It has two circles, and those two circles will be connected with arrows. And the two things that you will see apart from that is one circle will have the star here, and you can have more than one circle. OK, next week, we'll talk about when you can have two or different number of circles that can have dual circle within it, right? You have one circle within another circle. So the arrow indicates that this is your starting position. This is where we start. And double circle indicates where you end. So in this. Start and come back to itself before you end. What it says is if you get to move to T state, you're moving if you get A or B. OK, now if I get A, let's say I start, OK? OK, so let's say first you do nothing, epsilon. If it is epsilon, then you start and you stop at start only. Is that valid? Yes. How do you know it is valid? Because you are in the ending state. So when you stop, you should be in a double circle. If you are in double circle, that means you are in the ending state. If I start and don't have any place to move, OK? And this can start itself is your ending state also. That means it's a valid string. Epsilon is valid. Now, if I get A, for example, is A a valid string in this case? No, because with A, you move from state S to T. But is T your final state, ending state? No. Ending state is S. So you have reached a non-ending state. And when you reach a non-ending state, OK, it's not the valid string. It's something that I don't understand half written code. If I say int identify, let's int. That is invalid pattern. Read the pattern. Similarly, invalid pattern. If I say again A, A comma A. Now, on A from S, you reach to T. And again

from T, with A, you reach back to S. OK, so A comma A, is it valid? Yes, it is valid. A comma B, is it valid? Yes. Again, you reach to A. B comma B, is it valid? Yes. B, A, is it valid? Yes, because from S, you reach T. But it is B. And again, A. So all four, A, A, B, B, A, B, B, A, are all valid string. In this case, you come back to S. And S is a ending state. So it's your valid automata. This is what we're talking about when we talk about your regular expression. So this is just the beginning, OK? We'll talk more about it in the next session. But what do we have on the next slide? T, F, A. Deterministic. If we have a pattern, who checks it? Answer is automata. Automata checks your pattern. So if you want to enter a building, you have a security guard who has to check your ID card. And if your ID card is valid, you are allowed to get in. Otherwise, you are not allowed to go in. Similarly, when we had epsilon or we had double A, the guard allowed you to complete the string. But when we had single A, it did not allow. So your automata is like that guard. It's a system. It's a program. It's a computer which determines what is valid and what is invalid. Now, what is deterministic in it? Deterministic is you know what is the next state. Next state is determined. When you have an input, when I have an A, I know what is the next state. Where am I going? If you do not know where you need to go when you have A, let's say in the previous example, from S to T, you have both A, B. If I remove B, I have only A. So from S, when you get B, you know that you need to go to T. But if I have A, then I don't know where to go. Then we call it as non-deterministic. For all input values, I do not have a clear output. For some, I have. But for some, I don't have. I don't have clear output. And that's where your determinism will fail. And we call it as non-deterministic automaton, ND. We'll talk about it as we go along. So this is what it is. And here, DFA does exactly the same thing like security guard. So one thing is the state diagram that we see, transition diagram, where you can actually represent. And second thing, you can create a table as well. We create a table as well. So table will have current state, input, and the next step. So when you are at Q0, let's say you are at S, and you get from Q0, you can go to Q1, which is valid. Look at the previous diagram on the top that you see. So here, Q0 is your start, because arrow is on Q0. Then you have from Q0, you can go to Q1, or you can go to Q2. But Q2 is invalid. So if you go to Q2, then it's invalid. But Q1 is valid, accepted state. So from Q0, if I land up in Q1, Q1 is ending state. That's why it's valid state. So it's valid. But Q0 to Q2 is invalid. Similarly, when you are at Q1, which is your accepted state, to Q1 is valid, because you can be in the same state as well. So it is valid. But from Q1 to Q2 is not valid. If you talk about Q2, Q2 to Q2, access denied. OK? So we use table also to represent the value. We'll learn more about these diagrams and tables in the next session. Yeah. There are some examples here. see okay q0 q1 you have a state so in the first example okay so valid alphabet are 0 and 1 that means okay the state that we have it can only take your you know each state how much time we have 9 yeah let's finish up this quickly so q0 has only alphabet that we have so from q0 you can have you I mean from scuse me can have 0 or 1 so what it says is when you get 0 you remain in the same state when you get 1 you go to q1 so is 0 a valid string no it's not valid string because you remain in q0 which is not a ending state but if I get 0 0 0 0 0 0 1 it's a valid because any number of zeros you get you will be in same q0 and then moment you get 1 you reach to q1 now if I get 0 1 0 0 0 1 1 1 1 1 1 1 1 yes that's again valid because you know 0 you state you stay in q0 you get 1 you move to q1 and q1 again irrespective number of 1 because 1 it's the same right you remain in same state so if you get a number of 1 you remain in same state but the moment you get 0 again okay you go back to the q0 then since q0 is not a valid state I mean for not a finite sorry not finite not the ending state the input value is not valid so this automata accepts only you know if this if the string ends with 1 the string ends with irrespective of what you can have anything and have any number of zeros 1 0 1 of course alphabet has 0 and 1 so only zeros and 1 allowed any number of you know

any value 0 1 0 1 but at the end it should be 1 is accepted if it is not 1 if it is 0 it will not be accepted okay so you can test with multiple value that is what it is done on the right side you can test with multiple values you know which are accepted and which are not so how does DFA no string ends with 1 okay you end up in the last state that means you know it's a valid state okay so this is how we analyze and see if it is valid state or not okay then we have relationship between re and DFA okay yes does the DFA take into account the whole string so far or does it only take into account the previous two characters of the string because in in the previous case there were only two states so meaning only the history of the past two would be required right past one is required so you can have any number of state think of let's say you know let's say you know you think about that you are standing on floor in your home and let's say your floor has black and white or let's say you're standing on a chessboard okay white and black okay now every stage okay you you decide whether you can go to white state or black state okay now what matters to you is only the final state that you end up with right now if I say that if you end up with black state only it is valid so what matters the current state but you you have the previous state knowledge so it doesn't matter okay how many state you have but even if you have hundreds of states and hundreds of alphabet still it matters your you know the ending state okay so yeah next it depends upon where you are once you reach the next step then you don't need to know the previous step right so that's why here is no memory is necessary for going to the next stage but after that the previous knowledge is centrally required yes exactly so previous state is nothing but your current state right there's nothing called previous current and future so previous state is not required your current state is required and the next state is because that is what determines where you are understood yeah so it's about let's say you know vending machine and let's say you want to buy a chocolate which is worth 50 rupees now okay you can give one single 50 rupees you can give 10 rupee note 5 times or you can give 5 rupee 5 rupees note 10 times right or 1 rupee note 50 times now when you put the first point it needs to know the current state and state is zero rupees the moment you put 10 it needs to know that you know the next step okay so if I have zero and if I get a 10 rupee note then I'm at state 10 so you need to know 10 but once you move to 10 you don't need to know you know what was given to you before as long as you have a memory which stores your current state which is 10 now if I entered 20 rupee note now yes need to know current state because 10 plus 20 will be 30 so you once you know if once you go into future state which is 30 then again you don't need to know whether you know what was the previous it was it 20 or 10 or 5 you don't need to know you will roll in remember 30 so it'll tell you that I got 30 and I need 20 more of course right so when you say I'm you know when you stop entering node it will check is it less than 50 or more than 50 or equal to 50 if it is less than 50 you won't get your stuff but the moment it's become 50 or more than 50 then it will give you the your stuff right so the idea here is that it tracks only the current value and to know future value into no current value so current value and future value once your future value becomes a current value in the previous value is gone right yeah so yeah so that is the relationship between regular expression and DFA okay and there's some examples here okay language expression DFA in the final state or not if you're in the final state that means it's valid otherwise it's not valid so you have like DFA and program programming nothing but the automata that we spoke about okay so there's some questions okay and you know this is just to test your skills we are done with today's session let's finish off couple of questions here so what does a star contain you see a star what does it contain it contains lambda or epsilon it contains a a a a and so on perfect okay you know epsilon is also part of it yeah right because you know zero times and then a so now I have done done with one single string but you can apply same logic the first DFA that I showed you a be like any combination of a B is allowed

right as long as it is even a length match the following so language what is column B in the language okay anyway we are running out of time so let me share the answer and let's go through this so is your C set of strings okay and not any combination of string right valid combination of string that follows the grammar alphabet it's a finite set of symbols right and when you say finite it means not duplicated right and any order 0 or 1 1 0 means the same right there is no order and you can't duplicate this alphabet you can't have 0 comma 0 in alphabet then next something similar to what we did last week as what is string string is your sequence of symbols right from that alphabet okay so now all the symbols that you use has to belong to the alphabet so you need to know the alphabet then you have DFA right so what is DFA DFA deterministic finite automata talks about regular language and then you have regular grammar regular grammar generates regular language okay so with that we come to end of this session I hope we learn something new today and yeah if there are no other questions then we can end it here okay and we will take up those video thing we'll see okay what wrong what could have gone wrong in the back end we'll just check and we get back all right and prepare well okay for your next credit quiz anyway we'll meet before the quiz starts okay I think we starts on Saturday but again don't take my word check your calendar check your schedule or the content I think it starts after Friday okay Saturday it starts so we'll anyway meet one more time before but yes it keep checking your course website course structure and give it from the right thank you everyone have a good rest of the week welcome to week 3 yeah and I'll see you next Friday take care bye everyone so better that was the discussion sure we'll look

Recommended Reading: Introducing Regular Expressions Reading• .

Duration: 10 minutes

[Notes]

Brief: Regular expressions (REs) are symbolic notations used to define patterns within strings. They describe sets of strings over a given alphabet and provide a concise way to specify languages. Understanding REs is crucial for various applications, including lexical analysis, pattern matching, and designing finite automata. Alphabet: The set of symbols used to construct strings, typically denoted as Σ . Primitive REs: These include basic regular expressions like a symbol (a), empty string (ϵ), and empty set ($\emptyset$). Union (+): Describes the union of two sets of strings. Concatenation (·): Joins two strings end to end. Kleene Star (*): Denotes repetition of a string zero or more times. Example: Given the regular expression $(0+1)^*$, it describes the language of all strings containing any combination of 0s and 1s, including the empty string. Reference: Peter Linz, An Introduction to Formal Languages and Automata, Sixth Edition, Jones & Bartlett. Chapter 3, Section 3.1.

Recommended Reading: An Advice on Writing RE's Reading• .

Duration: 10 minutes

[Notes]

Brief: When working with regular expressions (REs), it is essential to pay attention to operator precedence to avoid ambiguity in the patterns you create. Incorrectly constructed REs can lead to unintended language definitions, which can cause confusion or errors in parsing. The regular expression $a.b+c$ can be interpreted in two ways: $(a.b) + c$ – This means the language consists of

either the string "ab" or the string "c".

$L((a.b) + c) = \{ab, c\}$ $a.(b + c)$ – This means the language consists of the strings "ab" or "ac".

$L(a.(b + c)) = \{ab, ac\}$ Without clear parenthesisation, the intended meaning of the RE can be lost, as the two interpretations yield different languages. Operator Precedence: Determines the order in which operations are evaluated in an RE. Parenthesisation: Recommended to explicitly state the intended pattern to avoid ambiguity. Example: The regular expression $a.b+c$ can be interpreted as either $(a.b) + c$ or $a.(b + c)$. These interpretations yield different languages, making parenthesisation essential to convey the correct meaning. Reference: Peter Linz, An Introduction to Formal Languages and Automata, Sixth Edition, Jones & Bartlett. Chapter 3, Section 3.1.

Recommended Reading: Definition of Regular Expressions Reading• . Duration: 10 minutes

[Notes]

Brief: A regular expression (RE) is a concise way to represent patterns over a given alphabet, useful in describing formal languages. Defining regular expressions involves formalising the connection between symbols and the strings they generate. REs allow us to describe sets of strings in a structured way, combining symbols and operators to define the desired pattern. Regular expressions are constructed using basic building blocks and combination rules. Primitive Regular Expressions: $\emptyset$, λ , and $a \in \Sigma$ (where a is any symbol in the alphabet) are valid primitive regular expressions. Combination Rules: If r_1 and r_2 are valid regular expressions, then: $r_1 + r_2$ (Union) represents the set of strings in either r_1 or r_2 . $r_1 \cdot r_2$ (Concatenation) combines strings from r_1 followed by those in r_2 . r_1^* (Kleene Star) allows any number of repetitions of r_1 , including zero. (r_1) (Parentheses) group expressions to ensure correct precedence. Given $\Sigma = \{0, 1\}$, we can define two regular expressions: $(0 \cdot 1) + (0 + 1)$ matches either "01", "0", or "1". $(0 \cdot 1)^*$ matches any number of repetitions of "01" (e.g., "", "01", "0101"). A string is considered a valid regular expression if it can be constructed through repeated application of these rules starting from the primitive regular expressions. Reference: Peter Linz, An Introduction to Formal Languages and Automata, Sixth Edition, Jones & Bartlett. Chapter 3, Section 3.1.

Recommended Reading: Languages Associated with Regular Expressions Reading• . Duration: 10 minutes

[Notes]

Brief: Each regular expression defines a specific language, which is the set of all strings that match the pattern defined by the RE. Understanding how to deduce the language from a given RE is fundamental to applying regular expressions in practice. Union of Languages: If $L(r_1)$ and $L(r_2)$ are the languages generated by REs r_1 and r_2 , then $L(r_1 + r_2)$ is the union of both languages. Concatenation of Languages: For r_1 and r_2 , the language generated by $r_1 \cdot r_2$ is the concatenation of strings from both languages. Kleene Star: The RE r^* generates strings by concatenating zero or more occurrences of the language described by r . Example: Given the RE $(0 + 1)^*$, the language includes all possible combinations of 0s and 1s, such as 0, 1, 10, 011, and so on. Reference: Peter

Linz, An Introduction to Formal Languages and Automata, Sixth Edition, Jones & Bartlett. Chapter 3, Section 3.2.

Recommended Reading: Problem Solving – Language for RE (1)

Reading• . Duration: 10 minutes

[Notes]

Brief: Regular expressions are powerful tools for problem-solving in various computational contexts. They can be used to specify patterns for searching text, validating inputs, or generating finite automata for language recognition. Practical Applications: Writing REs for real-world scenarios such as matching email addresses, validating identifiers, and searching for specific substrings.

Constructing REs for Problem Solving: Learn how to break down problems into smaller patterns and construct regular expressions to match specific criteria. Example: The RE

`([a-zA-Z]+@[a-zA-Z]+\.[a-zA-Z]{2,})` can be used to validate email addresses by specifying the general pattern for an email format. Reference: Peter Linz, An Introduction to Formal Languages and Automata, Sixth Edition, Jones & Bartlett. Chapter 3, Section 3.1.

Recommended Reading: Problem Solving – Language for RE (2)

Reading• . Duration: 10 minutes

[Notes]

Brief: This reading focuses on the process of deducing the language generated by a given Regular Expression (RE). It explains the foundational concepts of how regular expressions define sets of strings within a formal language. Through step-by-step guidance, readers will learn techniques for interpreting REs and translating them into their corresponding languages. An Introduction to Formal Languages and Automata, Peter Linz, Sixth Edition, Jones & Bartlett. Chapter - 3, Section 3.1 & 3.2. Introduction to Automata Theory, Language and Computation, John E. Hopcroft, Rajeev Motwani, Jeffery D. Ullman, Third Edition. Chapter - 3, Section 3.3.

Recommended Reading: Problem Solving – RE for a Language (1)

Reading• . Duration: 10 minutes

[Notes]

Brief: This reading offers a practical example of writing a Regular Expression (RE) for a specified language. It guides readers through the methodology of constructing REs, illustrating how to effectively translate language patterns into formal expressions. Through detailed examples, the content aims to enhance understanding of the application of regular expressions in formal language theory. An Introduction to Formal Languages and Automata, Peter Linz, Sixth Edition, Jones & Bartlett. Chapter - 3, Section 3.1 & 3.2. Introduction to Automata Theory, Language and Computation, John E. Hopcroft, Rajeev Motwani, Jeffery D. Ullman, Third Edition. Chapter - 3, Section 3.3.

Recommended Reading: Problem Solving – RE for a Language (2) Reading• . Duration: 10 minutes

[Notes]

Brief: These readings provide a comprehensive exploration of Regular Expressions (RE) and Deterministic Finite Automata (DFA), focusing on their role in language recognition. They include practical examples that illustrate the process of constructing regular expressions for specific languages and demonstrate how DFAs operate to recognise patterns within input strings. **An Introduction to Formal Languages and Automata**, Peter Linz, Sixth Edition, Jones & Bartlett. Chapter - 3, Section 3.1 & 3.2. **Introduction to Automata Theory, Language and Computation**, John E. Hopcroft, Rajeev Motwani, Jeffery D. Ullman, Third Edition. Chapter - 3, Section 3.3.

Recommended Reading: Regular Language (Using RE) Reading• . Duration: 10 minutes

[Notes]

Brief: A regular language is a formal language that can be expressed using a regular expression (RE). This means that every pattern described by an RE corresponds to a regular language, and conversely, every regular language has a defining regular expression. **Regular Language:** A language that can be generated using a regular expression. **Correspondence:** For each regular language, there is an equivalent regular expression that defines the set of strings belonging to it. **Lexical Analysis:** Used in compilers to identify tokens in source code. **Pattern Matching:** Applied in search engines and text editors for finding text patterns. **Network Protocols:** Useful for defining communication patterns in protocols. **An Introduction to Formal Languages and Automata**, Peter Linz, Sixth Edition, Jones & Bartlett. Chapter - 3, Section 3.1 & 3.2. **Introduction to Automata Theory, Language and Computation**, John E. Hopcroft, Rajeev Motwani, Jeffery D. Ullman, Third Edition. Chapter - 3, Section 3.3.

Recommended Reading: Introducing Finite Automata Reading• . Duration: 10 minutes

[Notes]

Brief: A Finite Automaton (FA) is a theoretical model of computation used to recognise patterns and solve problems through a finite set of states and transitions. It serves as a foundational tool in formal language theory and is widely applied in various areas of computer science. **Finite Automaton:** A system with a finite number of states and transitions that process input symbols to determine if a string belongs to a language. **States and Transitions:** Automata transition from one state to another based on the input symbol, following predefined rules. **Switch Control:** An FA can model a switch with two states—ON and OFF—and transitions triggered by a push. **Password Validator:** Automata are used to validate passwords by defining the rules for acceptable patterns (e.g., minimum length, character requirements). **Network Protocol Verification:** Protocols can be modeled with automata to ensure valid sequences of communication events. **Keyword Recognition in Programming:** Automata are used in compilers to identify keywords like "then" in source code. Finite Automata play a

significant role in pattern recognition, validation, and parsing tasks, such as compilers, network verification, and user input validation. Their ability to process input through states makes them crucial in both theory and practical applications within computing. Reference: John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Third Edition, Chapter 2, Section 2.4.

Recommended Reading: Deterministic Finite Automata (DFA) (1)

Reading• . Duration: 10 minutes

[Notes]

Brief: A Deterministic Finite Automaton (DFA) is a computational model used to recognise patterns and determine whether a given string belongs to a specified language. Each DFA has a set of states, a starting state, and transitions between states triggered by input symbols. DFA Definition: A DFA operates on input symbols from an alphabet and moves through a series of states in a deterministic manner—only one possible transition exists for each input symbol from a given state. $L = \{a^n b \mid n \geq 0\}$: This language includes strings that begin with zero or more 'a's followed by a single 'b'. q_0 : The initial state, reading the input symbol 'a' and transitioning to q_1 . q_1 : The state reached after reading one or more 'a's, transitioning to q_0 upon receiving a 'b'. q_2 : The final state that accepts the input once the conditions of the language are met. This reading introduces the concept of DFA using a simple example that recognises the language $L = \{a^n b \mid n \geq 0\}$. DFAs are essential in applications such as lexical analysis, pattern matching, and protocol verification, as they offer a precise way to validate structured inputs. Reference: John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Third Edition, Chapter 2, Section 2.4.

Recommended Reading: Deterministic Finite Automata (DFA) (2)

Reading• . Duration: 10 minutes

[Notes]

Brief: A Deterministic Finite Automaton (DFA) processes input strings by transitioning through a set of states based on the symbols it reads, ultimately deciding whether to accept or reject the string. $L = \{a^n b \mid n \geq 0\}$: This language consists of zero or more 'a's followed by a single 'b'. After reading one or more 'a's, it transitions to state q_1 . Upon reading a 'b', it transitions to the accepting state q_2 . Any further input (including 'a' or 'b') leaves it in state q_1 , which confirms acceptance. Result: Rejected – This string contains extra characters after the required structure. Result: Rejected – The string is missing the required 'b' to be valid. Result: Accepted – The string fits the pattern of 'a's followed by exactly one 'b'. Result: Accepted – Even with a large number of 'a's, the DFA still correctly accepts the input upon receiving one 'b'. Reference: John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Third Edition, Chapter 2, Section 2.4.

Recommended Reading: Deterministic Finite Automata (DFA) (3) Reading• . Duration: 10 minutes

[Notes]

Brief: A Deterministic Finite Automaton (DFA) is formally defined as a 5-tuple: δ : A transition function that defines how the automaton moves from one state to another based on the input ($\delta: Q \times \Sigma \rightarrow Q$). q_0 : The initial state from which the computation begins. F : A set of final/accepting states ($F \subseteq Q$). The DFA for the language $L = \{a^n b \mid n \geq 0\}$ is used to demonstrate how the automaton transitions through states based on input: $q_0 \xrightarrow{a} q_1$: Transitions to q_1 upon reading 'a'. $q_1 \xrightarrow{b} q_2$: The final state that accepts the input. This reading formalises the definition of DFA and illustrates how it functions as a language acceptor by processing input strings step-by-step according to defined state transitions. DFAs provide a precise way to model and verify input patterns in computing, making them essential in fields such as automated verification, lexical analysis, and communication protocols. Reference: John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Third Edition, Chapter 2, Section 2.4.

Recommended Reading: Language Accepted by DFA (1) Reading• . Duration: 10 minutes

[Notes]

Brief: DFA recognises patterns and describes its accepted language through state transitions and input strings. DFAs play a crucial role in pattern recognition and input validation in systems like parsers and lexical analysers. Reference: John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Third Edition, Chapter 2, Section 2.4.

Recommended Reading: Language Accepted by DFA (2) Reading• . Duration: 10 minutes

[Notes]

Brief: Through examples, the reading illustrates the practical applications of DFAs in computational theory, laying the groundwork for understanding more complex automata and their significance in formal languages. Reference: John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Third Edition, Chapter 2, Section 2.4.

Recommended Reading: Designing DFA Reading• . Duration: 10 minutes

[Notes]

Brief: This reading highlights the process of DFA design to recognise or reject input based on specific language rules. It focuses on designing a Deterministic Finite Automaton (DFA) for the

language $L(M) = \{w \in \{a, b\}^* \mid w \text{ does not contain three consecutive 'b's}\}$. The goal is to create a DFA that tracks the sequence of 'b's in the input string to ensure it remains valid. The language accepts all strings that do not contain three consecutive 'b's. Example valid strings: "aab", "ababa", "bba". Example invalid string: "bbb" (contains three consecutive 'b's). State 1: "I have not seen any 'b's so far" – The DFA starts here and transitions upon reading 'a' or 'b'. State 2: "I have seen one 'b'" – Moves here after reading a single 'b'. State 3: "I have seen two consecutive 'b's" – Reaches this state when two consecutive 'b's are read. Invalid State: "Three consecutive 'b's seen" – Rejected if this state is reached. If 'a' is encountered, the DFA can reset to earlier states. The DFA counts consecutive 'b's, but the sequence breaks upon encountering 'a'. The DFA successfully tracks the number of consecutive 'b's using distinct states. Any sequence with three consecutive 'b's is rejected. This DFA illustrates how automata can enforce complex language rules by maintaining a state-based structure. Reference: John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman, Introduction to Automata Theory, Languages, and Computation, Third Edition, Chapter 2, Section 2.4.

Recommended Reading: Grammars for Regular Languages Reading• . Duration: 10 minutes

[Notes]

Brief: Regular languages can be generated using various formal systems, including regular expressions, finite automata, and grammars. This reading focuses on the use of right-linear grammars to describe regular languages. A grammar $G = (V, T, S, P)$ is right-linear if all production rules are of the form: $A \rightarrow xB$ or $A \rightarrow x$, where $A, B \in V$ (non-terminals) and $x \in T^*$ (terminals). Right-linear grammars are used to generate regular languages. Consider the grammar $G = (\{S\}, \{a, b\}, S, P)$ with the following production rules: The language generated by this grammar, denoted $L(G)$, includes strings such as "a", "aba", "abababa". The general form is: Yes! Since the grammar is right-linear and generates only sequences of terminal symbols, $L(G)$ qualifies as a regular language. Right-linear grammars, with their straightforward structure, provide an alternative method to describe regular languages, complementing finite automata and regular expressions. Reference: An Introduction to Formal Languages and Automata, Peter Linz, Sixth Edition, Jones & Bartlett. Chapter - 3, Section 3.3.

Recommended Reading: Grammars for Regular Languages (2) Reading• . Duration: 10 minutes

[Notes]

Brief: In addition to right-linear grammars, left-linear grammars also generate regular languages. While the structure of the rules differs slightly, the result remains a regular language. A grammar $G = (V, T, S, P)$ is considered left-linear if all production rules follow the format: $A \rightarrow Bx$ or $A \rightarrow x$, where $A, B \in V$ (non-terminals) and $x \in T^*$ (terminals). In left-linear grammars, non-terminals appear at the beginning of the production rules. Consider the grammar $G = (\{S\}, \{a, b\}, S, P)$ with the following production rules: The language generated, $L(G)$, includes strings such as "a", "baa", "bbaaa". These strings follow the general pattern $^*(ab)a$. Given $G = (\{S\}, \{a, b, c\}, S, P)$ with the

following rules: The extended grammar generates more complex patterns, still following the rules for regular languages. Left-linear grammars provide an alternative way to generate regular languages by placing non-terminals at the beginning of production rules. This complements the right-linear approach, showing the flexibility of grammar-based methods in generating regular languages. Understanding both left- and right-linear grammars is essential for building language models and parsing algorithms effectively. Reference: An Introduction to Formal Languages and Automata, Peter Linz, Sixth Edition, Jones & Bartlett. Chapter - 3, Section 3.3.

Recommended Reading: Automaton for Left Linear Grammar Reading• . Duration: 10 minutes

[Notes]

Brief: This reading illustrates how a DFA/NFA can be derived from a left-linear grammar. The automaton reflects the rules of the grammar through state transitions. This section explores how to construct a finite automaton (DFA/NFA) for a given left-linear grammar. The connection between grammars and automata is essential in formal languages, as both describe patterns in structured data. The grammar $G = (\{V_1, V_2\}, \{a, b\}, V_1, P)$ generates the following productions: States in the DFA correspond to variables in the grammar plus additional states (including a final state). State Transitions are identified directly from the productions of the grammar. V_1, V_2, V_f (final state), and possibly V_3, V_4 (depending on additional states required by transitions). Reference: An Introduction to Formal Languages and Automata, Peter Linz, Sixth Edition, Jones & Bartlett. Chapter - 3, Section 3.3.

Review Questions & Practice Problems Reading• . Duration: 1 hour 1h

[Notes]

(No transcript found.)

Contact Session 2 Reading• . Duration: 10 minutes

[Notes]

(No transcript found.)

Direct Implementation of a Simple DFA Video• . Duration: 9 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to implement the computations represented in a DFA. Let's get started. What you see in the screen is a simple DFA. The DFA has two states q_0, q_1 . The DFA represents some computation for inputs on $\Sigma = \{0, 1\}$. The starting state is q_0 , final state is q_1 . And the language accepted by this DFA is, can you pause the video and try to identify that? The language accepted by this DFA is strings that always ends with 1. You can see that when you are in starting state, as long as you receive 0, you are in starting

state. The moment we see a 1, you go to an accepting state. At an accepting state, as long as we see a 0, we are at accepting state. The moment we see a 0, we don't want to see a 0 at the end of it. Right? So the moment we see a 0, go back to q_0 . So we are in an accepting state if the current input is 1. So this is a DFA that accepts languages over 0, 1 that always ends with a 1. Now, let us suppose we have a computer program that implements this DFA. In such case, what should the program do? It should take any string as input, any string over Σ as input. Let's say the string over Σ is x . It takes x as input and it returns an output yes if x is an element of L , else it says no. Right? So what should this C program do internally? Or in other words, how should this C program be written? We'll begin writing a pseudocode. Right? How should this pseudocode be written? Or how should the algorithm be arrived at? For which, we should quickly revise how the DFA accepts a string. Let's assume a string 0, 0, 1. Now, when the system is in the starting state, it receives the first input 0 and the system is in the starting state itself. It receives the next input and the system stays in q_0 . And it receives the third input and the system goes to an accepting state. The system comes to a halting state after processing the entire input. Hence, 0, 0, 1 is accepted. We want our algorithm to replicate or to implement this idea. Exactly. Let's see how to do that. We'll assume that our input is kept in variable x , rather an array x . So our input x will be kept in an array like this. The size of this array or the length of this input will be n . So x will be the input and n will be the length of it. And assuming a language like C, the index of the first element will be 0, second one will be 1, third one will be 2. These are basic assumptions we'll make. It is absolutely not necessary that we have to keep the input in an array and then we need an explicit length of x to be n to be initialized. This will make our implementation simple. Since this is our first attempt at implementing DFA, these assumptions are fine. The logic would remain the same irrespective of how do you really keep your input. Let's suppose we have two variables. In one variable we'll always remember the current state. In the other variable you'll always remember at what point are you in processing the input. So let us suppose your input is 0, 0, 1. While you begin, or the current element to be read and processed is 0. Alright? These are your initializations. The second part of the algorithm is to replicate the iterations involved in processing the input by DFA. How does it happen? Take the current input, look at the current state and the DFA says if in q_0 you see an input 0, the next state will be q_0 . Then read the next input. While reading the next input, refer the current state. Right? In q_0 , if you see a 0, the new state will be q_0 again. So after reading each input, consult the current state and then update the new. Right? If you are in q_0 , seeing 0, go to q_0 again. If you are in q_1 , seeing 1, go to q_1 again. If you are in q_1 , seeing 0, go to q_0 again. And we need to do this iteratively for each element in the input. Read the input and follow the DFA. What do we mean by that? We need to read the input as long as the index is less than n . n is the length of the input and index will always point to the current position referring to the character we are currently processing. So we need to repeat this until the end of the input. Starting from the first character to the second, third, fourth, all the way till the last. Within the while loop, what do you do? We need to update the state by looking at what is the current state and what is the current input. So we do this within the loop. After this is done, all we do is inside the loop, increment the index pointer so that you go to the next input. Right? So to visualize this better, again I take the example. So within the loop, all we do is refer to the current input and current state and then update the state. So q_0 , 0, it becomes again q_0 . Right? So that's what you do. Let's look at this. The first step within the loop is read the current input. Right? In 0, 0, 1, when the index is 0, we are going to read the first input. Right? Keep the current input always in A . All right? Now you know the current input is A . You know the current state is 0. Right? What should you do? Switch statement. Switch state. That is, perform the action according to the current state. If the

state is 0. Okay? Right? So state is 0. Under case 0, we have implemented all the transitions that goes out of state 0. If the input is 0, in state 0, then the new state is also 0. Correct? If the input is not 0, then the state is 1. In case 1, what if the state is 1? If the current state is 1 and the input is 0, go back to state 0. So the body of this loop is a very simple implementation of a DFA. Right? You will have a switch statement and you will have one case block for each state and within the case block, you will implement all the transitions for that particular state. Case 0, all the transitions for state 0. Case 1, all the transitions for state 1. If there are multiple states, repeat the same. Now, assume the input for 0, the processing the loop has completed its job. Now the index will increment by 1. You will go to the next pointer, right? Next location. And when you come back to the beginning of the loop, you will read the next element and you will repeat it. All right? The loop will continue until you read the entire string. The moment index becomes equal to n and once you come out, all you need to do is look at what is the value in the state variable. Right? If state is equal to 1, print yes. Right? Because state 1 is accepting state. So here we should write, if state equal to 1, print yes. Else, print no. I'm sure you understood this simple implementation of a DFA. Respective of the size of the DFA, you will be able to write this kind of an implementation by yourself. And if you have this implementation, or if you have an algorithm like this, you can directly write a C program. All right? And you can execute it and try it out. So you learned how to implement a DFA by directly implementing all the transitions in the loop. Right? The implementation that I've shown to you might look a little inefficient. Right? It's possible to simplify it a lot more. I'm leaving it all to you. Thank you.

Table Driven Implementation Video• . Duration: 8 minutes

[Notes]

Dear students, welcome to this video. We've learned how to implement a DFA by directly implementing all the transitions. It's time to learn an even simpler way of implementing the DFA. So in this video, we'll learn how to implement a DFA by representing the transitions as a table or a 2D matrix. Let's begin with a simple DFA. This DFA accepts language over $\Sigma = \{0,1\}$ such that a string always ends with a 1. Now the transitions in this DFA can be nicely captured as a table. We have one row for each state and one column for each input symbol. Right? So for state q_0 , we have a row. The first row corresponds to state q_0 . And from state q_0 , there are two outgoing edges. The other one is incoming. Ignore that. There are two outgoing edges. One outgoing edge has got a label 0. The other outgoing edge has got a label 1. So the first column, title 0, will capture this edge. And the second column, title 1, will capture an outgoing edge on 1. Similarly, the second row captures the transitions that goes out of second state q_1 . So the table nicely captures all the transitions there in the given DFA. And this transition table can be represented as a 2D matrix in any programming language. Let us suppose we create a table called a transition table. A 2x2 table, because we need to represent this part of it. Correct? It's a 2x2 matrix, in which the first row has got entry 0, 1. 0 representing state q_0 , 1 representing state q_1 . So the first row will have entry 0, 1. Correct? So if you consider a table, a transition table, then the first row corresponds to state 0, will contain entry 0, 1. Right? This entries. Second row will contain entry again 0, 1. So this table can be nicely represented as a transition table of two dimensions. Now assume that you have a transition table, let's actually assume this is your transition table. This transition table, the entries can be accessed using a row value and a column value. Right? So how would you access this entry? By giving the row value, which is a current state, and by giving the current input, which is a column

value. Correct? So if the table is accessed with entry 0, 0, that is transition table of 0, 0 refers to this entry. Okay? This entry means if the current state is 0, input is 0, go back to 0 state. And entry 0, 1 means, if you are in state 0, seeing input 1, then go to state 1. Correct? So the table can be used in a very simple way like this. Let's now look at how this table simplifies our implementation. We assume our input x is in an array. Let us suppose an input like 0, 0, 1 is kept in an array x . And the length of this array is n , which is 3. The array x is indexed from 0, 1, 2. Right? And we initialize two things. One is what is the current state? When we begin, the current state is always 0. The index is 0, signifies that we are going to read the 0th element in the array. Our first input is in the 0th element of the array. Alright? Now, the next step in the algorithm is to read the input and follow the DFA. So an input like 0, 0, 1 must be read from left to right, element by element, and be processed. Right? The current state is 0. The current input is in location 0. Correct? The current input is in location 0. What we should do first? We should do what happens if the current state is 0, input is 0. How can we use table to check it? We just go to 0, 0th entry of the table. That tell you what will be the state after consuming 0 in state 0. Alright? So this entry says, come back to state 0. Understood? So I've looked at the table and the table says, you'll be in state 0. Right? That's what the table says. Now advance the index so that you can read. Next input is again 0. Now you should check the entry 0, 0 in the table. The 0, 0th entry in the table says again, your new state is 0. Right? Fine. So go back, increment the index and read the next input. The next input is 1 and the current state is 0. Right? So if the current state is 0 and the next input is, and the input is 1, which entry will you read in the table? In transition table, you will read the entry 0, 1, 0, 1th entry says the current state is 1. Right? So you will update the current state is 1. And at this time, you have read the entire input. After reading the entire input, the current state is 1, which is a final state and you will announce that the string is accepted. Now let us look at the way we actually have written the iteration, considering that A transitions are represented as a table. So we read, we repeat as long as the input is not completely processed. It means as long as the index is less than n , we need to repeat. Take the current symbol, right? When the index is 0, current symbol means the first one, when the index is 1, the current symbol means second one, when the index is 2, the current symbol means so. So look at the current symbol and take the current symbol. After taking the current symbol, at this moment, you know what is your current state and you know what is the current symbol. Right? So you can directly go and look at the transition table for the new state. So update the state as transition table state comma A. So look at this transition table and update the current. So you got a new state. What should you do now? Go to the next step, go to index plus 1 and come back to the loop. Keep repeating this. Once you come out of the loop, if you see that the current state is equal to 1, then you will print yes, that the input is accepted, else you will say no. So the loop would still be very compact, right? If the transition table is larger, it means the DFA is larger, but the loop will still be the same compact loop, right? So this implementation is a lot simpler implementation. We've come to the end of this video. I'm sure you understood what is a table-driven DFA implementation and how it is simpler comparing to a direct DFA implementation. Thank you.

Direct vs Table Driven Implementation Video• . Duration: 4 minutes

[Notes]

Welcome to this video, we've learned how to implement a DFA using two different approaches. Directly implementing all the transitions or representing all the transitions as a matrix and go for a

table driven implementations. In this video, we're going to put both implementations side by side and see how they compare. Let's get started with a simple DFA. You know this DFA is accepting languages over 0 and 1 that always ends with 1. Your professor asked you to draw this DFA and then asked you to implement this DFA, right? So you have a DFA like this, and you have a code implementing this DFA. And now he comes back and say that, oh, I think the language has to be slightly changed. Right? So you change the language and you redesign the DFA that resulted in some changes, right? The state q1 from the original one or the current one, there's no change. The state q1, there is a minor change, right? On input 1, you stay in q1 here, you stay in q1 in the previous one, so there's no change. But on 0, there is a change, right? And there's a new state got introduced, a state called a skew error. Now you need to go back and implement these changes in the code, right? What if you implement the original DFA as a direct coded one, or what if you implemented the DFA as a table driven one? How easy to make changes. In the direct coded, which is actually in the left side of the screen, the transitions are written here, right? I'm directly writing all the transitions, switch state for each state, right? So to accommodate these changes, you will have to insert one more case for the new state and you will have to change, correct? You'll have to change this, change this state transition. So you'll have to work by changing the content of the value. In case if you have implemented the original DFA as a table driven one, do you need any change in the code? But you need to change the transition table. If the transition table is changed, then the code works seamlessly, correct? The table earlier was a two by two table, two states, two inputs. Now it will be a three by two table, three states, two inputs. And the algorithm does not change for the table driven implementation. That would give you an impression that the table driven implementation is a very simple implementation. Indeed, yes, it is. There's one important thing that you must look at from the table driven implementation. Every time we need to change the state, we need to go to the table, access the particular element and then update the state, right? A table lookup is involved every time you need to change the state. Whereas in direct coded, you don't need any table lookup. It's all there in the code directly. All right? Now, a larger DFA, for example, we have a programming language like C or Java or Python. We need to recognize a lot of keywords, identifiers, various elements in the language. So the DFA will be pretty large, right? In such case, the table will be extremely large. So the table size will be M by N. If you assume M to be the number of state, if you assume N to be the number of input symbols, allowed input symbols, then the size of the table will be big of M by N, right? So that size has to be dedicated for table and the overhead for accessing the table entry is also involved while you execute the code. But then when it comes to making changes, changes are required only in the table entries, not in the code. Having a larger DFA, implementing a larger DFA like this, using conditions for each of those transitions, right? Making changes would be difficult, it's error prone, and it's not quite reliable. So for reasonably sized DFAs where you actually can manage the complexity, you can think of using a direct implementation of the DFA. If you have a larger DFA where maintainability, reliably changing DFA for updating, for incorporating the new changes, then the table driven DFA is a good option. For our course going forward, I assume for whatever assignments or programming exercises that we give, we will use table driven implementation. So by now, you would have understood how does a table driven implementation and a direct implementation of a DFA compares. Thank you.

Structure of a Compiler Video• . Duration: 6 minutes

[Notes]

Welcome to this video. In this video, I'm going to introduce you to a high-level structure of a compiler or a language translator. Let us get started. A compiler is a program that translates an input program in a certain language into an output program in another language. You learned how to compile a C program into a target program earlier. In that case, you wrote a program in C language and then fed to a compiler and the compiler generates a program in target language. A target language is a language that's close to a machine language. A compiler is a large piece of software. The internal parts of the compiler are organized into two larger components, one called as front-end followed by another component called as back-end. It's important to understand that the source program is in a language called as C and the target program is in a language that's close to a machine language. It's as good as saying that you have a passage written in English and you would like to translate this passage into Spanish, right? So this requires understanding the English passage quite well and encoding the understanding in a form from where we can translate the passage into Spanish. So the front-end is largely concerned with understanding the source language or rather English here. The back-end is concerned with translating the intermediate representation into the target language or in other words, converting the intermediate representation into Spanish. So the front-end performs the task of analyzing the source program and verifying the source program is syntactically correct and it has semantics, it has valid meaning and this front-end can be seen as having three components. To understand a passage in English, before you go ahead translating this into Spanish, the very first thing that you do is to go through every word and see that each word used in the program is a valid English word, right? A lexical analyzer takes a program in C and then goes through the program from consumes the program character by character and see that every element used in the C program is a valid element as per what is recognized by the programming language's syntax. Assume we have an example, `xsint equal to sum plus y`, maybe there's a semicolon, right? Now the lexical analyzer reads this program character by character and then groups the character into meaningful elements that are recognized as a valid element by the syntax of the programming language. If you just look at `xsint` is a valid element as recognized by the syntax of the programming language. What element it is? `xsint` is an identifier, right? It recognizes `xsint` as identifier and then it recognizes `equal to` as an equal to symbol and then it recognizes this simple `sum` as identifier and then it is recognized as `plus` right or rather `op` and this is recognized as identifier again and this is recognized as `semi`, right? So an input `xsint equal to sum plus y` is recognized as an identifier followed by equal to followed by id followed by operator followed by id and then semi, right? So each of these element `id`, `eq`, `semi`, `op`, each of these element is called as a token. A token is a sequence of characters that can be treated as a unit in the grammar of a programming language, right? So grammar of the programming language treats `id` as a unit, `eq` as a unit, right? So a grammar of the programming language accepts this statement as a valid assignment statement because the syntax of the assignment statement is such the LHS of the assignment statement has to be an identifier and RHS of an assignment statement has to be an expression and an expression has to be of the form `id plus id`, `id plus number` and so on, right? So a lexical analyzer takes the source program as input and produces a sequence of tokens of this kind which the parser takes it and verifies that this sequence is valid according to the syntax of the programming language. Once after the parser verify the syntax and parser convinces that the program is syntactically correct, it gives this output as a syntax. Next state or next task of the front end is to verify that the program semantically correct, right? So it takes the syntax tree at input and then performs a semantic analysis and once after the semantic analysis phase completes its job, it produces its output in the form known as intermediate representation. So a source program is getting continuously transformed first into tokens by the

lexical analyzer and then into syntax tree by parser and then into an intermediate representation after a semantic analysis. The back end will take the intermediate representation as an input and then prepares the output program in the target language. I am sure now you understood broadly how a compiler translates a programming source language into target language. Thank you.

Working of a Lexical Analyser Video• . Duration: 8 minutes

[Notes]

In this video, we are going to learn about a lexical analyzer. Lexical analyzer is the first phase of translation in a compiler. Let's get started. A compiler is a program that takes a program written in a source language and then translates this into a program in target language. The first phase of translation happens at lexical analyzer. Lexical analyzer takes a source program as input and then produces a sequence of tokens as output. A token is a sequence of characters that can be treated as a unit permitted by a grammar of a programming language. Let's try to understand this better with an example. What you see in the screen is a simple proof. The first line is a line commenting that the program that follows has a function main. Right? So what you see in the first line is a comment. And then we declare the function main as a function that takes integer input and returns an integer output. And we define the main function as main equal to, it's a function which takes an integer as input and returns an integer. Here comes the body of the program. There's only one line in the body of the program. Return f of x. Right? And then finally, we have an end module keyword that closes the module. Now the lexical analyzer looks at this program from the very first character and it looks at every character from the first character onwards and it attempts to recognize the elements of this program. Elements as recognized by the grammar of the programming language. Right? So while trying to recognize the lexical analyzer understood that the program starts with a comment. Right? The moment the lexical analyzer sees that there's a slash and star, it understood, oh, there's a comment beginning and it waits until the end of the comment. Right? But it recognizes the whole comment and it does not return anything. It does not produce any output for that. It simply strips the comment. Right? Because comments are not necessary for translation purpose. Comments are necessary for us to understand a source language better, source program better. Right? So it looks at a comment from start to end and recognizes that this much part of the program is a comment and then it just ignores it. And then it actually proceeds to the next character and it finds that we're using VAR. VAR is a keyword in the program. So it returns a token keyword for variable is found. TK underscore key underscore VAR just stands for we've recognized a keyword named variable. As it proceeds, it understood that main is a name of the function. So it is an identifier, right? The category of main is identifier and the value is main and it proceeds. It recognizes the colon and it proceeds. It understands that a keyword integer has been used. So it returns TTK integer and then it recognizes hyphen hyphen greater than symbol together as a long arrow. So token long arrow and then it recognizes a keyword integer and then what follows integer is a semicolon and then the lexical analyzer continues to read the next line main, right? So main is an identifier. We're defining the function main. So main and this is an assignment statement. Whenever we see a colon and equal to in this particular programming language, it is considered to be an assignment statement. So TK assign fun. It's a keyword fun. in a keyword in X an identifier with value X colon keyword integer right parenthesis keyword return keyword integer left brace keyword return, right? And an identifier F with value F left parenthesis an identifier X and it goes all the way till the end of the program and return

the sequence of tokens. So you can understand that the syntax of the programming language recognizes a declaration to be a variable keyword must be used followed by an identifier that you want to declare, which can be main or anything identifier. If the declaration is of the form identifier colon int a long arrow float, it means that this variable declares a function, right? So this is a structure or a syntax as recognized by the programming and here in this statement variable is a valid syntactic element ID is a valid syntactic element an element that is recognized by the syntax of the programming language right colon is recognized by syntax of the programming language and so on. So the lexical analyzer has to go through the program from the very first character to the last character and then recognizes every element from start to end. In fact, this is the only phase in the entire compilation program that looks at the entire program character by character. So lexical analyzer is a software that takes source program as input and then returns a sequence of tokens as output. The input to lexical analyzer is a program as a text stream and the output from the lexical analyzer is a token stream. Look at this, the input is a text stream, right? And the output is a stream or sequence of tokens while it generates its output. The lexical analyzer gives all comments, all white spaces, it will all get filtered out. The white spaces comments are not necessary for the translation purpose, right? The white spaces comments are not necessary for the translation purpose and other symbols that you use only for indentation purpose. These are all things which are filtered out and in case if there is an error, it gives an error. For example, you're trying to define a variable, something like 5, 3, x, y. We expect a variable to always get started with an alphabet. It's 5, 3, x, y. It doesn't recognize anything. It doesn't recognize this is a number, it doesn't recognize this is an identifier. So the lexical analyzer directly would say that this is something I don't recognize, right? It will show error as and when it is required, invalid comment, invalid token, all those errors it actually gives. I'm sure by now you get a fair understanding of what lexical analyzer is and what is a token. Thank you.

Approach to Design Lexical Analyser Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, you are going to learn about the approach we take to design a lexical analyzer. Once if you understand the approach, you will be able to understand how lexical analyzers are designed. Let's get started. A lexical analyzer is a program that takes a program written in a particular programming language or in some programming language and then it produces a sequence of tokens. The lexical analyzer looks at the input program from the very first character and attempts to group the character into meaningful units, which we call as tokens. If you take this program as an example, it groups this sequence of characters into one group called as comment is a unit that lexical analyzer handles in a very peculiar manner. A lexical analyzer simply strips off comments. It does not produce any output for comment. It recognizes that a sequence of character qualifies to be a comment and then it strips off. It doesn't produce any output. It's important to know that the task of recognizing something as a comment is still rest with lexical analyzer. And then it recognizes are every possible tokens there in the input program. It recognizes a keyword like variable, integer, fun, return. It recognizes every single keywords. It recognizes every single operators. It recognizes every single identifies. It recognizes every tokens in the input program. A programming language by the way it is designed, it recognizes a certain set of tokens in the input program. So, a lexical analyzer must be able to recognize all

these tokens and be able to produce a corresponding output. So, for us to design a lexical analyzer, the very first job is that the lexical analyzer should be able to recognize each and every token. So, the first step in the design of a lexical analyzer is to be able to list all those tokens and be able to describe each of this token as a regular expression. So, the first job is to list down all the tokens and be able to describe it using a regular expression. For example, in case a token that we want to recognize is an identifier, then the identifier can be described as a regular expression as below. It always starts with a letter and what follows is a sequence of letter or digit. Right? So, it always starts with a letter and then what follows is a sequence of letter or digits. This is a regular expression that defines an identifier. A keyword like VAR can be defined like this. V-A-R. It's a regular expression that describes VAR. So, every token must be listed down and we must be able to write a regular expression for all the tokens. Once we have a regular expression for all the tokens, the next step is to design deterministic finite automaton for each of the tokens. So, we have a DFA that recognizes keyword variable, identifier, colon, l-parent, n-mod, every single token. Right? So, we have a deterministic automaton for recognizing every single token programming language. A lexical analyzer is a program that recognizes all the tokens in the input program. So what we must do after constructing the DFA for every single token is that we must combine all those DFAs into one large DFA that recognizes every token in the input program. This might look a little complicated. In the next video, I'll give you an example to make sure you understand this. I'm sure you understood the overall idea used in designing a lexical analyzer. It's a three-step process. Define the regular expression for each of the tokens. Convert the regular expression into DFA. Combine all the DFA into one. And then implement that DFA. Thank you.

Lexical Analyser Design Example-1 Video• . Duration: 13 minutes

[Notes]

Dear students, welcome to this video. In this video, you are going to learn how to design and implement a lexical analyzer. We are going to learn this with a simple example. Let us assume that we have a toy language in which there are only 3 tokens to be recognized. The first token to be recognized is a number, second to be recognized is an identifier, third to be recognized is a comment. Of these, when we recognize comment, we will not return a token which will figure in the output of a lexical analyzer. We would still recognize a comment and then we will strip the comments off. However, since our programming language allows us to write numbers, identifiers and comments, let's proceed to recognize all of them. An identifier is a sequence of one or more characters in which the first one is an alphabet. So sum is a valid identifier, sum 5 is a valid identifier, 5 sum, it is not a valid identifier. So the definition is self-explanatory. Now the next token that we want to recognize or the next category of an element that we want to recognize is comment. A sequence of characters that begins with a double slash and ends with a new line is a comment. The approach to design a DFA begins by defining the regular expression and then writing the DFA for each of those elements. So the first step is to come up with the DFA for each of those elements. In case if you can draw the DFA directly, it's well and good. Otherwise, write the regular expression and then arrive at the DFA. These elements are simple elements for which we can write or we can draw the DFA directly. So I'm going to give you the DFA for each of those elements. Let's begin with a number and number in the programming language is a sequence of one or more digits. So starting with the start state, if you see a digit. So the digits is a set 0 to 9. So if you see one of the digit, go to

state number 2. And in state number 2, if you keep seeing digits, stay in state number 2. This recognizes number. Let's look at the next identifier. It's a sequence of one or more characters in which the first character is alphabet. So assume we have a set alpha, which has got A to Z, lowercase and uppercase. So when we mark alpha in the transition or along the edge, it means you're seeing one of those symbols from alpha. So assume you are in start state, state 1, you see the alphabet, go to state number 2. State number 2 is a terminal state. If you see just an alphabet, you got an identifier. And if you see digits or alphabet after you are being in state 2, you stay in state, right? So identifier always begins with an alphabet. And what follows is a sequence of alphabet or digit. The last element that we want to recognize is a comment. A comment is defined as a sequence of character that begins with double slash and ends with a newline character. Let's write an example of a comment, right? So it starts with double slash and we write, okay, this is a row in C. It's not a good comment, but it's a comment. Starts with double slash and you write whatever you want to write and you enter a newline character. So it starts with two slashes. So after seeing two slashes, you are in state 3, sure that you're going to see a comment, right? Now, after seeing two slashes, stay in state number 3, irrespective of whatever you receive, except the newline character. That's what it means. As long as you receive anything except the newline, stay in state number 3, right? So when you look at first slash, you go to state number 2. When you look at second slash, you go to state number 3. for the entire sequence, this is a program, this is FLA. So after seeing the entire comment, we still stay in state number 3 until we see a newline character. So as long as you don't see a newline character, stay in state number 3. And the moment we see a newline character, go to state number 4. So we've completed designing the DFA for all three elements. The first DFA that we had is for a number. And the second one that we had is for identifier. And the third DFA is for comment. So we have three DFAs. We've learned a lexical analyzer is a single large DFA that recognizes everything. How do we combine all three DFAs into one? So we cannot have three start states, if we combine all of them into one. So let's create a new start state, okay? And then we'll combine all three DFAs. From the start state, if we see digit, then probably we are going to recognize a number, correct? And you can get rid of this part. And from start state, if we see a slash, then probably we are going to recognize a comment, right? So this part, we can get rid. So remove the start state, create a new start state and connect this new start state with every other DFA. And now take the next DFA, we get rid of this part by creating a new connection or edge with a label alpha, right? So let's proceed. We want to combine three DFAs into one. So we're going to create a new state, new start state. And we are going to combine this and this, right? This is how it looks like. We got rid of this start state and we got rid of this start state. So what does it mean now? You are in start state. If you see a digit, you're going to probably recognize a number. If you see an alphabet, you're going to probably recognize and identify, right? So from start state, if you land at this state, then you would return that you recognize token number, right? So in this state, you'd probably return that you recognized a token identified. Now let's see how to connect the third one. So how do we do that? We remove this part and we connect this, right? So we will remove that part and be connected. So this becomes our single DFA, right? It's not complete yet. We can see that there are a lot of states with state number two. So it's important that we renumber all the states. So this is a state number two and let's actually renumber all the states so that the states are actually unique. State number two, three, four, five, six. So what you see in front of you is a large DFA, a one large DFA that recognizes three tokens together, right? Now I have a large DFA that recognizes three tokens. How do we implement this? We know how to implement a DFA using either table driven approach or direct implementation. Let's assume that we are going to

take the route of implementing the DFA using a transition table. So let's represent all the transitions present in the DFA as a transition table. The first row correspond to first state. Second row correspond to second state, third row correspond to third state and so on. And the inputs are actually labeled as digit alpha slash backslash n, right? When we mark a column as digit, this is indeed a compressed form of writing 10 columns, one column labeled zero, one column labeled one, one column labeled two. So all the digits and we need to replicate everything in digits into all 10 columns. So it's a compressed way of looking at it, right? So this is a compressed way of looking at one column for every alphabet, compress it to one column, right? Come back. From state one, if we see digit, go to state number two. If we see alphabet alpha, go to state number six. If we see a backslash, go to state number three. If we see backslash n from the starting state, it is not handled yet, leave that entry blank. So now look at every edge in the DFA and then fill the corresponding entry in the table faithfully. Once after you complete that, you will see some of the entries in the table are still empty. For example, what happens when you are at the starting state? In other words, you're starting the program and this person keeps entering new line many times before he types anything. So from the start state, he keeps entering new line. For now, our limited setup does not really handle it, right? So in the starting state, the only characters we accept are digit, alphabet, backslash. Anything else is not handled. You can actually throw an error saying that unexpected character, right? So that is not handled. Now assume that you are in state number two, you already recognized a digit and now you see an alphabet, something starts with digit followed by alphabet, that's an error, right? So these entries which are missing are not handled explicitly in the DFA that you see in the screen. So what we can do is, we can transition the system to a particular state called as an error state, create a new state called as a zero state and then on those empty cells, in other words, from state one, seeing backslash n, go to state zero. From state two, seeing alpha, go to state zero. So all these blanks fill with zero, cases which are not handled, indicating probably an error condition, maybe as a language designer, you little closer, you might want to replace this zero with something else. You might want to handle certain cases, but for now, this is not handled in our DFA, right? So assume that your state zero is a dead state or an error state. DFA completes the two-dimensional table or matrix that represents the transitions in the DFA. All right? So we have a matrix that represents all the transitions in the DFA. How do we implement the DFA from now on? If you have a transition matrix or this table is stored in transition matrix XY, we can now use a table-driven lexical analyzer, MOTLY, to implement this lexical analyzer. So the routine for recognizing tokens using table-driven DFA implementation is a fixed routine, right? So we can start using that to implement this DFA. So we've come to the end of this video. You learned how for a toy programming language, we go through each step in the design of lexical analyzer and design the lexical analyzer and implement the lexical analyzer. The only thing that is left is to convert the pseudocode into a program. I'm sure you are smart enough. I hope you understood how to design a lexical analyzer by hand. Thank you.

Recognising Longest Match Video• . Duration: 9 minutes

[Notes]

Dear students, welcome to the video. In this video, we are going to learn about an important aspect in the design and use of a lexical analyzer. We are going to learn about Longest Match and its possible use in the use of lexical analyzers. Let's get started. To illustrate, we are assuming a simple

case in which we have to recognize two tokens. One is a keyword if, the other one is identifier. The keyword if is recognized by recognizing i and f and identifier is recognized by recognizing a sequence of one or more characters in which the first character is an alphabet. So let's draw the combined DFA directly because we have only two cases and let's attempt, let's see what's going to happen. To recognize if, starting from start state, if we see i in the input, we take the system into state i and from i, if you see an input f, we will take the system into state if. The state if is a final state. Now let's try to extend this DFA so that it can also recognize identifier. So from start state, if the input is alphabet minus i, what does it mean? If the input along this edge is any alphabet minus i, from zero, if you see i, you go to recognize if. If you see anything other than i, you go to recognize identifier, okay? Alpha minus i. So if you see alpha minus i, you proceed to recognize identifier. So we come to a state called as id, where we recognize identifier and if you see any alphabet after being in state id, you stay in state id. You can see that if and id are final states. Now for the input like in, we start from state zero, consuming i, come to state i and the moment we see n, we realize that we are not probably recognizing if, i in is also an identifier. Right? So if you see from i, anything other than f, you're probably recognizing identifier. Correct? So anything other than f from i, you're probably recognizing identifier. Assume you have an input i, f, b. After seeing i and f, you would reach the state if. From state if, if you see a b, probably you're recognizing something more than if, right? So take the system into state id. So now what you have is a DFA that recognizes the keyword if and identifier. You can pause the video and convince yourself that the DFA recognizes both the keyword if and identifier. Let's now consider an input i, f, g, a. And let's see how this DFA recognizes i, f, g, a. The DFA starts in the starting state, it accepts i and go to state i, it accepts f and go to state i, f. By now you've reached i, f, i, f is a final state. So after reading this much of input, it is possible that your lexical analyzer gives or announces the word that I've recognized token key if. I've recognized the keyword if. Then take the system back to state 0 and start recognizing g, a as t, k, i, d. This is not the right thing to do. The right thing to do is to look for the longest match possible, right? So i, f, g, a. After seeing i, f, we are in state if and after seeing g, we go to state i, d and after seeing a, we go to state i, d and we must return the entire token to belong to a token category i, d, right? So this i, f, g, a must be recognized as i, d. This i, d correspond to a longest match possible for this. It is not right to return for i, f, g, a that have recognized a token keyword if and then I recognized i, d as two tokens, right? Now let's look at another case. Assume that we have a lexical analyzer that recognizes integer and real. Integer is a sequence of digits and real is a sequence of digits followed by a dot followed by sequence of digits. So from state zero, if you see a digit, take the system into a state called int, signifying integer. In integer, as long as you keep seeing digits, stay in the integer state, but the moment you see a dot, you're probably trying to recognize a real number. So take the system into state one, if you see a dot and then if you see a digit, a sequence of digits, one or more digits, then recognize real. Now let's consider an input, an input nine, nine, nine, nine, dot. How do we recognize this number nine, nine, nine, four, nine, four times nine and a dot. So when we see the first nine, we will come to the state int because nine is a digit and we will see next three nines and we'll stay in the state int. So after completing first four digits, we will be in state int and now a dot comes in. When a dot comes in, right, the system comes to state one and the input ends and we are not in an accepting state, it's a state which is not an accepting state. It means that four nine followed by a dot is not a valid input, right? It's an error. In such case, we have two approaches. We can see that the entire string is an error string or invalid string. The other option is we backtrack to the most recent point where we recognize something correctly and we recognize string up to that particular point and then say a dot is unexpected. There are two ways of handling

this. As per this particular way of handling, we, after looking at four nines, we will announce to the world that we've recognized integer and then we are going to give an error message on the screen that are unexpected data, right? This looks to be a little more meaningful. So this is how we have to make decisions while we design lexical analyzer for programming languages. Longest match is an important case that you must consider while designing lexical analyzer. Next aspect that we want to consider in the design of lexical analyzer is how do we handle the cases that leads to error. It's always good to backtrack to the most recent point where I've recognized something and then go back into the string and recognize up to that and then say the rest of string is not expected. So it depends how the language is being designed and it's important as a designer, you must know these things. We've come to the end of this video. We've learned that a lexical analyzer works to find the longest match in the input for tokens. As we look for a match, we always remember the last match token and the position in the text so that when there is an error, we can backtrack to that particular point. We can announce to the world that we recognize something and certain other things are that comes after the last recognized point is unexpected. In case of reaching dead state or error state, we can always go back to the most recently recognized token so that your lexical analyzer will not end or stay in an error state. Thank you.

Recording of Formal Languages and Applications Week 3 - Live Session on 19-06-05 Video• . Duration: 47 minutes

[Notes]

Hello. Good, good evening, everyone. Welcome to Live Session 3, of course, our subject, formal languages and applications. Today, we'll talk about more from implementation perspective. So far we have learned about alphabet, strings, languages, grammars and finite automata, deterministic finite automata to be very specific. We'll see what is the application of these things. That's the topic for today. We'll also talk about lexical analyzer, what it is and how do we build it. It is the first part of your compiler, designing a compiler for programming languages. Lexical analyzer is the first part. We'll talk about it, but I think there's a reminder for all of you you have quiz coming up. You have your first grade quiz coming up. Please complete it as soon as possible. It's going to close automatically on the due date, we won't be able to reopen it. Check the dates, tick the timing, and tick the test. This will carry marks for your final evaluations. That's why. Please take the quiz. Let's talk about the Week 3, but before that, this is me. This with me, I have Shared Shubha, Devanshu Chirag we are here to help you in your journey, journey of learning formal languages. If you have any content related questions, assignment questions, technical issues, please use the discussion forum, and we will be more than happy to take your questions there. Just request do not keep questions to you. Learning anything, becomes fun and interesting when you ask questions and your questions are getting answered, and that's why we have set up these live sessions. You can ask us during this session using discussion forum, and we'll do our best to address your queries. With that, let's move on to today's session. Today, we'll talk about pattern recognition and how these influences our work that we do. But before that, I'll just show you a couple of examples. Let's say you type search in in Google search. What happens? It'll start giving you suggestions as sachin tendulkar. Of course, you might work with multiple other search in names, but Google will try to immediately recognize the word, and the search term that we give the word that we give and it'll start giving us suggestions. Isn't it? Now, on the right side, you see, that's a screenshot I have taken

from Jupyter notebook. I'm trying to write a Python code here. What happened if you see here? The colors that you see four is green, but bold, I is black. Again, N is bold green. Range is in green, print is in green. Hello is in red. How does this color scheme works? How is it able to say that four should be dark green? Hello should be red? What does this indicate? In a sense, the programming language in this case or even the editor in this case, Jupyter note book is able to recognize certain terms, and we'll talk about those terms in some time, but it is able to recognize these certain terms and based on that, it changes the output, isn't it? The question here is, how does the computer know what we are going to type or what we have typed, how does it know? The answer is, before any intelligence happens, even before Google search happens, even before your AI ChatGPT response, even before the program runs the code, a simple machine first recognizes patterns, and that simple machine is called as DFA or deterministic finite automata. Whether you talk about face unlocking system of your phone or OTP validation, email validation, ATM pin, all these pattern making is handled by your DFA. This is what we saw a little bit last week as well, isn't it? Now if you talk about specifically about AI or let's say, we can take ChatGPT as an example. Let me ask you a question. Do you think that AI can understand the meaning of any word or sentence directly? If I say what is AI question mark? Does it know what we are talking about? No. It doesn't understand the entire word or sentence. What exactly happens is your ChatGPT when it receives texts like this. What is AI question mark? It first converts it into tokens, and token could be like, what is one token. This is one, AI is another token. Question mark is another token. You have something called a tokenizer, which breaks text into smaller pieces, which are called as tokens. Tokens are nothing but the smallest unit of the input, is your token. The entire process of breaking this sentence into token is called as tokenization and tokenization actually uses lexical analysis concept. It uses DFA. You can imagine that without tokens, wouldn't have any AI system, isn't it? No ChatGPT, no LLM, so let me ask you a question. What do you think is easier for a computer? A full sentence or smaller pieces of tokens? What do you think is important for AI systems? It is just programmed to use logic. It is just programmed. When you say program, what does that program mean? Abigit? When you say it's just programmed, what do you mean by program? Program is not one single word. It's not one single concept. It's not a one single process. Program goes through multiple phases. For example, anything. Let's say if I ask you any tax that you do before doing any tax, first, you need to understand what you're supposed to do, isn't it? Even before we write the program, you need to understand what it is, and that's where token and keywords come into picture. First, you need to understand. Then you need to know what tool I need to use. Let's say in this case, you're talking about, say syntax evaluation. But the first step is understanding the input itself, and that's where what we are doing today actually comes into picture. Let's take another example. Let's take a traffic signal example. Now, in a traffic signal, the light that you have traffic light has three states, isn't it? Red, green and yellow. From red, it goes to green. From green, it goes to yellow. From yellow, it goes to red, and this process continues, isn't it? Here, when the traffic light is red, it knows that when time comes, it will be time based event, isn't it? When time comes, it has only one state to go, that is green state. The step is determined. To be state or the next state is determined from red. If you know the current state, you can predict the next state. That's why it's called deterministic finite automata. You can easily determine what would be the next step. If the current state is red, next step would be green. If current state is yellow, next step is red. You can implement that using C program, something like this, what I've shown here. If state equal to red, state equal to green. That means the new state value will be no green. Now, this is a simple one step implementation of DFA. But this is for small system. Think of vending machine. Something that we discussed last week, if

you remember. In vending machine, if I have to buy a chocolate worth 30 rupees, then the machine has to understand the current state. There you start current state with zero and from zero, you can have multiple input, but you know that you will enter into a definite state. If I enter a five rupee node, then the next state would become five rupees. Then if you enter 10 rupee node, it becomes 15 rupees. Again, you enter, let's say another 10 rupee, it becomes 25. Again, you can implement that using EFLs conditions. Something like this. What we just discussed, this is a simple example where DFA would be something like this. Your current state is red, and after the timer expires, the next step would go into green, and then if the timer expires, next step is yellow. This is simple. But when you talk about let's say junction has multiple roads, there are let's say six or eight such roads. Same concept, but on a larger scale. You might have signal controlled by pedestrian also. Most of the cities in India also has it and outside developed country has that where pedestrian, so you have this continuous red green and yellow. But if a pedestrian come, pedestrian can also press a light, and the light would then turn red for the traffic. Pedestrian light will be green and pedestrians when crosses, maybe they'll have 20 seconds or something at the time or after 20 seconds, automatically the red for the traffic will turn into green. You have multiple search conditions which we'll see in the next step, but what you see here is a simple implementation of our traffic signal, isn't it? This is part of DFA. This is DFA where you have current state red, goes to green, green to yellow. Now, this can become more complex. You might have now 100 intersections. There are three state, of course, red to green to yellow, and then again to red. But then would be different timings. Some intersection which are more busy might have longer wait time intersection which are less busy will have a smaller intersection. People crossing the road will have again, different timer. What happens in this case? Instead of writing 100 EFLs, we'll look for easier option and that's where your transition state becomes helpful. Instead of writing 100 EFLs or in this case, probably you might end up writing 100-5,200 EFLs. Instead, if we can have some kind of table where we define the current state and the future state and the system would read from the table and perform the desired output. If you have to write 100 conditions, you'll have at least 200 sentence lines of code, isn't it, or even more. But if I can create some kind of table. Let's see, let's look at a table here. It simplifies the entire effort. What it says is, if your current state is red, and then if timer expires, input is timer expiring. Timer expires. Let's say you have set 40 seconds for a signal. Every 40 second, the state will change. I freeze red, it goes to green. If it's green, it goes to yellow. Your current state is red, green, yellow, this is the action that you take on this state, and this is the next state. Now here, you might have some emergency vehicle. Ambulance, for example. Even though you may be in a red state, from red, you have to go to yellow, then to green. But let's say if emergency is there, and then what we need to do is you need to immediately open the road for ambulance. No matter which state you are in, the moment you get E emergency vehicle, you should go into green state. That's what E indicates. If you're in green, you continue to be in green. If you are in red, you get into green. If you're in yellow, you get into green. Even if you're in red, you don't have to get into your regular yellow to green. Directly it goes into green. That's another input to your current state. Third input could be pedestrian requesting to cross the road. If it is in green state, vehicle is moving, I want to cross the road, it should turn into red. That is your current state is red, then you continue to be in red. If your current state is green, then you become yellow warning for the vehicle that your light is going to turn into red, so be careful, and then it turns into red. It depends upon the input, and input decides the future state. Now, same thing happens with our programming world as well. This is the same slide which I had used the last week. And here you see you have alphabet, 01, and then this alphabet using that, you're going to generate strings. Strings are generated using your DFA. This is

state diagram, and the circles are your state. Simple circle is your state. Then the circle with arrow coming towards it, indicates the starting state. The state, which has double circle indicates final state, end of the state. Then you can see the input and this black arrow indicates your next state. If my current state is q_0 and you bring input as zero. It's like if you are in red, that may not a good example. Let's say you are in green, and then the input is zero, input is, let's say, emergency vehicle. Then you continue to be in green. Think of something like that. Q_0 , if zero is the input, you continue to be in your same state. Something like the last example that we saw. If you're in green and you have a emergency vehicle, you continue to be in green state. What it says is if you apply one, then you go to q_n state, and that is a final state. It's like if you are in green, and then timer expires, one is like a timer expiring, then you end up getting into red state. Based on these DFA water, valid input, but before we talk about valid input, valid string, you look at the table. Same diagram can be represented into a transition table. You have current state, input variable, and you have the future state. Now you can see these are your valid input. If your input is one, that means you are in zero and you get input one. What it says is end of the string, where are you? Are you in final state? If you're in final state, then you'll accept that string. If you're not in final state, then you will not accept the string. If I have just one, so q_01 , you will end into q_1 and q_1 is a final state, that means it's a valid string. You end up getting into a valid state. If you look at this Epsilon, Epsilon indicates that empty string. Empty string means no action. No action means you start from q_0 , you stay q_0 . Since q_0 is not the final state, Epsilon doesn't become a valid string. That's why you will reject Epsilon. You see, it says you are in q_0 state. It's rejected because for string to be accepted, you have to end up in q_1 state. Moving on. Sorry, I got cold. This is how it indicates your DFA, and this is how it indicates, how do you know what's the valid string or not? Now, what is the application of it, isn't it? Where do we use it? That's where things starts becoming interesting. Now we're introducing another concept called as lexical analyzer. What does lexical analyzer mean? Now, let's say, I get a message, something like this on my Whatsapp. Let's say I get a message saying that, hi Sop Nil meeting at 10:00 AM, tomorrow. What does it mean? Now if I have to read this message, if I have to interpret this message, I'll say, hi, so it's a greeting, saying hi. If I have to respond, I'll say hello, you respond back to the greeting. Hello. Sop Nil. By looking at it, it knows that somebody is trying to reach out to me. If I see somebody's else's name, then probably it could be a wrong message. But my name is there, so definitely it's not trying to read somebody else, isn't it? Name, a message for me. Then you have time 10:00 AM. That's the time. There's something that has to happen at a particular time. Then tomorrow, date reference. Tomorrow, I have to meet. When we see a message, this is how we try to interpret. We break information into meaningful pieces, isn't it? Computers do exactly the same with source code. When we type any code, exactly this is what computers do. The component that performs, this breaking of task into meaningful, like the meaning out of a code is called lexical analyzer. The come and performs this task. This task means, generating the meaning out of the code, it's called lexical analyzer. So what is lexical analyzer, simple definition? Is the first phase. That means there are definitely multiple phases that a compiler would use to interpret our code. The very first case here you can see the compiler structure. The very first stage is lexical analyzer. It is the first phase of a compiler that reads source code character by character and converts it into meaningful units called tokens. Right? And then, of course, you have other stages like syntax analyzer, semantic analyzer, code generator, and then finally, you have the .exe or class file in case of Java. The machine code. But the first step is where you convert the input characters or text into tokens. So for example, if you see this here, `int salary equal to 50,000;`. It recognizes IT as a keyword. It knows, int and there will be list of keywords already saved,

and it will match. That's why what we say is do not use keywords as variable names or do not use keywords as identifier. Because those have predefined meaning in it, Python or any programming language, for that matter, knows that it's a keyword and it has to perform a certain task. You see, it's a keyword. Then comes your next word, which is salary. Salary is treated as an identifier. Then you have equal to. Equal to is understood as an assignment operator. All the operator would have this red color. Let's assume that we are using a certain editor to type this program, equal to, greater than, less than, not equal to, all of them will be similar colors. All the keyword will be similar color. All the identifiers will be the similar color. Let's say, not dark, I think, dark blue, I think, maybe, or black or dark blue, whatever it is. Same color will be your identifier. Blue color is a number, it will remain as blue color. Then you have a semicolon indicating end of the line. Each component of your code is broken down, understood by the computer using lexical analyzer. Let's say your input is totally equal to amount plus 100. Step 1 reads each character. Space indicates end of the character. You read the character, recognize it. Oh, it's an identifier. Then continues to read until next space. It says, this is an operator. Then continues to read another identifier. If I had put in, then it'd read it as a keyword. But of course, now, is this a valid? We are not yet done. Lexical analyzer will not tell you if your code is valid or not. The syntax is checked in the second stage. Syntax is checked in the second stage, not in the first phase. First phase, we are still trying to learn how to read these values. You read it like a identifier, then you go to plus. That's again an operator. That's where you see total amount of same color, plus equal to the same color. Then finally, recognize a number. This is not a variable, not an identifier, but a constant number. It recognizes as a number and then semicolon indicates end of this line. These are different steps that lexical analyzer uses to read and understand what you are typing. It's like let's say when two people are talking and here you have machine and programmer trying to talk, isn't it. Now, let's say if we both are talking, I make a sentence. You try to break that into each word and try to understand the meaning, isn't it? If my grammar is not correct, if I don't use right word, then you'll see what is this trying to say, why I'm not able to understand. But we're not yet gone there to the understanding part. We are still at the reading the text part. Whatever word I say, you will try to hear and try to understand the meaning of each word. We're not even talking about the context yet. We're not talking about the meaning yet. When we give this input to the machine, to the compiler, the output is like identifier, assignment operator, another identifier, then you have plus operator, then you have a number, then you have a semicolon. Lexical analyzer reads the source code character by character and groups them into meaningful tokens. Next step is when you pass these tokens to syntax analyzer for further processing. That's where you'll try to understand the meaning of this. What is a lexical analyzer? Think of it as something that reads the characters, converts into words. Then understand the sentence. Now, each of these are done by different part of your compiler. Source code is we characters that you write. Lexical analyzer reads each word and tries to understand the meaning of it not trying to understand, try to read each word. Syntax analyzer will check the grammar or what do you call a syntax. Does that sentence make sense or not? Here you're only reading each part if count four, identifier, but next step, you're trying to see, do we have two identifier together, like total space count? No, it is not valid sentence. If you have an identifier, it should be assignment. Those are the rules, the grammar, the syntax. That is handled by syntax analyzer. Syntax analyzer is meaning, and then you're looking at the context, semantic analyzer. Does it make sense here? I'm saying total equal to count plus 100. Does it make sense? Perfect sense. But then you understand that count is not yet declared. We don't have the value of count. Semantic becomes problem. We are not able to find syntax is correct, but semantics is wrong. If semantics is also correct, finally, code is generated machine. This code is your machine

level code, your class file or .exe file. This is the structure of compiler and we are at the first stage which is lexical analyzer. Compiler takes a source code and transform it step by step from characters to action. What are different tokens? Tokens are your smallest meaningful units just a repetition of what we have seen. It is a keyword, while is a keyword, salary is identifier, 50,000 number plus is an operator, and semicolon is end of the line or delimiter. We call it as delimiter. This is how lexical analyzer will read each token and then group them into types of tokens. The code is then grouped as each token. Each word, each character is read and grouped as tokens. This is where DFA comes into pictures. Remember, DFA is, again, towards current state and future state. You know your current state, current character, and then you know the next upcoming character, and then you know if it makes sense or not. If you reach your empty space, that's when you know that you have reached your final stage or ending stage. If you look at count here, for example, let's say this is a rule, letter means it has to start with letter, then we're saying letter or digit. First, it has to be letter, then it can be letter or digit. To the power star means any repetition. That means first character has to be a letter, followed by whatever letter or digit. Again, it does not talk about space, comma, full stop, hyphen. You do not have any other special character. Count, of course, is a valid identifier. Salary1 is a valid identifier, student123 is valid. 1count is not valid because starting has to be a letter. At the right, salary not valid because first is anyway letter, and at the right is also not there here. DFA checks whether the input follows the rule or not. If your DFA accepts the order, that means you are good to read it as whatever, identifier or keyword or value or delimiter. This is very specifically for identifier rule. That's what it says, identifier rule. Identifier rule is compulsory a letter, and the bracket letter or digit, pipe to stand for or. Star means any repetition. It could be a Epsilon. Remember, a star accepts Epsilon also. That's what we did. That means it could be a single star. Single letter a, b is also a valid identifier. You have this starting state is q0. Letter, you go to q1 and that's when q1 is your ending state. You can't have Epsilon here because you need to have at least one letter. That's what it says. Then you go to q1, and q1, a letter or digit becomes accepting state. If qd is your dead state, dead state means from there, no way you can reach to the final state. For example, if I say ab at the rate cd123, it will send it to q1, b. Moment you get another red sign, you are sent here. In qd state, there is no way you can go to q1. If you're in q0, you can go to q1. But if you end up in qd state, no way you can go to the ending state, final. You are stuck there. The moment you get even one special character, special characters are not allowed, you are stuck and you cannot move. That's why you see from qd, there is no arrow to any other state. This is called your dead state. That means once you reach here, you cannot go anywhere. Once you get a special character which is not as part of letter or digit, then after that, whatever it is doesn't matter, you are stuck. This is your qd state. So q0 is starting state, q1 is the ending state, and qd is dead state because you cannot come out of this state. If you can't come out from certain state, and that is not the final state, that is called dead state. In this case, let's say zero was not this, zero was on this itself. For example, let's say. If zero is on this, zero or one, both are in q, and then q1 is also stuck. Once you reach q1, you will remain in q1 state. But q1 is a ending state, that means accepting state, then that is not dead state. The last concept is that lexical analyzer goes for longest valid token. For example, let's say you have int as a keyword. Let's assume introduction is also a valid keyword. If you get int, of course, it accepts. If you get introduction, which also has int in it, but since it goes to the maximum, bigger to this thing, then it will look at introduction as a valid string, not int. Similarly, let's say int is keyword, but let's say I use int123 as a identifier, it's valid because as I said, it goes for the longest valid token. Int is a valid token, and the token type is keyword. Int123 is also a valid token. Of course, it's not a keyword. It becomes an identifier in this case. That's why when you write

int123, it does not say that I found a keyword and it's wrong. It says, it's not a keyword, but it's a identifier. It always goes for the longest valid token, remember this. That's why if you say greater than or equal to, it will treat greater than or equal to as one single token, not as a two different token. Remember, lexical analyzer designs such a way that it goes for the longest valid token. With that, we end our discussion on lexical analyzer. Lexical analyzer is a pattern-recognition engine that converts raw characters into meaningful tokens. It is the first stage of every compiler and is built using concepts such as Regular Expressions, DFA, Transition Tables, and Longest Match Rules. By now, by end of Week 3, you should be comfortable with these terms: Lexical Analyzer, DFA, Regular Expressions, Transition Tables, and Longest Match Rules. With that, we come to end of this session. You have a quiz. Again, this is not a graded quiz, it is just for fun. I would say just go connect to the website, take the test. You don't have to put your name as well. The first homepage will ask you to put your name. You can choose not to put your name, just play this quiz. Before we end today's session, let me know if you have any questions. There are 10 questions on the quiz and those are easy, as I said. If you have listened to the session carefully, you will be able to understand and get this clearly. Any questions before we end the day? Of course, try to play this quiz. This will reiterate what we have seen today. Any questions on any topic, Week 1, 2, 3? For questions, I see a chat. No questions, sir. The session was straight on point. I'll definitely go through the quiz after the class. You can do the quiz now. It shouldn't take more than five minutes. But sure, I will keep that quiz open for a few minutes. Maybe till eight o'clock, I'll keep it open. Sure. Thanks for the visit. That is my intention. I try to make each class interesting. But beautiful, good that you found it interesting. Okay, guys, if there are no questions, then we can end it here. Please be ready for the graded quiz, online test. Don't forget to take it. It's easy and it'll add weightage to your final grade. If there are no question, then we'll end the day here. I'll wait for two more minutes before we end the day. Thank you so much for your time today. Again, take the quiz and I'll see you next week. Next week, again, we'll come up with something interesting. Thanks, Larry. Thank you. Good night, everyone. See you next week. Bye.

Recommended Reading: Direct Implementation of a Simple DFA

Reading• . Duration: 15 minutes

[Notes]

Brief: A DFA processes an input string and determines whether it belongs to a certain language by transitioning between states based on the input symbols. In a direct implementation, the DFA is represented explicitly using conditional statements in code to define transitions between states. This approach is suitable for smaller and less complex automata. **State transitions:** Defined explicitly using conditionals, mapping the current state and input symbol to the next state. **Final state:** A DFA accepts a string if, after processing all the input symbols, it ends in an accepting (final) state.

Efficiency: Direct implementation is straightforward but can become difficult to manage as the number of states and transitions increases. `if (state == 0 && input == '0') { } else if (state == 0 && input == '1') { } else if (state == 1 && input == '0') { } else if (state == 1 && input == '1') {` **Reference:** Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press, Section 2.3. **Practice Lab 1: Direct Implementation of simple DFA in C** In the attached file, you'll find a basic C program (.c file) that provides foundational code for this practice exercise. As you work through it, feel free to modify or enhance the code to deepen your

understanding and improve your skills.

Recommended Reading: Table Driven Implementation Reading• . Duration: 15 minutes

[Notes]

Brief: A table-driven implementation stores the transition function of a DFA in a two-dimensional array, where each state and input pair maps to the next state. This method is more scalable than direct implementation and is easier to modify, especially when dealing with large automata.

Transition table: Represents the state transitions as a 2D array where rows represent current states and columns represent input symbols. **Scalability:** The table-driven approach allows for easier handling of complex DFAs with many states and transitions, reducing the amount of hard-coded logic. **Code efficiency:** It simplifies code maintenance and is often preferred when DFAs have multiple states and inputs. `int transitionTable[2][2] = {{0, 1}, {0, 1}}; // DFA transition table state = transitionTable[state][input - '0'];` **Reference:** Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press, Section 2.3. **Practice Lab 2: Table Driven Implementation of simple DFA in C** In the attached file, you'll find a basic C program (.c file) that provides foundational code for this practice exercise. As you work through it, feel free to modify or enhance the code to deepen your understanding and improve your skills.

Recommended Reading: Direct vs Table Driven Implementation Reading• . Duration: 15 minutes

[Notes]

Brief: Direct and table-driven implementations of DFAs both serve the same purpose but differ in their scalability and ease of maintenance. Direct implementation is suitable for smaller DFAs, while table-driven implementation is more efficient for handling large state machines. **Direct**

Implementation: Hardcodes transitions using conditionals, making it less flexible as the number of states grows. **Table-Driven Implementation:** Uses a matrix of state transitions, making it more scalable and easier to modify for complex automata. **Choosing between implementations:** The choice depends on the size and complexity of the DFA being implemented. **Example:** In a scenario where a DFA has more than 10 states and 2 input symbols, a table-driven implementation is more efficient because adding new states would only require adding rows to the table, whereas direct implementation would require adding multiple conditionals. **Reference:** Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press, Section 2.3.

Recommended Reading: Structure of a Compiler Reading• . Duration: 15 minutes

[Notes]

Brief: A compiler is a program that translates high-level source code written in one programming language into a target program, typically in another language or machine code. This process

involves multiple stages, ensuring the program is syntactically correct and semantically valid. The lexical analyser is the first stage in this process, breaking the input program into tokens, which are meaningful sequences of characters like keywords, identifiers, and operators. The source program is the input to the compiler. The compiler processes the source code and generates a target program or machine code as output. The compilation process is divided into two main parts: This stage involves analysing the source code for syntactic and semantic correctness. Lexical Analyser: Breaks the input into tokens. Tokens represent keywords, operators, identifiers, and literals used in the source code. Parser: Converts tokens into a syntax tree. Semantic Analyser: Ensures the program has meaningful logic. Output: Intermediate Representation (IR) such as a syntax tree or intermediate code. This stage converts the intermediate representation into the final target program. Involves optimisation and code generation. Reference: Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press, Chapter 2, Section 2.1.

Recommended Reading: Working of a Lexical Analyser Reading• . Duration: 15 minutes

[Notes]

Brief: The lexical analyser reads the input program and generates a stream of tokens. It filters out whitespace and comments, as these are not needed for parsing. The main job of the lexical analyser is to simplify the input for the parser by breaking it into meaningful symbols. Tokenisation: The process of converting input characters into tokens. Common tokens include keywords, identifiers, operators, and symbols. Filtering: Removes unnecessary elements like comments and whitespace. Error handling: Detects illegal sequences of characters and reports lexical errors. Input: The source code is treated as a text stream. Output: A token stream is generated, containing valid tokens. Error Handling: If any invalid tokens or unclosed comments are found, the analyser reports an error. `main := fun (in x : integer) ret integer { TK_KEY_VAR, TK_IDENT (main), TK_COLON, TK_KEY_INTEGER, TK_IDENT (main), TK_ASSIGN, TK_KEY_FUN, TK_LPAREN, TK_KEY_IN, TK_IDENT (x), TK_COLON, TK_KEY_INTEGER, TK_RPAREN, TK_KEY_RET, TK_LBRAC, TK_KEY_RETURN, TK_IDENT (f), TK_LPAREN, TK_IDENT (x), TK_RPAREN, TK_SEMI, TK_RBRAC, TK_KEY_ENDMOD` Reference: Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press, Chapter 2, Section 2.1.

Recommended Reading: Approach to Design Lexical Analyser Reading• . Duration: 15 minutes

[Notes]

Brief: Designing a lexical analyser involves defining the tokens that the analyser should recognise, constructing regular expressions for each token type, and using a deterministic finite automaton (DFA) to identify these tokens in the input stream. Token definitions: Keywords, operators, identifiers, and literals are defined using regular expressions. Finite Automaton: A DFA is constructed to recognise the different tokens and return the corresponding token type for the input. Longest match principle: The lexical analyser is created by merging individual DFAs into one large DFA that can recognise the union of all defined tokens. The analyser selects the longest sequence

of characters that form a valid token. Example: The regular expression for an identifier in a C-like language is `[a-zA-Z_][a-zA-Z0-9_]*`, meaning it starts with a letter or underscore and can be followed by letters, digits, or underscores. Reference: Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press, Chapter 2, Section 2.1.

Recommended Reading: Lexical Analyser Design Example-1 Reading• .

Duration: 15 minutes

[Notes]

Brief: A lexical analyser is a critical component of a compiler that processes the source code to identify and categorise lexical elements such as numbers, identifiers, and comments. This reading walks through the design and construction of a lexical analyser using regular expressions (REs) and deterministic finite automata (DFA) for individual components. **Number:** A sequence of one or more digits. **Identifier:** A sequence of alphabetic characters or digits where the first character must be an alphabet. **Alphabets:** {a, b, c, ..., z, A, B, C, ..., Z}. **Example DFA:** Starts with an alphabet and can transition to further alphabets or digits. **Comment:** Begins with // and continues until the end of the line. **Example DFA:** Tracks transitions starting with // and ends upon encountering a newline character. **Combining DFAs:** Each DFA for numbers, identifiers, and comments is combined into one large DFA to handle multiple lexical elements within a single program. **Transition Matrix:** A matrix is created to define state transitions for each input symbol, including digits, alphabets, '/', and newline characters. **Dead State (State 0):** Indicates unhandled cases where no valid transition exists. This reading introduces the design process of a lexical analyser by constructing DFAs for different lexical elements and integrating them into one DFA. The transition table serves as a table-driven approach for efficiently recognising numbers, identifiers, and comments. This example demonstrates the practical application of automata theory in compiler design. Reference: Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press, Chapter 2, Section 2.1.

Recommended Reading: Recognising Longest Match Reading• .

Duration: 15 minutes

[Notes]

Brief: In lexical analysis, the longest match principle is crucial. When multiple tokens could be recognised from the input, the analyser selects the longest valid sequence of characters that match a token definition. **Longest Match Rule:** When multiple tokens are possible, the lexical analyser always selects the longest matching token. This ensures that identifiers, keywords, or numbers are recognised as complete units rather than partial matches. **Identifier:** Any sequence of letters and digits, starting with a letter. Although "IF" is a valid keyword, the analyser returns "IFGA" as a single identifier because it matches the longest possible token. **REAL:** Sequence of digits with a decimal point (e.g., "123.45"). The analyser treats "9999." as a REAL number because the longest match includes the decimal point, even though it ends with no digits after the point. If the analyser encounters a dead state or error, it backtracks to the most recent valid token and returns it. Reference: Andrew W. Appel, Modern Compiler Implementation in C, Cambridge University Press,

Section 2.3.

Practice Labs Reading• . Duration: 10 minutes

[Notes]

Practice Lab 3: Implementation of a Simple Lexical Analyzer In the attached files, you'll find a basic C program (.c file) that provide foundational code for this practice exercise and a PDF that provides the step by step guidance. To solve the assignment, please start by reviewing the associated tutorial PDF, then proceed to work on the code. As you practice, feel free to modify or improve the code to enhance your understanding and skills. Practice Lab 4: Extending the Implementation of a Simple Lexical Analyzer This practice builds on the concepts covered in Practice Lab 3 and is an extension to the same problem. Please ensure that you complete Practice Lab 3 before starting Practice Lab 4. In the attached files, you'll find a basic C program (.c file) that provide foundational code for this practice exercise and a PDF that provides the step by step guidance. To solve the assignment, please start by reviewing the associated tutorial PDF, then proceed to work on the code. As you practice, feel free to modify or improve the code to enhance your understanding and skills.

Review Questions & Practice Problems Reading• . Duration: 10 minutes

[Notes]

(No transcript found.)

Live Session Reading• . Duration: 10 minutes

[Notes]

The program team will be hosting a live session via Zoom. You can access the webinar from the Live Events tab in this course. This live session is designed as an interactive session for discussing the content covered in week 1 of the course. During this session, learners can clarify any doubts they may have on the covered content in the videos or discuss assessment and project-related queries, if any. NOTE: Ensure that you create a free Zoom account before you join the live session. Some features in Zoom are limited to registered account users.

Equivalent DFA's Video• . Duration: 6 minutes

[Notes]

Dear students, welcome to the video. We have learned that, if a DFA is given to you, the language that it accepts is unique. On the other hand, if I hand over a language to you, two of you can come up with two different DFA, yet these two DFA's accept the same language given to you. In this video, you are going to see an instance of this. Let's get started. You are seeing a DFA on the screen. This DFA, there are four states, Q0, Q1, Q2, Q3. Let's get an intuitive understanding of what language is accepted by this DFA. Right? Let's start with Q0 and right away, write down what is a possible language that it accepts. At Q0, we receive sequence of b's. In other words, we receive

zero or more b's, and then we receive an a, right? And we are coming to an accepting state, Q1. So b^*a is accepted. Further, zero or more b's also accepted. All right? So b^*a is accepted. In the state Q1, instead of seeing a b, if we see an a, then it goes to Q3, which takes b star and takes us into, takes a and goes into Q2, right? Q2, which takes b star, right? So just observe this, b^*a , right? b^*a , and if it takes an a, then b^*a , right? b^*a takes you to final state, correct? But if you see an a, it takes another b star, a b star to take to the final state. But now from Q2, if you see an a, then you take another b star, a b star to come to accepting state, all right? So what is accepted? So at this point, you are in the final state. At this point, you are in the final state, right? So b^*a , and if you see an a, another b star, a b star, and that's how the language goes. So now look at the DFA, a simple DFA, which has got two states, which is an incomplete DFA. Let's complete it. On a, it goes to Q1, and from Q1 to Q0, on a. Let's see what language does it accept, right? Look at the state Q0. It accepts or it receives 0 or more b, okay? And if it receives a, and it reaches Q1, an accepting state, at accepting state, it can receive 0 or more b's, okay? So it starts b^* , it accepts a b star, right? And if it sees an a, then b^*a is what it has to receive, right? If it sees an a at Q1, then it receives b star, a b star. It's easy to convince yourself that the language that DFA on the left hand side and the DFA on the right hand side accept are the same thing. I urge you to write down a simple form of the language that both DFA's accept. It's not a long string like this, b^*a , then a, b^*a , then a, b^*a , then a, b^*a , then a. How can I write it in a very simple manner? In other words, how can we write a regular expression that describes this language? Take it as an exercise, try this, right? So these two DFA's accept the same language. If you look at one more aspect about these two DFA's, the first DFA has got four states and the second DFA has got only two states. Theoretically speaking, there is no problem in having either the DFA on the left side or the DFA on the right hand side. But if we represent, if we try to implement the computation represented by the DFA, maybe a smaller DFA may allow you to write or have the entire code in the cache memory or have a very simple code or compact code that can be implemented very efficiently, right? So some applications having simple DFA makes a lot of difference, right? Now what is in front of you are another two DFA's, right? This is DFA one, let's say this is machine one and this is machine two. These two DFA's are taken from a textbook, right? The textbook for this course, written by Peter Litz. I want you to try understanding the language that M1 and M2 accepts and try to convince yourself that both M1 and M2 accept the same thing, same language. So it's actually possible that given a regular expression, you can have more than one DFA's that accepts a language and it's possible that one of this DFA's have the smallest number of states, right? We prefer smallest number of states when we want to go ahead implement this in computer. We've come to the end of this video. In this video, you've seen instances of more than one DFA's that accepts a single language and you also learned that we have a DFA that has got the least number of states which is a preferred DFA. Thank you.

Indistinguishable States Video • Duration: 11 minutes

[Notes]

Dear students, welcome to this video. Let us suppose we have a DFAEM and this DFAEM has a set of states q . We want to learn if two states in q are distinguishable or indistinguishable. We are going to use this particular aspect, distinguishability, to prove certain aspects about regular languages

later on. In this video, let's just learn when two states are distinguishable and when two states are indistinguishable. Let's get started. What you see is a DFA. This DFA has got four states. Let us now look at two states or let's now focus on two states. So you are given with a DFA. This DFA has got four states. And let's suppose you are given with two strings s_1 and s_2 . This is string s_1 and this is string s_2 . How do you process a string? You take the string as input and then you look at each character in the string from left to right and you will follow the DFA transitions. Now let's suppose you're processing string s_1 , processing character by character, character by character. At some point in the string, you have reached state q_0 . Okay, you reached state q_0 and you have not completely processed the string yet. Some part is left out. This is a part of the string which is left out, which is z . Let's say called left out part is z . Now let's keep that and let's come to string s_2 . You're processing string s_2 in a similar manner. You're processing string s_2 from the starting state q_0 , processing character by character. You come all the way at some point. Okay. After processing up to this particular point, the system take you to state q_3 and some part is left out and let us suppose this part is same as your z . So from q_3 , a part of string is left out, that is z and from q_0 , some part of string is left out. That's also z . How do you understand this? You are given with string s_1 and s_2 in which the suffix is z . Let us now assume a particular form for the suffix. First, let's assume your suffix is abb . It means it's abb in both places. Now tell me from where will you go from state q_0 on receiving abb . Right? We are not talking about the first string. You are in q_0 . If you see abb , right, from q_0 , you will come to q_1 on a and after seeing two b s, you would still be in q_1 and you will accept this string. Now come back to the second string s_2 . So you are in state q_3 and you are seeing the same abb . The same abb will take you to state q_2 , right? You will follow the edge once on a and you will follow this edge twice on b and you would still be in q_2 . If it is in q_2 , what will you do? Accept it. So what happened is, if you are in state q_0 or in q_3 , the same suffix makes you to accept in both cases. Now let's take another example. Look at the suffix b , not abb , but it is slightly different, okay? It is bbb . So here also bbb . Now the first suffix. So if you see bbb from q_0 , you would still stay in q_0 for string 1. Now if you just look at string 2 from q_3 , if you see bbb , you would still stay in q_3 . So if you see the suffix z , in both cases you would reject, right? You would not accept it, right? This string is not accepted. So what is that you infer from this small example in discussion? If you look at state q_0 and q_3 , right? The moment you reached q_0 and q_3 , respective of, let's draw it again, q_0 , q_3 . Let us suppose you reach q_0 , q_3 somewhere. The moment you reach q_0 and q_3 , if you see same suffix after q_0 and q_3 , either they both will get accepted or they both will get rejected, right? Do they behave in a similar way? The first case, they both accepted the z , they both accept. For another particular z , they both reject, right? The moment you come to q_0 and q_3 , from then on, if you give the same input to q_0 or q_3 , the behavior is same, right? So you can observe that q_0 and q_3 behaves in a very similar manner, right? Keep this in mind. Let's go to the definition. So let's look at the definition. Two states p and q of a DFA are called as indistinguishable, they are one and the same thing. Now, from p , if you see w , it takes you to a set of states f , implies that from q also, if you see the same string as input, you should also go to the same set of states f , where f is a set of final states, all right? And for certain strings, from p , for certain string w , if you reach a non-final state, then from q also, you should reach to the non-final state for whatever string w . If it behaves in this particular manner, then the state p and q are indistinguishable, right? In other words, p and q , the behavior from p and q are the same thing on a particular input or any input w . Now, assume that you have two states p and q . So here, this is your p state and this is your q state. If these two states are indistinguishable, then the states p and q are equivalent. In other words, if string w from p takes you to final state, then from q also, it'll take you to final state. If from p , w takes you to a non-final state,

then from q, w takes you to non-final state. So these two states are equivalent. Now, the indistinguishability has the properties of equivalence relation. Now assume that a set of states in a DFA are indistinguishable. You found that the DFA has got p, q, r. These three states behave in a very similar manner, right? They're indistinguishable, right? Then this equivalence class satisfies or this indistinguishability satisfies the properties of equivalence relation, right? Now you can see that p is related to p, right? And you can see that if p is related to q and implies that q is related to p, right? And you also can see that if p related to q and q related to r implies that p related to r. So what are those properties that we have mentioned here? One is reflexive, second one is symmetric, third one is transitive. You take a state p and the same state p, and you can see that these two states are indistinguishable, right? Property reflexive. Second one is symmetric. If you take two states p and q, then if p is indistinguishable to q, then q is also indistinguishable to p, right? If p are q, then q are p. And now you say that if p and q are indistinguishable, and q and r are indistinguishable. In other words, p and q behave the same way, and q and r behave the same way. You can also convince yourself that p and r behave the same way. So you can see that indistinguishability has the properties of an equivalence relation. So we've seen what is indistinguishable, one and the same thing, equivalent and so on. Now let's see what are distinguishable states. Let us suppose you have two states p and q, and you're applying the same string w from p and q. Now, from p, seeing w, assume the system takes you to a final state, and from q, the input doesn't take you to final state, right? If you can see one string, one w, such that the behavior of p and q are not in line with each other, then these two states are distinguishable states. In other words, these two are distinct states, right? Now look at this particular DFA. If you look at state q0 and q2, right, let's now assume from q0, you are seeing a, b, and from q2, you'll also see a, b, right? Assume this arc has got a, b, and these arcs should have a proper ending. Now from q0, if you see a, b, you will come to q1, you can look, you can take a pause and then verify this, which is a final state. And from q2, if you see a, b, you will reach q2, which is not a final state. This is sufficient to show that the state q0 and q2 are distinguishable states or two distinct states. We've come to the end of this video. I'm sure you understood the definition of distinguishability, when state x and y are distinguishable and when state x and y are indistinguishable, right? Thank you.

Approach to Simplifying DFA Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to reduce a given DFA to a minimal DFA. Reducing the given DFA means that reducing the number of states in the given DFA to a minimal number that's possible. To do this, we are going to use an approach that has got two steps. One is mark, the other one is reduce. And this mark inside, it uses the idea of indistinguishability. If two states are indistinguishable, we are going to merge them, right? So let's get started and try to understand this approach. We've learned that if two states p and q are indistinguishable, if DFA has same behavior from p and q, then the state p and q can be merged into one state. So the approach to simplify the DFA, which we call it as mark-reduce method in this course, internally it actually uses one mark algorithm, right? And then one reduce algorithm, it's a two-step algorithm. The first step, mark, works to identify all the pairs of indistinguishable states and mark them. Whichever states are indistinguishable, we need to find out. And then, once after marking all the indistinguishable states, what we need to do is to reduce all the indistinguishable

states into one state. For example, if state p and q are indistinguishable, combine them. x, y, z are indistinguishable, combine them, right? So the reduce step attempts to combine all the indistinguishable states. Assume that a DFA is given to you and assume that we identify using mark method that this state and this state are indistinguishable. Because the behavior of being in state q0 and behavior of being in state q3 are one and the same thing. And also, by some way, the algorithm figures out that state q1 and q2 are indistinguishable, right? By some way, it identifies that state q1 and q2 are indistinguishable. Can you pause the video and try to figure out if state q1 and q2 are indeed indistinguishable, right? How do you do that? Assume you are in state q1 and you are seeing an input BBB. You stay in accepting state and you are in q2 and you see BBB. And you will still be in accepting state. Now you see an input BBA. From q1, if you see BBBA, right, you will be in a non-final state. Similarly, from q2, if you see BBBA, you will be in a non-final state. And this is true for any string. If you see any string from q1, if that string takes you to final state, then from q2 also that string will take you to final state. And if a particular string from q1 takes you to a non-final state, from q2, the same string will take you to non-final state. So state q1 and q2 are also indistinguishable. The question is, how do we find that q0 and q3 are indistinguishable and q1 and q2 are indistinguishable? These are relatively smaller DFAs. You can identify those indistinguishable states by inspecting the DFA. So it's important that we have a method. The method mark that we are going to learn will mark all those pairs. Once those pairs are marked correctly, then we can merge those indistinguishable pairs. For example, if q1 and q2 are identified to be indistinguishable, merge them, right? If 0 and q3 are identified to be indistinguishable, merge them, right? This is the approach we are going to take, right? Two things we need to understand. One is how do we mark the indistinguishable pairs correctly? And then how do we combine them so that we get a final DFA? We've come to the end of this video. You understood how this mark and reduce methods are used to simplify a given DFA. The details of those mark and reduce, we'll learn in the next video. Thank you.

Mark in Mark-Reduce Method Video• . Duration: 13 minutes

[Notes]

Dear students, welcome to this video. Let us suppose you are given with a DFA and you want to minimize the DFA. And we've learned that we need to identify those states which are indistinguishable and then we need to combine those states and so on. The first step is to identify those states which are indistinguishable. The mark method is used to identify the indistinguishable states. In this video, we are going to learn about the mark method and we are going to learn this with an example. The first step in the mark method is to remove all the states which are inaccessible from the start state. Assume a simple DFA, right, a starting state, another state and another state and this is being the final state. Let's number the state. For example, this is state number 1, 2, 3. Now we can see that in this DFA, the state 3 can never be reached. From starting with state number 1, which is a start state, we can go up to 2. And how do we go into 3? So the state 3 can never be reached. If you see any such state in a DFA, you can right away remove them, right? In the second step and the third step which I am going to show, we are going to mark if a pair is distinguishable or distinct and we are going to pick those pairs which are not marked as distinct as indistinguishable, right? That's what the approach we are going to take now. Now assume you have a DFA with states q, okay? Let's say for example, a, b, c, d and a set of final states which is c, d. It's obvious to find out

that the state a and c, a is a non-final state, c is a final state. So they are obviously distinct, right? And b is a non-final state, so c is a final state. So b and c are distinct. So the very first pair of distinct state that you can identify right away from the DFA are those pair in which one of them is final state and the other one is a non-final state, right? So mark all these pairs as distinguishable, fine. Once after marking those pairs as distinguishable, let's look at the rest of pairs. You always consider a pair because we want to verify if a pair is distinguishable or not. To understand the third step, I meant to show the third step, but let me give you intuitively what this third step is before I showed you, right? Assume a pair is considered, okay? As state p, as state q, right? As state p and state q are considered. Now you're taking an input a on both state p and q, right? And on taking a, p goes to pa, q goes to qa. In the table that you made in already, you marked that pa and qa, that pair is already distinguishable, right? So you can now mark p and q are distinguishable because on the same input, it takes you to a pair that are distinguishable, right? That implies p and q are also distinguishable. Let me show you the steps. For all the pairs p and q, and for all inputs a and sigma, find out what happens if you see a on p and if you see a on q, right? So if you see a on p, you go to pa. If you see a on q, you go to qa. If pa, qa are already marked as distinguishable, then mark pq as distinguishable. Now the step 3.1 will be repeated until no more pair can be marked as distinguishable. Start with an example, right? We have to consider all pairs. So here there are four states, q0, q1, q2, q3. So we need to consider all pair of states. So how do we manage all the pairs? So we can consider a table, okay? And all those entries below the diagonal represents a pair. So if you look at this entry, this considers a pair q0, q1 or q0, q1, q1, q0, unordered pair, right? And this q0, q2, right? And this one q0, q3. So each entry represents a pair. Let's now begin. Those entry which are diagonal and above are marked as blanks because those are not necessary. Those are duplicate entries, right? So q0, q1 is this and q1, q0 is this. I just want one of them, right? Because we consider unordered pairs. The first step is mark all those pairs such that one element is final state, the other element is non-final state. So let's look at this, q1, q0, right? You're looking at q1, q0. Will you mark it or not? q1 and q0, these two are the states. So will you mark this entry or not? One of them is final, right? So q1 is final, q0 is not final. So this has to be marked, right? So I'm marking it. Now, the next one is q0, q2, right? So q0 is non-final, q2 is final. So this is also marked, right? And this entry, look at q2, q1. So this is q2, this is q1, right? We're looking at these two, q2 is final, q1 is final, both are final. We cannot mark, right? One of them is final, one of them is not final, we'll mark. So this is not marked, right? Similarly, look at all those pairs and make your marking. And you will see that only two places you will not be able to make any marks. One is q0, q3, right? So this is q0, this is q3. This pair, both of them are non-final, the other one, that case, both of them are final states. So there's no marking. Now, let's go to the next step. What we need to do is, we need to look at all those entries which are blank, where there's no marking. So this is a blank entry, and this is a blank entry, okay? Let's take the first blank entry, q2, q1, okay? So we will take q2, q1. What happens for q2 and q1 on input 0 and 1? Let's look at the first column. What happens if you see the input a and b, okay? From q2, if you see an a, you go to q0. From q1, if you see an a, you go to q3, right? So the pair q0, q3. From q1, q2, or q2, q1, if you see a, you would go to q0, q3 pair. Can you look at this table, q0, q3? Is it marked? q0, q3 is not marked, fine. Now let's look at what happens from q2, if you see a b, you go to q2. From q1, if you see a b, you stay in q1. So now look at, so q2, q1 pair is here. This is also not marked. So it doesn't change any marking. So this cell is considered now. Now let's actually go to the next empty cell, right? This one, q0, q3, okay? So q0, q3. We need to see what happens on input a and b. So q0, q3 on a. From q0 on a, we go to q1. From q3 on a, we go to q2. Now just look at, is q1, q2 marked? Answer is no, right? There is no change now. Now

from q_0 , we see a b, we are in q_0 . From q_3 , we're seeing a b, we are in q_3 . So q_0, q_3 , cell is empty. So there is no change in the marking status. Now, is there any other empty cells or is there any other pair which are yet to be considered? Answer is no. And the status is not going to change, right? Even if I repeat, I would still find this cell and this cell as unmarked one only. In other words, it means now that the pair q_1, q_2 , this one, corresponding to this, these two states are equivalent. And this one, q_0, q_3 , and these two states are equivalent. Now this is an equivalence class and this is an equivalence class. If you look at the original set of states, which is q_0, q_1, q_2, q_3 , the original set of states on relation indistinguishability partitions the original set into two subsets, right? The first subset consisting of q_1, q_2 , second subset consisting of q_0, q_3 , right, don't forget. Now, each subset is an equivalence class, right? So the original set is partitioned, each partition is an equivalence class. An equivalence class means that we can replace q_1, q_2 into one single state. In other words, we can combine q_1, q_2 and we can combine q_0, q_3 . So since there are no more markings, the mark method ends and these two cells indicate indistinguishable pairs and each indistinguishable set is an equivalence class and we divided or we partitioned a given state into a set of equivalence classes, right? So indistinguishability on the given DFA lead to partitioning the given states into a set of equivalence classes, right? The number of equivalence classes is the minimal number of states possible for the given DFA. In other words, the original DFA has got four states. The number of equivalence classes that we obtained are two, right? So this is the number of minimal states possible for the given DFA. So we've come to the end of this video. You learned what mark method is and you learned the working of mark method with an example. At the end of this video, you marked those states which are to be merged or reduced into one single state, right? Let's go and learn how the reduce method work on top of this to reduce the given DFA based on the input from the mark method. Thank you.

Reduce in Mark-Reduce Method Video• . Duration: 6 minutes

[Notes]

Dear students, welcome to the video, we are learning how to reduce a DFA into a minimal one. We learned how to use mark and reduce methods to reduce the DFA to a minimal one. Mark method identifies those indistinguishable states, reduce those equivalence classes into one state and reduce the entire DFA into a minimal one. Let's learn how reduce method works to accomplish this. Let's get started. In this slide, you see a DFA and the output of our mark method. The mark method identified state q_1, q_2 to be indistinguishable and state q_0, q_3 as indistinguishable. Now the reduce method must combine q_1, q_2 into one state and q_0, q_3 into another state and then modify all these transitions so that the resultant reduced one is a proper DFA. Okay. Now we need to combine q_1, q_2 into one state, q_0, q_3 into one state. Let's create a state, one for this equivalence class, two for the second equivalence class. Now we can name each of those states as anything like q_0, q_1 . Our original DFA also has got state named q_0, q_1, q_2, q_3 and the reduced DFA when we name it as q_0, q_1 , maybe we'll get confused. So the best way to name the states in the reduced DFA is to name it with all the states in the corresponding equivalence class, right? So the first equivalence class has states q_1, q_2 . So let's name it as a state q_1, q_2 and let's name this state as q_0, q_3 . So this will make us clear that the states, the corresponding equivalence classes. Now having identified the states in the reduced DFA, we need to identify next which of these states will be the starting state, which of the states will be the final state. A start state is a state that includes q_0 , which of the state includes

q0? So here in this state, we have q0. So this will be the starting state. A final state is a state that includes a state in the accepting state. So in the given DFA, q2, q1 are accepting states. So a state that includes either q1, q2 or both will be the accepting state. So this is an accepting state. Now we've identified the states, we identified what is the starting state, what are those final states? Done. So to identify the transitions in the reduced DFA, we must inspect each transition in the original DFA. Let's now look at the state q0 and we'll look at all the transitions that goes out of q0. The state q0, when it receives a, it goes to q1, okay? So here, this state has got q0 and this state has got q1. So on input a, the transition goes from q0, q3 to q1, q2. Now from q0, on b, we come back to the same state, right? So this is a state containing q0. So on input b, we come back to the state that contains q0, okay? Let's look at our next state, q0. Let's look at q1, okay? On a, q1 goes to q3. So this is q1, this state contains q3. So from q1, if you see a, you go to q3 in the original DFA. So in the reduced DFA, the edge is like this, right? It starts with a state, a reduced state containing q0 or q1 to a reduced state containing q3. So from q1 on a, the transition looks like this. Now from q1, the transition on b. So from q1, when we get b as input, we come back to q1, right? So from q1, if we get b, we come back to the state that contains q1, right? So we've handled state q0, state q1, all the transitions are looked at. Now look at q2. From q2, if you see a, we go to q0. So from q2, if you see a, you go to q0. So the edge is already present, right? Now from q2 b, so from q2, if you see a b, we come back to q2. So from q2, we should have an edge into q2 itself, right? So for this state containing q2, so the edge is already present. So q2 state is taken care of. Now let's look at q3. From q3, if you see an a, it goes to q2. So the edge is already present. From q3, if you see a b, we come back to q3. So this edge is also present. Now we've got a reduced DFA, okay? Is there any more edges that are left to be considered from the original DFA? The answer is no. All the edges in the original DFA are transformed into the reduced DFA. So the reduced DFA has got two states. One of them is a starting state, one of them is an accepting state. And you can convince yourself that the original DFA and the reduced DFA accepts the same line. We've come to the end of this video. What you have learned is the working of a reduce method. Now you know how a mark method is used to identify all those indistinguishable states. And you know how to use a reduce method that reduces the given DFA into a minimal DFA by merging all the states which are indistinguishable. I'm sure now if a DFA is given to you, you should be able to use the mark and reduce methods to reduce the given DFA into a minimal one. Thank you.

Mark-Reduce Method – Example (1) Video• . Duration: 11 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to reduce a given DFA into a minimal and equivalent one using markReduce method. Let's get started. What you see on the screen is a DFA and we are going to apply first mark method to identify all indistinguishable pairs, alright. The first step of mark method is to identify all those states which are inaccessible from the starting state. Look at this DFA. From the starting state, q1 is reachable, q3 is reachable, q2 is reachable, q4 is reachable. How do we reach q5 from q0? From q4, I cannot go into q5. If I cannot go into q5, then the state q5 is not reachable, right. So there is no simple path from q0 to q5 and hence q5 along with all its transitions can be removed from the DFA, right. So this is a DFA, after removing q5 is what we are going to work. Now we are going to look at pair of states such that one element of the pair is a final state, the other element of the pair is a non-final state. If you see such a

pair, then we can mark those pair as distinguishable pair, right. So let's look at the first pair, q_0, q_1 , q_0, q_1 , both are non-final states. So this need not be marked, q_0, q_2 . Should this be marked? Both are non-final, need not be marked, right, q_0, q_3 . So of this, q_3 is the final state, so this has to be marked, okay. I am marking this, q_0, q_4 . This has to be marked because q_0 is a non-final state, q_4 is a final state. So we are marking this, all right, done. Now q_1, q_2 , should we mark? No, not necessary because both are non-final state, right. So no mark. And q_3, q_1 , this one. Should we mark this? q_1 is non-final, q_3 is final, so let's mark them. q_1, q_4 , should we mark this? Yes, we need to mark this because one of them is final, one of them is non-final, so we are marking this. Now q_2, q_3 , yes, we need to mark and q_2, q_4 , we need to mark. And q_3, q_4 , both are final, no need to mark, right. So here are the markings, right. Now if we just look at, in the given DFA, there are five states, q_0, q_1, q_2, q_3, q_4 . If we can right away see, there are two groups, q_0, q_1, q_2 forms one group, q_3 forms other group. q_3, q_4 is a separate group because they are final states and q_0, q_1, q_2 are non-final states. We are attempting to come up with equivalence classes induced by indistinguishability on the given DFA. To begin, these are two equivalence classes that we can think of. May be right, may not be right, we are just working towards, right, we are working towards. Now all we did is, we considered $q_0, q_3, q_0, q_4, q_1, q_3, q_1, q_4, q_2, q_3, q_2, q_4$ and we marked these six pairs as distinct or distinguishable pairs, right. So we did this in step 2. Now what we need to do is to verify the remaining blank cells to see if they can be marked. How can we do that? Let's look at the pair q_1, q_0 , right. q_1, q_0 . Consider inputs 0 and 1. From q_1 , if you see a 0, we go to q_2 . From q_0 , if you see a 0, you go to q_1 . Now look at, is the cell q_1, q_2 is marked, q_1, q_2 is marked or not? q_1, q_1, q_2 is not marked, right. This is not marked, right. Now let's look at the other input, input 1. So how does q_0 and q_1 behave on input 1? q_1 , when we give input 1, it goes to q_3 . Now q_0 on input 1, it goes to q_2 . So now look at this pair, q_2, q_3 , right, q_2, q_3, q_2, q_3 . So this is marked. Since this is marked as distinguishable, we need to mark q_1, q_0 as distinguishable. So this is marked now, all right. Now we need to inspect the next empty cell, this one, right, q_2, q_0 or $q_0, q_2, q_0, q_2, 0, 1$. On 0, how does it perform and on 1, how does it perform? On 0, q_0 goes to q_1 , on 0, q_2 goes to q_2 . So q_1, q_2 is unmarked still, look at the behavior on input 1. So q_0 on 1 goes to, q_2 on input 1 goes to q_4 . So q_2, q_4, q_2, q_4 is marked, right. That means q_0, q_2 should be marked, okay. Now I just want you to verify this particular cell and this particular cell. In other words, verify the pair q_2, q_1 and q_4, q_3 . You will see that these two cells would not get marked. Let's try it now. One of them we'll try it now, we'll try q_2, q_1 , right, $q_1, q_2, 0, 1, q_1, q_1$ on 0, it goes to q_2, q_2 on 0 goes to q_2, q_2, q_2, q_2 are indistinguishable, right. So both are the same state. So they're indistinguishable, they're not marked. So ignore that. Now q_1 on input 1, it goes to q_3, q_2 on input 1, q_4 . So q_3, q_4 , both are final states, q_3, q_4 . So this cell, this is also not marked. Hence, the cell q_1, q_2 need not get marked, right. And you can repeat the same thing for this cell and you will also understand that this cell also need not be marked. Now if you just look at the result of mark method, we identified the states q_1, q_2 are indistinguishable, this one, right. And we identified states q_3, q_4 are indistinguishable. In our equivalence class that we tried to evolve, we identified q_3, q_4 as a class already. But if you just look at another class, which contains three states, q_0, q_1, q_2 , and the outcome is q_1, q_2 are indistinguishable, but not q_0 . So what we do is we split this set into two pieces, one containing q_1, q_2 , the other containing q_0 , right. So we split the equivalence class, which contained originally q_0, q_1, q_2 into two pieces, one containing q_0 alone, the other containing q_1, q_2 , right. Now we have three equivalence classes. This means in the reduced DFA, there are three states, one state containing q_0 , the other state is for q_1, q_2 , the other state is for q_3, q_4 , all right. Let's now draw the DFA. So we create one state for q_0 . We create one state for q_1, q_2 . We create one state for q_3, q_4 .

And now we're going to look at each edge from q_0 , if you see zero, you go to q_1 . From q_0 , if you see zero, you go to state containing q_1 . From q_0 , if you see one, you go to q_2 . So from q_0 , if you see one, you go to a state that contain q_2 . q_0 has been inspected. Now look at q_1 . From q_1 , if you see a zero, you go to q_2 . So from q_1 , if you see a zero, you go to q_2 . So you create a loop from the state q_1 , q_2 to itself on zero. So zero is okay. Now from q_1 , if you see a one, you go to q_3 . So from a state containing q_1 , if you see a one, you go to a state containing q_3 . In a similar manner, you inspect every other edge and you will arrive at this DFA. It's a reduced DFA that accepts the same language as the original DFA. The number of equivalence classes that you arrive at after the mark method indicate the number of states in the minimal DFA. Here in this case, we've got three equivalence classes. It means the reduced DFA has got three states. Fine. So we've come to the end of the video. What you see on the screen is an additional exercise. You can see a DFA on the screen. This is taken from the textbook for the course by Peter Lentz. And my request is, after this video, try to reduce it using MarkReduce method and verify your answer from the textbook. So we've learned how to use MarkReduce method to reduce a given DFA into a simpler one or a minimal one. Thank you.

0*1* vs. 0n1n (where $n \geq 1$) Video • Duration: 6 minutes

[Notes]

Dear students, welcome to this video. In this video, I'm going to show you two languages. One of them is a regular language. The other one is a non-regular language. And let's try to understand why one of them is regular, the other one is non-regular intuitively. Let's get started. The first language that we consider is 0^*1^* . And the DFA that accepts 0^*1^* is what you see on the screen. Is it sufficient to say that 0^*1^* is regular? If you have a DFA that accepts a language, then the language is regular, so 0^*1^* is obviously regular. Right? Let's try to understand how this DFA works. Assume the language 0^*1^* . The valid strings are sequence of 0s followed by sequence of 1s. So the sequence of 0s, the number of 0s could be n greater than or equal to 0. Here also n greater than or equal to 0. Any number of 1s. It's possible to have a string of entirely 1s, right? Without 0, we can have a string of entirely 1s. And we can have a string which consisting of entirely 0s, perfectly fine. If you have a string which consisting of 0s and 1s, the sequence of 1s should follow sequence of 0s. Now, if you just look at our DFA, the state q_0 receives sequence of 0s. The moment it actually sees 1, it takes a system or DFA into a state q_1 . When the DFA goes from state q_0 to q_1 , all it remembers is, I've seen sequence of 0s. It's time to see a sequence of 1s. Nothing else. That's the only thing that state q_1 remembers. And now, q_1 will only accept sequence of 1s. As long as it sees sequence of 1s, it will stay in final state. The moment it sees a 0, it will recognize, oh, this is not something that I expect, take the system to a dead state, okay? So it accepts sequence of 0s followed by sequence of 1s. Now, let's look at the other language, a DFA that accepts 0^n1^n , where n is greater than or equal to 0. Try to understand this better. When n is equal to 0, what you accept is an empty string, just nothing. Empty string can be accepted if you have a DFA in which the starting state itself is an accepting state, empty is accepted, not a problem. And then, when n equal to 1, we need to recognize 0, 1. When n equal to 2, what we need to recognize is 2 0s followed by 2 1s. When n equal to 3, we need to recognize 3 0s followed by 3 1s, 4 0s followed by 4 1s. And it goes like this, alright? If we try to use a DFA that accepts 0^*1^* , then it will not work for any of these cases. The reason being, we

need to receive n number of 0s, right? And we need to exactly receive the same n number of 1s after that. So when you come from q_0 to q_1 , you should remember how many number of 0s that you received. Let's look at little closely. When n equal to 1, the DFA that accepts it would be q_0 . You take a 0, go to q_1 , you take a 1, go to q_2 , q_2 , q_3 , whatever, right? Some state. So this accepts a language 0, 1. To accept a language 0, 0, 1, 1, we have a DFA like this, 0, 0, 1, 1, right? At q_2 , we exactly know we've received two 0s, so that we can have two more states to accept remaining 1s. Now, if we extend this n equal to 3, then we should have received three 0s and we should receive three 1s. We're not actually writing a language for three 1s, three 0s followed by three 1s, or two 0s followed by two 1s. But we are looking for a language which has got 0 power n , 1 power n , okay? Receive n 0s and then exactly receive n 1s after that. Now, this n is infinity. n can go all the way to a much larger number, infinity. It's an infinite language, right? Do you now understand that if n is equal to 1,000, you should have 1,000 transitions that accept 1,000 0s and then followed by 1,000 transitions that accept 1,000 1s, right? There's no other way we can actually play with it, all right? Now, if you apply, if you try to arrive at a DFA, you will see that you would need by a number of states. The number of states required is not finite, actually. It means you cannot have a DFA that accepts 0 power n , 1 power n . I'm sure in your data structures class, you would have learned how to recognize strings of the kind 0 power n , 1 power n . How do you do that? You would actually be using a stack, right? And assume there is a string. You will read the string left to right. As long as you see a 0, you will put it in the stack. Next 0, put it in the stack. And when you see a 1, pop the topmost 1. I remove the matched 1. When you see a next 1, pop the topmost 1. When you see the next 1, pop the topmost 1. When you receive, when you reach the end of the string, you would have popped everything in the stack, right? So if you have a little more memory than what a DFA gives you, you will be able to recognize 0 power n , 1 power n , correct? There are many languages for which we cannot have a DFA. These are not regular languages. We have come to the end of this video. Now, I'm sure you understood intuitively what makes a language not regular. Can you try to identify a few other languages which has a requirement that is required for recognizing 0 power n , 1 power n . Thank you.

The Myhill-Nerode Theorem Video • Duration: 5 minutes

[Notes]

Dear students, welcome to the video. In this video, I am going to introduce you to an important theorem in the formal languages called as the Michael Leroux theorem and we will also discuss its applications in formal languages. Let's begin by understanding the statement of the theorem. The statement begins by defining an equivalence relation RL . It says a string x and y are related by the equivalence relation RL if and only if, for all z element of Σ^* , xz is in L exactly when yz is also in L . L is regular if and only if, the equivalence relation induced by RL is of finite index. In other words, the number of equivalence classes that we get is finite. Let's now have a look at how this theorem is used in showing that a language is not regular. What you see on the screen is a language L is equal to 0 power n , 1 power n , where n is greater than or equal to 0. The valid strings of this language are sequence of 0s, a n number of 0s followed by n number of 1s. Alright? Now, let us consider a set S consisting of strings of the form 0 power n where n is greater than or equal to 0. So this set will contain strings empty, right? We can write empty as this way as well, as 0, double 0, triple 0, four 0s and so on. Now, what we do is we pick two strings from S . Let's say we are picking a

first string from S and second string from S . The first string, let's call it as 0^k . Second string, we call it as 0^m . So what we do is we attach an 1 which is 1^m . So we attach this 1^m to both the string 0^k and 0^m . This results in $0^k 1^m$ and $0^m 1^m$ being in the language, 0^k , $a^m 0^m$ and m times 1 . That's not in the language. All right? Now, can we take any other string such that for a single result, for a particular result, two strings are in the language together or not in the language at the same time? Not possible, right? It means the number of distinct states that you need to remember to recognize 0^n , 1^n is infinite. It just means for me to recognize 0^m , 1^m , I have to explicitly remember 0^m . In other words, to recognize $0, 0, 1$ I should, $0, 0, 0, 1, 1, 1$, I should explicitly remember triple 0 . To recognize $0, 0, 1, 1$, I should explicitly remember $0, 0$. To recognize 4 times 0 and 4 times 1 , I should explicitly remember 4 times 0 . It means the number of states is same as the number of elements in the set S . Our theorem says that the language L is regular if and only if the number of equivalence classes induced by the relation is finite. In this case, each pair is distinct. We cannot represent this using finite number of states, hence the language is not regular. Now let's consider the problem of minimizing the DFA. What you see are two DFAs on the screen. The one on the left is minimized to get the one on the right. We use mark reduce method to minimize the DFA on the left into the one that you see on the right. The key step in mark is to identify all indistinguishable states. If the set of states are indistinguishable, we will combine them into one state. So what are indistinguishable states? Let's say if q_1 and q_2 are indistinguishable. It means whatever string that you see after either being in q_1 or q_2 , the DFA behaves exactly the same way. Is it not? Are we talking about a similar relation on strings as defined by Michael Neroot? So we can consider the algorithm that we learned, mark reduce, as an instance of a particular variant of Michael Neroot theorem. We have come to the end of this video. In this video, we learned about Michael Neroot's theorem and its application to minimizing the DFA and verifying if the language is regular or not. Thank you.

Applying Pumping Lemma Video • Duration: 7 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to use pumping lemma to prove a language is non-regular. We are going to take a particular example, a language L equal to $a^n b^n$, where n is greater than or equal to 0 , as an example. Let us get started. What you see on the left is a statement of pumping lemma. Let L be an infinite regular language. Now, there exists M such that every string w in L whose length is at least M can be decomposed as $w = xyz$ in which the length of first two parts is less than or equal to M and the length of y is at least 1 such that $w_i = x y^i z$ is also in L for all the i 's. Let's now try to prove the language $a^n b^n$, where n is greater than or equal to 0 is not regular. The pumping lemma shows the conditions to be satisfied if the language is regular. But we want to prove that the language L is not regular. So if we try to fit the conditions to this L , then we should get a contradiction. Let's see how it works. Let us begin assuming that the language L is regular. Let us assume the language L is regular. $a^n b^n$ is regular to begin with. Let's assume that. Now, let us suppose we choose a value M . Now, let's see how to do that. The string, a sequence of a 's, that is n number of a 's, followed by n number of b 's is in the language. All right. So n number of a 's followed by n number of b 's is in the language. Now, we need to choose a M so that any string whose length is at least M can be decomposed. Right. So let us smartly choose M to be n . So this is

the string with n a's and n b's. If we choose M equal to n , then our x, y , the length of x, y has to be less than or equal to M . Right. Then our x, y will be entirely of a's. So we say sequence of a's and sequence of b's. Here, there are n a's and n b's. And we're now going to say that M equal to n . It means we should be able to find a decomposition of this string into three pieces such that x, y , the length of x, y is less than or equal to M . And we should also see that the length of y , which is at least one, can be pumped. Right. So this is our x, y . Then what will be our y ? y will be some suffix of this. Correct. And what will y comprise of? Entirely of a's. Let us suppose length of x, y is equal to M , the length of y equal to k . Then what will be the length of x ? Length of x will be M minus k . Right. So this, this and this should be clear. So we can write the given string as a power M minus k , a power k , b power m . Can we write a power m , b power m as this? a power m minus k , a power k , b power m . Is that okay? So in which what is the first part? This is the first part. What's the second part? This is the second part. And this is the third part. Now, what happens if we repeat the second part? What happens if we repeat the second part zero number of times, the string that you will get will be a power m minus k , b power m . If you repeat the second part one time you get a power m , b power m . If you repeat the second part twice, you get a power m minus k , a power $2k$, b power m . So if you observe this, we will not see that for all i 's, x, y, i, z is part of the language L . Right. So if you pump the second part zero time, immediately you will realize that the string is not in L . Right. If you pump it twice, this is not in L . If you see one instance where when you pump, the string is not in L , it means that we are contradicting with our assumption due to a pumping lemma. Right. As per pumping lemma, in a string whose length is greater than or equal to m , right, in a string whose length is, let's say this string is w , the length of the string is greater than or equal to m . And from this string, you take a part whose length is up to m . Okay. Let's say this part is x, y . You take a part whose length is up to m . In that part, the suffix y can be pumped. Suffix of length at least one can be pumped. Now when we try to do this for a power n , b power n language, we identified m , we picked a string and we picked our second part, first part and second part, and we try to pump the second part, we see that pumping results in strings which are not part of L . That means the language L is not regular. So we've come to the end of this video. We've seen one example of how to show a language is not regular using pumping lemma. Let's see a few more examples. Thank you.

Applying Pumping Lemma (2) Video• . Duration: 7 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to use pumping lemma as a game between two people to show that the language is not regular. We are going to take an example, a power n , b power n for this purpose. What you see on the left of the screen is a statement of pumping lemma for x . What you see on the right is a set of steps or a sequence of steps we took to prove the language a power n , b power n not regular. We assumed first the language is regular and we chose m equal to n and then we choose a string whose length is greater than or equal to m , okay. And then what we did is when we choose m equal to n , the string that we automatically choose is a m , b m , right. Do you observe now that this string length is more than m ? The length is actually $2m$, much more than m . Is that not? We chose a string whose length is $2m$. Now we are going to partition this string into three parts and the partition must be made such that the length of first two partitions should be less than or equal to m . It means the first two partitions can only include a's and the third partition can include just b's, right. So x, y can only include a's and

z can include only b 's, right. So now can you understand why choosing m equal to n is a smart move, okay. Now if you say that x, y is equal to a^m , then y will be entirely of a 's only, correct. So let us suppose the string a^m can be written as a^{m-k} and a^k in which a^k is your y and a^{m-k} is your x , right. So now how do you write the whole string? x which is a^{m-k} , y a power k and b which is b^m , right. Three parts x, y, z . Now the theorem says that if you pump y 0 times, 1 times, 2 times and so on, the resultant string should be in language here. Now we took i equal to 0. We decide to pump y 0 times. If we pump y 0 times, the resultant string consists of only first part and the third part. The second part will be cut off because y equal to 0, right. I mean the number of times y repeated is 0. That results in a string that is a^{m-k} , b^m . This is certainly not of the form a^n, b^n . This is not in language EL . Hence our assumption that the language EL is regular is incorrect. We arrived at a contradiction. This completes our proof that a^n, b^n is not regular. We're going to play this as a game between two people. The first person is you, yourself. The second person is an adversary. You will try to prove that the language is not regular. That's your intention. You want to arrive at a contradiction in the pumping lemma. Now your adversary will try to foil your attempt. In other words, he'll try to ensure that the language is proved as regular. You and your adversary play the game in four steps. The first step is your adversary's step. The adversary picks m . He can pick any m , all right. The moment adversary picks an m , you will pick a string whose length is more than m or at least m . Look at it. The theorem states there exists a positive integer such that any string whose length is more than m can be decomposed into x, y, z , right. So your adversary picks m and you pick a string whose length is more than m . Now your adversary will pick how to partition this string so that it meets this condition. So what is there for his choice? He has to choose what is x, y , and what is z , right. Or what is x , what is y , what is z . That's what he has to do. And now after he's giving you what is x , what is y , what is z , you're going to now show that for a particular i , if you pump y i times, the result is not in language L , right. So you will try to arrive at a contradiction in step 4. Let's try to play this game to prove our language A^n, B^n is not regular. Who makes the first step? Your adversary. What is he going to do? He's going to give you a m . Let us say the m he gives you is 10. We'll make it a smaller number which is m equal to 4, okay. Now you have to choose a string whose length is more than m . So you're choosing a string whose length is $2m$. Particularly you are choosing a string which is $aaa, 4a$'s and $4b$'s, right. So here m equal to 4. You're choosing a string whose length is $4a$ and $4b$. Now the next move is made by your adversary, right. Your adversary has to partition this string $4a$'s $4b$ into how many parts? Three parts, right. The length of first two parts should be at most m . So the length of first two part should be at most m . So the maximum length he can go to tell you or to give you an xy is 4. So this is your xy , the worst you can get and this will be your result. So since adversary give you an m , since you pick your string whose length is $2m$, that was a smart decision. So that whatever way he is going to give you a partition, he will not succeed. So you made your move smartly, right. Now your adversary place the string w into xyz . So the string $aaa bbbb$ is split into three parts such that this is your xy and this is your result and he also says that this much is your x and this much is your y and now it's your turn. You're going to now say that your x which is 3 times a and your y one time a result four times b . If I repeat this part zero number of times, what I get is three a 's and four b 's which is not in language. Hence you got a contradiction and you won. All right. If the language is not regular, you will certainly have a way to win for sure. Whatever m your adversary choose. Isn't it an interesting approach to prove a theorem? So we've learned how to prove a language L is not regular using pumping lemma as a game between two people you and your adversary. Thank you.

Applying Pumping Lemma (3) Video• . Duration: 4 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn how to use pumping lemma to show that the language ww^R such that w is in Σ^* is not regular. We are going to use pumping lemma. We are going to use pumping lemma as a game between two people to show that the language L is not regular. Let's get started. Let us now assume that the Σ that we consider is a Σ with just two symbols, zero and one. This is for simplicity. This works for any Σ . The valid strings in the language are of the form ww^R . If w is 0101, its reverse will be writing this in opposite direction 1010. So this string is a valid string in the language. Now, if you consider a string to be 1010, its reverse will be 0101 in the opposite direction. If you consider a string to be 1100, its reverse will be 0011. Right? So you know, what are the valid strings in the language L . And we want to prove that the language is not regular. You and your adversary are playing. Your adversary makes the first move. Your adversary chooses an m . All right? For illustration purpose, let us suppose this m is equal to 4. Or even simpler m , m equal to 2. Now, we need to choose a string whose length is larger than m , which makes the adversary's moves difficult. Okay? Let us choose a string whose length is more than m . Okay? We are choosing a string of the form $0^m 1^m 0^m$. All right? In other words, if you assume m is equal to 2, we are throwing a string 0011, 1100, our adversary. All right? So since the m chosen by the adversary is 2, the next job is that the adversary must take this string and come up with a split. How should he make the split? The adversary can make a split such that the length of first two parts can be at most m . So the length of xy can be m . Okay? So he makes a split so that he takes $0^m 1^m 1^m 0^m$. And now he suggests the following split. He says x and y is entirely of zeros and z is $1^m 1^m 0^m$. Correct? Now, the length of xy is m and length of z is a little more. That's okay. So your adversary gave you a split. Now, your job is to show that if you pump it in a particular way, the resultant string is not an L . Okay? Let us suppose as per the split, 0^m is split as $0^{m-k} 0^k$. Right? The remaining is as such. $1^m 1^m 0^m$. So what is your y part? 0^k . And now you're saying that if you repeat the y part 0 times, what you get is 0^{m-k} . If you're repeating y 0 times, 0^k is gone. And then what you have is $1^m 1^m 0^m$. This string is not in the form ww^R . And you can readily say that you arrived at a contradiction and you have won. It means the language ww^R is not regular. We've come to the end of this video. You learned how to use pumping lemma to show that a language is not regular. Thank you.

Closure Under Set Operations Video• . Duration: 2 minutes

[Notes]

Dear students, welcome to this video. In this video, we are going to learn about closure properties for regular languages. So let's get started. Let us suppose we have two languages L_1 and L_2 . Assume these two languages are regular. It is of our interest to understand if $L_1 \cup L_2$ is also a regular language. Given L_1 is regular and L_2 is regular, the question is, is $L_1 \cup L_2$ also regular? Let us suppose we have a regular expression R_1 that describes L_1 . Can we do that? Since L_1 is regular, we can always get a regular expression. Similarly, we can get another regular expression

that describes L_2 . Then, the language $L_1 \cup L_2$ can be described by regular expression $R_1 + R_2$. We have a regular expression $R_1 + R_2$ that describes $L_1 \cup L_2$. Since $L_1 \cup L_2$ is described by a regular expression, this is also regular. This proves that $L_1 \cup L_2$ is also regular. Now, let me ask you another question. How do you prove $L_1 L_2$ is also regular? So $L_1 L_2$ is a language that can be described by a regular expression $R_1 R_2$. So $L_1 L_2$ is described by a regular expression which is $R_1 R_2$ and it is regular. How about L^* ? This will be L^* of R_1 . So these theorems have simple proofs and we don't get into formal proofs of everything. It is certainly of our interest to understand given language L_1 and L_2 , what are those operations that result in the regular languages outcome. These are answered by closure properties. We have come to the end of this video. You learned what are closure properties. You learned the statement that states that the family of regular languages are closed under union, intersection, concatenation, complementation and star closure. We will not actually get into the proof of all of these properties but knowing that regular languages are closed under union, intersections, concatenation, complementation and star closure will certainly help you to apply this knowledge somewhere. Thank you.

Closure Under Reverse & Other Operations Video• . Duration: 5 minutes

[Notes]

Dear students, welcome to the video. In this video, we will understand the statement of these closure properties, so that we can use it whenever there is a requirement. These properties are largely of theoretical interest, hence, we will not do the proof of these properties as a part of this video and this is out of the scope of the course. Let's get started. Let us consider a language L and a language L^R . We form the language L^R in a way that we pick each string in the language L and we write it in reverse order and we keep it here. If this is W , we will reverse it as W^R . Right? W_1, W_1^R . We will take W_2 and reverse it and like this we continue this for all the strings in L . So our L^R is a language consisting of strings which are reverse of the strings in L . So what we did is to create a new language from an existing language L . Now a new language created by the operation string reversal is still a regular language. So if L is regular, then L^R is also regular. The theorem says the family of regular languages is closed under reversal. Our second closure property is on homomorphism. Let's first understand what is homomorphism and we will see an example of this. Let us suppose we have two alphabets, Σ and τ . The function h that maps symbols from Σ into τ^* is called a homomorphism. In simple, homomorphism is a function that maps a symbol from an alphabet into a string and the other alphabet. We will see an example. Let us suppose we have two alphabets, one is Σ , the other one is τ . Let us define the h function as below. The h function takes a symbol a as input and returns a b as output. In other words, it maps a into a b . a is an element of Σ , a is an element of τ^* . Similarly, we define h of b as b^c . Now assume we have a string $a b a b$ element of Σ^* . So h function takes this string as input and maps it into a string which is an element of τ^* . It does this by taking each symbol in the input and maps it accordingly. So a is mapped as $a b$, b is mapped as b^c , the next a is mapped as $a b$, the next b is mapped as b^c . So the mapping is simple, isn't it? Now let us suppose you have a language here. What we can do is take each string in the language and then apply the homomorphism and arrive at a new language which has got all the strings transformed, all right? So the language that we obtain as an outcome is called as homomorphic image of L . So we take $h(L)$,

apply homomorphism on each string and we get h of l . Now given h to be a homomorphism and given a language EL which is regular, then the homomorphic image of h which is called as h of l is also regular. In other words, regular languages are closed under homomorphism. The third operation is right quotient. Let's try to understand what right quotient is all about. Let us suppose we have a language $EL1$ and language $EL2$ and these two are the language on the same alphabet. Then the right quotient of $EL1$ with $EL2$ is defined as a set consisting of vx such that xy is an element of $EL1$ for some y element of $EL2$. For example, if $EL1$ contains a string 0101 , $EL2$ contains a string 01 , then the right quotient of $EL1$ and $EL2$ will contain 01 . Now given that we have languages $EL1$ and $EL2$ and we know that $EL1$ and $EL2$ are regular languages, then the right quotient of $EL1$ and $EL2$ is also regular. We have come to the end of this video. What we have learned is that the family of regular languages is closed under string reverse, homomorphism and right quotient. It's important that you understand this closure property well. Thank you.

Recommended Reading: Equivalent DFA's Reading• . Duration: 10 minutes

[Notes]

Brief: This reading explores the concept of equivalent deterministic finite automata (DFAs), demonstrating that multiple DFAs can exist to recognise the same language. While these DFAs may differ in structure, they accept the same set of strings, and simplifying them can result in more efficient automata. Two DFAs are considered equivalent if they accept the same language, meaning they both recognise the same set of strings over a given alphabet. Even though the state structures may differ, the input-output behaviour of equivalent DFAs is identical. By reducing redundant states and transitions, a DFA can be simplified while preserving the language it recognises. Simplified DFAs are easier to understand and more efficient to implement. Consider two DFAs for the same language L , with the following transitions: DFA 1: Has 4 states and more transitions. DFA 2: Uses fewer states to achieve the same functionality. Both DFAs accept the same strings, proving they are equivalent. A simpler DFA is often preferred in practical applications such as parsers and lexical analysers, where minimising states improves performance. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 2, Section 2.3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.4.

Recommended Reading: Indistinguishable States Reading• . Duration: 10 minutes

[Notes]

Brief: This reading introduces the concept of indistinguishable and distinguishable states in a deterministic finite automaton (DFA), which plays a crucial role in DFA minimisation. Indistinguishable states can be merged to create a more efficient DFA, while distinguishable states remain separate. Identifying and handling these states is essential for constructing optimised automata. Understanding when states are indistinguishable helps reduce the complexity of DFAs without changing the recognised language. Two states p and q in a DFA are called indistinguishable

if, for every input string $w \in \Sigma^*$: This means both states lead to either accepting or non-accepting states for any given input string. Indistinguishable states are equivalent and can be merged during DFA minimisation, which reduces redundant states while preserving the language the DFA recognises. Two states p and q are distinguishable if there exists a string $w \in \Sigma^*$ such that: $\delta^*(p, w) \in F$ but $\delta^*(q, w) \notin F$, or vice versa. This difference indicates that the two states cannot be merged. Consider two states q_1 and q_2 in a DFA for language L . If both states, given the same input, always lead to acceptance or rejection, they are indistinguishable and can be merged. If state q_1 accepts a string w but q_2 rejects it, the states are distinguishable. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 2, Section 2.3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.4.

Recommended Reading: Approach to Simplifying DFA Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading introduces the Mark-Reduce method for simplifying deterministic finite automata (DFA). The goal of DFA simplification is to minimise the number of states by identifying and merging indistinguishable states, ensuring the DFA maintains the same behaviour. The approach ensures that the minimised DFA remains equivalent to the original, recognising the same language with fewer states. If two states p and q are indistinguishable, the DFA exhibits the same behavior from both states. These states can be merged into a single state without affecting the language recognised by the DFA. Identify pairs of states p and q that behave identically for all inputs. Mark pairs that lead to different accepting or non-accepting behaviours, distinguishing them. Combine unmarked, indistinguishable state pairs into a single state to reduce the DFA's size. The new DFA preserves the original language but with fewer states. Given a DFA with states q_1 , q_2 , and q_3 , if states q_1 and q_2 behave the same for all inputs, they can be merged into a new state r . The reduced DFA is easier to analyse and more efficient in terms of memory and processing. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 2, Section 2.4. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.4.

Recommended Reading: Mark in Mark-Reduce Method Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading covers the Mark Method, a technique used in DFA minimisation to identify and merge indistinguishable states. This method systematically identifies pairs of states that behave identically for all inputs, allowing the reduction of redundant states in the DFA. The Mark Method involves checking all pairs of states (p, q) in the DFA. If one state in the pair leads to an accepting state and the other does not, the pair is marked as distinguishable. Example: If $p \in F$ (an accepting state) and $q \notin F$, the pair (p, q) is marked as distinguishable. For every pair of states (p, q) and every input symbol $a \in \Sigma$: If the resulting states are marked as distinguishable, mark (p, q) as

distinguishable as well. Repeat this process until no new pairs are marked. Reduction by Identifying Indistinguishable States: States that remain unmarked are considered indistinguishable and can be merged into a single state. Removing these redundant states produces a minimised DFA that retains the original language. Consider a DFA with four states: q_0, q_1, q_2, q_3 . For input symbols a and b , compute the state transitions and mark distinguishable pairs. Once the marking process is complete, merge unmarked pairs to minimise the DFA. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 2, Section 2.4. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.4.

Recommended Reading: Reduce in Mark-Reduce Method Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading introduces the reduce phase of the Mark-Reduce method for DFA minimisation, focusing on using equivalence classes of states to construct a simplified DFA. By creating a state for each class and mapping the original transitions, the reduced DFA preserves the original language with fewer states. This approach optimises DFA performance, making it more efficient for language recognition and parsing tasks. Use the Mark method to identify pairs of states that behave identically for all inputs. Marked pairs indicate distinguishable states, while unmarked pairs are indistinguishable and can be grouped into equivalence classes. Indistinguishable states are grouped into equivalence classes. Example: If q_1 and q_2 are indistinguishable, they form an equivalence class $\{q_1, q_2\}$. Step 1: Create one state for each equivalence class in the minimised DFA. Step 2: Map each transition from the original DFA into the new states of the reduced DFA. The start state in the reduced DFA corresponds to the equivalence class containing the original start state. If any state in an equivalence class is an accepting state, the entire class becomes an accepting state in the reduced DFA. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 2, Section 2.4. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.4.

Recommended Reading: Mark-Reduce Method – Example (1) Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading demonstrates the Mark-Reduce Method for simplifying a DFA by merging equivalent states. In the example, the original DFA with five states is reduced to a new DFA with three equivalence classes, resulting in a simpler structure and fewer transitions. Equivalence classes: $\{q_0\}, \{q_1, q_2\}, \{q_3, q_4\}$ New DFA: Each class is treated as a single state, resulting in fewer transitions. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 2, Section 2.4. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.4.

Recommended Reading: 0^*1^* vs. 0^n1^n (where $n \geq 1$) Reading• . Duration: 10 minutes

[Notes]

Brief: This reading explores the differences between regular languages (such as 0^*1^*) and non-regular languages (such as 0^n1^n (where $n \geq 1$)). The goal is to develop an intuitive understanding of how the structural properties of these languages affect their DFA implementations and recognition capabilities. **Description:** This language consists of any sequence of '0's followed by any sequence of '1's, including the empty string. Example Strings: "", "0", "1", "00011", "01". A DFA for 0^*1^* is straightforward to design, requiring a finite number of states to handle the transitions between sequences of '0's and '1's. **Description:** This language contains strings where the number of '0's matches the number of '1's, with all '0's preceding the '1's. A DFA cannot recognise this language because it requires the automaton to count the number of '0's and compare them with the '1's—a capability beyond the power of finite automata. **0^*1^* :** A DFA with a few states can handle any length of '0's followed by '1's, as there is no constraint on the counts. **0^n1^n :** To recognise this pattern, an automaton would need to store unbounded information, which is not possible with a DFA. While a DFA for 0^*1^* can be minimised to a small number of states, no finite DFA can recognise 0^n1^n . This is an example of a language that requires a pushdown automaton (PDA) instead of a DFA, as PDAs can handle context-free languages. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 2, Section 2.4. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.3 & 4.4.

Recommended Reading: The Myhill-Nerode Theorem Reading• . Duration: 10 minutes

[Notes]

Brief: The Myhill-Nerode theorem is a powerful tool for proving whether a language is regular and for minimising the number of states in a DFA. The theorem states that a language is regular if and only if the number of equivalence classes induced by the language is finite. It provides an algorithm for constructing the minimal DFA by merging equivalent states. **Myhill-Nerode Theorem:** The theorem identifies the minimal number of states required to recognise a regular language by finding the finite set of equivalence classes. **State Reduction:** The theorem facilitates DFA minimisation by combining equivalent states into one, thereby reducing the overall number of states without altering the language recognised by the DFA. Given a DFA with 10 states, by applying the Myhill-Nerode theorem, you can reduce the DFA to 5 states by identifying and merging equivalence classes. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Sections 4.4.1 & 4.4.2.

Recommended Reading: Introducing Pumping Lemma Reading• . Duration: 10 minutes

[Notes]

Brief: This reading provides an introduction to the Pumping Lemma, a fundamental concept in formal languages used to prove that certain languages are not regular. It outlines the key components of the lemma and explains its application through the decomposition of strings. If a language L is regular, there exists a positive integer m such that any string $w \in L$ with $|w| \geq m$ can be decomposed as: For all $i \geq 0$, the pumped string $w_i = xy^i z$ must also be in L . Since a DFA has a finite number of states, any sufficiently long string must cause the DFA to revisit some state. This leads to a loop that corresponds to the repeated segment y in the lemma. Using Pumping Lemma for Non-Regular Languages: To prove a language is non-regular, you assume the language is regular and apply the Pumping Lemma. If you find a contradiction (a pumped string that does not belong to the language), the assumption that the language is regular is invalid. Decompose $w = xyz$, with $|xy| \leq m$. Thus, y consists entirely of 'a's. Pumping y (e.g., setting $i=0$) results in a string with fewer 'a's than 'b's, which does not belong to the language. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 4, Section 4.2, 4.3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.1.

Recommended Reading: Applying Pumping Lemma Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading explores how the Pumping Lemma is used to prove that a language is non-regular, focusing on the language $L = \{a^n b^n : n \geq 0\}$. The Pumping Lemma is a key theoretical tool in formal language theory, helping to establish when a language cannot be recognised by a finite automaton. By strategically decomposing the string and pumping it, you show that some pumped versions do not belong to the language, establishing that no finite automaton can recognise it. This proof highlights the limitations of regular languages in handling complex patterns. If a language L is regular, there exists a positive integer m such that any string $w \in L$ with $|w| \geq m$ can be decomposed as: For all $i \geq 0$, the pumped string $w_i = xy^i z$ must also be in L . Proof of Non-Regularity for $L = \{a^n b^n : n \geq 0\}$: Choose $m=n$ to ensure the string $w = a^m b^m$ that belongs to L . The decomposition $w = xyz$, must satisfy $|xy| \leq m$ and $|y| \geq 1$. Since $|xy| \leq m$, the substring y consists entirely of 'a's. If y is pumped with $i=0$, the new string becomes $w_0 = a^{m-k} b^m$ (where $k = |y|$). This string does not belong to the language L because the number of 'a's no longer matches the number of 'b's. Therefore, the assumption that L is regular leads to a contradiction. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 4, Section 4.1. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.1, 4.2.

Recommended Reading: Applying Pumping Lemma (2) Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading explains how the Pumping Lemma can be applied as a game against an adversary to prove that a language is not regular. In this example, pumping disrupts the balance

between 'a's and 'b's in the language $L = \{a^n b^n : n \geq 0\}$, demonstrating that the language cannot be recognised by any finite automaton. If a language L is regular, there exists a positive integer m such that any string $w \in L$ with $|w| \geq m$ can be decomposed as: For all $i \geq 0$, the pumped string $w_i = xy^i z$ must remain in L . You must then choose a string $w = a^m b^m$ that belongs to L . The adversary decomposes the string $w = xyz$, ensuring $|xy| \leq m$ and $|y| \geq 1$. Since y contains only 'a's, pumping y results in strings like $a^{m+k} b^m$, where $k \geq 1$. The new string no longer has equal numbers of 'a's and 'b's, meaning it does not belong to L . By showing that some pumped versions of the string are not in L , you establish that the language $L = \{a^n b^n : n \geq 0\}$ is not regular. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 4, Section 4.3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.1.

Recommended Reading: Applying Pumping Lemma (3) Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading explores the use of the Pumping Lemma as a tool to prove that certain languages are not regular. Specifically, it focuses on the language $L = \{ww^R : w \in \Sigma^*\}$, which consists of strings that are followed by their reverse. If a language L is regular, there exists a positive integer m such that any string $w \in L$ with $|w| \geq m$ can be decomposed as: For all $i \geq 0$, the pumped string $w_i = xy^i z$ must remain in L . You then pick a string $w \in L$ such that $|w| \geq m$. The adversary decomposes the string $w = xyz$, ensuring $|xy| \leq m$ and $|y| \geq 1$. Your objective is to choose an integer i such that the pumped string $w_i = xy^i z$ is not in L . If successful, you demonstrate that the language is not regular. For a string in the language, the second half is the reverse of the first half. By pumping the segment y within the first half of the string, it disrupts the symmetry required by the language, proving that the pumped string no longer belongs to L . Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 4, Section 4.3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.1.

Recommended Reading: Applying Pumping Lemma (4) Reading• .

Duration: 10 minutes

[Notes]

Brief: This reading explains how the Pumping Lemma can be applied as a game against an adversary to prove that a given language is not regular. By strategically selecting decompositions and pumped strings, the prover shows that the language violates the conditions of the lemma, confirming that it cannot be regular. This method is a powerful tool for analysing the limitations of finite automata. It demonstrates how the lemma is used to prove that certain languages are not regular, focusing on the language $L = \{w \in \Sigma^* : na(w) < nb(w)\}$, where $\Sigma = \{a, b\}$. If a language L is regular, there exists a positive integer m such that any string $w \in L$ with $|w| \geq m$ can be decomposed as: For all $i \geq 0$, $w_i = xy^i z$ must also be in L . You then pick a string w from L such that $|w| \geq m$. The adversary selects a decomposition xyz with $|xy| \leq m$ and $|y| \geq 1$. Your task is to

find an integer i such that the pumped string $w_i = xyz$ is not in L . If you succeed, you prove that the language is not regular. The goal is to show that this language is not regular using the Pumping Lemma. You must construct a scenario where some pumped string violates the condition $na(w) < nb(w)$, demonstrating that the language cannot be recognised by any DFA. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 4, Section 4.3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.1.

Recommended Reading: Closure Under Set Operations Reading• . Duration: 10 minutes

[Notes]

Brief: Regular languages are closed under several operations, including union, concatenation, and Kleene star. This means that applying these operations to regular languages produces another regular language. These properties are crucial for understanding how regular languages behave under various manipulations. If both L_1 and L_2 are regular languages, their union is also a regular language. Example: If $L_1 = \{a, aa\}$ and $L_2 = \{b, bb\}$, then $L_1 \cup L_2 = \{a, aa, b, bb\}$ is regular. The intersection of two regular languages is also regular. Example: If $L_1 = (a+b)^*$ and $L_2 = a^*$, then $L_1 \cap L_2$ is the language consisting of only strings with repeated 'a's, which is regular. If L_1 and L_2 are regular, then their concatenation is also regular. Example: If $L_1 = \{a\}$ and $L_2 = \{b\}$, then $L_1 L_2 = \{ab\}$. The complement of a regular language is regular. Example: If L_1 is the set of all strings with even length, L_1 will contain all strings of odd length, which is also regular. If L_1 is regular, the star-closure of the language (zero or more repetitions) is also regular. Example: If $L_1 = \{a\}$, then $L_1^* = \{\epsilon, a, aa, aaa, \dots\}$. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 4, Section 4.1. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4, Section 4.2.

Recommended Reading: Closure Under Reverse & Other Operations Reading• . Duration: 10 minutes

[Notes]

Brief: This reading discusses the closure properties of regular languages, including their behaviour under various operations such as string reversal, homomorphism, and right quotient. Understanding these properties helps in proving that a language remains regular after applying these operations. Theorem: If a language L is regular, then the reverse of L (where all strings are read backwards) is also regular. Definition: A homomorphism is a function $h : \Sigma \rightarrow \Gamma^*$ that maps symbols from one alphabet Σ to strings over another alphabet Γ . If $h(a) = ab$ and $h(b) = bc$, then the homomorphic image of the string $abab$ is $abbcabbc$. Theorem: If L is a regular language, then its homomorphic image $h(L)$ is also regular. Definition: Given two languages L_1 and L_2 over the same alphabet, the right quotient of L_1 with respect to L_2 is defined as: Theorem: If both L_1 and L_2 are regular, their right quotient L_1/L_2 is also regular. Linz, P. (2016). An Introduction to Formal Languages and Automata (6th ed.). Chapter 4, Section 4.1. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Chapter 4,

Live Session Reading• . Duration: 10 minutes

[Notes]

(No transcript found.)

Limitations of Regular Languages Video• . Duration: 6 minutes

[Notes]

Hello everybody, we are at the module Context-Free Languages and the lesson we are looking at is Introduction to Context-Free Grammars and Context-Free Languages. The topic specifically we will look at is Limitations of Regular Languages. Now through this video, we will help understand the limitations of regular languages. What are the places where regular languages do well, that is, they are useful and what are the places where they do not do well, right, they have certain limitations. So let's delve deeper into this topic. So regular expressions, as you all know, and you have seen so far, they are a class of languages recognized by finite automata, right, and described using regular expressions. So you will write a regular expression that will describe languages and in order to process those expressions, you will have a structure, right, you will have some kind of mechanism and that's the finite automata. So using finite automata, you could try out, for instance, whether a given string belongs to that finite automata or not. Now there are notations involved in this. So there will be alphabets and there will also be some symbols like plus, star, etc. Now together these, the finite automata, that's the mechanism and that how you describe the regular expression itself, helps you define some set of language, right, and that language can be used and that language essentially can describe many strings that will belong to the language, right. This is something you have seen already. Now there are certain limitations of regular expression. First of the limitations is lack of memory. Regular languages have no memory beyond finite states. What do I mean by this? So finite automata will not be able to process some sequence of inputs, which might be necessary to identify certain patterns. It is true that it will be able to identify certain set patterns, but there will be certain patterns that it will not be able to identify due to the lack of memory. The second limitation of regular expression is inability to count. Now sometimes what you come across are some kind of counting problems. So in this counting problem, you need to, for instance, you might want to match the number of opening and closing parentheses. In that case, regular languages will not be able to handle this situation. So these are essentially called counting problems. An example of this counting problem is the language $a^n b^n$, where n is greater than or equal to 0. Now what happens is here you need number of a 's to match the number of b 's, right? And this is essentially a counting problem, which regular expressions may not be able to handle. Now to kind of, we should understand the first point, that is, there will be patterns like $a^2 b$, double a double b , triple a triple b . Now this is, of course, this can be accepted by regular expressions, but we cannot define a single automata which can identify the entire language. So for instance, the language of $a^n b^n$ will be hard to describe and not, rather not possible to describe using a single automata. If we make, now one can go and make individual automata to identify these specific patterns, but that is not the solution because there will be infinite number of those automata needed if we try to solve the problem this way. Thus, we need

a more powerful model than regular expression. So we are in search of a model that can solve these problems, right? Being able to identify these patterns such as the matching problems, matching a raised to n with b raised to n or certain kind of counting problems. So for that, we need something beyond regular expression. Now certain problems can be solved summarizing this problem again. There are problems that can be solved by finite automata and some can't. We need a more powerful model and that is where the context-free grammar comes in. We will see as the module progresses, we will see in the upcoming videos how context-free grammar can solve this problem. So it will be able to identify patterns in the languages such as $a^n b^n$ followed by b^n . The limitations help us realize the limits of finite automata. So we have seen the limits and this is a motivation for finding better models that can solve this problem. So in summary, if we try to look back at what we have seen in this video, there are limitations of regular languages. Of course, they are good, you know, help us solve certain problems. You can help identify strings in certain languages. There will be patterns it will not be able to solve. And thus arises the need for context-free grammars. So with that, we are at the end of this video. Thank you.

Revisiting the Definition of Grammar Video• . Duration: 8 minutes

[Notes]

So, we are at the module Context-Free Languages. The lesson we are looking at is Introduction to Context-Free Grammars and Context-Free Languages. So, let's look at the topic of definition of a grammar. So, through the course of this video, we will learn understanding the definition of context-free languages. That is, we will have a more formal definition of context-free languages, which will help us understand the power of these context-free languages. So, what is a grammar? Grammar is a formal system that specifies you have a set of rules for generating strings in the language. Now, this formal structure, this formal system will have a set of rules. As I said, this we will look at closely through the course of this video. And the formal system will also generate strings, and these strings will belong to that language. Now, the grammar, it's also said that the grammar defines some kind of syntactic structure of a language. So what do I mean by syntactic? This has more to do with what we see as the syntax part of the language, right? As we go through the videos, we will understand the syntax associated with the context-free languages. Now, here is the formal definition of a grammar. A grammar is defined with four tuples, okay? So, a grammar G will have V , T , S , and P . These four tuples together define the grammar, right? So, let's start with what V is, right? V is a set of non-terminal symbols or variables. We will see what, with some examples, we will look at what a non-terminal precisely means. And then there is T , the set of terminal symbols or alphabets of a language, right? So these are called terminal symbols. Then you have the start symbol, which is a non-terminal symbol, right? And finally, you have a set of production rules. Together, all four will help us define the grammar. Now, as you must have noticed, there is a difference between the terminal symbols and the non-terminal symbols, which is very key to the definition of a grammar. And one of the non-terminal symbols is the start symbol. So, we use the start symbol to start deriving the strings belonging to that grammar or rather the string belonging to the language. Each rule, now there is the production rules, right? Each rule in P will have the form such that will be of the form $A \rightarrow X$, where A is a non-terminal symbol and X is a string and X is, X belongs to V and T put together. Okay? So, the V and T put together in the intermediate derivations, we will see X appearing. So, we will understand this better through an example. Before

that, the context-free grammars, now that we have seen the overall structure of how a grammar is formally defined, let us look at what a context-free grammar is. So, this is a specific type of grammar where every production rule is of the form $A \rightarrow X$, A gives rise to X , where A is a single non-terminal and X is any terminal non-terminal. So, every production rule should have this form, right? There will just a single non-terminal on the left-hand side and there is X , which can be a string of terminal and non-terminal on the right-hand side. Now, this is why context-free grammars are called context-free, they are called context-free because on the left-hand side of production rule is exactly one non-terminal and there is no dependency on any of the context. In this case, the context means surrounding symbols. So, on the left-hand side, you will always have a non-terminal, a single non-terminal. So, this is the unique part of context-free grammars. Now, CFGs have a lot of applications in programming languages, mathematical expressions, etc. And we will see this through the course of, as we go through the module, we will look at some of the applications of this context-free grammar as well. So, let us look at one example of context-free grammar and this is a context-free grammar for palindromic strings only consisting of A and B . Now, there is the non-terminal B , a start symbol S , then you have the terminal A and B , part of the terminal, and you have the three production rules. So, let us look at the production rules, the three production rules in detail. So, S gives rise to A , S , B , then you have the second production rule, S gives rise to B , S , A , and then the third thing is S gives rise to an empty expression, right, that marks the, that will be used to end, as a last step of the production. Now, a language of context-free grammars is a set of all strings that can be derived from the start symbol of a CFG. So, the question is, how do you define the language, right? If you start with the start symbol and derive all the strings possible with that, the start symbol, that is essentially the language of the context-free grammar. Now, starting from the start symbol, repeatedly applying production's rule until a terminal symbol of the strings remains, right? This is the derivation process. So, you start with the start symbol and then you repeatedly apply the production rules, okay, until only terminal symbols of the string remain, that means you can't apply any other production rule, right? So, that's the process of deriving strings belonging to the language, right? And while doing that, what will happen is you will build essentially some kind of parse trees, okay? Some graphical representation of derivation of string can be done, and that will show the derivation step by step. So, in all the context-free language, right, for the context-free grammar, then you have this process for starting with the start symbol and repeatedly applying the rules, which is called the derivation process. And the third thing is you have the parse trees, right, which is a graphical representation for the string. In summary, right, we looked at the grammar, which is the formal system, and we saw that the formal system has the production rules, terminals, non-terminals, and the start symbol, right? That is what we mean by the grammar. Then there is the context-free grammar, which is a specific form where we restrict the set of production rules, right? And it allows recursive language generation. How do we restrict? If you remember, we keep on the left-hand side, there is only one non-terminal symbol is permitted. And finally, understanding CFG is crucial for language design, parsing, and computational theory. That is, as we saw, and as something we will look at in more detail ahead, is that CFGs will be used in many applications and essentially at the center of many computing applications. So with that, we are at the end of this video. Thank you.

Example of CFG's Languages Generated by CFG - 1 Video• . Duration: 5 minutes

Hello everybody. So we are at the module on context-free languages and the lesson we are looking at is introduction to context-free grammars and context-free languages. In this video, we will look at an example of context-free grammar and how we are able to use it to generate strings belonging to the language, corresponding to the context-free grammar. So at the end of this video, we will learn to derive a CFG language from a context-free grammar. So let's start with, you know, understanding what context-free grammars are. We have seen in the past video what that is. Let me just reiterate what a context-free grammar is. So context-free grammar consists of some production rules and it is used to generate all possible strings in the language. So which language is this? It is the language of the context, this is the context-free language. Now we have also seen in the past video that CFG is a much powerful model than the regular languages. Right? The kind of strings that you could derive with regular languages are limited and context-free grammar, we can go much beyond that. We also saw in the past video, the components that are part of the CFG model, right? So there is the terminal symbols, non-terminal symbols, production rules, and the start symbol. These all components put together help us build a CFG. Now let's focus on the particular example, right? So there is a language L , and that language L is equal to $a^n b^n$, where n is greater than or equal to 1, right? Now for this language, to describe this language, we have a set of production rules, right? The first production rule is $S \rightarrow a S b$. We can also say S produces $a S b$, right? That's the first production rule. The second production rule is $S \rightarrow a b$, right? These two rules together are defining this context-free grammar. And as you can see, there are non-terminals here and the terminals involved. The non-terminals is S and the terminals are a and b , right? This CFG generates strings with n number of a 's followed by same number of b 's, right? So that's the unique part of this CFG. We have seen in the past, again, this example shows the power of context-free grammar. A regular expression has had two limitations, right? The lack of memory and not being able to count, right? In this example, the CFG is able to count the number of a 's and match them with the number of b 's. And that's why this example is unique and interesting. Now, let's look at the process of string derivation. We can generate the string $aabb$ using the grammar in this example, right? So we are looking at an example here of a string that is $aabb$. Now this belongs to this context-free grammar, right? And we will see that we can use the production rules to derive this particular string. So let's start with how this is done. So what do you do in the beginning? You start with the start symbol, that is S , right? So you have S as the start symbol, and then you use production rule, right? Which production rule? Rule number one, which gives $S \rightarrow a S b$, right? If you apply that production rule, you get $S \rightarrow a S b$, right? So that's rule number one. Then you get $a S b$, and the question is, what's next, right? What's the next production rule? Apply rule two, right? Which is $S \rightarrow a b$. If you apply that, then you replace S with $a b$. And as a result, you derive a followed by $a b$, and then b again, right? In this fashion, we have shown that we apply the production rules appropriately to derive the final string a, a, b, b . Now what we have learned here is how to apply production rules appropriately to get a target string a, a, b, b , right? Similarly, you could have other strings. And then the question is, given the production rules, whether you can derive a particular target string or not, right? If you are able to derive the string, then you can safely say that these strings belong to the context-free language, corresponding to the context-free grammar. So in summary, we have seen an example of production rules of a context-free grammar, okay? Applying the production rules to generate a specific string, and in general, this can be applied, right? You could apply production rules like this to generate many strings belonging to that language. So we are at the end of this video, and we have seen clearly an example of CFG and how

it is used to derive strings, okay? So thank you so much.

Example of CFG's Languages Generated by CFG - 2 Video• . Duration: 7 minutes

[Notes]

Hello everybody. So we are at the module on Context-Free Languages. The lesson we are doing is Introduction to Context-Free Grammars and Context-Free Languages. The topic we have is we will look at another example for Context-Free Grammars languages. So in the past video, we looked at one example. Now let's look at another example. Now again at the end of this video, we will be able to derive Context-Free Grammar languages from Context-Free Grammars. So like in the past video, we saw one specific example in which we applied the production rules of that particular Context-Free Grammar and we could generate a string belonging to the Context-Free Grammar language. Now we will look at another example. This will give you some kind of practice how to use Context-Free Grammars effectively. So just to put the definitions in front of you. So Context-Free Grammar consists of production rules like we have seen and they are used to generate all possible strings in a language. So we have seen this in the past. What will vary from one Context-Free Grammar to another is there are specific production rules. And the different production rules of the Context-Free Grammar somehow will determine what kind of strings it will produce. Again, as far as the components of the Context-Free Grammar are concerned, there will be terminal symbols, non-terminal symbols, production rules, and the start symbol. And the grammar will be Context-Free. And we have seen what do we mean by Context-Free. Just to help you recollect again, for Context-Free, we mean the left-hand side of the production rules. There is no other context there. That means we have only a non-terminal on the left-hand side of the production rules. So let's look at this specific example. This is our second example we are looking at. And the language involved here is $a^n b^m c^n$, where m is greater than or equal to 1. So this is a unique pattern that this Context-Free Grammar is able to produce. One can have any n or any m and this particular Context-Free Grammar will be able to derive strings in this language. Now what's the unique part here? The unique part here is a^n and you have c^n . That is a 's number of a 's should match the number of c 's. And b will be some equal to m . Here we are not saying n is necessarily equal to m , right? So n and m are two different numbers, but the specific thing here is a^n and c^n means that the number of a 's and the number of c 's should match, right? So let's look at the four production rules. This particular grammar has four production rules. S gives or S produces a as c . The second rule is S gives $a b c$. The third rule is b gives small b followed by the capital B . That is the small b is the terminal symbol, capital B is the non-terminal. Then finally the fourth one is b gives the b terminal symbol. That is b is the part of the alphabet for this language. Now this language, as I said, I'll emphasize again that we have an equal number of a 's and c 's with any number of b 's in between. So that's the unique part of this context-free grammar. Now let's look at string derivation. So how do we start with string derivation? Let's say given you have $a a b c c$, this is an example, right? You are given this example, a string, and you are asked, does this belong to this context-free grammar's language or not, right? So that's the question in front of you. So how do you solve it? You start with trying to derive that string using the production rules of the context-free grammar. So you start with S , right, the start symbol, and then S gives you a as c , okay? You apply rule 1 in this case. If you apply rule 1, you get S gives a as c . Why do you

apply rule 1? You apply rule 1 because if you notice this string, right, it has a a b c c. So it has a and c, a in the beginning and c at the end. So applying these production rules, you apply it with the intuition that it will help you derive that string, right? And of course, this can be non-trivial at times, but in this example, if you practice doing deriving these strings, then you will get to understand which rule to apply and when to apply, right? So we are at the second step, right? S gives a as c, then the third step arrives, right? What is the third step? You apply rule 2, right? As per the rule 2, the S will be replaced by a b c, right, a capital B c, right? So what do you get at the end of it? You get a, then in brackets, a capital B c, close bracket, and c, right? So you start seeing some part of the string getting derived. The next step, you start with a followed by a capital B c. And then again, the question is, which rule you should apply? We apply rule 4, right? That is capital B gives small b. So if you notice, if you apply that rule, you get the given string, right? You get a a b c c, then here you are, right? By applying these rules appropriately, you get the target string. So this is, at the end of it, you get a a b c c. And this proves that the given string belongs to the context-free grammar language, because you could apply the rules appropriately and get to that target string. In this fashion, similarly, one can have many other examples of context-free grammar, and it can help you derive strings like this, right? So this is a unique power of context-free grammar. And we went beyond the simple example of a raised to n followed by b raised to n. And we could derive strings of this form, where the complexity of the string is more than what we saw in example 1. So at the end of this, strings of a language described by CFG can be generated, appropriately substituting the non-terminal symbols. Why you are substituting the non-terminal symbols? Because you apply production rules one after another, you choose the production rules, and you help derive the final string, right, the target string. And this was another example we saw. In example 1, we saw one string getting derived. In example 2, we saw another string derived. So we are at the end of this video, and we have seen how to use context-free grammars effectively. Thank you.

Example of CFG's Languages Generated by CFG - 3 Video• . Duration: 8 minutes

[Notes]

Hello everybody, so we are at the module on Context-Free Languages and we are looking at the introduction to Context-Free Grammars and Context-Free Languages. So in this, the topic is another example for Context-Free Grammars Languages, we are at the third example for Context-Free Grammars. Now at the end of this video, we will look at another example of Context-Free Grammar. This grammar is going to have an associated Context-Free Grammar Language. Now compared to the past two videos, where we looked at rather simple examples, this is a slightly more complex example of the use of Context-Free Grammar. So let's dive into it. So just before we look at the example, let's look at the definitions once more, there will be a production rules used to generate all possible strings in a language and that is defining the Context-Free Grammar. We also know the components, there will be terminal symbols, non-terminal symbols, production rules and start symbol. Together, they will create the Context-Free Grammar we need. Now we looked at two examples in the past, this is the third example. So fairly unique example, we will dive into it. But just to get it right for you that the production rules will vary a lot. You might have more number of production rules, complex production rules, but all of them have this property that they will help you derive the string belonging to the language. So let's look at this example.

Language here we are talking about is any string with zeros and ones and turns out this string contains an equal number of zeros and ones. So you can write any string, it is strings of zeros and ones, there will be equal number of zeros and ones, there is no specification of any particular order. This is why this is a tricky example compared to the past. You can derive a Context-Free Grammar that is able to meet this language. So what are the production rules? This particular language will have three production rules. You have S produces $1S0S$, then second rule S producing $0S1S$ and then the third rule S produces an empty string. Now as I said before, language consists of strings that have an equal number of zeros and ones. Using these three production rules, one is able to derive strings, any strings, no matter what the length is. It should have some equal number of zeros and ones, no order specified. So let's take a pause here and think about it. What is so nice about these production rules? So if you notice production rule 1, it starts with S and has 0 after the S it has a 0 . So in some way it is deriving a pattern where you have 1 followed by 0 . Then you have the second production rule, it has 0 producing 1 . So any pattern where you will have 1 followed by the 0 and third of course you have the empty string. So it turns out these two production rules are enough to produce this language. So let's to understand better, let's look at an example. So consider this string 0110 . Does it meet our criteria? Yes. It has equal number of zeros and with ones, right? That's nice. So we have this string which meets our criteria and now the question is, how do we apply the production rules to derive this particular string? So let's start. In the beginning, we have the start symbol, right? Followed by the second step, which is a crucial step. We notice that we apply rule 2, that is S gives $0S1S$. To derive from S , we get $0S11S2$. Notice these notations for S is given just for improved clarity. They are still the non-terminal symbol S , right? Just we are annotating them with numbers just for clarity. Now applying the rule 2, we get some sort of pattern. We have got a string which starts with 0 . We don't know what will happen to $S1$ and $S2$. So let's look at the next step. We apply rule 3, which is empty string, right? S gives empty string. So we apply it to get the first $S1$ disappears, we get $01S2$. That is the first $S1$, it disappears because we have applied rule 3 to it. Now we get $01S2$. The question is, what do we do next? We apply rule 1 on $S2$, right? With that we get $011S30S4$, right? So again, are we closer to the target string? Yes, looks like we are closer to the target string because we managed to get 011 , that is $S30S4$. So we have got the pattern just that we are still left with $S3$ and $S4$. So what happens next? We apply rule 3, right? Rule 3, if we apply, if you notice, if we apply rule 3, $S3$ will disappear and if we apply rule 3 again, right? $S4$ will disappear and we will get our target string 0110 . So with this, we are able to show that appropriately applying the rules, we are able to get a string with equal number of zeros and ones. Now you can, as a homework assignment, you could try deriving different strings of your liking with these three production rules. So what have we learned here, right? So strings of the language derived by a CFG can be generated by applying the production rules. In the next step, you are substituting the non-terminal symbols with the right-hand side of the production rules. Now the choice of production rule is important, right? By making those right choices, you can correctly derive the target string, right? So until now, we have looked at three different examples of CFGs, each with increasing ability. More complex strings can be derived. As we know, the CFG has some kind of memory and it can do some kind of counting, right? With that, even patterns like with zeros and ones, having equal number of zeros and ones anywhere in the string, even this was possible. So we are towards the end of this video now. I hope we have understood how we can use context-free grammar production rules to derive different kinds of strings, okay? So we are at the end of this video. Thank you.

Left Most and Right Most Derivation - 1 Video• . Duration: 8 minutes

[Notes]

Hello everybody. So we are on the module Context-Free Languages. The lesson is Introduction to Context-Free Grammars and Context-Free Languages. We will look at the topic Leftmost vs Rightmost Derivation in Context-Free Grammars. Now at the end of this video, we will understand the difference between leftmost derivation and rightmost derivation in context-free grammar. This is important because so far, we have looked at some of the derivations and sometimes what happens is we have an option, right, we have an option to do the derivation of the target string in a leftmost manner or the rightmost manner. So far, we did not dwell into deeper into this aspect. In this video, we will look at how this difference comes into place and we will look at it in detail through an example. For context-free grammars, as we have seen so far, has production rules that can generate possible strings in a language. Now for this particular video, we will focus on different ways to derive the target string and we will apply the production rules in different order. We may not apply the production rules in the same order. That's the focus of this video. Again, the components, they will be same as before, terminal symbols, non-terminal symbols, production rules, and a start symbol. Now the derivation is the process of applying production rules of a grammar on the start symbol to generate a string. We know this and we have seen it through several examples in the past videos. Now however, the derivation has two types, right, we call it leftmost derivation and rightmost derivation. This difference of leftmost versus rightmost arises in the way that when we look at the target string, we are usually in the process of slowly deriving the target string. Now you could derive the target string from the left-hand side or you can derive target string from the right-hand side. So at any given time, we are focused on the leftmost side or the rightmost side. And this essentially is the difference between the two derivations. Now to illustrate this better, we'll look at a language, right? So we have a language, $a^{2n}b^m$, followed by b^m . What does this mean? So this language represents strings which have an even number of a's followed by zero or more b's, right? So if a's appear, they will appear in pairs, that is an even number of a's. There could be a b or there may not be a b at the end. So here are some five production rules that describes this language. You have $S \rightarrow ab$, right, and followed by rule 2 where $a \rightarrow AA$, right? So the double a's here are the alphabets and the capital A is the non-terminal symbol. You have rule 3, which is $A \rightarrow \epsilon$, rule 4 where $B \rightarrow Bb$, right? You have the capital B followed by the alphabet B. Then the fifth rule, the last rule is $B \rightarrow \epsilon$. So let us try to derive the string AAB, right? So we'll look at this example and we will try to derive the string. Of course, this string will have to look at, given the context of this video, we'll look at it in two different ways as specified earlier. Now what's the first way? The first way is leftmost derivation. The derivation each step, the leftmost variable is the centennial form is replaced, right? So the leftmost variable in the centennial form is replaced at each time step. And we'll look at the rightmost derivation as well. Now let's look at the target string we were considering, that is AAB. How do we derive this? If we follow the leftmost derivation. So we apply rule number 1. So S gives AB, then we apply rule 2 wherein A is replaced by the alphabets AA followed by the non-terminal A, right? So in the target string, if you notice carefully, we have replaced the leftmost capital A by AAA, right? So this is at the end of applying the step, applying rule 2. In the third step, we again apply rule 3, right? So the A will disappear, A after the third, the capital A, right? So AA, the big A, the capital A will get replaced by the empty string. So at the end of it, you get AAB. Then as a next step, we apply rule 4, that is $B \rightarrow Bb$. Here, the leftmost non-terminal symbol is B. So we replace it by capital B followed by small b. So at the end of it, we get AA, capital B, small b, right,

the alphabet. Then as next step, we apply rule 5, which is B derives empty string. So in each step, as you see, we have replaced the leftmost variable to derive the required string. At the end of it, we get AAB and each step, since this is called the leftmost derivation, we looked at the leftmost variable. Now let's try to look at how we can derive AAB with the rightmost derivation. So what happens here, right? In the rightmost derivation, if we look at carefully, the rule 1 is applied, right? From the start symbol, S gives AB. But the next step is where the differences arise. We apply rule 4, right? Rule 4, we take the rightmost variable, right? So AB, we replace B with B, small b, right? So we get AB, small b. Then next step, again we replace the rightmost variable. So B is replaced by the empty string. We go to the next step. Here again, which is the rightmost non-terminal? It is A. So we replace A with rule 2. We replace A with AAA. At the end of it, we get AA, capital E, small b, right? Now we have reached almost the end. We have A replacing the empty string. As a result, we get AAB, right? So we did the same process, right? We applied production rules one after another, but different set of production rules. Then at the end of fifth step, right, we are getting AAB, that is the same strings. We followed a different process. In the earlier slide, we used the leftmost derivation. We kept on replacing the leftmost variable, right, with different production rules. On the other hand here, we have replaced the rightmost variable each time, but at the end of it, with both ways, we were able to derive AAB. So here in each step, the rightmost variable is derived. So what does it prove, right? We can do rightmost or leftmost derivation and we will still get the target string. So as a summary, we looked at leftmost and rightmost derivation and deriving string from production rules of a context-free grammar. And there are different ways, but we are able to show that both leftmost and rightmost can help us derive string. The example is also very interesting because this was a different example than what we saw in the past. It again shows the power of context-free grammar. So with that, we are at the end of this video. Thank you.

Context-Free Languages and Applications Video• . Duration: 7 minutes

[Notes]

Hello everybody, so we are on the module Context-Free Languages and we have been looking at Introduction to Context-Free Grammars and Context-Free Languages. The topic we will look at is Context-Free Languages and some of its applications. At the end of this video, you should get some understanding of the applications of CFGs and the related languages. So far, we have looked at Context-Free Grammars and also looked at the production rules of Context-Free Grammars and how do we derive strings that belong to those languages. We have looked at several examples and progressively we have seen the complexity of strings and the power of CFG that can derive such strings. In this video, however, we will look at some applications, we will talk about and give you a glimpse of the CFG and the related, the applications of CFG and the related languages.

Context-Free Grammars, right? So we have been looking at this and we know that Context-Free Grammar consists of production rules used to generate all possible strings in a language, right? The ability to derive these different strings is very important and therein lies the power of Context-Free Grammars. The applications of CFG is also centered around this power. We have also looked at the components of a CFG that is the terminal symbols, non-terminal symbols, production rules and the start symbol. Together, these gave the necessary power to Context-Free Grammars. In the last few videos, we have looked at several examples of Context-Free Grammars and each of those examples were very unique and through these different kinds of strings and the different kinds of

production rules, one can derive several applications of Context-Free Grammars. So this is what we will look at closely in this video. So we are looking at Context-Free Languages, right? And so far, we have seen the use of Context-Free Grammars and we have said that the associated languages are Context-Free Languages. But let's try to define what are Context-Free Languages. A Context-Free Language is a type of formal language that is generated by the Context-Free Grammar. Okay? Now, how do we define it even better, right? So let's say you have a language L , okay, and it is a Context-Free Language, right? You call it Context-Free basically. When can you call it Context-Free? If there exists a CFG G such that L is equal to L of G , right, where L of G is the language generated by the grammar G , right? So what are we talking here? Basically, the language is Context-Free Language. If you could find a Context-Free Grammar and if the language generated by that grammar is equal to the language itself, right, then you can safely call it Context-Free, right? So it is very much dependent on being able to find a CFG G and the CFG will generate the language and that will be, and if that is same as the language in question, right? So essentially, this brings both the concepts together in a nice way. The second part important here, what is the engine for Context-Free Languages? Pushdown Automata is the engine for Context-Free Languages, just like Automata, Non-Deterministic Finite Automata and Deterministic Finite Automata were the engines for Regular Languages. Similarly, Pushdown Automata is the engine. We are not going to go into details of the Pushdown Automata, but it's important you know about Pushdown Automata. Now moving to the core part of this video, right? We are interested in the applications of Context-Free Languages. Some of the applications of Context-Free Languages for Arithmetic Expressions, Languages of Balanced Parenthesis and Languages of Palindromes. Arithmetic Expressions, you all know what are Arithmetic Expressions, Simple Arithmetic Expressions with Multiplication Addition, they are best processed by Context-Free Languages. You have Balanced Parenthesis that is open bracket, closer bracket, close bracket and different forms of, you know, parentheses can be used and whether Context-Free Grammars can help you identify whether the parentheses are balanced or not. Then the third application is Palindromes, right? What are Palindromes? They are strings, if you read them from the front or from the back, they are essentially the same, right? So they have some kind of symmetry. All these kind of expressions and strings can be best identified with Context-Free Languages. Programming Language Syntax can also be described with Context-Free Languages. That's a very important application. Some other popular applications of Context-Free Languages include CFGs can be used to define syntax of popular programming languages like Java, C, Python, etc. The syntactic rules for compilers are made from Context-Free Grammars. CFGs are used to model the syntax of natural languages as well. And CFLs are used in formal verification to ensure conformity to syntactic rules. So they have application in verification problems as well. The popular XML and HTML documents are also passed with Context-Free Languages. So there is a huge area of applications for Context-Free Languages and you will find them in modern computing in many different places. In summary, the Context-Free Languages are defined by Context-Free Grammars recognized by pushdown automata and used in many areas like programming language design, document passing, etc. Key applications include syntax definition in programming languages and natural language processing. And that is so because, as we have seen so far, the ability to identify, to generate strings or defined strings that will belong to the Context-Free Grammar is one of the central abilities of Context-Free Grammar. It is beyond what is possible with regular languages. And with that power, as we have seen through some of the examples, these, therein lies the origin of applications for the Context-Free Grammar. So we are now at the end of this video. Thank you.

Introduction to Push-Down Automata Video• . Duration: 4 minutes

[Notes]

Hello everybody, we are at the module Context-Free Languages and we are looking at the lesson Introduction to Context-Free Grammars and Context-Free Languages. The topic we will look at here is Introduction to Pushdown Automata. At the end of this video, we will understand just what the basics of pushdown automata. We will not go into details. For the viewers, it is important to at least know what a pushdown automata is. So that's the goal of this video. Pushdown automata are computational models used to recognize context-free languages. PDAs are just like non-deterministic finite automata but have extra component known as a stack. So PDAs are the engines of context-free grammars like what is there in regular languages. Non-deterministic finite automata is also used in PDAs but they have an extra component in the form of a stack. Now how does this stack come to use? The unbounded stack provides additional memory beyond the finite amount available in the control which allows them to recognize some non-regular languages. So what PDAs essentially do is they will do certain operations on the top of the stack and this stack helps them give the additional power which is absent in regular language. As I mentioned earlier, we will not go into details of pushdown automata but it is good to look at at least have some idea what they are. Now every context-free language can be recognized by some PDA. For every context-free grammar, there is an equivalent PDA that can accept the same language and vice versa. So this establishes the equivalence of context-free grammars and pushdown automata i.e. PDAs. Example of languages for PDA can be balanced parenthesis palindrome languages or language L where you have n number of A's followed by n number of B's. These are all examples of some languages that a PDA can recognize. So essentially what a CFG and the PDA are equivalent. In summary, pushdown automata provides a way to recognize context-free languages using a stack. So it's important to understand that the engines behind context-free languages is the pushdown automata and they beyond the engines that are behind regular languages, they have an additional component called stack and thus typically these models will operate on the top of the stack and which will give them some special powers. The special powers may come in terms of being able to count certain things, right? The counting abilities etc. Every context-free language can be recognized by some PDA as I had mentioned earlier and PDAs are crucial in passing programming languages and other formal language systems. We have looked in the previous video, we looked at the applications of context-free grammars. Part of the application is also coming from being able to have PDAs which can do all the processing necessary to enable and equip context-free grammars, right? So this is what, so at the end of it you have to have some idea of what PDAs are and this short video is just for you to understand what essentially is PDA without going into details. So we are now at the end of this video, thank you.

Membership in Languages, Equivalence of Languages, Applications Video• . Duration: 7 minutes

[Notes]

Hello everybody. So we are at the module Context-Free Languages and the lesson we will look at is Decision Algorithms. The topic is about membership in languages, equivalence of languages and

how do applications origin from these relations. So at the end of this lesson, we will understand what is membership. What is membership of a string in a given context-free grammar. Determine if two languages are equivalent and some real-world applications of this. So what is membership? A membership problem asks whether a given string w belongs to a language L . More formally given a string w does it belong in L . So we have seen context-free grammars and the examples of it and we have also seen strings being derived using the production rules of a context-free grammar. All kinds of questions emerge. One of the problems is membership problem. So given a string w , can we say whether that string does it belong to the context-free language. So that is what we are talking about. And it's a very important question. About membership, what we tend to ask is whether, as I said before, we tend to ask is whether a string w belongs to that language. There are multiple approaches to answer this question whether it's a regular language depending on whether it's a regular language or a context-free grammar. Let's look at regular language. Let's say we construct a DFA for the language L and if the DFA reaches an acceptance state on reading a string w , it is then the w is part of L . So we could answer this membership question on a regular language using a DFA. After reading w , the DFA reaches an acceptance state. So we can clearly answer that question. On a context-free grammar, we construct a pushdown automata as has been told before. If on parsing the string w , the automata stack is empty, then we say that w belongs to L . So as I mentioned before, the pushdown automata has a stack apart from the state machine, it also has a stack. We are not going into the details of it. But if the stack is empty at the end of reading w , then we say that the string does belong to L . So this is the solution approach. Now there is another property which is of importance and this is also a kind of relation or a property which people tend to discuss and find out about, which is equivalence. What is an equivalence problem? It asks whether two languages L_1 and L_2 are equivalent or not. More formally, for all w_1 belonging to L_1 and for all w_2 belonging to L_2 , is w_1 equal to w_2 ? That is, all the strings of belonging to L_1 and all the strings belonging to L_2 , are they the same? So the equivalence problem asks whether two languages L_1 and L_2 are equal or not. How do we answer this question? Again, depending on regular languages or context-free grammars, we ask it in different ways. So for regular languages, if we can construct a minimal DFA for L_1 and L_2 , and if they are the same graph, then we say that L_1 and L_2 are the same, because we use the minimal DFAs and if this minimal DFAs are the same graph, then we can be rest assured that the languages are equivalent. What happens in case of context-free grammars? When are they said to be equivalence? The problem is that this particular special problem of the CFG equivalence is undecidable. That is, we cannot formally decide if two context-free grammars are equivalent. So it is, in computer science terms, it is undecidable. We cannot answer that question. So it's a hard problem. Now, having seen membership and equivalence, let's look at what are the applications. So there is the first application, which is formal verification, checking whether certain properties hold for systems. What are these properties? It could be safety properties or termination properties. And this is done by deciding whether behaviors of the system are part of the desired language. So we check for certain properties that hold for system. We do that by deciding whether certain behaviors of the system are part of a desired language. The next property is compiler design. So membership and equivalence checks are crucial in lexical and syntax analysis, ensuring programs are valid and equivalent optimizations are performed. So in lexical and syntax analysis, this kind of relation membership and equivalence becomes important. It ensures that programs are valid. Sometimes it is used to check optimized version of some program with the original program. That's where the equivalence checks come into play. So both for formal verification and compiler design, we see that the membership and equivalence relation may come handy and may have certain applications. For

artificial intelligence, where decision algorithms help in automated reasoning, for example, determining if certain logical formulas are satisfiable or not. Here too, some of these problems can be solved with the help of equivalence and membership relations. And this is also an area where context-free grammar and the whole language theory, which has been the focus of this module, it becomes important. So there are the relations and there are the applications. In summary, we have looked at the usefulness of membership and equivalence relation in languages. We have also tried to understand how to answer questions pertaining to membership and equivalence relation. And we have also seen some applications of CFGs based on those relations. So there are a lot of applications for context-free grammars and these relations, membership and equivalence relation, also come into play when we talk about applications of context-free grammars. And that's also central to the motivation for this course, where languages are playing, have played a big role in new applications in the computing domain and they will continue to do so in the future. So we are at the end of this video. Thank you.

Module Closure (Summary) Video• . Duration: 5 minutes

[Notes]

Hello, everybody. We are at the module context-free languages, and the lesson is introduction to CFL and context-free grammars as well. In this video, we will summarize what we have learned, all the videos so far. At the end of this video, you will be able to get a summary of all the concept taught in the current module. We have learned several new concepts, so let's look at what we have learned so far. Initially, we looked at the limitations of regular languages. Regular languages are mainly inhibited by their lack of memory and inability to count. We have seen that big problem with regular languages. These limitations help us realize the limits of finite automata and use more complex models whenever necessary. That is, what we have done is over time is by looking at context-free grammars as a solution to some of these problems, we have also explored what problems can be solved by context-free grammars. Therein, what languages are within the ambit of context-free grammars. What are context-free grammars? We looked at context-free grammar consists of production rules used to generate all possible strings in a language. It had many components. The terminal symbols, non-terminal symbols, production rules, and a start symbol. Understanding CFGs is crucial for language design, parsing, and computational theory. All in all, what we have looked at, we have looked at the definition of CFG, what CFG is, and what it constitutes. We have looked at each and every part of it, and we have understood that they play a very important role in language design, parsing, and computational theory, which is the basis of computer science and engineering. Now, we also looked at derivation of a CFG. That is, strings of a language described by CFG can be generated by appropriately substituting the terminal symbols with their respective production rules. That is, we looked at derivations we can do with CFG. If we start with the start symbol and apply production rules one after another, the end we are left with all the strings with the alphabet of the language, then essentially we are going through the derivation process. The derivation process is applying, as I said, production rules of the grammar on the start symbol to generate the string. Essentially, the terminals and the non-terminal symbols are used together to derive different strings belonging to CFG. We have also looked in the leftmost derivation versus the rightmost derivation for context-free grammars. We also looked at what exactly is context-free languages. We tried to define what context-free languages in relation to the context-free grammars. They are used in many areas

of programming languages and document parsing. We looked at some of these. This is key application areas. The syntax definition in programming languages and natural language processing. This is where context-free grammars are so useful. They can help define the syntax so precisely. Now, PDAs, push-down automatas, are something that help you recognize context-free languages using a stack. Now, the stack is the additional part which is there, apart from the state machine. Every context-free language can be recognized by some PDA. This is also very important, we can build PDAs, and hence, process context-free language. Now, what about PDAs and context-free grammars? Every context-free language can be recognized by some PDA, which is reassuring because we can build a PDA, and hence, recognize every context-free language. For every CFG, there is a equivalent PDA. The reverse is also true. You can derive a PDA for a context-free grammar. Now this establishes the equivalence of CFGs, and example languages for PDAs we have looked at is balanced parentheses, palindrome languages, and a string where you have a number of a's followed by equal number of b's. We have looked at all these different applications of PDAs as well. In summary, what we have looked in this module is we started with the limitations of context-free grammar, and that's where we discovered it started with the limitations of regular languages, and that's where we started looking at more powerful models. We looked at the definition of context-free grammar and all the production rules around deriving certain strings belonging to the context-free grammar. We also looked at applications of this theory. This is an exciting topic, and it's the basis of many of the language theory and the various applications we see in computer science and engineering. Now, through this series of lectures, we are at the end of this module. I hope we have understood the basis of context-free grammars and languages through all these videos, and a lot more to be learned about context-free grammars and which we will see in the subsequent. Thank all of you, and I hope it was interesting to get introduced to this topic. I wish we'll have more exciting videos and engaging content in the subsequent module so that we can enhance knowledge about language theory even more. Thank you.

What it Means to Verify if $w \in L$ Video• . Duration: 6 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context Tree Grammars and as you know we are looking at formal languages and applications. In that specifically the lesson we will look at is parsing and ambiguity. At the end of this video we will try to understand what membership means. Membership of a string w in a context free grammar. We have talked about membership in the past but here we will focus more in understanding what membership is in more detail. So context free grammars. As you know it consists of production rules which we have seen and it is used to generate all possible strings in a language. We have also seen the components of the context free grammar. They are the terminal symbols, non-terminal symbols, production rules and a start symbol. Right. Together all these make up a context free grammar. Now in this video we will look at specifically the membership problem in context free grammars. So what is the membership problem? So the membership problem asks if a given string w belongs to the language L generated by a context free grammar. Right. So if you are given a string the question you are asking is does it belong to the language generated by the context free grammar. Now let's say you have w belonging to L then w can be derived from the start symbol S of the CFG. Right. So we know we have seen how the derivations have worked where we apply the derivations on the production rules. So if w

can be derived from the start symbol you apply different production rules and you can arrive at w that means w does belong to the language L . Now as we have seen derivation involves applying production rules to the start symbol as I mentioned earlier and it's a series of production rules that are applied. So this helps us understand what the membership problem is. Right. So the question asks if a given string w belongs to the language L generated by a CFG and we have seen a definite process to arrive at this answer. Now going moving further on parsing. Right. We will encounter this term in this video and the subsequent videos a lot. The term called parsing. So parsing describes finding a sequence of productions by which a string w belonging to L is derived. Right. So there is a process of parsing through which we can you know it describes a process of finding a sequence of productions by which we will arrive at w . Parsing essentially the reverse process of derivation. So what do you mean by reverse process? See when we applied derivations we applied production rules and we generated different strings based on the derivation rule. Now we are doing the reverse process. We are saying given a string w how do we figure out what are the right production rules and that's essentially the process of parsing. The membership problem is decidable. It is known that it is decidable for CFGs. What do we mean by decidable? That is there exists an algorithm to verify membership. Right. For any given string w we can answer the membership problem. Membership can be checked using parsing techniques as I mentioned and there can be two types of parsing top down parsing and bottom up parsing. Now let's talk about parse trees. Right. This is an important component within the parsing process. A parse tree as we will see is a graphical representation of a derivation of a string from a CFG. Okay. A parse tree it takes the form of a inverted tree. Right. You start at the root which is the start symbol s we have seen of the CFG. The internal nodes represents the non-terminal symbols and at the leaf of a tree you will have the terminal symbols that will form the string w . Each level of the tree corresponds to an application of a production rule. So essentially we have seen production rules being applied but now what we are talking about is something called as a parse tree. If you apply these production rules you can get a form of a tree structure. Now here is an example of a parse tree. Right. The string $a b a b$ and the grammar being considered is $s \rightarrow a s b \mid s s \mid \epsilon$ gives a $s b$ or s gives $s s$ or else gives the empty string. For this example you see here for the string $a b a b$ one can build a parse tree of this form which has s on the top and s derives two s . Right. Based on the second production rule. And each of those s will generate a $s b$ based on the first production rules and the middle s for each of these will lead to the empty string. So this is how you can derive a $b a b$. Now there is another phenomena called ambiguity in CFG. So what is ambiguity? Is if a string w can be derived in more than one way that is using different sequences of production rules. This will be a big focus because this is where you hit problems with CFG. There can be more than one way to derive the string w and that is where the confusion happens. And through this module we will also look at potential solutions for this problem. This also results in string having multiple parse trees as I mentioned and it is still possible to verify the membership problem. You could still answer the membership problem but you will end up having multiple parse tree. So in summary given the string w belonging L means that w can be derived from the CFG start symbol. There are multiple techniques to implement derivation and parsing as we have seen in the past and the term ambiguity is introduced. What it means is if there are multiple derivations possible then you have ambiguity. However, it does not prevent us to do membership verification. So we are now at the end of this video. Thank you.

Motivation for the Grammar to Specify Languages Unambiguously Video• . Duration: 4 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammars. The lesson we are looking at is Parsing and Ambiguity. At the end of this video, we will understand what are ambiguous grammars. We will also try to understand the definition and the need of unambiguous grammars. So let us look, we talked about ambiguity. Now let's try to define ambiguous context-free grammars. So what is an ambiguous context-free grammar? A grammar is ambiguous if there exists at least one string in the language generated by the grammar that has more than one distinct parse string or derivation. That's what ambiguous CFGs mean. So it necessitates at least one string in the language for which you will have more than one parse string. So that makes a grammar ambiguous. Now let us take an example, this production rules which define a CFG, $S \rightarrow ASB \mid SS \mid \epsilon$ gives rise to the string AAB. There are two distinct parse trees as we will try to understand in more detail in the subsequent videos. Now this is an example of an ambiguous context-free grammar. Now let's take a pause here and try to understand the need for unambiguous context-free grammars. There are multiple needs for unambiguous grammars. Ambiguity can lead to confusion in understanding or processing the structure of the language itself. Right? So all the processing machinery, the understanding of, as we have seen, right, the production rules applying all that, there will be confusion if the grammar is ambiguous. That different derivations can imply different meanings for the same string, right? So if you have different derivations, it may mean different meaning. This is very important for the formal language, right, and for different applications. Ambiguity can also lead to structural problems when designing compilers and parsers for a language. So let's say you are a compiler designer or you are designing a new language, you don't want to have ambiguity because it will lead to structural problems when you implement that language. Then lastly, unambiguous grammars are critical in many applications where the structure of a string must be interpreted in a consistent way. So this also relates to the second point we talked about, right? There are differences in meaning, right? And so that structure of the string must be interpreted. There is difficulty in interpreting the string in a consistent manner. So unambiguous CFG. What is an unambiguous CFG? A grammar is unambiguous if there exists exactly one derivation for each string in the language described by that grammar, right? To keep things simple, it will be best if there is only just one derivation. For example, the grammar $S \rightarrow AS \mid \epsilon$ derives the empty string is unambiguous. We'll try to understand these examples more in the subsequent videos, but this is an example of an unambiguous CFG. Unambiguous grammars have a clear and consistent interpretation of the language structure and are easier to parse, right? So we are very interested in these unambiguous CFGs because it will make our life easy, interpretation of the language structure and the parsing will become simpler. So what we have seen is the grammar would be ambiguous, okay, if there exists multiple parse trees for a single string in the language and it is unambiguous if there is exactly one derivation for each string in the language described by that grammar. Having understood the word ambiguous, we have now seen what grammar, ambiguous grammar or unambiguous grammar means. So now we are at the end of this video. Thank you.

Example 1 - Ambiguous Grammar Video• . Duration: 3 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammars. The lesson is Parsing and Ambiguity. At the end of this video, what we will look at is understanding, what we will get to know is understanding the concept of ambiguous grammars. We have looked at ambiguous grammars so far. In this video, we shall look at specific example of ambiguous grammars. So as we have seen, a grammar is ambiguous if there exists at least one string in the language generated by the grammar that has more than one distinct pastries. So at least one string should be there and we show that there are more than one pastries. That makes a grammar ambiguous. Now here is the example. Let us take the grammar J with the following production rules. Now we will test whether this grammar is ambiguous or not. So let's pick a string. Pick a string aabb and we try to create multiple distinct pastries. Okay. So let's how this works. This is the first pastry. Right. For the string aabb. What we have done here is that this s creates asa. Then again you apply the same rule. It gives asb and the s in the middle, there is an empty string. Right. So this is one for this string. Right. Now let's look at another way the pastry can be built. This is another way to generate a pastry for the same string aabb. Right. What is done here is from s you get two s non-terminals. One of them becomes the empty string and then you continue with that the second s and you do the same as you did in the first example. But at the end of it, you see you have a different pastry than what we saw in the in pastry 1. Right. Clearly both are different. Now the last, that means for the last two slides, we saw that the string aabb can be derived using two different sequences of production rules from grammar G. Right. And we saw two distinct pastries. So the grammar G is ambiguous and it makes it difficult to pass. Now what we have seen here is a concrete example, a grammar with all its production rules and a string. We found a single string here wherein we could derive that string in two different ways with different pastries. All this means is that the grammar is ambiguous. Why is it so? By making two different pastries for the single string in the grammar. We shall look at more examples like this in the future videos. Thank you.

Example 2 - Ambiguous Grammar Video• . Duration: 3 minutes

[Notes]

Hello everybody, we are at the module Ambiguity in Context-Free Grammar and the lesson we are looking at is Parsing and Ambiguity. Through this video, we will learn the concept of ambiguous grammars. We have already seen an example. In this video, we will look at one other example. So what is ambiguous CFGs? We have looked at so far that the grammar is ambiguous if there exists at least one string in the language generated by the grammar that has more than one distinct past string. Right? So we have seen this as the definition and we have seen an example already. Let's dive into another example. So let's say the grammar J. Right? It has these production rules. You have S deriving E. You can also have S deriving S plus S or S can be S cross S and E in turn can be A, B or C. Right? This is the grammar we shall look at. Now we will test whether this grammar is ambiguous or not. So let's look at one string A plus B cross C. We will try to create multiple distinct past trees for this string. So past tree 1. Right? This is the string A plus B cross C with the grammar G. Now what do we do? We start with the start symbol S and we derive S gives S plus S. The first S will generate E followed by A. The second S on the other hand will generate S cross S. The first S here will in turn derive, will lead to a derivation of B while the second one will lead to C. Right? So this is, as you can see clearly, this is one correct past tree for this grammar for the given string. Now let's look at the second past tree. What is happening here? Over here you have S. Unlike in the

earlier past tree, you have S will derive S cross S. Right? And the first S goes on and derives S plus S and subsequently A and B for the individual S through respective Es. The second S will give E and then C. Clearly, this past tree is different from the one we saw earlier. Right? Which indicates that for the same string we were able to derive two different past trees. So again, the last two slides showcase the string A plus B cross C derived using two different sequences of production rules from the same grammar G and we got two distinct past trees. So G therefore is ambiguous, making it difficult to pass. So we have looked at two different examples now. Right? And how we can show that a given grammar is ambiguous and we make two different past tree for that, for a given string S. So all this says that if we can find out a single string and show two different past trees for that same grammar, we can prove that the grammar is ambiguous. So we are at the end of this video. Thank you.

Explanation of Lemma Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammar and the lesson is Passing an Ambiguity. At the end of this video, we will understand how to simplify context-free grammars. We'll see various occasions where we have a chance to simplify this context-free grammar. So simplifying context-free grammars, what are the consequences when we do this and how do you take care of the issues that arise? So simplification of context-free grammars is a process of reducing context-free grammar to an equivalent form. But the equivalent form is far more simpler. What do we do in it? We reduce unnecessary rules and symbols while maintaining the grammar's ability to generate the same language, that is, the strings of the same language. It works as a method to optimize the grammar and it ensures passing efficiency. We are seeking efficiency through simplification, the grammar essentially formally and in terms of its semantic meaning, it stays the same. Now let's look at Lemma 1. So various lemmas we will look at. One of them is Lemma number 1. In this lemma, the key focus is the substitution rule. So what is the substitution rule? Consider this example of a context-free grammar. In this, we see a production rule A gives X, B, Y, where A and B are non-terminals. And we have another set of production rules involving B. Now we can create a new grammar. We can apply this production rule and create a new grammar where we delete this production A gives X, B, Y, we essentially delete this production and instead modify the second production which was there, B gives G1, G2, etc. That we replace by, we get the final production, that is, A gives X, G1, Y, X, G2, Y, etc. So essentially, we have rewritten these two productions with one production, that is the substitution rule. Then the second lemma is Lemma 2, which involves removing useless symbols. Useless symbols are non-terminals that have either, there are two ways to look at it. They do not generate any terminal strings. So that is one case. The other case is it cannot be reached from the start symbol. So that for this reason, we call them useless. Now given the grammar, let us look at an example of grammar G, that is, S gives A as B or A or empty string and a small a followed by capital A. Now here, the production S gives A plays no role as the reason being A cannot be transformed into a terminal string, thus we can remove this production. So that is why this is called removing useless productions. Now let us look at the third lemma, Lemma 3. It is about removing empty production. Any production of a context-free grammar of the form A gives empty production is called an empty production. Now these productions have been found to be problematic in passing an analysis. So we can, we want to remove this empty

production, like $A \rightarrow \epsilon$ gives empty production, where A is not a start variable. We can do that. How do we do that? Let us look at it. For every occurrence of A on the right-hand side of a rule, add a new rule with that occurrence deleted. So what does this mean? Let us look at that with an example. Let us say $A \rightarrow A$, capital A and empty. Then we can remove essentially the empty production. What we get at the end is $A \rightarrow A$, where we removed that empty production followed by all A followed by capital A . So essentially these two productions do the same task as the earlier production, but we got rid of the empty production. Finally, let us look at lemma 4, which is eliminating unit productions. Unit productions are rules of the form $A \rightarrow B$, where both A and B are non-terminals, okay? Though they do not contribute to the derivation of terminal strings directly and can be simplified. It is obvious, right? They do not contribute to the derivation. So if $A \rightarrow B$ and $B \rightarrow b$, we can directly replace $A \rightarrow B$ with this production $A \rightarrow b$, right? So we have got rid of the capital B . So this is an example of eliminating unit production. So in summary, we saw four ways, right, of how to simplify context-free grammar. Production rules, removing useless symbols, removing empty productions, and eliminating unit productions. For every grammar G , we can arrive at another grammar G' , where we do not have these problematic cases, right, the useless symbols, empty productions, and unit productions. Thus, this video gave you an idea of what simplifying context-free grammars means. And we also looked at specific examples in the form of changes through the production rule, which led us to the same grammar effect as in the original grammar, but got rid of this problematic production. So with that, we are at the end of this video. Thank you.

Example 1 - Eliminating Productions Video • . Duration: 7 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammars and the lesson we are focusing on is Simplifications of Context-Free Grammars. At the end of this video, we will understand how to eliminate productions in order to simplify Context-Free Grammars. So that's been the focus right, understanding how we can simplify Context-Free Grammars. Now so far we have looked at simplification can be done with four different production eliminations right. So there is substitution rules, then there was removing useless symbols, then removing empty productions and eliminating unit productions. Substitution rule was about substituting a production rule in another at the end of it you get fewer production rules. Removing useless symbols was about trying to find out if some symbols are unnecessary. They are not leading to for instance any terminal symbols for instance. Removing empty productions was about eliminating empty productions that is where a non-terminal is directly leading to a empty production. In that case, we are trying to simplify because it helps in parsing and other cases. The lastly we had eliminating unit productions which is about if one non-terminal creates another non-terminal and this might be in the bigger chain of production, this might be unnecessary. So we were trying to eliminate unit productions. Now production elimination as we understand is a crucial step in simplifying grammars and leading to more efficiency in the parsing processes right. So that was our goal. So now let's look at some examples. Consider this grammar right. You have s , a and b involved as non-terminals. You have small a and small b as the terminal symbols. You also have this empty string being mentioned. This grammar does not contain any useless symbols. So we do not worry about that part. Let's look at how we could simplify this. One thing to be noticed is this has empty productions and unit production. We

will apply the process of eliminating productions to simplify this grammar considering that it has unit productions and empty productions. So let's start with empty productions. What are the steps to remove empty productions? Firstly, what you do is identify all non-terminals that can't derive the empty string. Try to identify, look at the grammar and try to identify non-terminals that are leading to empty string right. Then what you do is remove the empty productions and for that you create new rules that account for the absence of non-terminal that generated the empty string right. So again to reiterate, you remove the empty productions okay and how do you do that? By creating new rules right. You create new rules that will account for the absence of the non-terminal that generated the empty production. You have to be careful what non-terminal led to the empty production and accordingly adjust the other rules. So for instance, if you look at this example, A gives small a capital A and empty. We can remove this empty production and adjust other rules right and by adding options that include or exclude A. So the key part here is to focus on this A right which is the non-terminal okay and we look at for each occurrence of A on the right hand side as we will see through the example. Now in the earlier example, if we remove empty production, the grammar will become as follows right. So the start symbol you will continue to have a b followed by b but notice that the empty production in the second what we had for the all the productions related to the non-terminal A have been changed. Now you have A giving a capital A or A. So we got rid of empty production. However, we have to make some adjustments right. So B is giving b b and then b a right. This is also important because if you notice in the start symbol, we have a b. So it is possible that we will have just b as a rule right. So that is why in the production rules related to non-terminal B, we make that adjustment. So we have successfully removed the empty production, A gives the empty string and the grammar has been adjusted to preserve all derivations. So you have to relook at the original grammar and make sure we have not missed any derivations okay. So the next step after you apply the elimination of empty production is eliminating unit production. So in our grammar, the unit production which we should worry about is B gives A. Now this has to be removed right. So we replace B to A by substituting the productions of A into B. So what we mean by this is as follows right. So if you look at the resultant productions, what we have is we had this A gives A, capital A and small a, this set of production with non-terminal starting with A. We have to do adjust the last set of production with B as non-terminal with this A, with this capital non-terminal, the productions with starting with non-terminal A. So if you see what has happened to the productions with non-terminal B, you have B, B, capital B, then you have just the small b, A, A and A that has entered the set of production with starting with non-terminal B right. So we have simplified our original grammar G after removing all empty productions and unit productions. Finally, we have the grammar G as follows right. So this grammar generates the same language but is more concise and efficient. That was our goal. So in summary, to simplify a CFG by applying different production eliminations, what we did is we looked at a case where we applied empty production issue and then we looked at the unit production issue. We went step by step, transformed the grammar, we followed the rule, at the end of it, we got a grammar such that that grammar will create the same strings as the original grammar. So now we are at the end of this video, we will look at one more example in the following video. Thank you.

Example 2 - Eliminating Productions Video• . Duration: 5 minutes

[Notes]

Hello everybody, so we are at the module Ambiguity in Context-Free Grammars and the lesson we are looking at is Simplification of Context-Free Grammars. So far we have looked at one example for the elimination procedures. Now we will look at another example in this video. As we have seen Simplification of Context-Free Grammars, there are four ways to do it. There is the substitution rules, then there is removing useless symbols, then there is removing empty productions and then eliminating unit productions. Production elimination is key for simplifying grammars and it can also be a way to make the parsing more efficient. So this we have already seen. Now let's look at the second example. This example is slightly more involved than the earlier example. So let's try to just understand it and see what are the steps involved. You may work it out on your own to get a better idea of deeper idea of the steps involved. In this video, I will go over the key parts of this example. So this grammar contains empty productions, unit productions and useless symbols as well. So we apply the process of eliminating productions. Let's do it step by step. Okay, we are removing useless symbols. In this grammar, there is C. C is a useless symbol and we cannot derive a terminal string from it with C. And so we must remove C from the grammar to simplify it. So if you work out the grammar, you will notice that C is useless and it's best to eliminate it. And this is the first step in the simplification process. So after removing C, the grammar will look like this. You no longer have C in the grammar. So this completes the first part. Now the second part as we have seen in the past is empty production. So what we do in it is again to reiterate, we identify all non-terminals that can derive the empty string. Right? This is the key part. And then we remove empty productions by creating new rules that account for the absence of non-terminal that generates the empty string. Right? So these two steps are crucial to remove empty production. The new rules will account for the absence of non-terminal that generates. So we have to keep an eye on the non-terminal that is involved for the empty production. Right? That's the key here. Now, for the given grammar, we remove empty productions and adjust the other rules. In this case, we do it for the non-terminal under question is A. And if we follow the steps as we have seen, then we will get a grammar as shown on your screen. Right? This is the grammar we will get. Right? It looks slightly more elaborate, but it is much more simplified than the earlier, the original grammar. Now if you look at it, the empty productions have been removed. Right? So after removing empty productions, turns out that a new empty production is created, which is for B. So we need to eliminate that empty production as well. If we do that, then we get the grammar as shown on your screen. What's the next step after removing empty production? That's the unit production. Right? So we go after the unit production. In this case, B gives A is an example of such unit production. What we do here? We replace B gives A by substituting the productions of A. So we look at all the productions of A and substitute them in B. Right? This is how we take care of unit production. So if you notice this new grammar, which has come, it has a lot of rules for B. That is because we substituted A's production into B. Right? This is what led to simplification of the grammar. And it may look more elaborate, but it does simplify. So we have simplified our original grammar after removing empty productions and unit productions. We also, in this example, looked at useless symbols. So the final grammar, if you work out, will look something like this. This grammar generates the same language, but is more concise and efficient. Right? So that was our goal. So at the end of it, we have shown with another example, how to go step by step and do production elimination, simplify the grammar, and you could practice doing so for different example grammars. So now we are at the end of this video. Thank you.

What is Normal Form? Why is Normal Form? Video• . Duration: 4 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammar. In this lesson, we will look at normal forms. So by the end of this video, we should get an idea of what normal forms are when we look at them in the context of context-free grammars. We will understand why we need normal forms. So let's look at what normal form is. Normal form refers to the specific structured formats of a context-free grammar where the production rule follows a standardized pattern. So we have looked at context-free grammars so far, but we did not look at what structured formats these production rules should be. So context, the normal form brings some kind of structured nature to the productions that are used in context-free grammar. A context-free grammar is said to be in normal form when every production rule of grammar has some specific form restriction. Now the Chomsky normal form is one of the simplest and most useful normal forms for a context-free grammar. So let's look at what Chomsky normal form is. A context-free grammar is in Chomsky normal form if every rule is of the form $a \rightarrow bc$ or $a \rightarrow \epsilon$. So these two rules, if the context-free grammar has a form in these two rules, then we call it the grammar to be in Chomsky normal form. Now in these two rules, a is a terminal symbol while a , A and B and C are arbitrary non-terminals. b and c may not be the start variable of the grammar. This is something to note. We may also permit s giving an empty string in case language derives the empty string. So the Chomsky normal form exists for a particular context-free grammar when the production rules take this form. There are two rules, $a \rightarrow bc$, $a \rightarrow \epsilon$. If it follows this, then we call it in Chomsky normal form. So why Chomsky normal form? The key advantage of Chomsky normal form is that every derivation of a string of n letters has exactly $2n-1$ steps. As a result of this, one can determine if a string is in the language by exhaustively searching for all the derivations. Every context-free grammar has an equivalent context-free language in the Chomsky normal form. This is also something important. Now why do we use these normal forms? Normalization of CFGs simplifies the structure of a grammar, making it easier to analyze and process. Secondly, more efficient parsing algorithms can be designed for grammars in a normal form. Thirdly, normal forms help in proving properties of languages such as membership in the context-free language classes. And lastly, many applications of context-free grammars such as PDAs or compilers require grammar in the standardized format for processing. At the same time, there are some limitations of normal form. Conversion to a normal form may result in a large number of production rules, increasing the size of the grammar. There are multiple algorithms to convert a context-free grammar to its equivalent normal form. Thus, normal forms are not unique. So there are multiple algorithms to convert from CFG to equivalent normal forms. As a result, the normal forms may not be unique. Converting a grammar to an equivalent normal form may add to its complexity. Thus, we have this set of limitations as well. In summary, normal forms like Chomsky normal form and other normal forms are standardized way to represent context-free grammars. They simplify parsing, proof of properties and allow for easier algorithmic manipulation. Despite these limitations, normal forms are very important. So we have looked at normal forms in this video and we are at the end of it. Thank you.

Motivation with an Example Video• . Duration: 4 minutes

[Notes]

Hello everybody. So we are at the module in context-free grammar and the lesson we are looking at is pumping lemma for context-free languages. So at the end of this video, we will understand why we need a pumping lemma for context-free languages. So let's look at first what pumping lemma means for context-free languages. So pumping lemma for context-free languages provide a necessary condition that every string in a context-free language must satisfy. Remember it's a necessary condition. It is mainly used to prove that certain languages are not context-free, right? So that's the applied part of using pumping lemma. Unlike regular languages, the pumping lemma for context-free languages is more complex due to the nature of their recursive and nested structures, right? So their context-free language's recursive and nested structure is best captured by the pumping lemma. So let's try to understand the pumping lemma a little bit more. Context-free languages are powerful, but they have limitations. The pumping lemma helps to demonstrate those limitations. The lemma shows that certain strings in a context-free language can be pumped in specific ways while staying within the language. Now let's look at the pumping lemma statement in more detail, right? This will give us a formal definition. So let's say there exists an integer p greater than 0 such that for all w , w are the strings belonging to a language L of length greater than or equal to p , then there exists a partition of w which is u, v, w, x, y , right? This is a partition of the w string such that v, w, x , the length of v, w, x is less than or equal to p , the length of v, x is greater than or equal to 1 and for all i greater than or equal to 0, the string u, v raised to i, w, x raised to i, y belongs to L , right? What the last statement means is that string, you could pump essentially v i number of times or x i number of times and it stays, it still belongs to L . Now let's look at some examples. Consider the language L equal to $a^n b^n c^n$, n greater than or equal to 1. This language describes strings with equal number of a 's, b 's and c . Now this language may seem like a context free language, but it is not a context free language, okay? It is very important to understand that this string, this language $a^n b^n c^n$ is not a context free language. In the future video, we will prove, we have given this example to prove that this language is not a context free language and we will look at that in more detail in another video. Now coming back to limitations of pumping lemma for CFL, the pumping lemma for CFL provide a necessary but not a sufficient condition for language to be context free, right? It is just a necessary condition, not a sufficient condition. Thus, it cannot be used to prove that a language is context free. It can be used to prove that a language is not context free. So the pumping lemma for CFL is a powerful tool for providing that certain languages are not context free as I mentioned earlier. Using the lemma, we can analyze nested structure in languages and test accordingly. So we are at the end of this lecture on introduction to pumping lemma for CFLs. Thank you.

Pumping Lemma Statement Video• . Duration: 2 minutes

[Notes]

Hello everybody, we are at the module Ambiguity in Context-Free Grammar. We have been looking at Pumping Lemma for Context-Free Languages. At the end of this video, we will try to understand the Pumping Lemma for Context-Free Languages in little bit more detail. Now Pumping Lemma for Context-Free Languages. As we know, the Pumping Lemma for Context-Free Languages provide necessary condition that every string in a context-free language must satisfy. It is mainly used to prove that certain languages are not context-free as had been mentioned earlier. Context-Free Languages are powerful but they have limitations. The Pumping Lemma helps to demonstrate those

precise limitations. The Lemma shows that certain strings in a context-free language can be pumped in specific ways while staying within the language. Now we looked at this formal definition of context-free language of Pumping Lemma earlier. Let's try to look at it once more. So if there exists a integer p greater than 0 such that for all strings w belonging to L of length greater than or equal to p , there exists a partition of w which is $u v w x y$. In this, the length of $v w x$ is less than or equal to p and the length of $v x$ is greater than or equal to 1. Now for all i greater than or equal to 0, the string $u v$ raised to i $w x$ raised to i y will belong to L . Now v and x are precisely the parts that are pumped. Now let's look at the aspects or components of the Pumping Lemma. You had that p , the pumping length, right? That is the minimum length of strings in the language that can be decomposed and pumped. You had v and x , so here at least one of these substrings must be non-empty and the pumping part, which is repeating or removing v and x , keeping the strings within the language. If you can manage to keep the strings within the language, it is good. If you cannot, that is where you could prove that it is not, the language is not context free. So in summary, the Pumping Lemma for CFL is a powerful tool for proving that certain languages are not context free. This video, along with the other videos, tries to give more information and understanding of the notion of Pumping Lemma for CFLs. We are at the end of this video. Thank you.

Example 1 Video• . Duration: 7 minutes

[Notes]

Hello everybody, we are at the module in context-free grammars and we are looking at the ambiguity in context-free grammars. The lesson we are looking at is pumping lemma for context-free languages. So at the end of this video, we'll understand how to employ pumping lemma for context-free languages. We'll look at an example of application of pumping lemma principle, we'll look at a particular grammar and see how to use the pumping lemma principle. So let's look at the pumping lemma statement for context-free languages. So L be in a CNF form for context-free language. Then the conditions are as follows, there exists an integer p greater than 0 and you will find w , for all w belonging to L of length greater than or equal to p , there exists a partition of w . So you could divide w into a partition of u, v, w, x, y . Such that three things will hold. The size of v, w, x is less than or equal to p , the size of v, x is greater than or equal to 1 and for all i greater than or equal to 0, the string u, v raised to i, w, x raised to i, y will belong to L . The third statement here is the pumping action, right? You change the value of i , you will pump v and x , right, to get different strings and you will try to see if L , if that string still belongs to the language or not. So that's the definition of pumping lemma. So this is a nice example, there is a language L , which is $a^n b^n c^n$, n greater than or equal to 1, right? Actually what this means, this language consists of equal number of a 's, b 's and c 's, one after another, right? So we will use the pumping lemma for CFL to prove that L is not context free. Remember we use pumping lemma to prove that certain languages are not context free. The lemma is not used to prove that certain languages are in the context free realm, it is used instead to prove that L is not context free. So let's look at this example. To apply the pumping lemma, we first assume that L , the definition of L , which is $a^n b^n c^n$ is context free, okay? So we assume that it is so, right? So now why do we assume it? We will see how we start with the assumption, but at the end, if the assumption, if it turns out it is not context free, if this assumption breaks, then we can safely say that it is not context free. So let p be the pumping length given by the lemma, okay? We will choose a

string s from L such that the size of s is greater than or equal to p . So in this example, the first step of pumping lemma is essentially identifying this p , right? You have s will be equal to a raised to p , b raised to p , c raised to p . This string belongs to L and satisfies s greater than or equal to p because it has p occurrences of a 's, b 's, and c 's, right? So now we have this expression s , we are able to write it in the form of p 's. Let's see how we will use it, right? According to pumping lemma, we can split s , right? So we have defined s , now we are going to split s , right? So let's say a raised to p , b raised to p , c raised to p is in this form, u, v, w, x, y , right? Where you will have, as per the pumping lemma, v, w, x , the size of it is less than or equal to p and v, x is greater than or equal to 1, right? Does this hold true here? Yes, it indeed holds true. The v, w, x is less than or equal to p and v, x is greater than or equal to 1. We need to, let's see what that means. This means that v, w, x must occur within the first p , right? We have restricted the size of v, w, x to less than or equal to p . So it must occur within the first p symbols, right? The second, either the first p symbols, that is all a 's or the first p symbols all b 's or the third p symbols all c , but it cannot span across the three symbols, right? Because we have defined it like that. Now we will pump the string, right? This is where the pumping lemma comes in. We increase or decrease the number of occurrences of v and x . So we get a new string. So if we pump b and x , we get this new string. This will either add or remove a 's, b 's or c . In this case, it will add either a 's, b 's or c , right, based on how we defined s . We had defined s . The new string, the important part here is that the new string will no longer have the same number of a 's, b 's and c 's, right? So if we start pumping in that form, you will get into a situation where you would have increased either a or b or c . As a result, they will not be equal, thus violating the structure of the language. What is the definition of the language where you needed equal number of a 's, b 's and c 's, right? So after pumping, the new string does not belong to L because it no longer has equal number of a 's, b 's and c 's. This contradicts the assumption that L is context-free. Remember we started with the assumption that L is context-free, we reached through the pumping lemma, we reached a contradiction, we found that it is not satisfying the lemma, right? Therefore, we conclude that the language L as defined is not context-free language, it cannot be pumped as required by the pumping lemma CFL, right? So this is an excellent example to understand how to apply pumping lemma, right? So we looked at in this video, we learned the pumping lemma for CFL is a powerful tool, right? For proving certain languages are not context-free, right? So the language which was placed in front of us, this language through pumping lemma, we could prove convincingly that it is not context-free. That's how you will use the pumping lemma. And this is an important lemma and will tell you whether a language is indeed context-free or not, right? So that the essential use of the pumping lemma. Thank you.

Example 2 Video• . Duration: 9 minutes

[Notes]

Hello everybody, we are at the module Ambiguity in Context-Free Grammars and the lesson we are looking at is Pumping Lemma for Context-Free Languages. At the end of this video, we will be able to understand the pumping lemma. We are looking at one more example of pumping lemma applied to context-free language. So let's start with the definition of pumping lemma statement. You have a L in the cnf form for a context-free language and there exists an integer p greater than 0. You need to find an integer such that for all w of all the strings belonging to L of length greater than or equal to p , there exists a partition of w which is u, v, w, x, y . So what this says is you should find a p first and

then you should identify w . A w will belong to L and in this case the length of w will be greater than or equal to p . And then we start doing this finding the partitions of w in different ways. The other three conditions are listed below. You have v, w, x should be less than or equal to p . v, x together should be greater than or equal to 1. And the pumping part, the third part is the pumping part where you have for all i greater than or equal to 0. The string u, v raised to i, w, x raised to i, y should belong to L . So in order to apply the pumping lemma, few things should hold. You should be able to identify an integer p . So that's important. Then there will be w and its size should be greater than or equal to p . And what you do essentially is you try to find partitions of w such that of this partition you pick v, w and x that will be less than or equal to p and v, x should be greater than or equal to 1. You set this up like this and then you do the pumping process, the third step. So we have looked already one example. Let's look at another example. This is slightly more involved, right? So consider this language L . It consists of strings w and it has a 's, b 's and c 's and the number of a 's in this language is less than the number of b 's and that's less than number of c 's. So you have a string of a 's, b 's and c 's. What essentially we are focusing on is the number of a 's versus number of b 's versus number of c 's. They are in this particular order. That is the number of a 's less than number of b 's and number of b 's less than number of c . So this is the language. The question is, is this a context free language? So that's the essential question. And we'll use pumping lemma for CFLs to prove that L is not context free. How do we do this? So we apply the pumping lemma, we first assume L is context free, that's the initial step. Then we have to choose a p , right? The pumping length given the lemma. So we choose a string s from L such that string size is greater than or equal to p as necessary as required by pumping lemma. Now here is the important part. You have to identify a s . The expression for s here is a raised to p , b raised to p plus one and c raised to p plus two, right? This string belongs to L and satisfies s . Does it really belong to L ? Yes, right? The number of a 's are less than number of b 's and number of b 's are less than number of c 's. So it does. And it also satisfies that the size of s is greater than equal to p , right? So we are setting up the pumping lemma. Now we can split this expression into u, v, w, x, y , right? Such that u, w, x is less than or equal to p and v, y is greater than or equal to one. So what does this mean? v, w, x must consist of five possible combinations, right? So here is the tricky part. If you set it up like this, there are various ways in which that partitioning can happen. So what are the various ways? So that expression may have all a 's, all b 's, all c 's, a combination of a 's and b 's or a combination of b 's and c 's, right? So you don't know what will be there in v, w, x , right? So since there are so many cases, we will have to elaborate those cases, right? Which we will do shortly. Now we pump the string, right? And we increase or decrease the number of occurrences of v, x , right? We get a new string, right? So we will get this new string, including where we drop, right? It is equal to zero. So we might drop v and x also. So either increase or decrease a 's, b 's and c 's. So let's do a case by case analysis, which is needed for this example. So let's look at one case, right, which consists of v, w, x , consists of only a 's, right? What happens in this case? Pumping the a 's would either increase or decrease the, now we are talking about a 's, right? Remember the expression a raised to p , b raised to p plus 1, c raised to p plus 2, right? So what happens here is a 's would either increase or decrease, right? Since n of a 's must be strictly less than n of b and n of c , right? So what will happen if we pump? Too many a 's will be produced and this may violate the structure of the language. So here you go, right? You have done something, you have done the pumping and it has violated the language. Now case 2, right? v, w, x consists of only b 's. Here again, what can happen is too many b 's or too few b 's can happen, which again can violate the structure of the language. Again, you can work it out what happens for the case of b 's, it can violate the language. Then there are other cases, right? Case 3, where v, w, x

consists of only c's. Then you have case 4, v, w, x consists of a's and b's, right? You can try to work out these cases. And then we have the final case where, which is v, w, x consists of b's and c's only. Now here again, pumping the number of b's and c's will either change the account, right? This may lead to violation of the constraint of the language, right? So you run into various issues. If you do a case by case analysis, you see you will run into issues. You will not be able to maintain that relation. So after the pumping lemma is applied, the new string does not belong to L because it no longer satisfies this condition. This contradicts the assumption that L is context free. Therefore, we conclude that this language is not a context free language and it cannot be pumped as required by the pumping lemma, right? So we have proved, we have gone step by step, we chose p, we chose s, then we did the right partitioning. Then after doing the partitioning, we realized that there are various cases. And then we, case by case, we saw that if we pump, apply the pumping lemma, it might so happen that we start violating the language structure. As a result, we are now saying that this language does not belong to the context free language class, right? So in this video, we looked at another example of a language which through pumping lemma, we could prove that it is not context free. So this again shows the usefulness of the pumping lemma to analyze and to, in order to apply the lemma, however, you have to meet all the criteria, set all the constraints and all the steps set by the pumping lemma as discussed in the earlier part of the video. And this is, this was the second example. I hope you could work out for any given language. You could set the pumping lemma and be able to prove or disprove that the language is context free. So we are now at the end of this video. Thank you.

Example 3 Video• . Duration: 6 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammars and the lesson we are looking at is Pumping Lemma. So at the end of this video, we will understand what Pumping Lemma means in the context of context-free languages. We have already seen a few examples of usage of Pumping Lemma. Let's look at one more. So we have learned this about Pumping Lemma statement. So let me reiterate it again, right? So let L be a CFL for context-free language. Now there exists an integer p greater than 0 such that for all w belonging to L of length greater than or equal to p, there exists a partition. So you search for a partition of w which is u, v, x, y, z. That's the partition you will do for w. And now if the grammar is indeed context-free, then what you will see is the length of vwx less than or equal to p. You will have to find this split, right? You will have to find appropriate subset of that partition vwx less than or equal to p. Then the length of vx is greater than or equal to 1. And then the pumping clause, right? For all i greater than 0, the string u, v, i, w, x, i, y should belong to L, right? So we are saying for all, if you see, if you remember what is said earlier, for all w belonging to L, right? These three things should hold true, right? And no matter what i you try for all i greater than or equal to 0, this condition should hold true. As I said earlier, we looked at a couple of examples earlier. Now let's look at one more example. The example we are looking at is ww, where w belongs to $\{0, 1\}^*$. It has 0s and 1s, right? This language consists of strings that are made up of two consecutive copies of the same binary string, right, w followed by w. Now we will use pumping lemma for CFL to prove that L is not context-free. This particular language is not context-free. We need to prove that through pumping lemma. So to apply pumping lemma, we first assume L is context-free as usual. We find, let p be the pumping lemma length, right? The key part

here is to figure out a string s such that length of s is greater than or equal to p , right? That is what s should hold. And we have to arrive at some expression for s , right? The nice expression for s is $0^p 1^p 0^p 1^p$. So 0^p times 1^p times followed by again 0^p times and then 1^p times. This string belongs to L , we know that, right? It does meet the criteria and it also satisfies the criteria in terms of length. Now according to pumping lemma, if we split this string, right, 0^p is 1^p followed by 0^p is 1^p . If we split this with this u, v, w, x, y and the two conditions, right? This means v, w, x , right, must consist of three possible combinations. What can happen? Either inside the first block of zeros, right? So where v , the what are these three possible combinations, either inside the first block of this fits into the first block with all zeros or is inside the first two blocks consists of zeros and ones that is $0^p 1^p$ or it straddles somewhere in between. Now each of these cases, the pumping lemma should hold and let's see what happens. Now we pump the string, right? This is again, as per the definition, we increase or decrease the number of occurrences of v and x . This will either add or remove zeros and ones. Let's do a case by case analysis. The first case is when v, w, x consists of only zeros, right? In this case, the pumping zeros would either increase or decrease their count. Now the key part here is if we decrease the count to nil, the string s will violate the structure, right? Remember, if we decrease zero, it will not hold this condition of w, w , where the second w is the second copy of the first, that structure will be broken. So this case will be a problem. The second case, v, w, x consists of zeros and ones, right? In this, if we pump that would either increase or decrease the count, when we increase the count of them both, right? The second half will not mirror the first, right? The first $0^p 1^p$ will not exactly mirror the second. Thus, the string s will violate the structure of the language. Again there is a problem. In the third case, where v, w, x straddles the middle, now your pumping zeros and ones in the middle would either increase or decrease their count, right? When we decrease the count to nil, the string s again will violate the structure of the language. So all these cases, there are problems, right? What does this mean? After pumping, the new string does not belong to L , because it no longer has and contains two similar halves. So if we do apply the pumping lemma, we end up in situations where the two halves are not the same. This contradicts the assumption that L is context free. Therefore, we can conclude that this language is not context free, right? As required by the pumping lemma for CFA. Now we have looked at the third example successfully. So in this video, we looked at pumping lemma for CFL, is a powerful tool for proving that certain languages are not context free, right? So we have now looked at another example and we have seen the applicability of pumping lemma. Again, to apply pumping lemma, certain conditions should hold true. We should choose appropriate s , right? And then there will be addition and deletion of the substrings and we should see if the language still fits in, the structure of the language is still retained or not. If it is not retained, that means this language is not context free. Thank you. So this is the end of the video.

Closed Under Operations Union, Concatenation and Kleene-Closure

Video• . Duration: 4 minutes

[Notes]

Hello everybody, we are at the module Ambiguity in CFGs. Now let's look at the lesson Closure Properties of Context-Free Languages. At the end of this video, we will be able to understand some of the closure properties of CFLs which are important to understand CFLs better and their

properties. Ok. What are closure properties? So a language is closed under an operation, if applying that operation to the language within the class results in another language within the same class, right? So you apply some operation, right, to a language and then whatever language results from it, if it belongs to the same class, then we say it has this good closure property. They help define the stability of context-free languages under operations and help us prove whether or not certain languages belong to the class of context-free language. As we have seen in the past, for instance, we have used pumping lemma to prove that a language is not context-free. However, what closure properties offer is, it can be used to prove that certain languages are context-free, right? If they fit, if they have nice closure properties. So let's look at what closure properties of CFGs indeed hold, right? There are three of them, the union, the concatenation, and the clean star. We will show that these three properties have indeed the closure attribute in them. So the closure under union operation, right? When we do the union operation, we will show that the closure property holds. So let's say there is L_1 and L_2 , these are CFLs, then we do a L_1 union L_2 . That's now we have to see if L_1 union L_2 is a CFL, okay? So what's the proof idea here? Given CFG L_1 and L_2 , we can construct a CFG for L_1 union L_2 . So the idea here is if we can construct a CFG for L_1 union L_2 easily, then that means we can prove that the closure property holds, right? That is, the resultant context-free grammar does belong to the context-free realm, right? For that, we introduce a new symbol, S dash, and two new production rules, right? S dash gives S_1 or S_2 , where S_1 is the start symbol for L_1 , and S_2 is the start symbol for L_2 . With these simple production rules in place, we have now created a new CFG, right? This is why we are saying that L_1 union L_2 , the closure property holds. Now, what about concatenation, right? If we concatenate L_1 and L_2 , it is also a CFL, right? So let's look at the proof idea here. So given a CFG for L_1 and L_2 , we can construct a CFG for L_1 concatenation L_2 , right? So what do we do here? We modify the grammar by making the start symbols S dash, right? Generate S_1 , S_2 , right, where S_1 and S_2 are the start symbols for L_1 and L_2 . So we do the start symbols, we simply concatenate them, and here you are. We get a closure property, we get new set of production rules for the new language, which also falls under as a context-free language. Now look at clean start. For clean start 2, what we do is, this also has the closure property, we modify the start symbol S dash to produce S repeatedly, right? In this way, we can get S one or more number of times, right? With this simple proof idea, we can prove that closure holds under clean star as well, right? So in this video, we have learned that closure property allows us to show that new languages generated through this operation remain in the CFL class, right? And CFLs are closed under union, concatenation, and clean star, right? So these are nice properties to have. This enables us to analyze context-free languages more deeply. Thank you.

Not Closed Under Intersection and Complementation Video• . Duration: 3 minutes

[Notes]

Hello everybody, we are at the module Ambiguity in Context-Free Grammars and we are looking at closure properties of context-free languages. So at the end of this video we will look at some more on the closure properties of CFLs. So far we have seen closure properties of CFGs earlier. So we have looked at union, concatenation etc. Now we will look at intersection and complementation. These turn out to be not closed under the following operations. CFLs are not closed. Let's look at it in more detail. Non-closure under intersection. If L_1 and L_2 are CFLs then L_1 intersection L_2 is a set

of strings that belong to both L_1 and L_2 . CFLs are not closed under intersection. Turns out they are not closed meaning that the intersection of two CFLs is not guaranteed to be context-free. Now how do we prove this? We will not go into the details of proof but what we will see that we could use pumping lemma for CFLs to demonstrate non-closure for certain sets of languages L_1 and L_2 . We can use possibility of using pumping lemma to demonstrate non-closure exists. So that could be explored. Now non-closure under complementation. What is complementation? Complementation of a language L is the set of strings not in L . So CFLs turns out are not closed under complementation meaning the complement of CFL is not necessarily a CFL. Non-closure under complementation can be proved via De Morgan's law. So that's also something to be explored. So we will just leave at that. Now in summary what we have seen in this video that closure properties allow us to show that languages generated through this operation remain in CFL as we had seen in the past operations. Some of the past operations we have looked at are union and other operations and union also concatenation. Now CFLs are not closed under intersection and complementation. That's what we looked at in this video. There are also other interesting facts like if L_1 is a CFL and L_2 is a regular language then turns out $L_1 \cap L_2$ will be a CFL. So there are many interesting aspects like that under closure properties which will be useful. We have delved into some of that and give you a glimpse of how to analyze closure properties. And as I said there might be more room for understanding these closure properties closely because they help you understand whether certain languages belong to the CFL class or not. So we have looked at some more of the closure properties in this video. We will stop there. Thank you.

Examples Video • Duration: 3 minutes

[Notes]

Hello everybody, so we are at the module Ambiguity in Context-Free Grammar and the lesson we are looking at is Closure Properties of Context-Free Languages. Now at the end of this video, we'll get to know a little bit more about this Closure Properties of Context-Free Languages. We have already seen a few of these properties. Let's dive into understanding a bit more about these properties. So let's take an example. So let's say there is a language L of W . The W are the strings where number of A's in W is equal to number of B's in W and that is equal to number of C's in W . So this is number of A's, B's and C's are the same. Now we can prove that this is not context-free using closure properties. Earlier we had seen using pumping lemma to prove that some language is not context-free. Now we will use closure properties. So let's say suppose L is indeed context-free. Then L intersection another language $A^* B^* C^*$ will give us $A^n B^n C^n$. Because the earlier language which was equal number of A's, B's and C's, if you intersect that with $A^* B^* C^*$, what you get is the specific strings which meet the criteria A and B and C. $A^n B^n C^n$. n times A, n times B, n times C. n greater than or equal to 0. Essentially you get there. Now, this would be also context-free. However, we know that this is not, this language we have seen earlier, we have proved that this $A^n B^n C^n$ is not context-free. We have proved that in the earlier videos. Thus L , the starting language is not context-free. So we have seen how we can use closure properties as well to prove that certain languages are not context-free. So in this video, we have learned and not just this video, in the past videos, we have looked at closure properties that allow us to show new languages generated through these operations remain in the CFL class. So applying those while we use

certain operations, whether the closure properties hold or not is something we have seen. CFLs are closed under union, concatenation and the clean start. This we have seen in the past and CFLs are not closed under intersection and complementation. So the closure properties is something important to understand and because they also help you analyze context-free languages better. We are now at the end of this video. Thank you.

Deciding Whether $L(G)$ is Empty and Infinite Video• . Duration: 4 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammars and the lesson we are looking at is Decidable Properties on Context-Free Languages. So at the end of this video, we will try to understand some of the decidable properties of context-free languages. So what are these decidable properties? A property is said to be decidable if there is an algorithm that can provide a yes no answer for any input in a finite amount of time. So given a system, in this case we are looking at context-free languages, we want to know if certain things, certain properties are decidable or not. The condition is that if we can have an algorithm that can give us a yes no answer, but that algorithm should run in a finite amount of time, in that case we will call it a decidable. Now there are two such properties, one is L of G is empty or L of G is infinite. So we will look at these two properties. Deciding whether L of G is empty. So this is one of the decidable properties. In this case, the context-free grammar G generates an empty language, L of G generates an empty language if no string can be derived from the start symbol. In that case, we call it the empty language. Now we need to decide whether L of G is empty or not. So there is an algorithm for this, a simple algorithm where in the first step you remove all non-generating and non-reachable symbols from the grammar, that is the first step. Then check if the start symbol S is still reachable. Start symbol should be reachable, if not, L of G , if the start symbol is not reachable then L of G is proved to be empty. So that's what the algorithm does. Now let's apply it for some grammar. We have this grammar in front of us. It has two non-terminals S and A , and there are two production, and from that arise some production rules. So let's go step by step. Identifying non-generating or non-reachable symbols. Start A is not reachable, so we can remove it. That's the first step. As a second step, check if S can generate a string. After removing A , S can still derive the empty string, which implies that L of G is not empty. So L of G in this case is not empty as proved by following the two steps. So this is an example of decidable property and we have shown through an example how to answer the decidable property question. Now deciding whether L of G is infinite. A context-free grammar G generates an infinite language if it can produce infinite many distinct strings. So starting with the start symbol, if we can have infinite number of strings, which will come as a result of applying the production rule generating the different strings, then we call that language infinite. The decidable property in this case is asking this question, is L of G infinite or not? The algorithm for this is you convert it. This may not be necessary, but let's say we convert it to CNF form. Now then what we do is you check if there is a cycle in the derivation. A non-terminal can derive a string that includes itself. If there is such a cycle, it is going to be infinite. The other thing to be noticed is if there is a non-terminal that can generate arbitrary long strings, in that case L of G is also infinite. These are the things you check through the algorithm. So at the end of it, we have looked at some of the decidable properties of context-free grammars and context-free languages. And a grammar,

we understood that L of G is, any language L of G is empty by checking if the start symbol can be derived, it can derive any terminal string. We looked at infinite property as well by detecting cycles and checking for non-terminals that can generate arbitrary long strings. Now both properties have practical applications in language design and parsing. So this is how you observe and study decidable properties of CFLs. We have looked at two of such properties. So now we are at the end of this video. Thank you.

Module Closure (Summary) Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module Ambiguity in Context-Free Grammars and the lesson we are looking at is Decidable Properties on CFLs. Now the key topic here is since we are closing the module, we are at the end of this module. In this video, we'll look at the summary, right? So at the end of this video, we'll revisit all the concepts taught so far, right? And we'll try to get a broader perspective on what all we went through as we proceeded through the module. Now Context-Free Grammars, right? That was the key focus. We looked at what it means, what are the different parts of Context-Free Grammars. So a Context-Free Grammar consists of a set of production rules. We saw the production rules and the components involved terminals, non-terminals and a start symbol, right? So Context-Free Grammars define Context-Free Languages and they generate strings for formal languages, right? So we kind of, through Context-Free Grammars, we understood Context-Free Languages. Now the derivation involved applying production rules from the start symbol to obtain a string, right? So we looked at both leftmost derivation and rightmost derivation. These are different strategies for deriving strings and again important for parsing and syntax analysis. Then we also looked at Ambiguity in Context-Free Grammars. Why does this ambiguity arise? We looked at examples of that. So what is ambiguity, right? A Context-Free Grammar is ambiguous if there are more than one derivations for the same string, right? So there is multiple parse strings. Ambiguity must be avoided in parsing to ensure clear syntax structure in languages. So this is something we should avoid and to remove ambiguity, one must simplify the Context-Free Grammars to remove unnecessary complexity, right? As I said, we looked at examples of these situations where ambiguity arise in the form of, for instance, when we have different parse trees for the same string, right? So we looked at this topic as well. Then we looked at Simplification of Context-Free Grammars. There were four methods we looked at. Using useful substitutions, eliminating useless symbols, eliminating empty productions and removing unit production, right? So these are the strategies we followed. We looked at examples as well for Simplification of Context-Free Grammars. Simplified CFGs are easier to work with for proving properties and constructing algorithms. Simplified CFG may take a normal form like the Chomsky normal form, where every production is either of the form $A \rightarrow BC$ or $A \rightarrow a$, right? We looked at this. So in general, simplification will help us to prove properties or construct algorithms better. After that, we looked at Pumping Lemma for CFLs. So what is Pumping Lemma? It provides a technique to prove that certain languages are not context-free. A string in a context-free language can be pumped or broken into parts and certain sections of the string can be repeated without leaving the language, right? So the Pumping Lemma holds for languages, but if it is not holding, then it becomes non-context-free. It is useful to show that certain strings cannot be generated by the CFG as a result. Then we also looked at closure properties of context-free languages, right? Some of them

being union, concatenation, clean star, under which CFS remain within CFS, right? There is a closure property. On the other hand, CFS are not closed under some properties like under some operations such as intersection and complementation, right? We looked at this through some examples. We studied closure properties as well. We looked at decidable properties of context-free languages. For a context-free grammar, if it is decidable that L of G is empty by checking if the start symbol can be derived any terminal string, right? So that is what proves whether some grammar and the associated language is empty or not. This is one decidable property. Then we also looked at the other decidable property that is infinite, whether L of G is infinite or not. This is done by detecting cycles and checking for non-terminals that can generate arbitrary long string, right? We looked at this as well. So in summary, we looked at context-free grammar and associated with it lot of theory and in the form of examples, we looked at how that theory applies for concrete examples. What are the different aspects of context-free languages? Properties of context-free languages, the operations on it, what are the properties that hold true? All that we studied in this module. So we have revisited the important concepts of topics in this particular video, ambiguity, closure properties, etc. So the idea here was to understand context-free grammars in a holistic perspective and get a good sense of that. In the subsequent modules, we will use this theory, you know, to move to topics such as parsing, etc. and gradually towards applications of formal languages. With that, we will end this video. Thank you.

Introduction to Parsing Video• . Duration: 6 minutes

[Notes]

Hello everybody, so we are at the module introduction to parsing and we are looking at the lesson recognizing context-free languages. At the end of this video, you will understand what parsing process is. We will also understand how to apply parsing and where to apply parsing. Now what is parsing? So parsing is the process of analyzing a stream of symbols. This is done either in natural or programming languages and this is done according to the rules of formal grammar. During the parsing process, we check if the input string belongs to the language and provides structure to the input. The parsing while it is done helps provide some kind of structure to the input. Now in terms of applications, parsing is used in compilers, interpreters, natural language processing, and nowadays even in data processing systems. There are two types of parsing, broadly speaking, you have top-down parsing and bottom-up parsing. Let's talk about the top-down parsing first. Top-down parsing, why it is called top-down parsing? Because we start from the root i.e. start symbol and in this process we try to generate the input string by starting from the root symbol. An example of this is recursive descent parsing which we will study in this as we go along the videos for this module. Then there is bottom-up parsing as the name suggests it starts from the input string and attempts to build up to the start symbol. That is why it is called bottom-up. An example of this is shift-reduced parsing. Now let's look at top-down parsing. As I said, works by expanding non-terminals into terminals based on grammar rules. The focus is always to expand non-terminals into terminals. A key example of this is recursive descent parsing. What do we do in recursive descent parsing? A set of recursive procedures that processes the input string. We will call procedures in a recursive fashion. Backtracking can be used but this can be inefficient. So we try to avoid it. So there will be some kind of recursive structure, one procedure calling itself or another procedure things like this to enable recursive descent parsing. Bottom-up parsing. This attempts to reduce the input string to the

start symbols by reversing the production rule. So it is in some sense the opposite of top-down parsing. An example of this is shift-reduced parsing. Now what is the mechanism for shift-reduced parsing? There is shift operator which moves input to the stack and there is reduce which replaces a series of symbols in the stack with a non-terminal according to some production rule. So this is the shift-reduced parsing and is an example of bottom-up parsing. Now we will also come across terms like LL parsing or LR parsing. Let's try to broadly understand the difference between LL vs LR parsing. We will go into details of this in some of the later videos. Now LL means left-to-right and leftmost derivation. This is what the two Ls stands for, left-to-right and leftmost derivation. This falls under top-down parsing. This is easier to implement but it handles a subset of context-free grammars only. Then there is LR parsers. What LR stands for here? Stands for left-to-right and rightmost derivation. Now this is an example of bottom-up parsing. This is more powerful and handles a larger class of grammars. Some examples LR parsers are LR0, SLR, LALR, LR1 etc. So we will go into details of this later. Now coming to the applications of parsing. As we said one of the big applications is compilers and interpreters where source code is parsed for syntax checking or translation. If you are given a program, compilers will use parsers to understand whether your program is correct or not and also to translate your program into machine code. The other big application of parsing is natural language processing where we parse sentences in human languages like English or any other language. And lastly, we have data validation also where parsing is used to interpret files such as XML, JSON etc. This is also emerging as a big application of parsers. So in summary, we understood that parsing, we analyze a set of inputs according to roles of a formal grammar. And there are types of parsing, bottom-up, broadly speaking bottom-up and top-down. And we looked at what are some of the examples under these categories. There are multiple applications of parsing, from compilers to natural language processing etc. And so parsing plays a very important role in computer science and through this module we will try to understand parsing, what are the types of it. We will go in detail into some of them and also little bit we will understand the implementations as well. So we are now at the end of this video. Thank you.

Approaches to Look for Derivation Video• . Duration: 6 minutes

[Notes]

Hello everybody, we are at the module introduction to parsing and the lesson we will look at is parsing itself. At the end of this video, you will understand the types of parsing. We will look at parse trees and understand ambiguous versus unambiguous grammars as well. Let's start with the type of parsing. So there are various approaches to look for derivation. Derivation in formal languages refers to generating a string from a grammar starting with the start symbol and then applying production rules. There are two main approaches to derivation, leftmost derivation or rightmost derivation. So what's the difference between the two? In leftmost derivation, we replace the leftmost non-terminal first. On the other hand, with rightmost derivation, we look at the rightmost non-terminal first. These are the two broad approaches of derivation. We will look at examples of each of them. Now leftmost derivation. So at each step, the leftmost non-terminal in the current string is replaced using a production rule. Ok. So the process goes as follows. You start with the start symbol and at each step apply a production to the leftmost non-terminal. Whatever leftmost non-terminal you see or the current process string so far, you choose the leftmost non-terminal. Continue until only terminal symbols remain. So that's the process for leftmost derivation. So let's

look at an example. So here is a grammar with s , a , and b as non-terminal symbols and small a , small b , the empty string being the terminal symbol. Now we try to do a leftmost derivation for the string a, a, b . This is the string we are trying to do a leftmost derivation. So we start with the start symbol, s gives a, b . Now which is the non-terminal on the leftmost side? It is capital A . Right? So we apply a production rule for capital A and we get small a , capital A , followed by b as it is, what was present up to the previous step. Now this is an example of leftmost, we picked the leftmost non-terminal. We continue this process. Let's say again we scan this current derivation so far and we find a again to be the leftmost non-terminal. So we apply the production rule for a again. We get a, a , small a , a , followed by non-terminal a , then non-terminal b . Remember we are trying to derive the string a, a, b . We continue the process. Now we need only two a 's, we apply the rule a gives empty string and a is the leftmost non-terminal. We are left with a, a , capital B . Which is the leftmost non-terminal now? It is B . So we apply a production rule for B which is B , capital B and thus we get this a, a, b , capital B . Now we have got the string we needed. We apply the empty string derivation for B and we get the resultant string. So this is an example of leftmost derivation. Now let's look at rightmost derivation. Here at each step the rightmost non-terminal in the current string is replaced using a production rule. As before we start with the start symbol, apply a production rule to the rightmost non-terminal and continue until only terminals remain. Let's look at an example. This is a grammar, the same grammar and let's try to do the rightmost derivation for a, a, b, b . How do we do that? We start with S, a, b . Now rightmost non-terminal is capital B . We apply production rule with capital B . We get a, b , small b followed by capital B . We do it again for B and we get a, b, b , capital B . And remember b is the rightmost. Again we replace b by empty string, b is again the rightmost. Then we look at, we scan that derivation so far, which is the rightmost. Now there is no b , there is a left. Although a is on the extreme left, but it becomes eligible for the rightmost derivation. So we replace a by small a , capital A . Right? We apply it one more time, the same rule. And since we want to get a, a, b, b , that's our derivation, we reach at the, almost at the end and we replace capital A by the empty string. Now you see here, we have applied the rightmost derivation and we were able to get this string. So this is an example of rightmost derivation. Now while we have looked at leftmost and rightmost derivation, it's also important to understand that ambiguity may arise in derivation. Okay. So what is ambiguity? A grammar is ambiguous if there exists more than one leftmost or rightmost derivation for the same string. If we are in a situation where more than one leftmost or rightmost derivations are there for the same string, then there is trouble. Ambiguous grammars can lead to different parse trees for the same string, leading to confusion during parsing. So this is an example, like let's say this grammar, E gives E plus E , E cross E , ID . There can be multiple parse trees for the same derivation, right? We will later look at some examples for this, but such grammars can cause problems when we follow the derivation process. So in summary, we understood leftmost and rightmost derivation and its role in parsing. Both approaches provide different perspectives on how a string is generated from a grammar. And this concept is foundational for constructing parse trees and detecting grammar ambiguity, as we will learn more about in the forthcoming videos. So with that, we looked at leftmost and rightmost derivation, which is a stepping stone, as I said, for us to move on in the topic of parsing. With this, we are at the end of this video. Thank you.

How to Write Recursive Descent Parser Video• . Duration: 5 minutes

[Notes]

Hello everybody. So the module is introduction to passing and we are looking at recursive descent passing. At the end of this video, we will try to understand more about the recursive descent passes. We will walk through an example of recursive descent pass. Now we will just try to get an idea of this example. In a later video, we will look at a concrete implementation of recursive descent passing. For now, let's just look at an example. So what's recursive descent passing? We have looked at it. It's a top-down passing technique that we have understood. The other thing is for each non-terminal symbol in the grammar, they correspond to a recursive function in the passer. So there is some sort of mapping between the non-terminal and a function implemented inside a pass. What are the steps of the passing? It goes in these following three steps. Each non-terminal in the grammar is implemented as a separate recursive function. These functions call each other to match production rules against the input tokens. Then, the passer consumes input tokens one by one and applies to the corresponding grammar production until all input tokens are processed. So these are the standard three steps. Let's look at the following example. Through this grammar, you generate a string of any number of a's followed by any number of b's. Now the passing flow will be as follows. You will start with the start symbol, the non-terminal s. You will pass all the a's first by passing the function corresponding to the non-terminal a, followed by passing all the b's using the passing function corresponding to non-terminal b. Then, you match the terminal symbols and handle non-terminals recursively. So how are you able to handle all the a's that you do by matching? You consume the non-terminals and the forthcoming non-terminals, you handle them by calling the functions a and b, respectively, given the terminal symbols and you call them recursively. You stop passing when no more matching characters are found. So this is just giving a general idea of how the passing will work. Let's say this is the input string, a, a, b. Now you will start with s, s we will call a and b. You start with passing for a. Input starts with a. So you match a and recursively call parse a again. Now you are able to parse, you have already consumed one a. Now the input is a, so you match and consume another a. Then you recursively call a. Now this time when you call a again, it does not match a. So you stop passing using the ϵ production, empty string production. Now then you move to parse b. In this case, there is one b, you match it, you recursively call parse b. At the end of it, there is no more input. So you stop passing. You are done with all the symbols. Now what we just did was, some key concepts I will explain in what just now we described. Non-terminals call themselves. So this is one central theme. So each non-terminal a and b were calling itself. Terminals match inputs. Terminal symbols a and b will match the current input character. So there is some sort of consumption happening. And non-terminals can finish passing early and there is no input left to match. So in a way, you call the empty production when you do not encounter the necessary terminal symbol or when you are at the end of the input string. So in summary, we just got a high level idea of how the recursive descent passing might work in the presence of these parse functions for respective non-terminals. Now, the key features are that recursive descent passing involves recursive functions for each grammar rule. It is easy to implement for small grammars and directly follows the grammar structure. The limitation, however, is it cannot handle left recursion. That we will go into details in some of the later videos. And it is not suitable for complex or ambiguous grammars. This gave us a good sense of what recursive descent passing is. We will continue to look at this topic more and we will also look, as I said, at a concrete implementation of recursive descent passing. Now we are at the end of this video. Thank you.

Writing a Recursive Descent Parser in C Video• . Duration: 10 minutes

Hello everybody. So we are at the module Introduction to Passing and we are looking at the lesson Recursive Descent Passing. Now at the end of this video, we will try to understand a concrete implementation of recursive descent passing. This will be done in the C language. So what is recursive descent passing? So we have learnt already that it's a top-down passing technique, where each non-terminal symbol in the grammar corresponds to a recursive function in the parser. So there is some kind of mapping between a recursive function and the non-terminal in the grammar. The parser will work as follows. So each non-terminal in the grammar is implemented as a separate recursive function. These functions call each other to match production rules against the input tokens. And the parser consumes input tokens one by one and applies the corresponding grammar production until all input tokens are processed. So these three steps define how recursive descent passing works. Now in this video, we will look at a concrete example and go through the step-by-step process of how this is done. For that, we will consider this grammar as an example. So we have looked at this grammar in the past as well, where s gives a b and then a gives small a capital A or null symbol and b gives small b capital B or null symbol. So this essentially generates a string with any number of a 's followed by any number of b 's and that is what this gives. Now the code structure. So each non-terminal will have a separate function. So we will look at how the parser's code is organized. For this grammar, our functions will be, there will be a function for the non-terminal a and for this rule and hence for the rule of a . Then for b and one again for the start symbol, right? So we will look at the code organization, the code structure for this implementation. In C, additionally, we'll have to handle the input string, right? For that, we have character consumption and some memory, right? For instance, we have the current character function. This gives the current character in the string. It just manually returns the current character in the string. Then there is the consume function. It consumes the character if it matches, right? Only on the condition it matches and then you will have the pointer for the current position that will also get updated, right? So here is the actual implementation. So you have this C code, right? You have this include various libraries. Then you have a constant input, right? Which is the input character string. Then you have the position, where is the position currently. This gets updated as you process the string and you have two functions, current character which returns the current character which is at the current position and then you have the consume function. What it does is it consumes. So as you see, if the current character matches the character, the input character of the consume function, then position is incremented. That means the character gets consumed, right? So these two functions will be used in the implementation. Then for non-terminal functions, we have three functions for the three non-terminals. For instance, for boolean s , we have a function, then a boolean a , right? That's another function and then boolean b , that's another function. So we have these three functions. What does s do? s will process a , right, by calling the function a and then it will process b and if it successfully processes a followed by b , it will return true. If not, it will return false. Then a , the way a is processed is it tries to make the current character with the terminal a . Because that's what is there in the rule, right, for a . If it is able to match with a , it will consume a and will call itself again, right, return a . Now, if the current character is not a , let's say it's some other character like say empty string or let's say it is some other character like b etc, it will return false. Let's go to function b . Here again, if the current character is equal to b , then it will consume b and it will call itself again, return b . Else, the other condition, if it is not b , then it will return false, right? So these are the functions which will be used by the parser. What does the main do, right, the main function? So we have set up the input as triple a followed by bb , right, this is a test input. Now, if say you call s and s returns true and

current character equal to the end of the string, that means we are checking if we are at the end of the input, then you will print string is valid according to the grammar, else you will print string is invalid according to the grammar. So that means we have shown how the main function is, we have shown how the individual non-terminal functions is and we have also shown how the supporting functions are, right, current character, get consuming the character or checking the current character, right? So this is how the whole code is organized. So now let's look at a try run for input a bb, right? How this will work? Given the code I have already shown, this is what happens when you process a bb, right? Suddenly you have s being called, okay, s in turn will call a. Now a will look for the character small a, right? So it consumes a at position 1, right? Then a will call itself again. Now when a sees b, it will return true. So a, if it sees a, it will, it was consuming a and calling a itself again. In the other case, it was returning true, right? So it returned true. Now since s had called b as well, a b is invoked, right? What b does is, it consumes b, position moves to 2, again a b gets called, the position moves to 3. Then the third time when b is called, at this moment, since b has consumed all the characters, it will return true, right? Because it sees the end of the string. Now when you get to the main, remember if you get to the main function, the s is invoked, s is returning true now, right? So s has already returned true because a and b has done. You will see that the main function, there was s returning true, you are at the end of the string. This is the condition which is met. As a result, we will see that this is a valid string that has been processed. So to reiterate, from starting with s, then moving with a's, a's will consume all the small a's, b's will consume all the small b. And at the end of it, when you will see that you are at the end of the string and a and b have been successfully processed, it is safe to say that the main function will realize that everything has got processed when we are at the end of the string. So we are good. Now if there was any other characters we had encountered, then we would have run into different scenarios where such a flow would have not happened. As a result, you would have discarded that string not to be present in this grammar. Those scenarios can be worked out. This is a good example of a good scenario. So in summary, the recursive descent parser in C was shown to you and the parsing strategy if you have noticed, what it does is there is recursive functions called for each non-terminal. And each function checks for the corresponding production rules. The other thing which is repeatedly being used is empty string is handled by returning when no more characters match. These are the techniques which are used to implement this. For functions, again for s, you will call a and b in sequence. For a, you will consume a's and recursively call itself as I have shown or returns true by encountering an empty string. Similarly, for b, at the end of it, input is valid if s succeeds and the entire string is consumed. That means at the end, you are at the position is end of the string. So with this, we have built a good understanding of how to implement recursive descent parser. Broadly, this is the structure it is following. And the implementation is quite simple for any grammar of this nature. You could implement a recursive descent parser and you can make it work and it will lead to a working implementation. At the end of this video, thank you.

Issues in the Expression Grammar Example Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module introduction to parsing and the lesson we have been looking at is recursive descent parsing. At the end of this video, we will understand why recursive

descent parsers cannot handle ambiguous grammars. So ambiguous grammars is a problem for recursive descent parsing and we'll look at this in detail. So recursive descent parsing, we have looked at this, right? So it's a top-down parsing technique, right? And here, for each non-terminal symbol in the grammar, it will correspond to some kind of recursive function in the parser, right? This is the general structure and it follows some three steps. Each non-terminal in the grammar is implemented as a separate recursive function, okay? So that's the first part. And these functions can call each other to match production rule against the input token. That is something we have seen so far. Finally, the parser consumes input tokens one by one and applies the corresponding grammar production until all input tokens are processed, right? So this is what a recursive descent parsing, the overall structure of recursive descent parsing is. Now let's look at, to understand this ambiguity problem, let's look at an example of a grammar. So here is a grammar where you have E derives, right? E plus E or E cross E or E in brackets, right? Or a number, right? So this is a grammar for general arithmetic expression, right? Now this has some issues, right? One of the issues is operator precedence. This grammar does not enforce operator precedence between addition and multiplication. So this leads to ambiguity in how expressions like 3 plus 4 cross 5 are parsed, right? So let's say you have an expression 3 plus 4 cross 5. Should we evaluate 3 plus 4 first or should we do 4 cross 5 first, right? We don't understand it by looking at that expression. And the grammar should be such, the parsing infrastructure should be such that it should solve this problem or should not have this ambiguity in the first way. Now if there is operator precedence issue for this expression, namely, you will, if you do the parsing first way, you will get 23 as result. If you do it the other way, you will get 35 as result, right? This is a problem. So two valid parse trees and two different results are obtained, right? So this is a great example of the problem in when you do recursive descent parsing and the grammar has this kind of structure as we see here. So apart from operator precedence, there is left recursion problem. The grammar has left recursion in the rules for, right, this particular grammar because let's say you have A gives A plus E and A gives A cross E , right? Now this makes it difficult for recursive descent parsing to handle as they might get stuck in the infinite recursion. We try to expand the leftmost recursion over and over again. So what can happen is when you expand this grammar and try to implement it with recursive descent parsing, E will lead to, the non-terminal E will lead to another non-terminal E and you might get into some kind of infinite recursion loop, right? So more concretely, when parsing E plus E , the parser will continuously call the rule for E on the left-hand side, right, leading to infinite recursion. So this is another problem with this technique. Now lack of associativity. This is another problem. It does not specify associativity for operators. For example, for the expression 3 plus 4 plus 5, it is unclear whether this should be grouped as 3 plus 4 together plus 5 or 3 plus and 4 plus 5 grouped together. So one case it is left associative, the other is right associative, right? So this associativity problem also exists. So the other problem is lack of associativity, right? So let's say you have an expression 3 plus 4 plus 5. Then in one case, you might group 3 plus 4 together and the other case, you may group 4 plus 5 together and then 3, right? Over here, the results are the same, but sometimes you may get different results. This is again an example of ambiguity. So in summary, multiple parse trees can be generated and this result in different evaluations, right? So this is a problem with some grammars. In the context of when we are employing recursive descent parsing, then there is undefined behavior. The grammar does not dictate the intended order of operations or how the operations should be grouped, right? There is some kind of undefined behavior, there is ambiguity, right? And there is parser inefficiency also due to left recursion that the grammar is unsuitable for direct use because it leads to infinite loops. All these are some of the problems that have to be

solved if we want to employ recursive descent parsing effectively, okay? So we have looked at the problems, the following videos will look at some of the solutions to this problem. We are now at the end of this video. Thank you.

Rewriting the Grammar: Eliminate Left Recursion Video • Duration: 4 minutes

[Notes]

Hello everybody, so we are at the module introduction to parsing and we are looking at recursive descent parsing. Now at the end of this video, we will understand how to remove left recursion in ambiguous grammars. As we saw, this is a major problem with using recursive descent parsing and we will look at some ways how we can address this issue. Left recursion. It occurs when a non-terminal in a grammar has a production rule where the non-terminal appears on the left-hand side of its own definition. This causes parser to continuously try to expand the non-terminal indefinitely. This leads to an infinite recursion. So we have looked at this problem and this is a major issue with when we try to use such grammars for recursive descent parsing. Let's look at an example. In the rule, in the following grammar, you have $E \rightarrow E + T \mid T$. Now the non-terminal E is on the left-hand side of its own production, making this a classic case for left recursive. Now the problems with left recursion is that for recursive descent parsers, this causes a problem since the parser may get stuck trying to expand the left recursive loop indefinitely leading to non-termination. So we have looked at this. The parser structure for recursive descent parser is based on you having this recursive call for the non-terminal and if it continues to exercise this issue, it will cause some kind of infinite loop. So steps to remove left recursion. Right? So firstly, you identify left recursive production. So look for rules of the form $A \rightarrow A \alpha$ where α is a string of terminals and non-terminal. Right? Then you introduce a new non-terminal. Define a new non-terminal A' to remove this recursion. Right? Steps to remove left recursion. So you replace the left recursive rules with two new rules. Right? For instance, you put $A \rightarrow \beta A'$. So why you do this? You start with a non-recursive term in the beginning followed by the new non-terminal A' . Right? So A' in turn, you can give as $\alpha A'$. Right? And or the empty string. So this handles the recursive part without left recursion using the new non-terminal. In this way, you can rewrite the grammar. So we are at the step of removing left recursion and we are showing an example. Right? So we looked at the production $E \rightarrow E + A \mid A$ and $A \rightarrow A * A$. Right? Now let's look at how to rewrite this grammar. We introduce a new non-terminal E' to handle this recursion. Now we have the new rules are as follows. Right? $E \rightarrow T E'$ and E' in turn gives $+ T E'$ or $* T E'$ or the null string. T in turn gives open bracket E' close bracket or the number. Right? In this way, we have eliminated left recursion. Now, let's quickly understand how this will help. How the change rules will help. Firstly, E now produces a term T followed by E' . This ensures that we process terms first and then handle additions and multiplication without starting with E again. Okay? So that's the first part. Secondly, E' handles the recursive addition and multiplication operation or it terminates with empty string, meaning no further operation. Right? This is the crucial second part. And thirdly, T handles the basic units of expression, numbers and parenthesize expression. In this way, the new grammar generated to handle left recursion can address all the issues which were present earlier. So at the end of this video, we have looked at left recursion is replaced with right recursion, which is recursive descent parser can handle efficiently.

And the new grammar processes terms and operators sequentially, allowing for correct parsing without infinite recursion. So we are at the end of this video. Thank you.

Rewriting the Grammar: Left Factoring Video• . Duration: 5 minutes

[Notes]

Hello everybody, we are at the module Introduction to Parsing and let's look at Recursive Descent Parsing, some more topics within that parsing category. At the end of this video, we will learn how to perform left factoring on a grammar. This will solve some of the problems in recursive descent parsing. So let's try to understand what left factoring is. So left factoring is a grammar transformation technique used to eliminate ambiguity when two or more productions for a non-terminal begin with the same prefix. So two or more productions for a non-terminal, they will begin with the same prefix. That is when you face this issue of left factoring. In these cases, the parser might struggle to decide which production to apply since there is the same prefix. By factoring out common prefixes, the grammar becomes more suitable for passes, particularly top-down passes like recursive descent pass. Let's look at an example. So you have A gives alpha beta 1 and it also gives alpha beta 2. In this case, both productions begin with alpha, making it hard for the parser to choose between beta 1 or beta 2 without looking ahead. So left factoring rewrites the grammar to factor out the common prefix alpha. Now here are the steps to remove left factoring. Firstly, you have to identify the common prefixes. Look for production rules that share a common prefix or a non-terminal. Factor out the common prefixes. How do you do that? You replace the non-terminal's production rule with a common prefix followed by a new non-terminal. So essentially what you do is, you have to replace the production's rule with a common prefix. Instead of many production rules having that common prefix, you have that common prefix in a production rule. So that will help in a certain way as we will discuss. Create new production rules. The new non-terminal will handle the rest of the alternatives. There will be more production rules needed for the new non-terminal which handles the rest of the alternatives, as we will see. So let's say you have this grammar, right? E gives TE' and E' gives plus TE' or star TE' or null string and T can give open bracket E close bracket or the number. For this particular grammar, we will try to understand what is the left factoring problem for this grammar. So look for common prefixes, right? So in the production E' both plus TE' and star TE' start with common operators plus and star. But they both follow the same form of operator followed by TE'. So the prefix here is the structure operator followed by TE', right? So that's the problem here. You factor out the common structure. We factor out the common structure and replace it with new non-terminal that handles the operators and terms, okay? So here is the new grammar, right? Essentially if you see here, we have E' gives YE', right? And Y in turns give plus T or star T. So we do not have that common prefix problem. Essentially we have that Y, Y will then diversify into two new productions, right? And we have that E' still there which will take care of the rest of the part. So the explanation for this is as follows. So you have E' the production plus TE' and star TE' were factored out into new non-terminal Y, right? which now handles any operator plus or star followed by the TE' term, right? The second part is after the operator and the term E' is called again to process the remaining parts of the expression, right? As I said, E' will remain there still. Y, right? What to do with Y? This non-terminal deals with choosing between plus T and star T, right? The choice of the operator is handled here. Now since the choice of the operator is local, this simplifies the rest of the grammar, right? As a result, any code structure that has to process this Y, it

is again simplified and we can apply the easy principles of recursive descent passing for this grammar. So in summary, why we apply lab factory? What we mean by that is there are common prefixes in the grammar are factored out, right? So it makes it more efficient and easier for the passer to decide which production rule to use. The resulting grammar is non-ambiguous and works well with top-down passes like recursive descent passes. So now we have looked at, earlier we had looked at lab recursion problem, now we have looked at lab factory. In this way, we have come up with solutions to the structure in the grammar, ambiguity, etc. And how to address those issues and make the grammar applicable for recursive descent passes to work on them, right? So this is essentially what we have learned. We are now at the end of this video, thank you.

Rewriting the Grammar: Non-Terminals in the Grammar Video• .

Duration: 2 minutes

[Notes]

Hello everybody. So we are at the module introduction to passing and we have been looking at recursive descent passing. Now at the end of this video we will also look at how to take care of useless symbols. We have done similar exercises for some of the past modules. We will just refresh some of those ideas and sometimes it might be applicable for recursive descent passing as well. So to identify useless symbols from a grammar we have to do a few things. Firstly, identify non-generating symbols. These are non-terminals that never lead to terminal strings. That is, they can't produce any valid input string. So we have to identify these non-generating symbols. Secondly, we have to identify unreachable symbols. These are non-terminals that never be reached from the start symbol. There is no path to read to them so that is the reason they are useless. So let's look at this grammar. For this grammar in particular, which we have been seeing so far, there is no useless symbol. So this is the grammar we have been looking at. Just let's do a quick pass to see if there are no useless symbols in it, in fact. So because n, there is no non-terminal which is not able to generate anything. So this grammar passes the useless symbol test, right? So in our example, the symbols are generating and no symbol is unreachable. So we are good to go. So useless symbols are addressed in the grammar by looking at non-generating symbols and unreachable symbols. For our grammar, fortunately, we did not run into these issues. But if you run into these issues, we know the steps and we have looked at how to address this in the past modules as well. Now at the end of this video, thank you.

Recursive Descent Parser for Rewritten Expressions Grammar Video• .

Duration: 10 minutes

[Notes]

Hello everybody, so we are at the module introduction to parsing and we are looking at recursive descent parsing. So let's look at recursive descent parsing in more details. At the end of this video, we'll make a recursive descent parser for the corrected grammar. In some of the last videos, we worked on trying to correct the grammar to make it more suitable for recursive descent parsing. Now in this video, we will operate on that corrected grammar. So just as a recap, right, recursive descent parsing. So it consists of a set of mutually recursive functions where each function implements one

of the grammar rules. This is what we have seen so far. These functions attempt to match the input string tokens to the grammar rules, right? And the consumption of tokens happens alongside. Now each non-terminal corresponds to a function in the parser and each terminal is matched directly against the input tokens. This is something we have seen and looked at closely so far. Now this is the corrected grammar. We will look at and examine this corrected grammar and we'll try to build a parser for this corrected grammar. If you notice how this grammar is structured, you have $E \rightarrow T \mid E \text{ dash}$, right? Now T here, if you notice towards the end, T is either in bracket an expression or is a number, right? So T 's role is towards being an expression or a number. Then T is followed by $E \text{ dash}$, right? $E \text{ dash}$ again can give $Y \text{ E dash}$. Now what's the Y there, right? So you started with T and then got $E \text{ dash}$ and $E \text{ dash}$ gives Y . Y gives plus T or star T , right? So there was T followed by Y which in turn gave plus T or star T . So in this way you can create, you can think that how this would, could imagine how this could create some expressions like 3 plus 2, 4 cross 5, things like that, right? So this together, this can create such expression. Also notice that you have $E \text{ dash}$ there, right? What is the role of $E \text{ dash}$? Now $E \text{ dash}$ gives $Y \text{ E dash}$ and $E \text{ dash}$ can be null. So $E \text{ dash}$ can be used to mark the end of that expression, right? In that case. So that's also something very important to understand, right? Then you also have these brackets around T gives to open bracket, close bracket $E \text{ dash}$. In this way you can create nested expression. So let's look at different roles and how the parser is constructed around those roles. So the first rule is $E \rightarrow T \mid E \text{ dash}$, right? Now if we think about this rule, we start by parsing T , right? Because T is in the front. So which is either a parenthesized, T can be either a parenthesized expression or a number. We know this, right? When we see T there, when we start to parse for T , we know it is either a parenthesized expression or a number. So after we successfully parse T , we proceed to parse $E \text{ dash}$, obviously, right? To check if the expression continues. That is, there might be another plus or cross followed by another term, right? So exploration of $E \text{ dash}$ after we parse T is to look for whether the expression continues. By itself, the T , right? The T just stands for opening close bracket expression or T will stand for some number, right? So we have to look for continuation of the expression. That is what the meaning of continuation of expression is. So this part, this rule is clear, right? Let's look at the grammar. So we looked at $A \rightarrow T \text{ E dash}$, right? We have to now slowly start looking at other rules. So the other rule is $E \text{ dash} \rightarrow Y \text{ E dash} \mid \text{null}$, right? So let's try to understand this rule, right? $E \text{ dash}$ can either be a continuation of the expression $Y \text{ E dash}$, right? Or as we have discussed $E \text{ dash}$ before, in case of the parse, the rule we just had discussed, we know that Y is an operator followed by another term, or it can be $E \text{ dash}$ empty, meaning if the expression ends, right? So Y , as you know, again will lead to some plus T or cross T , etc, right? So to parse $E \text{ dash}$, we check if the next token is either plus or cross, right? This is the operator defined by Y , as we understand all the rules. If we proceed to parse Y , right? Followed by another call to $E \text{ dash}$, this will handle multiple operators, right? If the next token, and if the next token, if it turns out, this is what I had emphasized earlier. If the next token turns out not to be plus or cross, then it means $E \text{ dash}$ is null and we can stop there. This gives us the idea of where this will terminate. Now let's proceed, right? There are more rules which have to be explained to fully understand this. Now Y in turn gives plus T or cross T , right? Remember, we had the first rule where E somehow led to $E \text{ dash}$, right? And $E \text{ dash}$ again had led to $E \text{ dash}$ involving $E \text{ dash}$ and Y . Now we are talking about Y . So Y handles the operator part of the expression, right? So it should match either the plus operator or the cross operator followed by another term T . So this is clear. So if the next token is plus or cross, match it and proceed to parse T , right? This allows the parser, so far based on what the rules we have seen, this allows us to parse 5, 3 plus 4, 3 cross 5, things like this,

right? Simple arithmetic operation. Now let's continue. The only thing remaining is T gives opening bracket E, close bracket and the number, right? So T represents a term which can either be a parenthesized expression, that is in brackets E, in which case the parser needs to match the opening parenthesis, then recursively parse the entire expression E and finally the closing parenthesis. So this is one possibility. The other possibility is a number which should match directly as a terminal, right? So 3, 5, etc. So we have looked at all the rules now, right? So we should have a good understanding of how the parser works. Let's just briefly look at the strategy for T. So if the next token is parsed a full expression inside the parenthesis by calling E, then match the closing parenthesis, right? So this is clear and if the next token is a number, we match it directly. So now let's look at a complete example. 3 plus 4 in brackets cross 5. We begin with parsing T. So the next token is opening bracket. So we match this opening bracket and then recursively call A to parse inside the parenthesis. So it should be clear that we are looking at T where that opening bracket 3 plus 4 closing bracket, we are interpreting it as T. Now however inside the T there is a E, right? So we recursively call E to parse inside the parenthesis. Then we reach parsing the T step. Now what we do is parse E dash to see if the expression continues, right? We have seen this, how T and E dash plays a role. The next token is a plus. So what we do is we parse Y, right, to match plus and then parse T, right? Then we parse E dash again. Now we are E dash as there's no more continuation. So E dash will lead to empty string. So we are at the end of this expression. The next token, the closing bracket, right, we have to worry about closing bracket. The closing bracket once it hits after parsing the parenthesized expression, we now parse E dash, right? And the next token is cross, right? So we parse Y again, match star, right, and parse T for the number 5, right? So this continues and we parse E dash again since it is null. So there is no more continuation. As a result, you see through this example, you could work out this example on your own and refer the rules, how the rules come into play, how the different production rules come into play and how the matching happens. If you work out this example, you will see how exactly such expression can be parsed with this grammar and this is an excellent example of employment of recursive descent parsing. So we have looked at recursive descent parsing, how it breaks down each part of the grammar into corresponding recursive function. The parser follows the structure of the grammar, okay, checking the expected tokens and making recursive calls when needed. By handling each non-terminal through function calls, the parser systematically processes expressions and ensures that they conform to the rules of the grammar, right? So we have looked at a corrected grammar in use, how recursive descent parsing comes into play, how the implementation works and we have looked at some examples. I encourage you to work on this example and some other examples also which you can think of and apply the same rules. So now we are at the end of this big example and complete working of recursive descent parsing. Thank you.

Drawbacks of Recursive Parsing Video• . Duration: 5 minutes

[Notes]

Hello everybody, we are at the module Introduction to Parsing and the lesson we are looking at is Recursive Descent Parsing. At the end of this video, we'll understand some of the drawbacks of Recursive Descent Parsing. So far, we have tried to understand what Recursive Descent Parsing was and how to use it, how to implement it, what kind of thing goes on behind the scene in terms of a concrete implementation, how the working of the parser is etc. In this video, we will look at the

drawbacks of Recursive Descent Parsing in specific, some of which we have already encountered so far, but we'll try to put them together in this video and give a more elaborate view of the drawbacks. One of the first problems with Recursive Descent Parsing is left recursion. Recursive Descent Parsers cannot handle left recursive grammars. When it happens, a non-terminal refers to itself as the first symbol in one of its productions. $a \rightarrow a\alpha$ etc. This is the case. What's the impact of this? If there is left recursion, then the parser will enter an infinite recursive loop calling the program to crash because the function for that non-terminal a will keep calling that function. It will happen in loops and this will crash the program. The grammar as part of a solution, the grammar must re-write to eliminate left recursion, which can be cumbersome and may not be straightforward at times. So this is the problem of left recursion. There are other problems as well. Inefficiency with ambiguous grammars. So what's the problem? Recursive Descent Parsers struggle with ambiguous grammars. What happens in ambiguous grammars? There are multiple parse trees possible for a given input. Ambiguity may cause the parser to backtrack extensively and fail altogether. When will the parser backtrack? Say one of these, it will try to explore one of these trees and it will fail. Then it will try to backtrack and explore the other tree. That is what backtracking means in this case. What is the impact of this? It may result in exponential time complexity due to backtracking, leading to very inefficient parsing. What's the solution to this? Ambiguous grammars need to be carefully refactored or avoided. Some of the examples we have seen on this. Now this might not be possible in all situations. The next problem, backtracking overhead. Recursive Descent Parsers often rely on backtracking when they encounter choices in the grammar. For example, if multiple productions are possible, the parser may try one production and backtrack if it fails and then try the next production. What is the impact of this? Excessive backtracking may make parsing inefficient, especially for grammars that have multiple possible paths. This leads to significant performance issues. Now as solutions, look-ahead techniques like LLK parsing, where the parser looks ahead of multiple tokens to decide which production to apply can reduce backtracking but will add a lot of complexity. Now limited to LL1 grammars. Now what's the problem here? Recursive Descent Parsers are typically designed to work with LL1 grammars, meaning they require only one token of look-ahead to make parsing decisions. Right? Only one token of look-ahead. Now many useful programming languages and grammars are not LL1, meaning they require more than one token of look-ahead or have more complex rules. Right? So what's the solution to this problem? The grammar may need to be modified. Right? Left factoring, removing ambiguity, or different parsing strategies like LR Parsers or LLK Parsers may be needed. Right? So we have seen so many of the problems. Let's look at one more. Error Handling. So what is error handling? Error handling is often poor in recursive descent parsing, where a syntax error is encountered, it is not always clear how to recover gracefully and continue parsing. Right? The impact of this is that the parser may stop prematurely after encountering an error or provide unclear error messages, or making debugging and diagnostics difficult for large inputs. Right? So when such impacts tend to occur, it is very difficult, the recursive descent parsers find it very difficult to give a clear indication to the user. Right? And they might provide unclear messages or they may make debugging or diagnostics difficult. So now what's the solution then? Implementing robust error recovery mechanisms like panic mode or phrase level recovery can be done but is challenging. Okay? In case of recursive descent parsing. And this may involve complex logic as well. Now, this means we have seen all these problems, right? And there are ways to handle this problem. Some of those we have addressed through the course of the videos seen so far. So these are the drawbacks of recursive descent parsing and there is always a trade-off involved. Right? You can go for those solutions, but there might be some issues, lingering

issues. All those solutions might be hard to implement. So we have seen this side of recursive descent passing as well. So now we are at the end of this video. Thank you.

Summary Video• . Duration: 4 minutes

[Notes]

Hello everybody, we are at the module introduction to parsing and the lesson we have been looking at is recursive descent parsing. The objective of this video is to revise some of the things we have learned in recursive descent parsing. Right? So let's start with ambiguous grammars. As you know, ambiguous grammars can lead to multiple parse trees and make parsing decisions difficult. It requires backtracking or sometimes manual grammar adjustment. Right? So the issue with that is ambiguity will lead to inefficiency in recursive descent parsing and ambiguous grammars need to be rewritten for clarity as some of the exercises we have done so far. Left recursion. Left recursion is also a problem. It occurs when non-terminal refers to itself as the first symbol in a production. Right? So steps to remove some of the things we have seen is to eliminate left recursion, we introduced new non-terminals and rewrote the grammar. Right? This was shown with a custom grammar. We built some custom grammars. There's left factoring also we discussed. It's used to refactor grammars where two or more productions have common prefixes. Right? Making parsing decisions easier by factoring out the common part. Right? So we worked on left factoring and the whole process around it. Now we discussed for similar grammars where choices start similarly and demonstrated how to rewrite them to avoid ambiguity. Right? So the other problem we looked at retakes slide left factoring. So left factoring. In left factoring we have seen the process. Right? Of how to refactor grammars where two or more production rules have common prefixes. Now after left factoring, making decisions becomes easier and that is through factoring out the common part. Right? So we discussed left factoring and the choices start similarly. That is, once we peel out the common part, then it's much easier to go in different directions. Right? Choose the subsequent part of the strings and the subsequent production rules can easily peel out the choices and this makes handling ambiguity easy. Now useless symbol removal. This is also a process and we have seen in useless symbols we derive string or terminal that can't be reached from the start symbol. Again we have looked at the process. How do we do this? We check if any non-terminals are unreachable or non-generating and remove them. Right? So this is also something we have seen. Then under recursive descent from passing we have something called as custom grammar. A new grammar was created and left recursion was removed and no left factoring was required. So we read the grammar. Right? To address many of these issues. Now we explained the recursive descent process design outlining how each non-terminals process step by step. So once we have a custom grammar which does not have any of the problems mentioned earlier, then we can easily go step by step and apply the principle of recursive descent passing. At the same time, we have also discussed the drawbacks of recursive descent passing, some of which are inability to handle left recursion, backtracking inefficiency, the concept of LL1 requiring look-ahead, lack of weakness in terms of error handling, and not suitable for complex languages etc. In spite of all this, we will say that recursive descent passing is still simple and easy, but the caveat is that it may not work well for all grammars as we have seen. At the same time, we have seen how to address the issues in grammars to work well with recursive descent passing. Right? And of course, while doing that, as we have seen with the issues, the issues we discussed lately on the about recursive descent

passing, it may not be always possible to address the issues like this. It may be difficult. And while addressing it, we may inadvertently increase the complexity. Right? So we are now at the end of this recursive descent parsing concept and implementation videos, and hopefully we will learn more such strategies in the future, in the future modules to come. So, we are at the end of this video and the end of this module. Thank you.

What is LL(1) Parsing? Video• . Duration: 3 minutes

[Notes]

Hello everybody, so we are at the module parsing and we will look at LL1 in specific. The lesson we are looking at is LL1 parsing and we are at the introduction side of the parsing. At the end of this video, we will get an introductory idea of LL1 parsing and why is it used. So let's look at what LL1 parsing is about. First of all, it's a top-down parsing method. So it starts from the start symbol and generates current strings from the rules. So since it is top-down, you start from top to bottom, that is, you start with the start symbol and generate strings from the rules. It is called L. The first L is left-to-right scanning of the input. The second L is leftmost derivation. That means each time you look at the leftmost non-terminal and that is replaced at each time step, followed by 1, right, LL1. What is this 1? It means one symbol of look-ahead, that is, look-ahead refers here to the ability of the parser to peek at one or more upcoming input symbols. So here you are, LL1 parsing and it's a top-down parsing method. We will be looking more into this type of parsing technique. Now top-down parsing as I mentioned, and there are other examples like the recursive descent parsing. We also know there is bottom-up parsing, right? That is, you start from the input string and attempt to build to the start symbol. For that, there are examples such as shift-reduce parsing, etc. Now we'll talk a little bit about the working of LL1. So it is based on context-free grammar and it will use a parsing table to determine which production rule to apply. So this is important and a lot of the focus here is to understand what this parsing table is about. Then the parser also uses a stack, right? So you have the input symbol and you have the top-up stack and you have to determine which production rule to apply. So this is going to be the process for working of LL1. Now there are limitations of LL1 as well, right? It cannot handle what do left recursive grammars do. They create infinite loops in the parsing process. So that's a problem. Then this is limited to one look-ahead symbol, right? So making it difficult to parse certain context-free grammars with ambiguous choices since you are looking at only one look-ahead symbol. Finally, it does not support ambiguous grammars where multiple derivations exist for the same input and making LL1 parsers unsuitable for such cases. So these cases are where multiple derivations, we have looked at some of these, where ambiguity can arise due to multiple derivations. LL1 are not good at this. So we are at the beginning of understanding LL1 grammars, the concept of look-ahead and needs for the predictive parsing table, more of which we will try to understand in the upcoming videos. This is limited to deterministic grammars such as LL1 grammars. We are now at the end of this video. Thank you.

Introducing (Predictive) Parsing Table and Using it to Parse Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module passing with LL1, right? And we are understanding the LL1 passing, we are at the introductory steps of this. At the end of this video, we'll understand the building blocks of LL1 passes. So let's look at the structure of the table we mentioned, right? LL1 will use a predictive parsing table, right? So let's try to understand what this table will be like. So the rows, the table will have rows, which will be for the non-terminals, okay? And we will have columns that will represent the terminals, right? Or the look ahead symbol and the end of input symbol as well, right? Now you have the rows and you have the columns. We have to understand what individual entry is for. So each individual entry, right, let's say T A capital A comma small a in the table contains the production rule to apply when the non-terminal A is on the top of the stack and A is the look ahead symbol, right? So this is important to understand. We shall refer to this table when we are doing the process for LL1 and which particular cell to look at is determined by this following rule, that is what production rule to apply, right? When the non-terminal A is on the top of the stack, right? So that corresponding row we will look at. And if since A is the look ahead symbol, we will look at that particular column. So for each non-terminal A, we find a production rule, A gives alpha, right? And for each production A to alpha, we have to do the following. First of all, we have to find A such that A is in the first set of alpha. Then we add A to alpha in the table under A. Then if alpha derives the empty string, something more we have to do, that is add the rule in the columns corresponding to the symbols in the follow set of A. Now what is this first and follow? We will know more about it in the forthcoming videos. Now let's look at a particular example. We have this grammar S gives a small a followed by capital A and A gives B on null string, right? Now here if you see what we have done is we have added S gives A capital A under A, right? Because A is in the first set, right? Then we have added A gives B because B is in the first set, right, of A. And we have also added A gives null string under A. The next step is you parse using the table. We'll look at more examples of creating the table, but let's assume the table is there. Then we use parsing using this predictive parsing table. So what do we do here, right? To parse an input string using the predictive parsing table, we follow these steps. Firstly we initialize the stack containing the start symbol followed by end of input symbol. And then we start the lookahead process, that is examine the input symbols, lookahead, right? So we will have the stack at one end and the table. Of course we have the input string as well. And we use all this to do the LL1 parsing, right? Now to parse an input string using the predictive parsing table, we will look at the top of the stack, right? So we use the predictive parsing table as been described so far. We'll look at two other aspects. The top of the stack can be a non-terminal. In that case, we refer to the parsing table to find the corresponding rule based on the lookahead symbol. Now if it is a terminal, if the top of the stack is a terminal, we match it with the lookahead symbol and move forward in the input, right? So this is regarding the top of the stack based on what is there at the top of the stack. Then there is apply rule on match. Sometimes we replace the top of the stack with the right-hand side of the production, okay? Or we pop the terminal if it matches the lookahead, right? So these are some of the actions we do. Now we have tried to understand what goes on in some of the actions we will do in the predictive parsing table, which is central to functioning of LL1 parsers. So in summary, we have looked at what role the predictive parsing table will play, right? Which is central to LL1 parsing. We are guided by the parser to apply the correct production rule, right? Based on the lookahead symbol, etc, right? Then we also understand that parsing proceeds by expanding non-terminals using the table and matching terminals with the input symbol, right? We have seen the role of the stack and also the role of the table, right? So this is how the parsing will proceed. We will look at more examples and we will also try to understand how to build this parsing table itself, okay? So we are at the end of this video.

Thank you.

Construction of Nullable, First and Follow Sets Video• . Duration: 4 minutes

[Notes]

So, hello everybody, we are at the module parsing for LL1. And in this video, we'll look at the lesson nullable first and follow tables, right? So this is would be the focus. At the end of this video, we'll get an idea of what this first follow nullable sets are and how to build them. So let's try to understand the need for these sets, right? Predictive parsing relies on parsing tables, we have looked at, tried to understand what these parsing tables are. Now LL1 parsers use first and follow sets for non backtracking parsing, right? We don't want to do backtracking. So these first and follow sets are very important. Nullable sets are used to determine if a symbol can derive the empty string or not. So you have the first, then you have the follow set and you have the nullable set. What is nullable set, right? So we'll look at the definition of nullable set. So for instance, if there is a non terminal x , we call it nullable if it can derive the empty string, okay? And this helps in deriving in determining whether other rules need to be considered in case of empty production. Now the algorithm for this is as follows. If x directly derives the empty string, then we can mark x as nullable. If x gives y_1, y_2 so forth as y_n and all y_i s are nullable, then again we can mark x as nullable. Okay, so that's the nullable set. Let's look at first set. The first set of a symbol x , okay, now here x can be terminal, non terminal, or a sequence of symbols. It is the set of terminals that can appear as the first symbols in any string derived from x . So if you are deriving some string from x , the first terminals that appear will be part of the first set. Now about the rules for calculating the first set, terminals. If x is a terminal, then you can include first x equal to x directly. Then if x is a non-terminal, if x is a non-terminal, x give y_1, y_2, y_n , etc. Then the first, you can add first of y_1 among this, okay, to the first of x , okay? If y_1 can derive the empty string, then you can add y_2 as well and so forth, right? So this is how you build the first set. Then there is the empty string itself. If x can derive the empty string, right, directly or indirectly, then you include empty in the first of x . Now the algorithm for first set is hence for a terminal a , first of a equal to a directly. If it is a non-terminal, x , if x derives y_1, y_2, y_n , etc., initially we add y_1 , first of y_1 to first of x directly, but if y_1 is nullable, then we add first of y_2 as well. If x can derive empty string, we add empty string to the first of x , okay? So we looked at nullable, right, and the first set as foundational to constructing the predictive passes. Understanding these sets enable creation of efficient non-backtracking passes. So we looked at the first, the nullable, and the first. And we will look at more of how to create these such sets and their use of it in the forthcoming videos. We are at the end of this slide. Thank you.

Construction Algorithm - Nullable, First and Follow Video• . Duration: 3 minutes

[Notes]

Hello everybody, so we are at the module parsing for LL1 and the lesson we are looking at is all these sets of nullable first and follow. At the end of this video as we have understood so far we will look more at the first follow and the nullable sets which are needed for LL1 parsers. So what are the

need for these sets? So predictive parsing relies on these parsing tables. LL1 parsers use first and follow sets for non-backtracking parsing and nullable sets are used to determine if symbols can derive the empty string. So the first follow and the nullable sets together will help in how the LL1 parsers will process. Now let's focus on the follow set. We have looked at nullable and first now let's look at the follow set. So what is this follow set? A follow set of a non-terminal a is a set of terminals that may appear immediately after a in a valid string derived from the grammar. So if a can be the last symbol in the derivation in that case there will be the end of input symbol the dollar sign included in the follow set as well right. So what we are looking at is a set of terminals that appear immediately after a in a valid string derived from the grammar. Now there are set of rules to attain the follow set. So the start symbol we always include the end of input symbol dollar in the follow set of the start symbol okay that's one of the rules. So now let's look at some of the other production rules. If a gives $\alpha b \beta$ now with such a production we will add first of β excluding the empty string to follow of b . Now if β can derive empty string then we add follow of a to the follow of b . So if we have a production rule of the form a gives $\alpha b a$ in that case also we add follow of a to follow of b . So for the algorithm for the follow set here are the rules. We add the dollar that is end of the string to follow of the start symbol. And for each production a gives $\alpha b \beta$ we add first of β to follow of b . If β can derive the empty string we add follow of a to follow of b . If we have a derivation if we have a production rule of the form a gives $\alpha b a$ we add follow of a to follow of b . These are the steps needs to be followed to arrive at the follow set right. So we have looked at nullable first and follow sets and how to derive them and understanding this sets enable the creation of efficient and non backtracking passes. In some of the forthcoming videos we will see how to employ these sets to complete the LL1 passes. Now we are at the end of this video. Thank you.

Nullable, First and Follow - Example 1 Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module parsing with LL1 as the parser with focus. The lesson we are looking at is nullable first and follow tables. Now at the end of this video, we will learn about first follow and null sets. And so far we have seen how they are built right. And so we will elaborate with some examples how this works. So consider this grammar right. So we'll look at first follow and null set, nullable sets for this grammar. So let's apply the rules for nullable set right. First of all you have this A gives null string right. You have that rule there. So A is nullable because you have this production rule. Similarly B is also nullable because there is a appropriate production of null string with that you have B included in the nullable set. Then you have the production rule for C . There again you have C gives null. So C is also in the nullable set. D similarly is also in the nullable set. Now S gives A, B, C and since A, B, C are nullable, S is also nullable. So we have S, A, B, C, D all included in the nullable set. Now let's apply the rules for first set. A gives A or null right. So we can include A in the first of A right because of the production A gives small a right. Since A is nullable we can also add null to the first set of A . Let's look at B . Similarly we can add small b because of the production B gives B to the first of B . Since B is nullable we can also add the empty string. Now let's look at C . We can add small c right to the first of C since we have that production where C comes right in the beginning. Since C is nullable we can add empty string as well. Let's look at D because we have this production for D we can add small d right and since D is nullable we can add empty string right. Now the first of S depends on A, B and C ok and since A, B and C are nullable we have

a first of S is equal to union of first of A, first of B and first of C. This we are doing because of the production S gives A, B, C right and since A, B, C are nullable. So at the end of it the first of S is A, B, C and null ok. So this way we got the first set. We can look at this first set we got for S, A, B, C and D. Now let's look at the follow set right. We have the rule S gives A, B, C. We can add directly add the dollar sign right to the follow of S because S is the start symbol as per the rules. Now let's look at the follow set ok. So we apply the follow rules. In this we can add the dollar directly to follow of S because S is the start symbol. Then we can take first of B and make it add it to follow of A right. Then we can again take first of C and add it to the follow of B. Now since C is nullable and follow so we can add follow of S to follow of C right. So follow of C is equal to the dollar sign. We can continue like this right. We have follow of A equal to B, C and dollar. Follow of B equal to S and dollar and follow of C equal to dollar. Now let's look at what happens with the production. C gives C, D. We can add follow of C which is dollar to follow of D right. As a result we get follow of D equal to dollar right. Now as a result we have this different follow set right. Right from S which is dollar, for A we have B, C and dollar, for B we have C and dollar and for C and D we have dollar and dollar. So in summary we have applied first, follow and nullable sets towards LL1 passing. We have seen the applications of first, follow and nullable set towards LL1 passing. So we are at the end of this video. Thank you.

Nullable, First and Follow - Example 2 Video• . Duration: 4 minutes

[Notes]

Hello everybody. So we are at the module parsing for LL1 grammars. And the lesson we are focusing on is nullable first and follow tables. We will try to understand how to arrive at this nullable first and follow sets. So at the end of this video we will understand how to build this first follow and nullable sets. So this time we are looking at some other grammar. So here is the grammar in front of you that is S, A, B. These are the non-terminals and you have the production rules. Now let's look at nullable set in the beginning. So you have a production with A. Here as you can see A cannot be nullable as neither production leads to the empty string. Now let's look at B. B is nullable because we have this B gives empty production. Now what about S. S gives A, B. Since A is not nullable but B is, S cannot be nullable. So we have this final nullable set. B is as shown here, B is nullable. S and A are not nullable. Now let's look at the first set. You have A gives small a then capital A or A gives B. What is the first of A. That's the question. Right. You can easily see that you can add A and B, small a and small b because these are the terminals A and B. The two productions as of the two productions of A. Right. So you can easily add the terminals A and B. Now B. Let's look at B. We can make out that C can be included in the first of B and null string as well because of the two productions. Let's look at the first set of S. Right. Now first S depends on first of A. Right. Since you have this production A gives A, A or B you can include first of A in first of S. Right. So in summary we have this final first set S has A, B. A also has A, B and B has C and null string. Now let's look at the follow set. We will apply the follow rules. Right. So the rule says that we can include the end of input marker, the dollar sign to the follow of S since S is a start symbol. Right. So we get the follow of S. Now S gives A, B. We can add first of B to follow of A. Right. Since because of this production. Now we know first of B has C. So C of course goes in the follow of A. Right. Now since B is nullable, we can also add follow of S to follow of A. Hence we get C and the dollar sign. Now let's look at the production B for B. Right. Now we can add D to the follow of B. It directly follows from the production. Right. Secondly, since B can give empty string, we can add follow of S to follow of B as

well. So we get follow of B as D and the dollar sign. Right. Now, as a result, the final follow set is you have S has dollar, A has C and dollar and follow of B has D and dollar. Right. So we have seen now how to derive the first follow and nullable sets. This will be applied in the coming slides and the coming videos towards the LL1 passing. So this gave us one more example of how to apply the first follow and nullable set rules. So we are at the end of this video. Thank you.

Table Construction Algorithm Video• . Duration: 6 minutes

[Notes]

Hello everybody, so we are at the module LL1 passing and the lesson we are looking at is LL1 table construction. So at the end of this video we will be able to create a passing table for LL1 grammars. So let's see how this goes. So given a grammar you have this set of non terminals N, set of terminals T, set of production rules P and a start symbol. We know we have understood this since the beginning. Now the construction rules are as follows. Firstly you compute the first and follow sets. We have seen how to do this. There are rules to create the first and follow set. So we calculate the first and follow sets for all non terminals in the grammar. These sets are used to determine what symbols can appear in a derivation. Then the second step is create an empty table. So we create a table with rows corresponding to each non terminal and columns corresponding to each terminal plus an extra column for the end of input symbol which is the dollar sign. Now how do we fill the table? So we follow the construction rules. For each production A gives alpha in the grammar we have these rules. So for each terminal A belonging to first of alpha, what we do is we add the production A gives alpha to the table entry A comma small a. So this is how we fill the table. So if the current input symbol is A and the top, so we look at the current input symbol in the string and we look at the top of the stack and if that is non terminal A and the input symbol is A, then we apply A gives alpha. This is how that particular entry of the table comes into play. Now also for each production A alpha in the grammar, if empty string belongs to first of alpha, then what do we do? So for each terminal B belonging to follow of A, we add A to alpha to the table entry A comma B. So earlier we talked about the first of alpha and now we are talking about if null empty string belongs to first of alpha, then we have to be worried about the follow of A. So we add an entry A gives alpha to the table A comma B where B belongs to follow of A. Now if dollar belongs to follow of A, then we have to do something more. We add the production rule A to alpha to A comma dollar entry. This allows the parser to accept empty string at the end of the input. So this is what we do for the condition that empty string belongs to first of alpha. Now the construction rules, there may be some issues of conflict. So for instance, if any table A comma small a, the terminal non terminal A comma followed by terminal A contains more than one production, then the grammar is not LL1. So we shouldn't be run into this conflicting situation. LL1 grammars must be conflict free, meaning only one production can be applied for any non terminal and the terminal pair. This is something we will figure out when we try to construct the table. So if such a conflict happens, that means it is not a LL1 grammar and that means either we have to change the grammar itself and adjust it to solve this issue or we cannot, it might happen that we cannot use this passing strategy at all. So let's look at an example. You have your grammar S gives A, B, non terminal A, B, A in turn A and B in turn have their production roots. So let's compute first and follow for this. So first of A is A and the null string, first of B is B and the null string, first of S is A comma B comma null string, follow of S is the dollar, follow of A is B comma dollar and follow of B is dollar and this is because for the reason stated right

there in the slide and we have seen how the different rules apply. Now let's try to fill this table. For S gives A, B, firstly we look at first of A, B. It contains A and B, the terminal A and B. So we place S, A, B in both S comma A and S comma B. Now since B is nullable, we also place S, A, B in S comma dollar. Now secondly we see the production A gives A. Place first of A which contains A, so we place S gives A in the entry for A comma A. Then we have A comma empty string. Since empty string belongs to first of A, we place A gives empty string, we place A gives empty string in as you see we have placed it in both A comma B as well as in A comma dollar. Now moving on we have B gives B. Here we place B gives B in the entry B comma B because of based on first of B. Then finally we have B gives empty string. So for B gives empty string, we place B gives empty string in the entry B comma dollar. So this follows from the rules explained earlier. So we have learned table construction algorithm and we have actually built a table. I suggest you practice this table construction for the example just stated and we shall look at more examples like this through the course. So we are at the end of this process of starting from creating first follow and nullable sets and then following the table construction rules to use that to fill the parsing table which is a very important step for the LL1 parsing. So we are at the end of this video. Thank you.

Parsing a String Using Table Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module parsing with LL1. We have been looking at the lesson LL1 table construction. We will continue to look at this topic. Now at the end of this video we will be able to use the parsing table to parse a string. We will be using the parsing table and the stack. Now let's start with the initial setup. So we have this input AB followed by \$ and we append this \$ symbol to denote the end of the input. The stack we have put capital S \$ that is the start symbol and the end of the input marker. On the right hand side you can see the table, the parsing table constructed which has already been shown how we arrived at this parsing, this particular parsing table. So there is three non-terminals S, A, B in the parsing table and then we have the terminals A, B and \$ and associated with each of these non-terminal and terminal there is some kind of production shown either for instance under A and small a we have A that means A gives A production. Then with capital A and B we have empty string that means A gives empty string. In this form we have these different productions and we will employ, we will use this table in this running example to parse this input AB\$. So let's start. So step one as you know we are the top of the stack was S and the input is A. So since the top of the stack is capital S and the input is A we refer to the table for the entry S, A and we push, we see that it basically refers to the production S gives A, B. So we push A and B into the stack. The new state of the stack is as follows right, A, B, \$. The new input, the input stays as it is. Let's move to step two where the top of the stack now is A and the input is A. So since this is the configuration we refer to the table entry A, small a and which makes a push A into the stack that is what that table entry will guide us. So the new stack becomes A, small a, capital A, capital B, \$ and new input again as it is. Now at step three since there is a rule for parsing that if the input symbol and the top of the stack matches then we pop the terminal if it matches the lookahead and move forward in the input. So as a result we have popping A from the terminal, A from the stack and also from the input. This is as per the parsing rules described earlier. Now we move with step four, there is a top of the stack now is A and the input however is B. So we check for the table indeed there is a entry for capital A, small b according to the table we apply the rule A gives empty string. So here we

replace A with the empty string which means we just pop A from the stack without consuming any input. As a result the new stack becomes B, \$. We proceed further, now top of the stack is B and the input symbol is B. We check for the table under B, capital B, B. According to the table we apply the rule B gives B. So the new stack becomes like this B followed by capital B and \$. We move ahead step six again we encounter a situation where top of the stack is B and input is B which means these are terminals and they match. So in this case we know we have to match and pop from the stack and the input. As a result we are left with new stack with B, \$ and new input is we are left with the end of the string marker. So we proceed to step seven where the top of the stack is B and there is end of string marker the \$. So we check the passing table for B, \$. According to table we apply the rule B gives empty string. So that means we replace B with empty string which means we just pop B from the stack. So we are left with just \$ now. Now we are proceeding with the next step in the passing which is step eight and the top of the stack turns out to be \$ and the input symbol is also \$. So both the top of the stack and the input symbol are the end of input markers which is great news that means the passing has completed successfully. So in this way we have done a step by step operation of passing using the passing table and the stack. The table construction let us build the passes which we have gone through and that table we used in these passing steps and we went to a step by step process different rules applied based on what is there on the top of the stack versus what is there on the input. As a result we have successfully passed this particular input and that is an excellent and very good illustrative example of the passing process. So thanks for watching this video we are end of this step by step passing process.

Code Demonstration for LL(1) in C Video• . Duration: 5 minutes

[Notes]

Hello everybody, so we are at the module LL1 passing and we will look at LL1 table construction. Now so far we have looked at the theory of LL1 passing. How do we construct the null set, follow set, the first set, etc. And how do we create a table, the passing table. We also looked at how do you process the input. Let's say given an input string, how you will use the stack and the table together to pass that input. Now in this video we will look at LL1 passing written in C. This is a simple instantiation for a particular grammar. We will give you an idea of how the real world implementation may look like. So we will do it for this particular grammar where we have non-terminals as A and B. And we have looked at this grammar in the past. So how will the LL1 C code for this grammar look like, let's look at it. So first of all let's look at some code utilities. So we have this input string, then we have the current pointer, and then for each of the non-terminals we have a routine for S, a routine for A, and a routine for B. We will also have a void match function that we will look at more detail what that does. We have an error function which will point out to any error circumstance and there is a match. What does the match do? If the input in the current string is equal to expected, then we move the current pointer plus plus, else if the current input does not match, we raise an error. So this is, we will see how the match function will be used. Let's start with the non-terminal S. We have looked at a similar approach of how to use functions recursively for the recursive descent parsing. In the similar spirit, we are now looking at for LL1 parsers. So if you have S, then to parse for S, you need to invoke the function for A, which will parse the A part of S gives AB, and then B, which will parse the B part of S gives AB. So now let's look at what happens inside the function A. Since there is a rule for A, which says A gives small a, the terminal A, followed by capital A, the non-terminal A.

What we check is we check if the input symbol, the input character in the current position is equal to equal to A. If there is a match, then we print the rule applied for A, which is the relevant rule production, and we do the match. The match will consume the terminal A. Then we recursively parse the next occurrence of A, based on the fact that the production A gives a small a, followed by capital A. So we have to recursively call A. For any other circumstance, else if the current input is B, or if the current input is N, you have this end of string, etc. And for those circumstances, what we do is we print the production A gives empty string production, because that's the relevant production. In case of an else, if we are inside the routine A, if there is any other condition besides this, then we just route outputting error message. Now, as you can notice, the current input can be B as well, which is the follow of A. So input current can be B as well. Now let's look at B. In B, what we do is, if we see B, the small b, the terminal B, we print the production rule B gives B, D, N. We do a match for B, that means we consume B, and again recursively call B. Else, if we are the current input is equal to all end of string markers, then what we do is, we print B gives the empty string. In rest of the cases, it is an error. So what we have seen is a recursive application of A and B, wherein N given the production rules, stated production rule. This is what LL1 parsing will look like as far as a concrete implementation is concerned. Then going back, the last important part here is the main, where we look at some input string is matched. Obviously, we call S and S in turn will call A, B, etc. At the end of the string, that means we add the dollar sign, then we say that the parsing is successful. If not, it is a failure. We have taken ideas from what the LL1 tables, etc, whatever infrastructure we learned from, and some ideas from implementation of a recursive descent, top-down parsing, and we came up with this implementation. We showed a working code for LL1 parser, and you can experiment with such a code to build parsers for different grammars. We did it for a particular example, and I encourage you to try to build parsers for other similar grammars. So we are at the end of this video. Thank you.

Example 2 – LL(1) Table Video• . Duration: 5 minutes

[Notes]

Hello everybody, so we are at the LL1 passing and then we will look at LL1 table construction. So at the end of this video we will look at a more complex LL1 passer. We have already looked at a few examples. Let's look at one more. So here is the grammar we will focus on. Right. There is S, E, E-, T, F etc. These are the non-terminals involved. And we have looked at this grammar in the past as well. So let's just quickly run through how we will proceed to build a LL1 passer for this grammar. So as I said these are the non-terminals. The terminals are also shown. You have the start symbol and you have the empty string. Now the first step is creating the first and follow sets. So you have the first of S which is equal to the first of E which is equal to first of T which is equal to first of F and that is equal to ID and the opening bracket. You have first of E- it is equal to if you derive it you will get it equal to plus and the empty string. First of T- on the other hand is star and the empty string. So if you follow the first creation of first and follow and null sets process, right, here is what you will get for the first sets. Then similarly if you follow the process for follow sets, you will get follow of S as dollar, follow of E as equal to follow of E- which is closing bracket and dollar and follow of T equal to follow of T- which is equal to plus closing bracket and dollar and then follow of F is equal to plus star closing bracket and dollar. Let's look at table construction, right. We initialize the passing table. We create an empty table with non-terminal as row labels and terminal as column label. Now the

question of filling the passing table using the grammar roots, right. So let's say we have this production rule $A \rightarrow \alpha$. Then what we do is for each terminal T belonging to $\text{first}(\alpha)$, right, we and if T is a terminal then we place $A \rightarrow \alpha$ in the table entry for A, T , right. Now it might so happen that null string also belongs to $\text{first}(\alpha)$. In that case for each terminal T belonging to $\text{follow}(A)$, we place $A \rightarrow \alpha$ in the table A, T , right. Now we also place $A \rightarrow \alpha$ in A, ϵ if ϵ is in the first set, right, that is indeed so and ϵ is in the follow set of A , right. So again this is something we have seen, right. So if we follow all this process, right, we will end up getting a table like this of this form where we have the non-terminals in the along the rows and we have the individual terminals along the column, right. So you will get some table like this. So now that we have constructed the table, let's get an idea of how we will use the table, right. For instance, if the top of the stack there is a non-terminal and the current input is ID , then what we do is we use the entry ID in the row for that non-terminal. For example, if E is that non-terminal and the input is ID , then we have to apply the following production rule that is $E \rightarrow TE$, right. So that is how we refer, we refer to the non-terminal, right, we pick the appropriate non-terminal which is at the top of the stack and depending on the current input ID , we refer to the appropriate column, right. If the input symbol is plus and the stack top is E , we would have applied $E \rightarrow TE$, right. Now for epsilon or empty productions, right, $E \rightarrow \epsilon$, for example, $E \rightarrow \epsilon$ gives empty string. We consult, in that case we have to consult the follow set, right, of E and if the current input is in the follow set of E , we can apply that epsilon production. So this is how we will keep processing the input, we manipulate the stack and we apply these appropriate production rules which are there in the table. So as we saw, we have understood a step-by-step process for LL1 passes, right, we went from creating the follow first and nullable sets, then we created a passing table and then we process this input by using a stack and a passing table. So this is how you would go about processing even more complex grammars using LL1 passes. So we are at the end of this video. Thank you.

What are Conflicts in LL(1) Parsing Table? Video• . Duration: 4 minutes

[Notes]

Hello everybody. So we are at the LL1 passing module. The lesson we are looking at now is conflicts in LL1 passing table. Now let's try to understand the points of failure for LL1 grammar and recognize the methods of evaluating. Okay, so we will look at some of the issues with LL1 grammars and LL1 passing in general. Now there is an issue of conflicts, right, in LL1 grammars, right. So there is something called as first future conflict, right, which occurs in LL1 passes when two or more productions for a non-terminal, so there will be some overlapping first sets and two or more productions for a single non-terminal will lead to that common first set. This will lead to uncertainty about which production to be used based on the next input symbol. So this is one of the major problems. Left recursion is also a problem. If a grammar is left recursive, it can lead to infinite loops as we have seen so far in top-down passes, right. This isn't a conflict in the traditional sense, but it prevents successful passing. Now let's talk about conflicts for LL1, specifically the first future conflicts. This occurs when two or more productions for a non-terminal have overlapping first sets. For example, if we have $A \rightarrow \alpha$ and $A \rightarrow \beta$. If both α and β can derive strings starting with the same terminal, in this case when the parser sees that terminal, it won't know which production to choose, right. So this is an example of first or future conflict as it is called. Now, there

is also issues with left recursion. A grammar that is left recursive can lead to infinite loops as we have seen so far. So for example, if A gives A alpha or beta, this type of grammar needs to be transformed to eliminate left recursion to become LL1, okay. So this is also something we have looked, something similar we have looked for recursive descent passing. Same applies for LL1 passing. Now ambiguous grammar. If a grammar can produce the same string in multiple ways, it's considered ambiguous. So an example of that is E gives E plus E or E cross E or ID, right. This grammar does not define clear operator precedence, leading to ambiguity in passing expressions like if you have ID plus ID cross ID, then multiple pass trees can be generated. As a result, this is an ambiguous grammar. Now there are other issues as well, something called as epsilon production. So when a non-terminal can derive the empty string, it can cause conflicts if it can derive it in multiple ways, okay. So if a production can lead to empty string and other production can start with the same terminal, the passer will face difficulty determining which rule to apply, right. So this is about empty string production. There is non-determinism as well occurs when the passer cannot uniquely decide which production to use based on the next input symbol. This usually arises from overlapping first and follow sets leading to multiple valid passing actions, right. So these are all the issues, right. This is the intention of this video is to make you aware of all these issues that come. So at the end of this, we saw multiple failure scenarios for LL1 passing, right. So all these conflict scenarios, epsilon production related scenarios, etc. We just tried to, the attempt was just to introduce to you all these failure scenarios. We are at the end of this video. Thank you.

Resolving Conflicts Video• . Duration: 3 minutes

[Notes]

Hello everybody, so we are at the LL1 passing module and let's look at the lesson Conflicts in LL1 passing table. At the end of this video you will get to understand how to resolve conflicts in LL1 passing. We have seen how conflicts can arise in LL1 passing. In this video we will look at some of the solutions. So there are many ways to resolve conflicts. One of them is refactoring the grammar. That is we convert the grammar to eliminate left recursion and ambiguity. We use refactoring to factor out common prefixes in production as well. This way we can create unique paths for terminals. Yet another way is to use LLK passing. We looked at LL1 passing but there is something called as LLK passing. In this we move to LLK passing where we look ahead more than one symbol to resolve the conflicts. Remember in LL1 we are just one symbol right in front of us. We can also modify operator precedence. We can define rules for operator precedence and associativity to handle ambiguous expressions. Now let's look at left factoring. So whenever there are two or more productions of the same non-terminal and it begins with the same terminal then the passer has a first slash future conflict. This is called first future conflict. What's the solution to this? We can rewrite the grammar so that the common prefixes are factored out allowing the passer to make a clear decision. So left factoring here is an example. Consider this grammar where A gives the terminal A followed by the non-terminal B and then again A gives the terminal A followed by the non-terminal C. As you can see these both productions starts with A. So we can rewrite this where A gives A, A dash and A dash in turn can be can derive B or C. In this way we can address left factoring. The other way is as mentioned earlier is LLK passing. In LLK passing we look ahead one symbol in the input to decide which rule to apply. So that's where LLK limitation comes. In some cases this is insufficient as the next input symbol may not provide enough information to resolve the conflict. So

this is where exactly where LLK passing helps. As a solution we use LLK passing. The passer looks ahead k symbols to resolve the conflict. This can make the passer more powerful but also more complex. Of course it brings some complexity. Then there is refactoring. So the problem is ambiguous or complex grammars can lead to various conflicts in passers. What we can do about this? We can simplify the grammar by reconstructing rules to eliminate ambiguity or overlapping productions. In general this needs introduction of semantic terms and relations to ensure that the ambiguity can be removed correctly. So we looked at many ways to address the limitations of LL1 passers. Refactoring then look ahead etc. So these are some of the ways by which one can address the limitations of LL1 passers. We are at the end of this video. Thank you.

LL(2) and LL(k) Parsers Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module LL1 passing. Let's look at the topic conflicts in LL1 passing table. At the end of this video, we'll get an idea how to create more powerful passes, right? So we move from LL1 passes to LLK style passes, right? So let's try to understand why this is needed and what are the issues involved. So what is LL1 passing, right? The L stands for left to right scan of input. Then the second L stands for leftmost derivation of the grammar. And then one stands for one look ahead symbol to decide which production rule to apply. In LL1 passing, what we do is based on the current symbol on the stack and the next input symbol, the passer decides how to proceed, right? This limits the grammar types that can be passed, right? Because we are just checking one symbol at a time. And since many grammars require more than one symbols to resolve conflicts, this may not be always sufficient, right? So this is LL1. It is simple, but it has its limits. Now let's take a step forward and think about LL2 passes, right? So there is LL2 passing, which extends LL1 passing by allowing the grammar to look ahead of two input symbols instead of just one. This gives the passer more context to decide. Now what is look ahead of two symbols? What does it mean? So in LL2 passes, the passer examines the current input symbol and the one that follows it, right? It not just checks the current input symbol, it checks one more, the ones that follow it. And this provides better insight into which production to apply, right? This can help resolve ambiguities that LL1 cannot handle, especially when two or more rules have overlapping first sets, but differ on the second symbol. We'll look at an example for this as well in the coming slides. Now conflict resolution, by looking at two symbols ahead, LL2 passes can differentiate between rules that share common prefixes, but differ afterwards, right? This reduces first set conflicts. Then we can extend this idea to LL2. We can extend this idea to LLK passing as well, which will bring its own added complexity. So LL2 passing can handle more complex grammars. They are also more computationally expensive. The passer's decision making process becomes more complicated. It has to consider multiple symbols simultaneously. The size of the passing table also grows exponentially with K, making the construction and storage table more challenging for larger values of K. So here is one example, right? You have S gives ABX, S gives ABY and S gives ACZ. As you can see, this will require two lookaheads because A is common and the next symbol B or C will decide what is the word for the course of action, right? Because the non-terminal afterwards, after those two terminals differ, they are respectively X, Y and Z. So let's do a comparison, right? There are many, along the rows you can see many axis of comparison and along the columns you have LL1, LL2 and LLK, right? Firstly, they will differ in terms of lookahead as I just described, one

symbol, two symbols or K symbols. Conflict resolution is limited in LL1, you can resolve more conflicts in LL2 and many more in LLK. Grammar complexity, it can handle simple grammars, handle more complex grammars and LLK can handle very complex grammars. The parser complexity, on the other hand, is LL1 is simple and efficient, LL2 is more complex and LLK becomes increasingly complex exponentially with K. And of course, as a result, table size also, you have small, large and grows exponentially with K. The use cases for these differ as well. You have basic programming languages, more advanced programming languages and then very complex grammars, right? So this has given us some good idea of the difference between LL1, LL2 and LLK and along these various axis, right? So we have briefly discussed LLK parsing, right? This should give you an idea what the K stands for, how you could develop your own passes starting from 1, 2 till K, right? But of course, if you move towards more higher extent of look ahead, there will be more complexity also that will be introduced and the table sizes will also grow. And there is a trade-off here when you, hence there is a trade-off here when you decide to use one of these. So we are at the end of this video, thank you.

Summary Video• . Duration: 6 minutes

[Notes]

Hello everybody. So we are at the module on LL1 parsing and the lesson we will look at is the summary of this topic and we'll try to bring some closure to it. So we'll revise all the ideas around LL1 we have looked at in some of the past videos. So while we went about understanding LL1 parsing, one of the key parts we learned is first and follow sets. So what is first set? Then what is follow set? And what is nullable sets? So let's look at the idea of first set. First of x. It is a set of terminals that appear as the first symbol in any string derived from x. If x is a terminal, then first of x is equal to x. If x derives empty string, then empty string is included in the first of x. So this is what we learned about first set. Now let's understand follow set. Follow of a. What is this? It is a set of terminals that can appear immediately to the right of a in some derivation. We have to always include dollar end of input marker in the follow set of the start symbol. If a non-terminal a is followed by b in a rule, so if a non-terminal a is followed by b in a rule, we add first of b to follow of a. So these are all the rules around the follow set. Then the nullable set, where nullable set is where a non-terminal can derive empty string. So this is what first follow and nullable set is all about. Now LL1 parsing. L here stands for left to right scan of input. Then the second L stands for leftmost derivation of the grammar and one stands for one look ahead symbol to decide which production rule to apply. In LL1 parsing, based on the current symbol on the stack and the next input symbol in the input string, the parser decides how to proceed. So this of course limits the grammar types that can be parsed since many grammars require more than one symbol to resolve conflict. So this gives a gist of LL1 parsing. Then there is the table construction algorithm. For each production a gives alpha, for each terminal t in first of alpha, we add a gives alpha to the table entry a,t. If the null string, if the empty string belongs to first of alpha, then we add a gives alpha to the table entry a,b for each b in follow of a. So this again is a keep these two are the key parts to the table construction. Now if multiple productions map to the same table entry, the grammar is not LL1 and refactoring is required. So this is while we are building the table, if we get into this situation, that means there is a problem and the grammar is not LL1. If no rule applies for table a,t, mark the entry as an error to handle unexpected input. Now then we also discussed the LLK approach, right? It increases

complexity, how? So although LLK parsers can handle more complex grammars, they are also more computationally expensive. The parser's decision making process becomes more complicated. It has to consider multiple symbols simultaneously. The size of the parsing table grows exponentially with k , making the construction and the storage table more challenging for large values of k . We are also seeing an example here of all the productions with s and there is an inherent problem here because all starts with the non-terminal a and you have to do one look ahead that is LL2 where you look at b , b or c to decide which production you would want to apply. Now there is something called as passing conflict. There is something called as shift reduce and reduce-reduce conflicts in LR parsers. We'll look at this more in the future. There is also something called as first future conflicts in LL1 parsing. There is overlapping first sets that we have already looked at in some of the earlier videos and there is left recursion causes infinite recursion in LL1 in LL parsers. This is also a familiar problem. How do we resolve conflicts? You do left factoring, you factor out common prefixes to resolve first set conflicts. The process of which was also discussed earlier. You eliminate left recursion, rewrite left recursive rules to avoid loops, this is also something we have seen and the LLK parsing uses k look ahead symbols to resolve complex conflicts. So these are some of the ways to resolve conflicts in LL1 parsing. So we have looked at this topic of LL1 parsing and it's quite an interesting parser. It looked at top-down approach, it was simple, we understood the table construction process and then we understood how to use that table and it's a very simple parser. It's easy to understand, it's a good place to start. Of course, there are complex parsers and we talked about LLK for instance, we talked about LLK parsers. This is one of the key topics in parsers after recursive descent parsing. This is something we focused on and tried to understand in depth. I encourage you to look at different grammars for LL1 and how the table construction works, practice the creation of first and follow sets etc. which will help you build a good understanding and working skills in LL1 parsing. So we are now at the end of this video. Thank you.

What are the Weaknesses of LL(k) Parsers Video• . Duration: 3 minutes

[Notes]

Hello everybody. So now we will move to the module passing LR0 and illustration of LRK. The lesson we will start with is weaknesses of LLK passes. So let's try to understand the weaknesses of LLK passes because that's where the motivation for other passes comes from. Here is an overview of LLK passes. We looked at LLK passes. It's a top-down passes that read input left to right and construct a leftmost derivation of a string with K tokens of lookahead. So this is what we have understood already. LLK passes are widely used for deterministic passing of context-free languages, especially in programming language compilers. This is one application area for LLK passes. They are efficient in terms of space and time for small k and they use a predictive passing method. So this is just an overview of LLK passes. Now one of the weakness is limited lookahead. So let's try to understand what that is. So LLK passes rely on a fixed number of lookahead tokens K to make passing decisions. For some grammars, even with large K the passer may not have enough information to resolve ambiguities or determine which production to apply. So this is in spite of large K you may still run into issues. This results in passing failures for languages that require more than K tokens of lookahead. So apart from that there is other weaknesses. One of them is handling of left recursion. LLK passes cannot handle left recursive grammars where a non-terminal calls itself on the left hand side of the production. We have seen examples of this. The grammar needs to be

transformed to remove left recursion which can complicate grammar design. So it is not always easy to remove left recursion. The third weakness is language coverage. LLK passes can only pass a subset of context-free languages called LLK grammars. More powerful passing techniques such as LR passes are required for these languages and which reduces the utility of LLK passes for modern compiler implementation. So there is advanced passing technique LR passing and we will venture into that. Since there are these limitations with LLK passes, we are in search of more advanced powerful passing techniques. So the limited lookahead of K tokens in LLK passes may not be sufficient for resolving complex passing decisions. LLK passes fail with left recursive grammars and require manual grammar transformation. LLK passes can only handle LLK grammars limiting their use in real-world languages. So as a result of this, we are in search for other passing techniques and it is good to know other passing techniques because they might be most suitable in some other application domain or in some other context. So this should lay some foundation on why we are exploring other passing techniques. So we are now at the end of this video. Thank you.

Demonstration of Shift Reduce Parser Video• . Duration: 6 minutes

[Notes]

Hello everybody. So we will look at the module on LR0 and LRK passing in general and we are at the lesson shift reduce parser. So at the end of this video let's try to understand this what this shift to reduce passing is about. We have looked at other passes in the past. Now let's look at this type of parser namely called shift reduce passing. So what is shift reduce passing? Shift reduce passing it is a type of bottom-up passing unlike the other passes we have seen like recursive descent passing this is bottom-up passing. The goal is to reduce the entire input to the start symbol of the grammar. So since it is bottom-up we start from the input and try to get to the start symbol of the grammar. The parser works by either shifting symbols from the input to the stack. So that's the shift part or reducing the symbols on the top of the stack into non-terminals based on the grammar rules. So that is why this is called shift reduce passing. I will say it again you either shift symbols from the input to the stack or you reduce the symbols and the top of the stack into non-terminals based on the grammar rules. So this is the essence of this passing. Now there are four key actions in shift reduce passes. The one task is shift wherein you move the next input symbol onto the stack. Then there is reduce where you apply a production rule to replace symbols on the stack with a non-terminal. So there will be a production rule you will apply that production rule to replace symbols on the top of the stack with a non-terminal. Then there is accept. The passing is complete when the stack contains the start symbol and the input is empty. That's when you know that everything has gone well and you can accept. There is also error that occurs when no valid action can be applied and the parser cannot proceed. So shift reduce can be applied for many grammars. So this is one example of a grammar on which we can apply shift reduce passing. Let us try to reduce input sequence id cross id plus id to the start symbol as using the grammar rules. So we'll go step by step and try to apply shift reduce passing to this. You have the input as you can see you have the input on the extreme right column. Then you have the stack how the stack is at any given time. And then you have the action type which can be shift reduce etc. So and mainly it will be shift or reduce. Let's see how this goes. Shift action right and the stack is empty at the moment. Then what you do is shift means you move id onto the stack right that's where you get the id on the stack. Then you apply the production rule f gives id as a result you replace id with f. Then you proceed and you apply reduce again

wherein you reduce you apply the production $t \rightarrow f$. So you replace f by t . Then you get into shift again where you move $*$ onto the stack right then further you what you do is you apply you move id as well to the stack right. So there are two subsequent shifts happening. Then you do the reduce where you replace id by f . So f gives id . Reduce one more time where you replace t you apply the production $t \rightarrow t*$ and as a result you replace the whole stack with t . Like this you proceed further you can apply $e \rightarrow t$ you get e on the stack. You shift $+$ onto the stack and then next step you shift id . Now as you see your input is empty but you still have things on the stack. Basically you have e plus id remaining on the stack. You apply reduce $f \rightarrow id$. So as a result you get e plus f on the stack. Then you reduce by applying the production rule $t \rightarrow f$ you get e plus t . Now you reduce again wherein you apply the production rule $e \rightarrow e + t$ and so you get e and finally you apply $s \rightarrow e$ so you will get s on the top of the stack. At the end of this you have the start symbol and the top of the stack and you have the input as empty. That means this expression the input expression could be applied for this grammar and it match the grammar and we have the correct accept state at the end. Let's talk about the advantages of shift reduce parsers. It is efficient for many programming languages. It can handle wide range of grammars including ambiguous one with proper conflict resolution of course. A shift reduce parser can easily handle arithmetic expressions with proper operator precedence like we just saw. There are some disadvantages as well. It requires careful handling of shift reduce conflict and reduce reduce conflicts. Handling tables for large programs can be complex and difficult to debug but with a small example we could appreciate the usage of this shift reduce parsers. So in summary we saw shift reduce parsing which is a powerful technique for bottom up parsing. It enables efficient handling of wide variety of context free languages especially when paired with good conflict resolution strategies. So we are at the end of this quick example and good overview of shift reduce parsers. We will look at other aspects as well of this particular parser and other bottom up parsing as well in the forthcoming videos. Thanks for listening and we are at the end of this video. Thank you.

The Need of LR Parsing Table Video• . Duration: 3 minutes

[Notes]

Hello everybody, so we are at the module passing, we have been looking at some of the bottom up passes like LR0, LRK etc. And the lesson we are focusing on is shift reduce passing. Now at the end of this video, we'll understand the need for LR passing table, right? We started with shift reduce passing in the earlier video. Let's dive deeper into this topic. Now LR passing, right? What are LR passes? The type of bottom up passes that read input left to right and construct a rightmost derivation in reverse. Right? Unlike the LL passer, if you remember, the term LR stands for left to right scanning of input and rightmost derivation in reverse. LR passes are stronger models than LLK passes, right? It can cover a much larger set of context free grammars. Now what is the role of the passing table in LR passing? A LR passing table is a pre-computed structure that directs the passers action whether to shift or reduce or accept or declare an error. Without the table, the passer would have no guidance for making decisions and would not be able to pass efficiently. The passer uses the table to decide its next move based on the current state and the next input symbol. So the LR passer table helps in efficient decision making. The passing allows the passer to efficiently decide its action based on the look up, right? Static resolution is also managed through using the table. It helps manage shift reduce and reduce reduce conflicts. There is also this predictive passing

you get through the table. The table pre-computes all possible shifts and reductions ensuring the parser operates in a deterministic way. Now there are advantages of the LR parsing table. Using LR parsing table has many advantages, right? One is one, tables are pre-computed once making parsing faster during run time, deterministic parsing strategy and allows us to handle complex grammars. So we just kind of introduced LR parsing tables which provides essential guidance for the parser determining shifts, reduce and state transitions. Without them LR parsing would be inefficient and prone to errors, particularly in handling conflicts. So we are at the end of this video on LR parsing table. We will continue to explore this topic further in the forthcoming videos. Thank you for listening.

What is LR(0) Parsing? Video• . Duration: 2 minutes

[Notes]

Hello everybody. So we are at the module on parsing. We were looking at LR0 and other types of bottom up passes. The lesson we are focusing on now is LR0 parsing. Now at the end of this video, we'll get some understanding of what LR0 parsing is about. So what is LR0 parsing? LR passes are a type of bottom up passes that read input left to right and construct a rightmost derivation in reverse. LR0 is the simplest form of LR parsing, which uses no look ahead symbols, zero look ahead. It's one of the easiest and the most basic algorithms for constructing shift reduced passes. Now, what is LR0 parsing? What does it involve? LR0 passes uses a DFA to guide its shifts and reductions. This means the parser decides whether to shift or reduce without looking at the next input symbol. The parser processes input by moving through a set of states and making decisions based solely on the current state and the content of the stack. Now there is something called as LR0 item. So LR0 item is a production rule with a dot indicating the current parsing position. Take for example, this rule $E \rightarrow E + T$, where we have these following LR0 items. In the first one, we have $E \rightarrow \cdot E + T$, which indicates that the dot is placed before parsing any symbol. This is the state, the current position is before parsing any symbols. In the next one, current parsing position is after recognizing E . So the dot is after E and then the dot is after plus. So it is after recognizing $E +$. These items track how much of a production rule has been passed. So this is called LR0 items. Now what are some of the advantages and limitations of LR0 parsing? First, the simplest form of parsing LR0 is the simplest form. It can be implemented efficiently. Now there are some of the limitations. Cannot handle all context-free grammars, prone to conflicts due to lack of look-ahead, especially reduce-reduce conflicts, and only suitable for simple grammars with minimal ambiguity. So we are at the beginning of understanding LR0 parsing. This is again a foundational parsing technique that uses a state machine and a parsing table to make parsing decisions. So we are now at the end of this video. Thank you.

Example: LR(0) Parsing Table to Parse a String Video• . Duration: 13 minutes

[Notes]

Hello everybody, so we are at the module on parsing for LR0 and we are looking at illustration. The module also involves illustration of LRK. The lesson we will look at specifically is LR0 parsing. So at

the end of this video, learners will be able to understand how to use LR0 parsing table to parse an input. So we will give an example of that. So we will look at this example. It's a simple grammar with three productions, S gives A, A gives small a terminal A followed by a non-terminal A and then again non-terminal A gives B, right, the terminal B. This grammar will be used to build the LR0 parsing table and parse a string. So let's quickly try to understand this example and see the overall flow. The individual parts covered in this step-by-step process are further illustrated in other videos. But let's try to see all of these processes coming into action through this grammar and in this video. So LR0 items, right, this is central to the LR0 parsing technique. So what is LR0 item? It is a production rule with a dot indicating the current parsing position. For a rule, for example, E gives E plus T, we have items where dot could be in the beginning, that means before parsing any symbol, before the E you see, that means after recognizing E or it could be after the plus sign, which means after recognizing E plus. So these items track how much of a production rule has been parsed. So one of the key pieces in the LR0 parsing is LR0 items for the grammar. So LR0 items for the example grammar can consist of these productions where the dots are placed at different places, right, and LR0 items track the position of the dot in the production rules showing progress in the parsing process. So here are some examples of production with the dot. However, when you apply the LR0 automation, right, the construction of the states and the corresponding transitions, which is illustrated more in a separate video, you will, at the end of that process, you will get these states, right, shown in front of you, and there will be transitions. For instance, here if you can see, the state S0 is crucial because that is where the dot is right in the beginning of many of those productions, right? And if you see that the transition from S0 to S4 is on a terminal symbol B, where you end up seeing this production A gives B dot, that means this is where this is a separate state by itself, and it shows that A gives B, this production has been applied fully because the dot is after the B. Similarly, if you see the transition from S0 to S3, you see the dot moving one step ahead, right, because you see A gives A dot A, right, so S3 itself is a state. Similarly, the other states, right, you can look at this closely and see how the transitions happen from one state to another, right, and as I will say that again, that this step by step process of constructing is explained in a separate video. So, let us assume this is correct and let us proceed further how you will do the passing, right. So, there is the LR0 passing table. It consists of action table which guides shift, reduce and accept actions, and there is a goto table that guides transitions to new states after the reductions. The action table tells us when to shift on different terminals like A, B, etc. and reduce based on specific LR0 items. The goto table tells us how to transition between states based on non-terminals. Again several examples have been shown and this video also will look at one example. So, this is the LR0 passing table for the grammar under consideration. As you can see here, there are different states all the way from 0 to state 5 and you have under action, you have specific terminals, those are the terminals you have to look carefully. They are A, B and \$ and the states along the rows and the terminals along the column. What does this indicate that looking at the state, right, on the stack and looking at the current input, we decide to do either the shift actions or the reduce actions. And when we say a shift action, let us say we say shift 3, what we mean, for instance, if you look at state 0 against action A, you have S3, it means we shift the input to the stack and we do 3, we go to state 3 directly because it is a shift. On the other hand, let us say you look at state 1 and action you have R1, what does that indicate, we, that means we reduce, we shrink the stack and that 1 here indicates the production, appropriate production we use to reduce the stack where we replace the right-hand side of the production with the left-hand side. And now the question arises, however, is when you do a reduce, what state would be the next state? So basically, as per the rules, you look at what state remains on the top of the stack. Let us

say, for instance, the top of the stack is 0, right, then you refer to the go-to part of this table for state 0 and you look at what is the recently reduced production. So let us say it was A, then you check 0 against A and you see 1, that means you have transitioned to state 1, push that state into the stack and start the process all over again, right. So again, several examples have been shown and we will look at one more example, step by step processing in this video, but this is the passing table and the construction of this passing table itself is also explained as a separate video, right. So you start with the LR0 items, the state table and the transition, then the different states and the transitions, and then you arrive at the LR0 passing table. The next step is actually doing the parsing, right. For that, let us say, assume we have to parse the string AAB using the LR0 passing table constructed, then this is how it will proceed, we will quickly run through this example. So here are the steps, right, on the extreme left column, there are the different steps and there is the stack, right, which I talked about. So there is stack for the states, the first stack, and there is a symbol stack, they can be merged as well, right, in this figure, they are shown separately. Then you have the input string, right, and then the different actions taken given the state and the input string, looking at the state stack, symbol stack and the input string together, we look at all of this to decide the action and at times we have to go to, on reductions especially, we have to go to a particular state. So that is also described in the go to column, right. So let's start the process. So initially we have state stack 0 and symbol stack \$ and we have the input AAB\$, right. In this case, we will do the action S3, right. So as a result of S3, you see, we push 3, state 3 into the stack and we have already processed 1, we have pushed the input A into the symbol stack, right. So we have applied S3 action. We move forward, again we see the input A and we see the top of the stack is 3. We refer to the table which action to take, it turns out it is S3 again and we therefore push A into the symbol stack and we push state 3 again into the state stack, right. We proceed further, we have B as the input, this time we do S4 action, which means we will push B and we will put B into the symbol stack and we will push state 4 into the state stack, right. Now for this particular state where you have the state 4 at the top of the stack and you have the input dollar, turns out we have to do a reduction as per the table. So we do R3 reduction. When we are doing R3 reduction, then in that case, we replace B by A. Remember this, the third production is A gives B, right. So we replace B by A on the symbol stack and we replace and what we do is we pop 4, right, from the symbol stack, we refer to the GOTO table to find out what is that we have to push to the state stack. In this case, it turns out to be 5, right. Remember again, we refer to the non-terminal. In this case, since A production was involved, A was on the right hand side, we will refer to A against the already present input that was 3. 3 and A will give us GOTO of 5. So we push 5 onto the stack. We proceed further. In this case, we reduce R2, which is replacing A gives small a to capital A, right. So we are able to reduce two symbols basically this time with R2 reduction. So the stack also shrinks, we are able to replace 3, 5 on the stack with just 5 and this time, if you notice, we have pushed 5 again onto the stack, right. So we are left with state stack of 0, 3, 5 and the symbol stack as \$A, capital A, right. So again, we continue the process, we do a third R3. In this case, we replace, again, this is AA replaced by A, capital A. The terminal, the production involved is capital A gives small a and capital A, right. So we get 0, 3, 5. Third reduction, 3, 5 is replaced by 1, we have to GOTO to state 1 and again, AA replaced by A. So we are left with state stack of 0, 1 and the symbol stack of \$A, right, and the input is \$A again. Now we do the R1 reduction. In this case, we replace 0, 1 by, the 1 is replaced by 2 and A is replaced by S because we are considering the production, S gives A. Now we are at a moment where the stack state and the input points out to an action in the action table, which is ACCEPT. This is excellent, right. And action means we are able to successfully parse the input AAB and hence we have reached the action

state, right. So that means we have successfully parsed the input, right. As mentioned earlier, we have gone through the step-by-step process of an example and starting with LR0 items all the way to being able to reach the final ACCEPT state. So LR0 parsing provides a basic deterministic method for parsing context-free grammars using a pre-computed parsing table as we have seen. In this example, we saw input string AAB of shifts and reductions for perform using the parsing table. So we are at the end of this complete example of how this LR0 parsing works. I hope this gave you a good glimpse of what LR0 parsing involves, right. Each of the steps were discussed in this video. Thanks for watching this video and we are at the end of it and the subsequent videos will dive deeper into each of these processes. Thank you.

Constructing LR(0) Parsing Table Video• . Duration: 7 minutes

[Notes]

Hello everybody, we have been looking at LR0 passing and the variance around it. So let's focus on the lesson LR0 passing. So we'll take some initial steps to understand the steps needed to construct a LR0 passing table. So let's recap what this LR0 item is about. LR0 item is a production rule with a dot indicating the current passing position. So for a rule $E \rightarrow E + T$, we have item E , the dot can be before the E , right? That means before passing any symbols, it can be after the E , that means after recognizing E or it can be after the plus as well. That means $E + T$ has been recognized, right? So this is used to create LR0 items, right? And we can track how much of a production rule has been passed based on this. There will be a LR0 passing table that will consist of the action part. It will guide shift, reduce and accept actions and there will be a goto part that guides transition to new states after the reduction step. The action table will tell us to do shift on terminal A or B , right? So given the state and the terminal, we will decide to do some shift operations and we will also decide to do reduce based on specific LR0 items, right? So we might also end up doing reductions when we reach a particular state. We might choose to do reductions. Now the goto table tells us how to transition between states based on non-terminals. So after we do a reduction, then we will refer to the goto table to help us transition to new states, okay? And that will be based on non-terminals and not terminals. So we have understood already that there is the shift action and there is reduce action. Reduce action typically will refer for a particular production rule, right? Following which we do the goto, we will refer the goto table to find the next state. Now here in this example, we will follow this simple grammar, right? So here is the grammar in front of you and the step one will be to augment the grammar by adding new production, right? $S \rightarrow S$. We will augment this grammar and then what we do is we have this canonical collection of LR0 items that represent the states of the LR0 parser. So we begin with the closure of the augmented grammar. For our grammar, the first table becomes, this is the table. What we do is for the very first item, we put a dot in front of S and then since we have put a dot in front of S , we have to find the production starting with S . So $S \rightarrow E$. So we put a dot in front of E . Similarly, we find the production for E . Then since $E \rightarrow T$, we find the production for T , F , etc. and we put that dot, right? Which represent that where we are in the current processing of the input, right? How much of the input we have processed vis-a-vis the non-terminals and the terminals process, right? Corresponding to the input string. Now LR0 item transitions move the dot based on the input symbol, terminal or non-terminal and example transition is $S \rightarrow S$, right? The dot will move beyond S or on plus we will, $E \rightarrow E +$, the dot will move beyond the $E +$, right? So and we do shift or reduce based

on this transition, right? So at the end of it, once we have the full, all the states and their transitions, what we build is an action table and a goto table. In the action table, what we do is for a terminal, we use shift if there's a move, right? Or reduce if the dot is at the end of the production, right? So shift we do when we have a move, right? We move some kind of terminal, we shift the dot and reduce if the dot is at the end of the production. We accept when S dash gives S dot, right? We reach that state, right? So that is the accept. Regarding goto, what we do is for a non-terminal, we transition to the corresponding state as indicated earlier. Remember, when we put the goto, we specify a production rule to be applied and when we do a shift, we decide, we tell the state to be transitioned. Now at the end of goto, next state is also important for that after the reduction, the next state to be obtained is also important for that we refer to the goto table, right? So here is an example. So for instance, if you see i1, right? On if we are in state i1 and plus we encounter plus then we have to do shift operation, move the input to the stack and we have to transition to state 6, right? On the other hand, if you see i4, for i4 on E, we have i8, right? That means on i4, if we have i4 and recently we have done a reduction, we have done a reduce operation, then if E is the non-terminal that is derived, we might put the next state as i8, right? So I'm talking about i4 and capital E non-terminal, right? So that transition. You also see some reduce, right? For instance, i7 in i7, if you see a star, you do r1, right? You do a reduction, right? And here, when you do a reduction, that means you will reduce a set of right-hand side of a production rule with the left-hand side, right? So right-hand side may have be terminal, non-terminal and you reduce them with a non-terminal and then this one is specified r1. What does that one mean? The one means which production you are reducing, which production is being employed. And then you can refer to the production to find out how many symbols are there on the right-hand side and what is the non-terminal on the left-hand side. That terminal on the left-hand side is again relevant because you will refer to the stack again what is left on the top of the stack and that non-terminal to refer to the go-to path, the right-hand side of this table to decide what is the new state you will put on the stack. So we have looked at LR0 passing, right? That provides a basic deterministic method for passing context-free grammars using a pre-computed passing table. We have briefly looked at the table construction algorithm and an example of constructing it. In the coming videos, we will look at the construction, some examples for the construction also. We will look at some examples of step-by-step processing of input by referring to the table and there will be a stack also alongside the table that will be used to process. So thanks for listening and we are at the end of this brief introduction to the various parts. As I said, forthcoming videos will lay out the other building blocks of LR0 passing. Thank you.

Parsing a String Using LR(0) Video• . Duration: 10 minutes

[Notes]

Hello everybody, so we are at this LR0 parsing and the lesson we will look at is how we go about using the LR0 concepts and being able to parse some grammar. Now we'll take a string as example and we'll go through the LR0 process. So let's take this simple grammar that is S gives E and you have either the plus operator or the cross operator and you have IDs and the non-terminals involved are E, T, F, etc. and the terminals involved are ID, star, plus, etc. So we will parse this string in brackets ID plus ID cross ID. Let's try to see step by step how the parser operates. So we assume here that we have been able to using the LR0 items and the creation of the state machine we assume we have the table with us. If you look at the table there are 18 from 10 to 18 states then you

have different terminals and under those terminals from ID all the way to the null terminal all the way to the end of string marker we have as you can see you see lot of shift actions or reduce actions. You also see this accept action at I1 and the \$ sign. So that is the part of the table we will use to decide whether we will do a shift operation or a reduce operation. Then there is the GOTO part of the table. You see non-terminals E, T and F. Under this if you see there are states written there again. For instance under I0 you have I1, I2, I3. And you have some states written under I4 and I6. These are the states where after the reduce operations you will do the GOTOs. So these are relevant for the GOTO action. Now let's try to proceed with the step by step process of parsing. Let's try to understand how to interpret this step by step process. So what happens is I will give you few examples to illustrate the process. And you will be able to generalize from it how this LR0 process works. So at the beginning you have at the top of the stack you have I0 and you have this input the full expression and the first action we choose is shift open brackets because at the input you have the open bracket. And we transition, we decide to transition to state I4 on seeing that open bracket based on the table shown earlier. So from I0 we have to go to state I4 on the opening bracket. So what do we do? We have to do two things. Firstly we have to shift this open bracket inside the stack. As you can see in the next row we have I0 followed by the open bracket. That means the shift has happened. And since we are transitioning to state I4 we will also put I4 in the stack. Now where do you get that I4? You get that I4 because in the table it is mentioned shift as 4. That means you shift and then you choose state 4 as your next state. Similarly as you see the current symbol on the input is ID. Again we refer to the table, we check what action we need to do for I4 state when we encounter ID. Turns out we need to shift ID. So we shift ID and we transition to state I5. So in the next row you see I0 open bracket I4 the ID is now shifted in the stack and you have state I5. So this is how you proceed. Now what happens is at the end of this process now you have I5 and you see plus and the table tells you you have to reduce the state diagram also tells you that you have to reduce. So you use F gives ID that is the production under consideration. So you reduce with the production F gives ID. So in the subsequent row you can see ID being replaced by F. So because you are reducing with that expression. Now there are a couple of other things happening here. What is also happened is before you once you reduce that expression you also have to decide what will be the new state of the stack. Typically what you do is since F gives ID, ID is the only symbol on the right hand side you have to replace only one symbol from the stack. That is why we replace I5 and ID and we replace ID by F. Now the question is what should be on the top of the stack. So what we do is we look at ID and we refer I4 that is the previous state. We look at I4 and ID and decide what is the next state should be. In this context since ID is replaced by F in the GOTO table we refer F and I4 and that GOTO table tells us to go to state I3. So under F the non terminal we see that we need to go to state I3. As a result you put I3 on the stack. Similarly you proceed like this further. In this case you reduce with the expression T gives F. As a result F is replaced by T and now you move to state I2 now. In this case it is I2 earlier it was I3. So you put I2 on the stack. To move further you now reduce by E gives T. With that you replace T by E and now I8 is on the stack. So you proceed like this. Next under I8 you see plus. As a result you do the shift again. This time you shift the plus as you can see the plus got shifted and the state put is I6. Again you do one more shift and this time you move state I5. So we will continue with the table. So you have the states and the terminals and sometimes the non terminals also on the stack and we have the input partially processed. Now the next step is we reduce with F gives ID. So we reduce ID to F and we go to state I3 on F. So if you see ID is been replaced by F. We continue like this. We now next depending on I3 and then closing bracket we decide to reduce T to F and we do the similar process. That is we have to pop from the

stack the state I3 and then we have to replace F by T. And then we have to look at the entry for I6 for T and then decide which state we'll have to go to. In this case it turns out to be I7. So as in the next row you can see I7. Now as we move further we get to reduce this production. E gives E plus T. So in this case there are three symbols on the right hand side. So it's a quite a big reduction. As a result in the next row you can see E plus T. The whole of it getting reduced to E. And then we decide based on the goto table what should be on the top of the stack which state which is I8. We continue like this. We do a shift. We shift the closing bracket. We do the transition. Again we do in the next step we do a reduce with F gives open bracket E closing bracket. And we reduce with that. As you can see here what happens is since we have reduced we will see that in the next slide if we reduce by that whole expression. So you see that brackets disappearing. You have reduced entirely. You proceed like this. And as you do this shift operations the input will be processed. So you have this only a small part of the input star ID remaining. So you keep proceeding like this. You do reduce. Then you do the shift star ID star. You do couple three more reduce. And then at the end of it if you see what what is happening is by the way the dollar sign is also a terminal in the table. You could refer the transition under the dollar sign. At the end of it you see I1 against dollar. And in the initial table that was the accept state. So you have reached the accept right. So in this way we have seen a detailed we have worked out how to use the table. How to do the shift. How to do the reductions. And in the reductions what has to be done is the right hand side has to be replaced by left hand side. The top of the stack also have to be figured out based on the goto table. So at the end of this LR0 operation and given that we had the table we have reached the accept state. So we have seen of any comprehensive grammar to parse this expression. I suggest you try to work out this example again and which will give you a good understanding of how the LR0 parsing works step by step. While you work out the example you can also refer to the table. In this example we assume the table was already derived. So you look at the table and try to go step by step. As a result you will get a good understanding of how the LR0 parser works. Thanks for watching the video.

What is LR(1) Parsing and LR(k) Parsing Video• . Duration: 4 minutes

[Notes]

Hello everybody, so we had the module parsing for LR0 and we will also give an illustration of LRk parsers. The lesson we are looking at is introduction to LR1 parsing and LRk parsing in general. Now we will try to understand what are LR1 and LRk parsers and differentiate between LR0 and these. So what is LR1 parsing? So LR1 parsing is a type of bottom-up parser that reads the input from left to right. The L stands for that and it produces a rightmost derivation in reverse. The 1 in LR1 indicates that the parser uses one look-ahead symbol to make decisions. Now which means it considers the current input token and one token ahead to determine the passing actions. So the key concepts here are the L stands for scans the input left to right, R stands for produces a rightmost derivation in reverse, and 1 stands for parser looks ahead one symbol to decide the shifts, reductions, and transitions. Now let's look at the components of LR1 parsers. There is the states which represents parsing configurations and there is look-ahead. A single look-ahead symbol that helps to determine when whether a rule should be reduced or not. There is an action table as well. It dictates whether the parser should shift. Shift means read the next input symbol and move to the next state or reduce that is apply a production rule or accept successfully parse the input or error which detects a syntax error. Here is an example of LR1 parsing example. But typical LR1 item is of

the form $A \rightarrow \alpha \cdot \beta, A$. Now what does this mean? Here $A \rightarrow \alpha \cdot \beta$ is a production rule with a dot indicating the current position in the rule. And A is the look-ahead symbol which is used to guide the parsing. So there is LR1 item example. So the look-ahead helps distinguish between the situations where the parser might otherwise be confused about which production to apply, especially when there are ambiguities in the grammar. So LR k differs from LR1 in that it has k greater than 1 look-ahead symbols for parsing instead of LR1's single look-ahead symbol. So you could generalize LR1 to LR k . So LR0 there are no look-ahead symbols. The parser decides based on the current state and the input. So that is LR0. Then there is LR1 uses one look-ahead symbol making it more powerful and capable of handling more complex grammar. And then you can generalize to LR k where you have k look-ahead symbols giving even more context but increasing the complexity of the parsing tables. So in summary there is LR1 parsing which uses one look-ahead symbol to decide shift reductions and transitions. It's powerful enough for most grammars and is used in many parser generators like Yacc and Bison. There is LR k parsing also which uses k look-ahead symbols. For k greater than 1 it allows more context for decision making but increases complexity making it less common in practice. So we are at the end of this video. Thank you for watching.

Hierarchy of Grammar Classes Involving LL and LR Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module parsing with LR0 and the related parsers. Lesson we will look at is hierarchy of grammar classes. So at the end of this video we will try to understand what this grammar hierarchy is about. How do we rank LL versus LR grammars. So there is some sort of hierarchy of grammars. The hierarchy of grammar classes involving LL and LR parsers is structured based on the type of grammars they can parse. This hierarchy helps in understanding the relative power of different parsing strategies. So let's look at LLK grammars first. So LLK grammars are parsed by LLK parsers where K denotes the number of look-ahead symbols used to make parsing decisions. These parsers read input from left to right and produce a leftmost derivation. That is why they are called LL. Now this LL1 grammar are the most common and simplest using only one single look-ahead symbol. Now the key characteristics of LLK grammars are as follows. They are top-down parsers. They try to match the grammar rules with the input string from the start symbol down to the terminals. That's why they are called top-down parsers. LLK grammars are more restrictive than LRK parsers. This is true in general. LLK grammars are more restrictive. They cannot handle left recursion for instance. If a grammar has left recursion, it cannot pass by an LL parser. So the class hierarchy will go something like this. You have LL1 is a subset of LL2. That is less than the subset relation subset or equal to LL2 like this all the way to LLK. As K increases, the grammar can handle more complex grammars but require more look-ahead symbols. LR are zero grammars. These are left-to-right and rightmost derivation. They are passed by LR0 parsers which use no look-ahead symbols. These parsers scan the input from left to right and construct the rightmost derivation in reverse. Their key characteristics are as follows. They are bottom-up parsers. They try to reduce the input string to the start symbol by matching rightmost derivation. They can handle left recursion. Unlike LL parsers, they can handle left recursion. They are more expressive than LL parsers but cannot resolve all parsing conflicts without look-ahead. Again, as far as the

class inclusion is concerned, you can say that LL1 is a subset of LR0. So any grammar that can be parsed by an LL1 parser can also be parsed by a LR0 parser. Not the other way round though. So it is a strictly less than subset relation. Now let's look at LR1 grammars. LR1 grammars again left-to-right and rightmost derivation. LR1 grammars parsed by LR1 parsers which uses one look-ahead symbol to make parsing decisions. LR1 parsers are more powerful than LR0 parsers because they use the look-ahead symbol to resolve shift-reduced conflicts. So LR1 grammars, the key characteristics are more general parsers used in practice. They can handle all deterministic context-free grammars. Look-ahead symbol helps distinguish between ambiguous situations where LR0 cannot handle. They can handle left recursion, complex constructs, and ambiguous grammars that require look-ahead for disambiguation. So we have this class inclusion where LR0 becomes a subset of LR1. That means every LR0 grammar is also an LR1 grammar. But not every LR1 grammar can be parsed by LR0 grammar. So if we try to see all this relation, then we can say that there is LL1 as a subset of LL2 all the way to LLK because LLK is more powerful than LL1. And then you have LR0, right? It might be even more powerful, but LR0 itself is a subset of LR1, LR2, all the way to LRK. So this is how roughly the class inclusion, how these grammar families are related to each other. So we have looked at LLK parsers. These are top-down and generally more restrictive. They cannot handle left recursion and are easier to implement by hand. LRK parsers on the other hand are bottom-up and more powerful, able to handle left recursion and more ambiguous grammars. So this should give us a general idea of where LL style parsers compare to LR style parsers. What is their relative power, etc. So with this, we are now at the end of this video. Thanks for watching.

Summary Video• . Duration: 4 minutes

[Notes]

Hello everybody, so we are at the module on LR0 passers and the related LR0 passers. The lesson we will look at is conflicts in LL1 passing table. This video however, we have seen many of the aspects like conflicts etc. and also the hierarchy of languages that these passers can address. In this video, we will try to revise the concepts covered. So let's start with parsing and basics. What is parsing? The processing involves analyzing a sequence of tokens to determine grammatical structure. We have looked at LL passing and in that LLK is a form of passers where we do left to right and leftmost derivation first, there is K symbols of lookahead. This is the entire LL class fits into top-down passing that means matches grammar rules starting from the root, the start symbol. Among them, there is LL1, most common and uses one lookahead and can handle but it cannot handle left recursion. Under LR passing, it is left to right, rightmost derivation in reverse and uses K lookahead symbols. We have looked at LR also along with LL. Now this is bottom-up passing that means it reduces input to the start symbol by recognizing rightmost derivation. Now there is LR0 and LR1 passing. Let's try to recap LR0 passing. There is no lookahead in LR0 passing. We decide shift reduction based on the current state and the input symbol. It can handle left recursion and more powerful than LL1 but can fail without lookaheads in ambiguous situations. So that is about LR0. Now there is LR1. It looks at one lookahead symbol which provides additional context to resolve ambiguities. Handles more complex grammars than LR0 and LL1. So compared to LR0 and LL1, LR1 can handle more complex grammars. Generalization in practical passing algorithms for complex languages. This is used in practice, right, the LR1 and as I said before, it handles more

complex grammar than LR0 and the LL1. Now LRK grammars, right, LRK passing, so generalization of LR1 uses K lookahead symbols, more expressive than LR1, right, capable of passing more ambiguous grammars. Generally used K, we don't use however K greater than 1 because that increases the complexity in passing table and this includes all deterministic context-free grammars. Now if we try to map this hierarchy, we will start with LL1 is a subset of LL2 so forth and you will get LLK. Then you will have LR0 and LR1, LR2 up to LRK, right, this is how you could build a hierarchy. The key difference is between LLK and LRK, right, LLK is top-down passing, starts from the top. It cannot handle left recursion and more restrictive simply and but simpler to implement, right. Whereas LRK which is bottom-up passing, reduces input to the start symbol, can handle left recursion and ambiguous grammars more expressive but higher complexity. Remember this can handle left recursion and ambiguous grammars better than the LL category. So we are at the end of this revision video wherein we looked at all the LL as well as LR passes. We tried to compare between the two and each of that we had explained with examples, what are the steps, how the passing tables are built, how the passing happens step by step etc. Thanks for watching this video and completing this module with me. We are at the end of this session. Thank you.

Introducing POS Tagging - The Task and Applications Video• .

Duration: 5 minutes

[Notes]

Hello everybody. So we are at this new module on part of speech tagging. And the lesson we will look at is application in part of speech that is POS tagging. So let's try to dive into this topic which involves understanding the basics, the tasks and the application of POS tagging. So what is POS tagging? The definition of POS tagging is that it is a process in natural language processing that involves identifying and labeling the parts of the speech such as nouns, verbs, adjectives etc. in a given text. So you identify these nouns, verbs, adjectives etc. that is called POS tagging. Example is as follows. In that sentence, the quick brown fox jumps. Each word would be tagged as determiner, the is a determiner. Then you have the adjective quick, then you have the other adjective brown, then you have the noun fox and then you have the verb jumps. This is how you would do the part of speech tagging. Extracting this text from that sentences, looking at that sentences, analyzing that sentences is called POS tagging. Why is this important? Because it provides foundational syntactic information crucial for high level NLP tasks. So this is very foundational to do high level NLP task with that information a lot can be done in NLP that is natural language processing. Now there are many techniques for POS tagging. The first is rule based approaches. Use predefined linguistic rules. This is what they are predefined rules. It is simple but may lack accuracy with complex text. Now in recent years, there has been quite an interest in statistical and machine learning approaches as well, which are hidden Markov models, that is probability based tagging or conditional random fields where you consider entire sentence context. And there has been neural network based approaches as well, which uses embedding and deep learning for improved accuracy. Now state of art models have also evolved into using transformers, they are contextualized embedding with high tagging accuracy. So lot is happening in this area and this connects to the core of this course, that is formal languages. So we will have a formal languages viewpoint on this topic. Now what are the applications of POS tagging? There is syntax analysis that is foundational for syntactic parsing and contributes to tasks like grammar checking. There is syntax analysis. Then there is named entity

recognition, NER, which helps identify named entities by distinguishing them from common nouns. This is another application. Then there is sentiment analysis, which distinguishes opinions by focusing on adjectives and adverbs. So this is another. So if you notice, there is syntax analysis, which focuses on foundation for syntactic parsing, which contributes to tasks like grammar checking. So there are range of applications from syntax analysis, named entity recognition and sentiment analysis. All these are diverse tasks, but all of them have this central idea that they emerge from being able to look at the grammar, being able to do POS tagging with that. Now there are other applications as well like information retrieval. It enhances retrieval precision by understanding word roles in queries. And there is machine translation as well, which improves translation quality by aligning syntactic structures across languages. This is also a big area by itself. So in summary, we have looked at, we got introduced to what this POS tagging is about. It's a key NLP technique for identifying word roles, enabling various downstream applications like we have discussed. The future directions involve enhancing context understanding with transformers, multilingual POS tagging for diverse languages, integrating POS tagging in broader NLP pipelines for real-time applications. So there is a lot which is happening and we are doing this module at the end of our formal languages. And as I said, we'll try to connect the theories of formal languages with the appropriate theories for POS tagging. Thanks for watching. We are now at the end of this video.

CFG and POS Tagging Video• . Duration: 5 minutes

[Notes]

Hello everybody. So we are at the module on POS tagging and we will continue to talk about applications in POS tagging. At the end of this video, we will try to understand the relevance of context-free grammars in POS tagging. So we ventured into POS tagging and since we looked at context-free grammar, it's important to ask the relevance of CFG in POS tagging. So what is context-free grammar? A grammar that defines a set of rules for sentence structure in a language. It was developed by Noam Chomsky to explain the syntax of sentences without relying on context. Now, for example, colorless green ideas sleep furiously. And this is grammatically correct but semantically nonsensical. So context-free grammar, we try to define a set of rules for sentence structure in the language and was developed by Chomsky to explain the syntax of sentences without relying on context. And this is an evolving area, we will try to understand the relevance of CFG more. So why CFG context-free grammars matter? So importance in computer science essentially for designing parsers that analyze code on natural languages used in NLP to interpret sentences structures syntactically. The relevance to POS tagging is that it helps in deciding parts of speech based on sentence structure, that is, whether run is a noun or verb in different contexts. So in some way, we can use the context-free grammar and all the theory to help in deciding parts of speech based on sentence structure. Now the basic structure of CFG, CFG consists of rules defining how words and phrases combine to form sentences. For example, a sentence is S equal to noun phrase plus the verb phrase, referred as NP plus VP. Now we focus on syntax, CFGs operate at the syntactic level, analyzing structure, not meaning. Very important. The structure itself is of the focus here and the meaning is something we are keeping aside. CFGs for NLP helps build a rule-based approach to parse sentences. Now what are parse trees and sentence structure? What is parse tree? Parse tree is a hierarchical structure representing the grammatical structure of a sentence. Each branch represents a rule from a context-free grammar. So for instance, the

sentence, I shot an elephant in my pajamas. Now there are multiple interpretations show different parse trees for the same sentence, as we will discuss. What is the inherent ambiguity in the parse trees? In the previous example, I shot an elephant in my pajamas, there could be two interpretations. The one is, I am in my pajamas when I shot the elephant, or it could be the elephant was in my pajamas. So these two interpretations are possible. Now how parse trees handle ambiguity? Different parse trees represent the distinct syntactic interpretations and shows the challenges in building accurate parses. So this is a challenge, but well-designed parse trees can handle those ambiguities. So what is the role of POS tagging and CFGs in NLP? So using POS tagging in CFGs, what we get is, we can identify parts of speech helping in disambiguating meanings. CFG defines valid combinations for the structures. We use word phrase, noun phrase, other so-called non-terminals we can use. So here is a simple example of grammar rules, where S gives NP, VP, N you have a determinant etc. you have all of these. It defines a grammar and we can process sentences under this grammar, as we will see through the forthcoming videos. So in summary, we have understood that CFGs are essential for understanding the syntax in natural language processing. Parse trees help visualize sentence structures and resolve ambiguities. POS tagging complements CFG by providing part of speech context for words. This is an exciting topic and we will venture more into this in the forthcoming videos. Thanks for watching.

POS Tagging as Parsing Video• . Duration: 8 minutes

[Notes]

Hello everybody, so we are at the module POS tagging and the lesson we are looking at is application of that, application in POS tagging. Now at the end of this video we will get an understanding of how to do POS tagging with context free grammars. So as mentioned earlier we will start, we are, we have looked at context free grammars in the past and now we will venture into context, the sensitive grammars. Let's look at this, what this POS tagging is about. So introduction to POS tagging as parsing. What is POS tagging? So it is a process of labeling words in a sentence with their part of speech, like noun, verb, adjective etc. So POS tagging as parsing, it can also be seen, the process of POS tagging can also be seen from the perspective of parsing. So in this classical approach we treat POS tagging as a parsing task using rules from context free grammar. So the objective here is in the classic approach we will treat POS tagging as a parsing task and we will apply some context free grammar. Now context free grammar and POS tagging. So what is context free grammar? We have looked at context free grammar. It defines a syntactic structure of sentences through grammar rules and CFGs in POS tagging. So each rule in a CFG helps identify possible POS tags by analyzing sentence structure. So that means CFG will help us in identifying a sentence structure and that will lead us to give this POS part of speech tags. Take for example simple rule S gives NP VP. That is a sentence can be broken down into a noun phrase followed by a verb phrase. This nicely fits into the CFG definition. Then naturally we ask why CFG for POS tagging? Because it provides a structured rule based approach that systematically can tag words. Instead of coming up with ad hoc ways to solve this problem, approach where we can use CFG is much better as we will see. So let's take an example of a sentence. The cat sat on the mat. This is the sentence and now let's try to apply the CFG rules for POS tagging. For that we have a context free grammar to analyze such sentences. Here the different productions are as follows. S gives NP followed by VP. So sentence consists of a noun phrase plus a verb phrase. So any sentence or

sentences of this form can be split into noun phrase followed by a verb phrase. The noun phrase itself can be seen as a determiner plus a noun. That is the noun phrase. Now if you go down further, that is the verb phrase that is left. How do you define a verb phrase? The verb phrase itself consists of a verb plus a prepositional phrase. And then we can break down the prepositional phrase into preposition plus a noun phrase. So this is a beautiful example of application of context free rules for POS tagging. Now at the end of it, if we do this process, then we can assign the following tags. The is a determiner, cat is a noun, sat is a verb, on is a preposition and the is a determiner again and mat is a noun. So at the end of it, we can get these tags, which are called the POS tags. Now the role of parsers in POS tagging, so using parsers with context free grammars. So the parsers can help analyze sentence structure using context free grammar rules to generate POS tags as we just saw. And there are different types of parsing, there is top-down parsing and bottom-up parsing. The top-down parsing starts from a sentence level breaking down into smaller, bottom-up parsing will start with individual words and build up to form the full sentence structure. The classic view is that we treat POS tagging as a parsing problem, where parsing rules guide tagging. So we can use different parsers for doing, different parsers and the context free grammars for doing POS tagging. So we will look at the example further, right? This the cat sat on the mat, right? So we begin with the sentence rule as gives NP, VP, right? Break down NP as determiner and the noun, right? So which is the cat, then we can break down VP as the verb and the prepositional phrase, right? So which is sat on the mat. Further, we can parse EP as a preposition followed by the noun phrase, which is on the mat, right? Now we can continue like this and at the end of it, we can, right? You get a parse tree where you for S, under S you have the NP and the VP, right? So NP again is the determiner and the noun, VP again breaks down into V the verb followed by the prepositional phrase recursively. The prepositional phrase itself can be broken down into the preposition followed by the noun phrase, right? And in this case, the noun phrase itself is broken down into the determiner and the noun, right? So as a result, we using a parse tree. This is how the parse tree for the earlier example will look like, right? And the POS tags are as follows, right? Which are listed here as also discussed earlier. So at the end of it, you build a parse tree and you get the POS tag. So that is the application we are trying to understand here. Now there are limitations of POS tagging on parsing as well, right? Some of the challenges are rigidity. CFG rules may not cover all syntactic structure in natural languages. Certain words, for instance, the problem of ambiguity as well, right? For instance, in certain words like run can be both nouns and verbs. So CFG alone may not resolve this, right? And there is complexity. Real world sentences often involve non-structures and making mutual CFG creation expensive. So modern solutions, we have moved towards machine learning and statistical models which have largely replaced rule-based POS tagging, but still it is a very foundational. The POS tagging using context-free grammar is very foundational and very efficient way to parse this kind of strings. So at the end of it, we learned POS tagging as an use case for parsing using context-free grammars. We looked at parsers to systematically assign POS tags. While effective, these rule-based approaches have limitations in handling ambiguity and complexity. But as I said, this is a good starting point to understand what POS tagging is about and for simple grammars and languages, these such solutions may suffice. So thanks for watching, we are at the end of this video.

Introduction to CSG Video• . Duration: 7 minutes

[Notes]

Hello everybody. So we are at the module POS tagging and the lesson we are looking at is context sensitive grammar. The end of this video will understand the reason for introduction of context sensitive grammar. And we are doing this in the context of POS tagging. So what is context sensitive grammars, right? So it is a type of grammar in the Chomsky hierarchy more powerful than context free grammar. We have looked at context free grammar in the past. This context sensitive grammar are much more powerful than context free grammar. They can capture language structures where the context around a symbol affects its production rules, right? So this is very different from what we have seen here. The context around the symbol will also affect the type of production rules. One of the key feature here is that the production rules can depend on the surrounding context of words making context free grammars more flexible for complex language. This power which is obtained by looking at surrounding context makes it more flexible for complex languages. That is why we are looking at it and it's also part of the general scheme of formal languages. Then this is one of it. Let's try to define context free grammar more precisely. Of course, like we with that POS tagging application in mind. So a context free grammar consists of four components, right? One is the set of non terminal symbols, set of terminal symbols, set of production rules and a start symbol. Now however, each rule in context sensitive grammar has the form $\alpha \rightarrow \beta \gamma$, right? Now where the non terminal α , β are strings, the symbols in this grammar and γ is a non empty string, right? Now what this means is, so let's look at the length requirement first. In any rule, the output right inside is at least as long as the input. And ensuring no reduction in string length. So this is very important and we'll see why it is defined in this fashion as we go through the example. Now consider this language $a^n b^n c^n$, right? Where the number of each symbol must match respecting the context. Now if we try to derive the production rules, right? Then for this particular grammar, we'll get something like this, right? So the goal here is to transform s into a sequence where the counts of a , b , c are identical and we have four productions here. Now the context governs how symbols are replaced. So this is the distinct part for context sensitive grammar. Here the context governs how the symbols are replaced, ensuring balance. In this case, for this example, ensuring balance across the string. So for instance, you see the position of B , capital B relative to capital C in the second production, right? You have CB replace BC , right? So you are playing around with the context in which C and B exist and you have a production around that. And this defines, such productions define context sensitive grammars. The application of context sensitive grammars, there are, one is the programming languages, of course. So some features in complex programming languages require context sensitive rules such as type checking, variable scoping. This cannot be done with context free grammars and there is natural language process useful in languages with agreement rules like gender or case matching where surrounding words affect interpretation, right? This is ample scope for context sensitive in natural language processing as well. Now this context sensitive grammar has also found applications when we try to model dependencies in DNA or RNA sequences where certain patterns depend on surrounding elements as well. Let's try to compare this to other grammars. Comparison with context free grammars. Context sensitive grammars allow the rule to be context dependent and in more complex languages but requiring more computational resources, right? Now the context free grammars rules are independent, are independent of surrounding symbols making them less flexible but computationally simpler, right? So there is trade off here of the complexity of the languages versus the computational resources needed, right? Only context sensitive grammars can handle more complex languages but require more computational resources. Now in the larger scheme of CHOMP's hierarchy, context sensitive grammars or CSGs

are type 1 grammars. They sit between context free grammars type 2 and unrestricted grammars type 0. Now there are challenges of complexity as I mentioned. So computational complexity is one such. Modeling context sensitive languages is generally more complex than context free languages often requiring non-dataphenistic linear space. There are practical limitations as well. Few efficient passes context sensitive grammars exist making them impractical for real time applications, right? And due to their complexity many languages modeled in context sensitive grammars are approximated by simpler grammar types where possible, right? Now these are some of the practical limitations. Now context sensitive grammars in theoretical studies, they are often studied in theoretical computer science to understand the limits of computational language models, right? They have been studied, you know, in a theoretical sense as well. So the thing to be highlighted here is the computational complexity and the practical limitations, right? So in summary, we looked at a formal definition of context sensitive grammars and a type of formal grammar that allow production rules to depend on the surrounding context of symbols making them more powerful than context free grammars. And we looked at a language, right, $a^n b^n c^n$. The production rules shown for this language, I encourage you to expand on those production rules and see how such a string, right, belonging to this language is derived, right? Here the number of each symbol must match, right? So context sensitive grammars rules ensure this balance by respecting context. And when you work out this example, you will yourself see how context sensitive grammars play a role. Also, as a summary, we looked at applications, right? Programming languages for type checking and scoping. That's where the applications are. Natural language processing is another application area and biological sequence analysis. Finally, the challenges, right? Passing context sensitive grammar is computationally intense, making them impractical for many real time applications. But they remain important for theoretical computer science studies. So now we are at the end of this video. Thank you for watching.

Real World Example 1 Video• . Duration: 6 minutes

[Notes]

Hello everybody, so we are at the module POS Tagging and we will look at the lesson Context Sensitive Grammar. So let's start with the example for understanding Context Sensitive Grammar. So there is something called as type checked arithmetic expression and this can fit well into Context Sensitive Grammar. So what would be the parts of this Context Sensitive Grammar, we will see now. There is non-terminals. So what are the non-terminals in this type checked arithmetic expression? Firstly you have the program that is the main structure that hold all the statements, that's the program. Then individual statement is assignment of a variable to a value. Under the statement you can have expressions which will involve variables and values and then there is variables and the types for type annotation. So these are all the non-terminals for type checked arithmetic expression. Then you will have terminals as well which could be variable names, literal values for example integers or strings etc. Then there is types, specific types like integer, string etc. operators and assignment. So this put together these are all the terminals that are possible. Now we will continue looking at the grammar in more detail. So if we see largely the program structure, the program is split into statements. So you could define program, give statement, program or null. In this way you can recursively build a program which consists of many statements. Then statement itself can be variable equal to expression of a certain type. And expression itself can be integer, for

instance it could be integer gives expression plus expression and it is of the type integer. Each integer alone can be of different numerals and then expression can be a string itself. So you have variables which is of type integer and then variables of the type string. Type itself can be integer or a string. So in this way you use notion of context sensitive grammar for building an entire program. We will see why this is called context sensitive shortly. So again, so that is the question, that is the key question, why is this grammar context sensitive? Now the grammar is context sensitive because for two reasons, the type of each expression and variable depends on the surrounding context. So in the surrounding context there could be specific type which will dictate what is the type of each expression and variable. Then rules apply differently based on the specific type of variables, ensuring only compatible types are used in operations and assignments. We will look at this how this plays out with some examples. But these are the two reasons why we will want to use a context sensitive grammar for a program. Here are few examples. So you start with, you have x equal to 3 plus 5 int or you have s equal to hello string. Now let's try to parse each of this. So how would we parse x equal to 3 plus 5 colon integer? How will we parse this? So statement will give x equal to, the statement is of the form x equal to expression plus expression and it is of the type int. So this in turn, expression plus expression int is 3 plus 5 gives, so we get to the terminals, 3 plus 5 int. Both these values and operations are integers. So the context here in this specific example, the context determines what 3 plus 5, what is the type for 3 plus 5 and rightly it is integer. Each of the expressions are integers and the overall expression there also indicates that this expression, this x resulting from this addition of expression will also be int. Now let's parse the other expression, s equal to hello string. This can be written as statement gives s equal to expression string, where again expression string translates to hello. So the assignment in this case, the assignment matches the declared type string. We have done an assignment here and it also matches the declared type string. So in this way, we see why context sensitive grammars apply for programs and also for individual parts of the program like statements, expressions, etc. So what is the comparison to other grammars? So let's think about comparison to context free grammars. So context sensitive grammars allows rules to be context dependent, handling more complex languages but requiring more computational resources. Then what about context free grammars? So rules are independent of surrounding symbols making them less flexible but computationally simple. So essentially there is a trade-off here, we would rather go for context sensitive grammars for defining a program because it gives us the ability to handle more complex languages. Of course, it comes at the cost of higher computational resources. And the next question is what is the placement in the Chomsky hierarchy? Context sensitive grammars are type 1 grammars in the Chomsky hierarchy. They sit between the context free grammar and unrestricted grammars. So we have looked at a few examples of usage of context sensitive grammar to define a program. We looked at a few expressions of the program and this is where context sensitive grammar plays a role. So thank you for watching this video, we are at the end of it.

Real World Example 2 Video• . Duration: 4 minutes

[Notes]

Hello everybody, so we are at the module POS tagging and we are looking at context sensitive grammar. We will try to look at one more example of where context sensitive grammar comes into play. So the example we will look at is the grammar taxonomic classification. So the grammar that is

used for taxonomic classification, right? Here the non-terminals will be classification, the main structure holding the full taxonomy. So there is that classification non-terminal. Then you have the family, the family name, genus, genus name under a specific family and then you have the species, the species name under a specific genus, right? Now the terminals itself, there will be various family names like Latin based names, like Felidae or Canidae, etc. There will be genus names like Felis, Canis, etc. Species names like Catus, Lupus, etc. So here the Latin name rules like genus and species names must be italicized and lowercase is used for species, right? So how will the grammar look like? The grammar will look like as follows, there is classification gives family colon genus and species, right? Again classification and the null string, this will continue. So the family genus species put together create some kind of hierarchical rules. This ensures that each species is classified under the correct genus and family, right? And context sensitive rules also prevent a species from being classified under genus outside its family, right? So this is where the context sensitive nature of the grammar becomes handy. Now let us look at a few rules, such rules we follow to describe this grammar. So we have a family which can be Felidae gives genus Felis. Now genus Felis itself can give species of like Catus or Silvestris, etc., right? Then we have another example, family Canidae gives genus Canis and then genus Canis gives species which can be Lupus, Familiaris, etc., right? So this is how, this is one example. So what are we doing here, right? So there is some kind of enforcement of genus names, you know, they are capitalized and species names are lowercase and italicized. So here are some examples, right? Genus can be Felis or Canis and species can be these four, the listed like Catus, Lupus, etc. Now it is also important to understand that species names must be placed within species context under their respective genus. So this is also where context sensitive nature comes into play. So genus Felis gives species Catus or Silvestris, right? So this is an example and we have another genus Canis example also shown. Now let us look at a few examples. So we are given the task of parsing this expression, Felidae, Felis, Catus and then one more. We will see how those rules for this grammar are come into play to parse this expression. So let us first start with Felidae colon Felis Catus, how will we parse this? So classification gives family genus species, right? So family Felidae is a family and then it gives genus Felis, right? Similarly, genus Felis gives species Catus. So we can use the structure as you see the structure is maintained of the grammar and this helps us parse this input expression. Similarly, we can do for the next expression Canidae colon Canis Lupus and we can understand which of these in this expression what is the species, what is the genus, what is the family, right? So in this way, this is another application of context sensitive grammar, right? So we have looked at one more example of context sensitive grammar in this case for taxonomy and for doing this kind of classification, right? So thanks for watching this video, we are now at the end of it.

Applications Video• . Duration: 6 minutes

[Notes]

Hello everybody. So we are at the module POS tagging and the lesson we are looking at is context sensitive grammar. Now in this video we will look at some of the applications of context sensitive grammar. We have already looked at a few and in this video we will try to summarize and get a overall idea of what kind of applications are come into play when we talk about context sensitive grammars. So let us revisit context sensitive grammars first. So context sensitive grammars allow rules that depend on the context around symbols. Unlike simpler grammars, context sensitive

grammars can capture relationship between elements that rely on position or neighboring symbols and context sensitive grammars are more powerful than context free grammars as we have seen, describes languages that require context for certain rules to apply. Applications of context sensitive grammars involve complex systems where context is essential such as languages, biologies, structured data processing etc. So one application is natural language processing. Context sensitive grammars are used in NLP to handle linguistic rules that require context especially in cases of agreement like the number or the gender agreement as we will shortly see. For example, let us look at the number agreement. So sentences in English must have agreement between subjects and verbs based on the number. For instance, we will say the dog runs which is singular versus the dogs run which is plural. So context sensitive grammar can enforce this agreement by defining rules that depend on the subject's plurality to apply the correct form of the verb. Then you have gender agreement also. So in languages like French, adjectives must agree in gender and number with nouns requiring a grammar that can respond to these context based rules. So this we were talking about natural language processing and how context sensitive grammars come into play. Now there is also biological sequence analysis. So in bioinformatics, context sensitive grammars help model complex biological sequences like the NA and RNA which have dependencies not easily captured by simple grammars. For instance, when you talk about protein folding in that you look at protein structures, they have hierarchical rules based on amino acid sequences that dictate their 3D structure. These folding rules depend on the presence of certain amino acids in specific contexts. That is where context sensitive grammars can come into play. So context sensitive grammars can model these dependencies to predict how proteins fold based on the surrounding amino acid sequences. The benefit is that it enables understanding and predicting the structure and functions of proteins critical for fields like drug discovery and genetic research. Then we talk about third application which is markup languages and formatting. So a context sensitive grammars can define complex rules in document formatting system especially when formatting depends on nested or contextual elements. For example, in HTML or XML validation, web languages like HTML or XML have nested structure where elements must be correctly nested. Now context sensitive rules can enforce that certain elements only appear when specific tags making sure the document structure follows strict validation. So that's an example that's a third example of context sensitive grammar. What's the benefit here? It helps ensure structure and consistent document layouts especially for applications that involve automated document generation or validation. Now there is fourth application as well which is compilers and programming languages. So context sensitive grammars are applied in programming language compilers to manage scope rules and variable declarations that depend on context. For example, variable scope and type checking. Compilers and programming languages often have strict rules on where variables can be declared and assessed which vary depending on the location in the code. So CFGs can enforce these context sensitive rules ensuring validations are only used in appropriate scopes and help in type checking for correctness. What is the overall benefit we see in compilers and programming languages? It ensures correct syntax and logical structure in programming reducing error and maintaining code integrity. The fifth one is hierarchical data modeling. So CFGs can describe hierarchical data models especially where child parent relationships have strict contextual dependency. So some examples are taxonomies and classification systems some of which we have already seen. For instance, in biological taxonomies each species belongs to a specific genus, family and so on and names follow specific Latin based conventions. So CFGs can ensure that each level follows the correct structure enforcing rules based on the context. For instance, species must match the genus. So benefits here

are it facilitates consistent data organization which is essential for fields like biological research, library sciences, organizational hierarchies etc. So in all we have looked at several of the applications for context sensitive grammar. It should give you an idea of where context sensitive grammar are useful which is a major focus, one of the key focus areas for this module. I hope you have got a good sense of the applications and as illustrated with few examples and now a broader view of the applications. So now we are at the end of this video. Thank you.

Intro to Grammar Hierarchy Video• . Duration: 6 minutes

[Notes]

Hello everybody. So we are at the module POS Tagging and the lesson we are looking at is Chomsky Hierarchy. So let's dive into this topic. So at the end of this video, we will try to get an understanding of what Chomsky Hierarchy is and what's the connection between this and some of the other topics we have looked at so far. So the Chomsky Hierarchy was proposed by linguist Noam Chomsky and it classifies types of formal grammars based on their complexity and power. So it's a theoretical framework used to understand different language classes in computer science, linguistics, and automata theory. The hierarchy consists of the following different types of grammars. So there is type 0, which is unrestricted grammar, then there is type 1, context-sensitive grammar, type 2, which is context-free grammar, and type 3, which is regular grammars. So it starts with 0, 2, all the way 3. Now it can be diagrammatically explained as well where type 0 is the unrestricted grammar, it's a larger set, then you have the context-sensitive grammar, followed by the context-free grammar, and at the very center you have type 3 or regular grammar. So type 0 is unrestricted grammar. So definition of unrestricted grammar is as follows, so type 0 grammars have no restriction on their production rules, making them the most powerful. And production rules are of the form $\alpha \rightarrow \beta$, where α and β are strings of variables and terminals, with α containing at least one non-terminal. So that's the formal definition. Now it is unrestricted grammar, so type 0 grammars can generate any language that is recursively enumerable. So these grammars correspond to Turing machines, making them as powerful as any algorithmically describable process. That's the computational model side of unrestricted grammar. Now let's look at context-sensitive grammar. So it's a type 1 grammar and have rules where each production must have at least as many symbols on the right-hand side as on the left, so which I am preserving context. So productions are of the form $\alpha A \beta \rightarrow \alpha \gamma \beta$, where A is a non-terminal, and α and β are strings of terminals and non-terminals, and γ is a non-empty string. So the key part here is the type 1 grammar have rules where each production must have at least as many symbols on the right-hand side as on the left, and it should preserve context which is explained through the formal definition. Now languages, what are the languages that are generated by context-sensitive grammar? These grammars generate context-sensitive languages, which include certain complex patterns not possible in the lower levels. Now as far as computational model is concerned, CFGs correspond to linear bounded automata which are Turing machines with limited memory. And there is type 2 which is context-free grammars. The definition of it is as follows. So type 2 grammars have rules where a single non-terminal symbol is transformed independently of surrounding symbol. So productions are of the form $A \rightarrow \gamma$, where A is a non-terminal and γ is a string of terminals and non-terminals. So what are the languages generated by CFGs? So CFGs generate context-free languages, which are commonly used in

programming languages and syntactic structure in natural languages. So it can describe these programming languages and syntactic structure in natural languages. CFGs are recognized by pushdown automata, which use a stack-based memory structure. So that's the computational model for CFG, that is pushdown automata based on stack-based memory structure. Now finally, there is regular grammars. What's the definition of regular grammars? So these are type 3 grammars, have very strict rules where productions involve a non-terminal transforming into a terminal optionally followed by a non-terminal. So more formally, this can be explained in the form of a production rules where $A \rightarrow a$, $A \rightarrow aB$ or $A \rightarrow AB$. So A and B are non-terminals and a is a terminal. So this is the formal definition of regular grammar. What are the languages generated? Regular grammars generate regular languages, the simplest language class, which includes patterns like sequences and repetition. What is the computational model? Most regular languages are recognized by finite automata, both deterministic and non-deterministic finite automata. So we have looked at the Chomsky hierarchy and we looked at how the hierarchy described different types of language structures, right from simple to complex. We looked at regular grammars, which are type 3. So they are recognized by finite automata and used for simple patterns. We looked at context-free grammars. These are recognized by pushdown automata and handle nested structures. We also looked at context-sensitive grammars, which are recognized by linear bounded automata and capture complex dependencies. And then finally, unrestricted grammars also we saw, which are recognized by Turing machines and capable of expressing any computable language. So thanks for watching this video till the end and with this we have a good summary of the Chomsky hierarchy. Thanks for watching.

Overview Video• . Duration: 3 minutes

[Notes]

Hello everybody, so we are at the module POS Tagging and let's look at view of what we have seen so far. This video can also act as a good summary of what we have seen throughout this course. So as I mentioned, this can also be seen as a revision of all the modules of this course. So what is the course overview? So this course provides a foundational understanding of formal languages theory and automata with a mix of theoretical and practical perspectives as we have seen. The emphasis on things like regular languages, context-free languages, context-sensitive languages and the Chomsky hierarchy. We have looked at applications in lexical analysis, parsing, POS, tagging for NLP as we have explored some of these topics. Now as far as regular languages are concerned, the topics we looked at is regular expression, the definitions and pattern matching, deterministic finite automata, structure, transition and the language acceptance, regular grammar rules that generate regular languages. The connections we established for regular languages are defined by DFAs and regular expressions and regular grammar. We did not go into non-deterministic finite automata to keep the scope. Now we also looked at context-free languages. In that we looked at structure and generation of context-free languages. We also looked at closure properties and the language operators. Now what is the connection with the other topics? So CFGs are foundational for implementing passes. We looked at several types of passes. We introduced top-down parsing and bottom-up parsing techniques etc. Now more about the parsing techniques, there is top-down parsing, we looked at predictive parsing and recursive descent parsing. The bottom-up we looked at shift-reduced parsing and LR parsing. So what is the parser construction technique? Introduction to

basic techniques for generating parsers, which is a part of the parser construction process. Emphasis on parsing as a core application of context-free languages. Now then to give a broader perspective we looked at Chomsky hierarchy in that there is type 3 which is regular grammars, then type 2 context-free grammars, type 1 context-sensitive grammars and type 0 is unrestricted grammars. What was the purpose? To understand the classification of languages and the corresponding automata models. As you know the corresponding automata models are respectively finite automata, CFGs. Then you have context-sensitive grammars which define more complex syntax rules and then the type 0 corresponds to Turing machine level complexity. So with that we are at the end of this course. I hope the course was enjoyable and we were set with the target of introducing formal languages and also looking at the applications of formal languages. We chose topics that emphasize these aspects and we have laid down several techniques and we have laid down several tools that you could use in some of the other courses you will go through as you progress through these programs or in a work setup or an internship setup you could apply some of the techniques you have learnt in this course. Once again I hope this was informative, educative and enjoyable. Thanks for watching all the modules. Thank you.